

Undergraduate Topics in Computer Science

James T. Streib  
Takako Soma

# Guide to Data Structures

A Concise Introduction Using Java



 Springer

The Springer logo consists of a stylized white chess knight piece on a black square, positioned to the left of the word "Springer" in a serif font.

---

# Undergraduate Topics in Computer Science

## Series editor

Ian Mackie

## Advisory Boards

Samson Abramsky, University of Oxford, Oxford, UK

Chris Hankin, Imperial College London, London, UK

Dexter C. Kozen, Cornell University, Ithaca, USA

Andrew Pitts, University of Cambridge, Cambridge, UK

Hanne Riis Nielson, Technical University of Denmark, Kongens Lyngby, Denmark

Steven S. Skiena, Stony Brook University, Stony Brook, USA

Iain Stewart, University of Durham, Durham, UK

Undergraduate Topics in Computer Science (UTiCS) delivers high-quality instructional content for undergraduates studying in all areas of computing and information science. From core foundational and theoretical material to final-year topics and applications, UTiCS books take a fresh, concise, and modern approach and are ideal for self-study or for a one- or two-semester course. The texts are all authored by established experts in their fields, reviewed by an international advisory board, and contain numerous examples and problems. Many include fully worked solutions.

More information about this series at <http://www.springer.com/series/7592>

---

James T. Streib · Takako Soma

# Guide to Data Structures

A Concise Introduction Using Java



James T. Streib  
Illinois College  
Jacksonville, IL  
USA

Takako Soma  
Illinois College  
Jacksonville, IL  
USA

ISSN 1863-7310                      ISSN 2197-1781 (electronic)  
Undergraduate Topics in Computer Science  
ISBN 978-3-319-70083-0              ISBN 978-3-319-70085-4 (eBook)  
<https://doi.org/10.1007/978-3-319-70085-4>

Library of Congress Control Number: 2017957684

© Springer International Publishing AG 2017

This work is subject to copyright. All rights are reserved by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.

The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.

The publisher, the authors and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, express or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Printed on acid-free paper

This Springer imprint is published by Springer Nature  
The registered company is Springer International Publishing AG  
The registered company address is: Gewerbestrasse 11, 6330 Cham, Switzerland

---

# Preface

## Purpose

The purpose of this text is to provide the reader a concise introduction to data structures and associated algorithms. This text is intended for a second-semester course in programming using the Java programming language prior to a course on advanced data structures and algorithms. It is a continuation of the text *Guide to Java: A Concise Introduction to Programming using Java* written by the authors and published by Springer Verlag London Limited [2].

## Comparison and Need for This Text

There are a number of introductory texts on data structures using the Java programming language. Many of these texts are very comprehensive, but unfortunately, they sometimes seem to cover so many details that might make it difficult for a beginning programmer to discern which concepts are the most relevant. The old adage that “one can’t see the forest for the trees” might apply in some instances because if one is too busy with the many details when trying to learn data structures one might not have a clear grasp of the fundamental concepts necessary to completely understand the material. There are also some shorter texts, but sometimes they seem to cover important concepts very quickly which can confuse readers too. This text attempts to fulfill the need for an introduction to data structures by helping the reader to concentrate on the fundamentals which in turn allows the text to be more concise and help the reader remain focused on the key concepts. The result is that the reader can learn data structures quickly and also have a good foundation to learn more complex topics later.

## Features of This Text

As mentioned above, this text is a concise introduction to data structures which is accomplished by concentrating on the fundamentals. This text is written in the same style as the previously mentioned *Guide to Java* text. It provides many examples and illustrations and uses visual contour diagrams to illustrate object-oriented semantics as needed. Also as before, in some paragraphs of the text, questions are asked of the reader to help them interact with the material and think about the subject matter presented.

The text starts with data structures using arrays to reinforce array concepts learned previously and then introduces linked data structures to compare with array-based structures. A data structure is first introduced with a simple data type to help with understanding basic concepts, and then, it is reinforced using generic data types. In addition, elementary algorithm analysis is introduced and discussed as needed throughout the text.

To help further reinforce concepts, each chapter has one or more complete programs to illustrate many of the concepts presented and to also help readers learn how to write programs on their own. In addition, for review and practice, there are summaries and exercises provided at the end of each chapter. Further, in the appendices at the end of the text, there are answers to selected exercises and a glossary of important terms. A summary of all these features is listed below:

- Stresses the fundamentals.
- Provides many examples and illustrations.
- Begins with array-based data structures to reinforce array concepts learned previously.
- Follows with linked data structures for comparison and to reinforce methods.
- Uses both primitive and generic data types in each chapter.
- Uses contour diagrams to illustrate object-oriented concepts.
- Asks readers questions to help them interact with the material.
- Contains one or more complete programs in every chapter.
- Provides chapter summaries.
- Includes exercises at the end of each chapter, with selected answers in an appendix.
- Has a glossary of important terms.

## Overview of the Chapters

After an overview of preliminary concepts, this text introduces stacks and queues using arrays along with a discussion of array-based lists. This is followed by an introduction to linked lists and the implementation of stacks and queues using references. Next, there is an introduction to binary trees, a discussion of various

sorting techniques, heaps, and hashing. The appendices include a glossary and answers to selected exercises. Lastly, there is a reference and useful Web site section and an index. The following provides a brief synopsis of the chapters and appendices:

- Chapter 1 reviews and discusses various preliminary concepts.
- Chapter 2 introduces stacks using arrays.
- Chapter 3 illustrates queues using arrays.
- Chapter 4 discusses lists using arrays.
- Chapter 5 introduces lists using references and objects.
- Chapter 6 examines linked lists.
- Chapter 7 explores stacks and queues using references.
- Chapter 8 introduces binary trees.
- Chapter 9 explores sorting algorithms.
- Chapter 10 discusses heaps.
- Chapter 11 introduces hashing.
- Appendix A contains a glossary of key terms.
- Appendix B provides answers to selected exercises.

Note that the above order is the authors' preferred sequence; however, it is understood that some instructors, professionals, and independent students might want to pursue some topics in a different order. As given, all linked list structures follow the array-based structures, but alternatively, one could have Chaps. 5 and 6, following Chap. 1, and then, Chaps. 2–4 could be examined later. Further, parts of Chap. 9 can be introduced earlier at the instructor's discretion. Of course, other combinations are possible given the preference of the instructor or the reader's background.

## Scope

As mentioned previously, this text concentrates on the fundamentals of data structures such as stacks, queues, lists, (using both arrays and links), sorting, and elementary binary trees, heaps, and hashing. Since it concentrates on the fundamentals, it might not cover all the details that are found in some other texts, and if necessary, these topics can be supplemented by the instructor or reader, or covered in a subsequent text and/or course.

## Audience

This text is intended primarily for readers who have had a previous course or used a text in programming using Java. Some of the concepts and specific skills needed are discussed and reviewed in Chap. 1. As mentioned previously, this text sequentially

follows the authors' previous text *Guide to Java*, but this does not preclude a reader from having used other texts as well. Note that although one might be able to read this text with knowledge of another language such as C++, this text does not review the Java programming language and previous knowledge of Java is highly recommended. In addition to being a classroom text for a second-semester course in programming in preparation for a subsequent advanced course in data structures and algorithms, it can be used as a self-study guide in either academe or industry.

## Acknowledgements

The authors would like to thank the reviewers Mark E. Bollman of Albion College, James W. Chaffee of the University of Iowa, and Naomi E. Hahn of Illinois College. Also, the authors would like to acknowledge the students of Illinois College who have read and used various sections of the text in classroom. On a personal note, James Streib would like to thank his wife Kimberly A. Streib and son Daniel M. Streib for their patience, and Takako Soma would like to thank her family and friends, near and far.

Note that Java is a registered trademark of Oracle and/or its affiliates and that Windows is a registered trademark of Microsoft Corporation in the USA and/or other countries.

## Feedback

The possibilities of errors exist in any text, and therefore, any corrections, comments, or suggestions are welcome and can be sent to the authors via the e-mail addresses below. In addition to copies of the complete programs presented in the text, any significant corrections can be found at the Website below.

Website: <http://www.jtstreib.com/GuideDataStructures.html>

September 2017  
Jacksonville, IL, USA

James T. Streib  
[james.streib@jtstreib.com](mailto:james.streib@jtstreib.com)  
Takako Soma  
[tsoma@ic.edu](mailto:tsoma@ic.edu)

---

# Contents

|          |                                                                 |           |
|----------|-----------------------------------------------------------------|-----------|
| <b>1</b> | <b>Preliminary Concepts</b>                                     | <b>1</b>  |
| 1.1      | Introduction to Data Structures                                 | 1         |
| 1.2      | Prior Concepts and Skills Needed                                | 1         |
| 1.3      | Elementary Algorithm Analysis                                   | 2         |
| 1.4      | Abstraction and Interfaces                                      | 8         |
| 1.5      | Java API                                                        | 12        |
| 1.6      | Complete Program: Implementing Interface                        | 13        |
| 1.7      | Summary                                                         | 14        |
| 1.8      | Exercises (Items Marked with an * Have Solutions in Appendix B) | 15        |
| <b>2</b> | <b>Stacks Using Arrays</b>                                      | <b>17</b> |
| 2.1      | Introduction                                                    | 17        |
| 2.2      | Analysis and Design                                             | 17        |
| 2.3      | Stack Class: Data Members and Methods                           | 21        |
| 2.4      | Reversing Integers                                              | 29        |
| 2.5      | Generic Types                                                   | 32        |
| 2.6      | Prefix and Postfix Expressions                                  | 37        |
| 2.7      | Complete Program: Checking for Palindromes in Strings           | 41        |
| 2.8      | Summary                                                         | 45        |
| 2.9      | Exercises (Items Marked with an * Have Solutions in Appendix B) | 46        |
| <b>3</b> | <b>Queues Using Arrays</b>                                      | <b>49</b> |
| 3.1      | Introduction                                                    | 49        |
| 3.2      | Analysis and Design                                             | 52        |
| 3.3      | Queue Class: Data Members and Methods                           | 58        |
| 3.4      | Program Using the <code>QueueArray</code> Class                 | 62        |
| 3.5      | Complete Program: Simulating a Scheduling Algorithm             | 67        |
| 3.6      | Summary                                                         | 73        |
| 3.7      | Exercises (Items Marked with an * Have Solutions in Appendix B) | 73        |

|          |                                                                          |     |
|----------|--------------------------------------------------------------------------|-----|
| <b>4</b> | <b>Lists Using Arrays</b> .....                                          | 77  |
| 4.1      | Introduction .....                                                       | 77  |
| 4.2      | Analysis and Design .....                                                | 78  |
| 4.3      | ListArray Class: Data Members and Methods .....                          | 82  |
| 4.3.1    | The Insert Method .....                                                  | 83  |
| 4.3.2    | The Delete Method .....                                                  | 86  |
| 4.3.3    | The Search Method .....                                                  | 89  |
| 4.3.4    | Complete ListArray Class .....                                           | 92  |
| 4.4      | Simple Test Program .....                                                | 92  |
| 4.5      | Complete Program: Checking Opcodes<br>in Assembly Language .....         | 97  |
| 4.6      | Summary .....                                                            | 105 |
| 4.7      | Exercises (Items Marked with an * Have Solutions in<br>Appendix B) ..... | 106 |
| <b>5</b> | <b>Lists Using Objects and References</b> .....                          | 109 |
| 5.1      | Introduction .....                                                       | 109 |
| 5.2      | References .....                                                         | 110 |
| 5.3      | Objects .....                                                            | 111 |
| 5.4      | The Node Class .....                                                     | 114 |
| 5.5      | Creating Links .....                                                     | 116 |
| 5.6      | Output .....                                                             | 121 |
| 5.7      | Test Program .....                                                       | 124 |
| 5.8      | Output Using Recursion .....                                             | 126 |
| 5.9      | Complete Program: Outputting Items in the List Using<br>Recursion .....  | 128 |
| 5.10     | Summary .....                                                            | 130 |
| 5.11     | Exercises (Items Marked with an * Have Solutions in<br>Appendix B) ..... | 131 |
| <b>6</b> | <b>Ordered Linked Lists</b> .....                                        | 135 |
| 6.1      | Creating the Insert Method .....                                         | 135 |
| 6.1.1    | Inserting in the Middle of the List .....                                | 135 |
| 6.1.2    | Inserting at the End of the List .....                                   | 142 |
| 6.1.3    | Inserting at the Beginning of a List .....                               | 146 |
| 6.1.4    | Inserting into an Empty List .....                                       | 148 |
| 6.1.5    | The Insert Method .....                                                  | 150 |
| 6.2      | Creating the Delete Method .....                                         | 151 |
| 6.3      | The LinkedList Class .....                                               | 154 |
| 6.4      | Test Program .....                                                       | 154 |
| 6.5      | Doubly Linked Lists .....                                                | 157 |
| 6.6      | The Node Class as an Inner Class .....                                   | 160 |
| 6.7      | Complete Programs: List of User Defined Objects .....                    | 163 |
| 6.7.1    | Generic LinkedList Class with External<br>NodeGeneric Class .....        | 163 |

|          |                                                                                |            |
|----------|--------------------------------------------------------------------------------|------------|
| 6.7.2    | Generic <code>LinkedList</code> Class with Internal<br>NodeGeneric Class ..... | 167        |
| 6.8      | Summary .....                                                                  | 170        |
| 6.9      | Exercises (Items Marked with an * Have Solutions in<br>Appendix B).....        | 171        |
| <b>7</b> | <b>Stacks and Queues Using References.....</b>                                 | <b>173</b> |
| 7.1      | Stack.....                                                                     | 173        |
| 7.2      | Stack Test Program: Reversing Integers .....                                   | 177        |
| 7.3      | Queues .....                                                                   | 178        |
| 7.4      | Queue Test Program.....                                                        | 183        |
| 7.5      | Comparison of Arrays Verses References.....                                    | 186        |
| 7.6      | Complete Program: Undo Button .....                                            | 186        |
| 7.6.1    | Graphical User Interface .....                                                 | 187        |
| 7.6.2    | User Defined Frame.....                                                        | 187        |
| 7.6.3    | Placing Buttons .....                                                          | 189        |
| 7.6.4    | Handling Button Events.....                                                    | 192        |
| 7.6.5    | Generic Linked Stack .....                                                     | 193        |
| 7.6.6    | The <code>UndoButton</code> Class.....                                         | 194        |
| 7.7      | Summary .....                                                                  | 196        |
| 7.8      | Exercises (Items Marked with an * Have Solutions in<br>Appendix B).....        | 197        |
| <b>8</b> | <b>Binary Trees .....</b>                                                      | <b>199</b> |
| 8.1      | Introduction .....                                                             | 199        |
| 8.2      | Binary Expression Trees.....                                                   | 201        |
| 8.2.1    | General Concepts .....                                                         | 201        |
| 8.2.2    | Output in Prefix Form.....                                                     | 203        |
| 8.2.3    | Output in Infix Form.....                                                      | 211        |
| 8.3      | Creating a Binary Expression Tree from a Prefix<br>Expression .....            | 216        |
| 8.4      | Generic <code>if</code> Statement for Recursive Algorithms.....                | 220        |
| 8.5      | Binary Search Trees.....                                                       | 221        |
| 8.5.1    | General Concepts .....                                                         | 221        |
| 8.5.2    | Inorder Traversal.....                                                         | 222        |
| 8.5.3    | Creating a Binary Search Tree.....                                             | 226        |
| 8.5.4    | Finding the Minimum and Maximum .....                                          | 230        |
| 8.5.5    | Removing an Item from a BST.....                                               | 231        |
| 8.6      | The Complete <code>BinarySearchTree</code> Class .....                         | 240        |
| 8.7      | Test Program .....                                                             | 242        |
| 8.8      | Complete Program: Implementing a Binary Expression Tree ...                    | 244        |
| 8.9      | Summary .....                                                                  | 247        |
| 8.10     | Exercises (Items Marked with an * Have Solutions in<br>Appendix B).....        | 247        |



|           |                                                                 |     |
|-----------|-----------------------------------------------------------------|-----|
| <b>9</b>  | <b>Sorting</b>                                                  | 249 |
| 9.1       | Introduction                                                    | 249 |
| 9.2       | The Insertion Sort                                              | 249 |
| 9.3       | The Quick Sort                                                  | 255 |
| 9.3.1     | General Concepts                                                | 255 |
| 9.3.2     | Potential Problems                                              | 256 |
| 9.3.3     | The Sort3 and SwapElements Methods                              | 257 |
| 9.3.4     | The QuickSort Method                                            | 258 |
| 9.4       | The Radix Sort                                                  | 273 |
| 9.4.1     | General Concepts                                                | 273 |
| 9.4.2     | More Pockets                                                    | 275 |
| 9.4.3     | Fewer Pockets                                                   | 275 |
| 9.4.4     | Memory                                                          | 277 |
| 9.4.5     | Radix Sort Program                                              | 280 |
| 9.5       | Complete Program: Sorting String Items                          | 286 |
| 9.6       | Summary                                                         | 290 |
| 9.7       | Exercises (Items Marked with an * Have Solutions in Appendix B) | 290 |
| <b>10</b> | <b>Heaps</b>                                                    | 293 |
| 10.1      | General Concepts                                                | 293 |
| 10.2      | Creating Heaps                                                  | 295 |
| 10.3      | The insertItem and moveUp Methods                               | 300 |
| 10.3.1    | The insertItem Method                                           | 300 |
| 10.3.2    | The moveUp Method                                               | 301 |
| 10.4      | Priority Queues                                                 | 306 |
| 10.5      | The removeMin and moveDown Methods                              | 309 |
| 10.5.1    | The removeMin Method                                            | 310 |
| 10.5.2    | The moveDown Method                                             | 311 |
| 10.5.3    | Walk-Through                                                    | 314 |
| 10.6      | Test Program                                                    | 316 |
| 10.7      | The Heap Sort                                                   | 319 |
| 10.7.1    | The Simplified Heap Sort                                        | 319 |
| 10.7.2    | The Modified Heap Sort                                          | 320 |
| 10.7.3    | The makeHeap Method                                             | 321 |
| 10.7.4    | The heapSort Method                                             | 324 |
| 10.8      | Test Program and Output                                         | 328 |
| 10.9      | Complete Program: Priority Queues for Printers                  | 329 |
| 10.10     | Summary                                                         | 335 |
| 10.11     | Exercises (Items Marked with an * Have Solutions in Appendix B) | 335 |

|                                                                                  |     |
|----------------------------------------------------------------------------------|-----|
| <b>11 Hashing</b> . . . . .                                                      | 339 |
| 11.1 General Concepts . . . . .                                                  | 339 |
| 11.2 Mid-square Method . . . . .                                                 | 341 |
| 11.3 Hash Function for Strings. . . . .                                          | 342 |
| 11.4 Collision Resolution . . . . .                                              | 343 |
| 11.5 Implementation of a Hash Table. . . . .                                     | 344 |
| 11.6 Implementation of Hash Tables Using the Java API. . . . .                   | 347 |
| 11.6.1 The HashMap Class . . . . .                                               | 348 |
| 11.6.2 The HashSet Class. . . . .                                                | 352 |
| 11.6.3 The hashCode and equals Methods . . . . .                                 | 355 |
| 11.7 Complete Program: Hash Table Using the HashMap Class . . . .                | 357 |
| 11.8 Summary . . . . .                                                           | 360 |
| 11.9 Exercises (Items Marked with an * Have Solutions in<br>Appendix B). . . . . | 361 |
| <b>Appendix A: Glossary.</b> . . . . .                                           | 363 |
| <b>Appendix B: Answers to Selected Exercises</b> . . . . .                       | 365 |
| <b>References and Useful Websites.</b> . . . . .                                 | 373 |
| <b>Index</b> . . . . .                                                           | 375 |

---

## 1.1 Introduction to Data Structures

Having learned how to store data in a computer's memory from a previous text or course using variables, arrays, strings, and objects, it should be seen that there are a vast number of possibilities to store and manipulate data. However, instead of having to "reinvent the wheel" each time a program is written, others have found that there are some common techniques to store data that are quite helpful in a wide variety of situations. Some of these include stacks, queues, lists, and trees, all of which will be discussed in this text. The different ways data can be stored, along with the specific operations that can be performed on data, is known as *data structures*.

Although individuals might sometimes refer to an array as a data structure, this is not very precise. It is not the mechanism of storing the data that is a data structure, but rather it is the combination of the way the data is stored and the associated methods that can be performed on the data that define the data structure. For example, an array itself is not a data structure, but rather the order of the data (for example, integers in ascending order) and the associated operations that can be performed on the data which might be implemented using an array that constitutes a data structure.

---

## 1.2 Prior Concepts and Skills Needed

The prior concepts and skills needed for this text should have been learned in a previous introductory class or text using the Java programming language. In particular, the reader should have a good grasp of input/output, assignment statements, arithmetic and Boolean expressions, selection structures (`if` and `switch` statements), and iteration structures (`for`, `while`, and `do-while` statements). Furthermore, the reader should have a working knowledge of primitive data types,

strings, and one and two-dimensional arrays. Lastly, a good background in object-oriented programming is needed including data members, value-returning and void methods, constructors, private, protected, and public data members and methods, sending and receiving objects from a method, instance and class constants, variables and methods, inheritance, abstract classes, and polymorphism.

The authors understand that learning data structures can be a bit intimidating at times, so in many cases the authors will remind the reader of previous subtle points and concepts to help in understanding the topic under consideration. However, if at any time the reader feels deficient in any of the areas mentioned above, it is recommended that time be taken to review these concepts in an introductory text such as *Guide to Java* [2] or other similar text. Although some readers might have had some exposure to elementary algorithm analysis, abstraction, and interfaces, these topics are reviewed here in this preliminary chapter to ensure that the reader is ready for the first chapter on stacks.

---

### 1.3 Elementary Algorithm Analysis

Just as there are different ways of storing data, there are different ways of processing data. If an algorithm is running slow, what can be done to speed it up? A first response might be to get a faster processor and if a processor is twice as fast, then one might expect the algorithm to be approximately twice as fast too. But what if the goal was to have an algorithm run 1000 times faster? It might be difficult and expensive to find a processor that runs that fast. If possible, an alternative is to write a faster algorithm. Is there a way to determine whether one algorithm is faster than another algorithm? The answer is yes, and it is through the process of algorithm analysis. For example, if a program had two statements as follows, how fast is the algorithm?

```
total = total + x;  
i = i + 1;
```

Although some operations might take longer than others and speed might vary from computer to computer, assume that each operation (such as assignment, comparison, and arithmetic) will count as one unit of time. In the current example, each arithmetic statement would take 2 units (one for the arithmetic and one for the assignment) for a total of 4 for the two statements. Continuing, what if there were three statements as shown below?

```
x = y * z;  
total = total + x;  
i = i + 1;
```

The additional statement adds 2 time units, which increases the total to 6. Although the first code segment is approximately 33% faster, is this a significant amount of time? In one sense it is, but in comparison to other algorithms it is not a significant increase. As will be seen shortly, regardless of how much data is processed, each code segment is executed in a fixed amount of time and it can be said that both of these code segments are executed in constant time.

A common notation, called *Big O* notation (pronounced Big Oh), uses the capital letter O to compare the relative order of magnitude of various algorithms. Algorithms that are executed in constant time are said to be of order 1 or  $O(1)$ . The number 1 is used regardless of how many actual time units the algorithm took and to indicate that the time is constant. So in answering the question whether one of the above code segments is significantly faster than the other, neither is because they are both  $O(1)$  algorithms.

However, what if the above three lines of code were placed in a `while` loop along with another line to initialize the variable `i` as shown below?

```
i = 1;
while(i <= n) {
    x = y * z;
    total = total + x;
    i = i + 1;
}
```

In the above example, the first assignment statement will take one time unit. As before, the three arithmetic statements in the body of the `while` loop would take 6 time units. If the value of `n` was 3, then each statement in the body of the loop would be executed three times, for a total of 18 time units. Further, how many times does the `while` statement itself get executed? If one answered 3, that answer would unfortunately be incorrect. Don't forget that the final value of `i` is checked one last time before control is transferred to the bottom of the loop. The correct answer is 4 times. Again, assuming the value of `n` is 3, the result is that it would take 23 time units to execute the code segment as shown below:

```
i = 1;                                // 1 operation
while(i <= n) {                        // 3 times thru loop + 1 last check of i
    x = y * z;                         // 2 operations * 3
    total = total + x;                 // 2 operations * 3
    i = i + 1;                         // 2 operations * 3
}
```

An astute programmer might recognize that the first assignment statement in the body of the loop does not need to be calculated each time through the loop. As a result, that statement could be moved prior to the execution of the loop as follows:

```
x = y * z;  
i = 1;  
while(i <= n) {  
    total = total + x;  
    i = i + 1;  
}
```

Given this change, how many units of time would it take to execute this code segment assuming again that the value of  $n$  is 3? Using the same method of analysis discussed above, the correct answer is 19. Another way to look at it is that since the  $x = y * z$  is executed only once instead of three times, there are 4 fewer operations that need to be performed.

What if the value of  $n$  in the above two program segments were 10? The answers would be 72 and 54, respectively. Again, the  $x = y * z$  would be executed nine fewer times which accounts for the speed up of 18 units in the second segment. What about 100? The answers would be 702 and 504, respectively. In the first segment, notice that there are 7 operations for each iteration of the loop (the comparison, the multiplication, the two additions, and the three assignments), where 7 times 100 is 700. Plus there is the initialization of  $i$  and the last check of  $i < n$  prior to exiting the loop, for a total of 702. Likewise with the second example, there are 5 operations for each iteration of the loop (the two additions, two assignments, and one comparison), plus the 4 extra operations outside the loop (the two assignments, the multiplication and the last comparison before exiting the loop) for a total of 504.

The result is that by careful programming, there can be some change in algorithmic efficiency. In this case there is a 28.2% speed up. Is this a very large change? Further, could the answers above be more generic in terms of  $n$ ? The answer to the second question is yes, where one might see the pattern in the numbers above, especially when  $n$  equals 100. The first algorithm would be  $7n + 2$  and the second algorithm would be  $5n + 4$ . In looking at these two expressions, does the +2 or +4 make any significant changes to its efficiency? It can be seen as the value of  $n$  gets higher, say one million, the effects of these values are rather insignificant.

Further, whether the result is  $5n$  or  $7n$ , they are both linear. Regardless if the coefficient is a 5 or a 7, the algorithm depends on the value of  $n$ . The result is that both of these algorithms are considered to be order  $n$  or  $O(n)$ . So in answer to the first question in the preceding paragraph, in both cases these algorithms are considered to be the same order of magnitude and it is not a significant change.

Does this mean that one does not need to do a detailed analysis of an algorithm? Although these two algorithms can be determined by inspection to be of order  $n$  because they both loop  $n$  times, there are some other very tricky algorithms where it might be necessary to take the time to do a line-by-line analysis. Further, does this mean that one should not try to increase the efficiency of an algorithm from  $7n$  to

$5n$ ? As will be seen there are other algorithms that are faster than  $O(n)$ , but if a faster algorithm is not available and speed is a concern, then one should try to increase an algorithm's efficiency, especially inside a loop or nested loops. However, as will be seen later in this text, sometimes there can be a dramatic increase in the speed of an algorithm by using an alternative method.

In looking at some other algorithms, what would be the efficiency of the following code segment?

```
for(i=1;i<=n;i++) {  
    // some statements  
}  
for(i=1;i<=n;i++) {  
    // some statements  
}
```

Since the specific statements in the body of the loops are not included, it is not possible to do a detailed analysis. However, by inspection it should be clear that the first loop would be  $O(n)$  and the second loop would also be  $O(n)$ . But what is the efficiency when they are combined? Since they are both of  $O(n)$  and one follows the other, the two are added and the result would appear to be  $n + n$  or of order  $2n$ . Further, as discussed earlier the coefficient can be dropped and the algorithm consisting of the two non-nested loops is  $O(n)$ . In another example, consider the following algorithm that also consists of two `for` loops. What is its algorithmic efficiency?

```
for(i=1;i<=n;i++)  
    for(j=1;j<=n;j++) {  
        // some statements  
    }
```

Unlike the previous code segment, the loops in this segment are nested. If the answer is not readily apparent, it is sometimes helpful to pick a value for  $n$  to help determine how many times the statements in the inner loop are executed. The key is not to pick a number too small where the pattern might not be evident, nor pick a number too large in case one has to manually walk through the code. If the value for  $n$  was 3, how many times would the statements in the inner loop be executed? The outer loop would execute three times, and for each time through the outer loop, the inner loop would execute three times too. The result is that the statements in the inner loop would be executed nine times. What if the value of  $n$  was 10? Then the statements in the inner loop would be executed 100 times. The result is that for any value  $n$ , the inner statements would be executed  $n \times n$  or  $n^2$  times and thus the segment is  $O(n^2)$ . Given the previous two example, consider the following:

```
for(i=1;i<=n;i++)
    for(j=1;j<=n;j++) {
        // some statements
    }
for(i=1;i<=n;i++) {
    // some statements
}
```

At first it might appear that the answer might be  $O(n^3)$  because there are three loops. However, notice that the first two loops are nested and the last one is not nested. Since the first two are nested they are  $n^2$  and since the last loop is not nested it is  $n$ . The result is that the two are added to form  $n^2 + n$ . But what is this in Big O notation? When there are more than one term in the resulting analysis, only the most dominate term is used in Big O notation, or in other words the slowest term is used. The result is that the above algorithm is  $O(n^2)$ . In another example, if an algorithm was analyzed to be  $3 + \log n + 2n$ , what would it be in Big O notation? The answer would be  $O(n)$  because  $2n$  is the dominate term (slowest) and as before the coefficient is not included.

Continuing, what would the efficiency be for the following loop?

```
i=n;
while(i>1){
    // some statements
    i = i / 2;
}
```

At first it might appear that the inner statements are executed  $\frac{1}{2} n$  times due to the division by 2, but this would be a mistake. As mentioned above, it is sometimes helpful to pick a number for  $n$  and walk through the code segment. If one chose 4, then one might have also come to the conclusion that the answer is  $\frac{1}{2} n$  since it would loop 2 times. However, as mentioned previously, one does not want to pick a number that is too small where a pattern might not be seen. Instead, try the number 16 for the value of  $n$ . The first time through the value of  $i$  would be 16. The second time through the value of  $i$  would be 8. The third time through it would be 4, followed by the value of 2 the fourth time through the loop. The loop does not iterate a fifth time because the value of  $i$  would be 1 which makes the relation in the `while` statement `false`. The result is that when  $n$  is 16, the loop iterates 4 times. How many times would it iterate when  $n$  was 32? As one might suspect, the answer is 5. If  $2^4$  is equal to 16, then  $\log_2$  of 16 is 4. Likewise, if  $2^5$  is equal to 32, then  $\log_2$  32 is equal to 5. The result is that the above code segment is logarithmic and is of order  $\log_2 n$  or  $O(\log n)$ . Notice that if the subscript 2 is left off the logarithm, because in the field of computer science,  $\log$  is assumed to be  $\log_2$ .



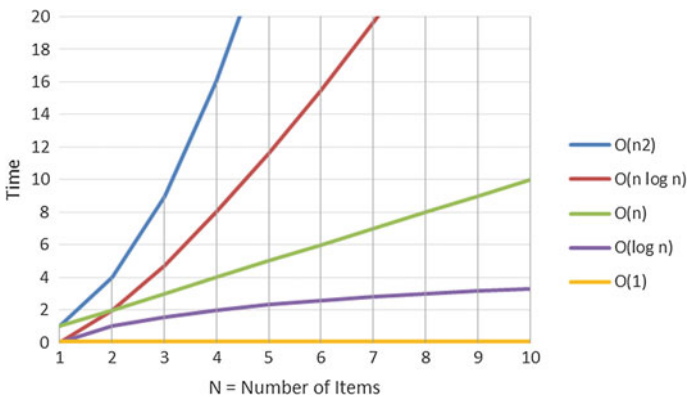
In one more example, what is the algorithmic efficiency of the following code segment?

```
for(i=1;i<=n;i++) {
    j=n;
    while(j>1) {
        // some statements
        j = j / 2;
    }
}
```

As with the nested `for` loops previously, where the two nested  $O(n)$  loops were  $n \times n$  or  $O(n^2)$ , similarly the outer loop is  $O(n)$  and the inner loop is  $O(\log n)$  which combine to give  $n \times \log n$  or  $O(n \log n)$ . As will be seen in Chap. 9, some of the fastest sorting algorithms are  $O(n \log n)$ .

In all the previous examples, regardless of the coefficients in a particular instance, as  $n$  gets sufficiently large, an  $O(\log n)$  algorithm is faster than an  $O(n)$  algorithm, an  $O(n)$  is faster than an  $O(n \log n)$  algorithm, and an  $O(n \log n)$  algorithm is faster than an  $O(n^2)$  algorithm. Of course the fastest is an  $O(1)$  algorithm because it does not depend on the value of  $n$  as shown in Fig. 1.1.

This concept of comparing algorithms is very important in the field of computer science, since the relative speed of algorithms can be compared with each other. Although briefly introduced here, this concept will be seen again throughout the text.



**Fig. 1.1** Comparison of  $O(n^2)$ ,  $O(n \log n)$ ,  $O(n)$ ,  $O(\log n)$ , and  $O(1)$

## 1.4 Abstraction and Interfaces

In addition to the ability to create various data structures using an array, many can also be implemented with objects that can be linked together using references, which will also be discussed in this text. How these data structures are implemented is often hidden from the person using the data structure and this concept is known as *data abstraction*. For instance, whether a list is implemented using an array or using references is an example of data abstraction. Further, how the details of an algorithm are implemented is known as *procedural abstraction*. As might have been learned in a previous course or text, how a number is calculated using either iteration or recursion is an example of procedural abstraction. Together, through the use of data abstraction and procedural abstraction, one can create a data structure known as an *abstract data type* (ADT).

As an illustration, whether a vehicle has a four-cylinder engine or a rotary engine is an example of data abstraction and whether the vehicle uses a mechanical or electrical steering process can be an example of procedural abstraction. Although there are advantages and disadvantages to various types of engines (for example, a diesel engine has a lot of torque and is often used in trucks), the driver of a vehicle might not know what type of engine is being used and all that he or she cares about is that when the accelerator is depressed the vehicle accelerates. Likewise, the driver of a vehicle might not be concerned whether the steering is implemented through a mechanical or electrical process, but rather only cares that when the steering wheel is turned to the left, the vehicle turns to the left. The combination of various mechanical parts and processes in the vehicle make it an abstract data type.

Just as there are advantages and disadvantages to using different mechanical parts in a vehicle, there are advantages and disadvantages to how data is stored. Some ways of storing data are fixed and they cannot change or it is difficult to change their size during the execution of a program and are known as *static*, such as arrays. Others have the ability to change during the execution of a program, such as using references, and these are known as *dynamic*. Further, dynamic structures can be faster than static ones. At first, it might seem as though the dynamic structures would be the best choice under all circumstances. However, it should also be pointed out that dynamic structures generally take up more memory than static structures. The result is that determining which type of structure should be chosen depends upon the circumstances.

Consider a simple application which calculates and outputs the average of three exams. How could the three scores be stored? One way is to create three separate memory locations and another is to use a one-dimensional array of size 3. How the scores are stored, either using three separate variables or a one-dimensional array, could be hidden from the application programmers. They could use the class without needing to know the details of how it was coded. To facilitate this, a Java *interface* structure can be used to separate the headings of the methods from the implementation of the methods. A Java interface lists a set of abstract methods without their code. A class that implements an interface provides definitions for the

methods. The important point is that the particular implementation which is used will not affect other programs that interact with it because every implementation is written to satisfy method headings in the interface. The reason for having several different implementations for the same set of methods is that one class may be more efficient than another at performing certain kinds of operations with certain type of data. Programmers can choose the best implementation depending on the nature of the application.

The following code is a Java interface which deals with exam scores. The definition starts with the visibility modifier `public` followed by the keyword `interface`, which is followed by the name of the interface `ScoresAvgADT`.

```
public interface ScoresAvgADT {  
    // asks user to enter test scores  
    public void enterScores();  
  
    // computes and outputs average of test scores  
    public void computeAverage();  
}
```

As can be seen, it has two method headings, `enterScores` which should ask a user to enter the scores and `computeAverage` that should compute and output the average of the test scores. These methods can be implemented with three variables or a one-dimensional array in separate classes. The classes that implement an interface must provide all of the methods listed in the interface, and those methods must be declared as `public`.

The keyword `implements` is used to declare a class that implements the interface. To illustrate, a complete class using three separate variables is shown below.

```
import java.util.*;  
public class Scores implements ScoresAvgADT {  
    private final static int NUM_EXAMS = 3;  
    private Scanner scanner = new Scanner(System.in);  
    private int exam1, exam2, exam3;  
  
    public Scores() {  
    }  
  
    public void enterScores() {  
        System.out.print("Enter three scores: ");  
        exam1 = scanner.nextInt();  
        exam2 = scanner.nextInt();  
        exam3 = scanner.nextInt();  
    }  
}
```

```
public void computeAverage() {  
    double sum;  
    sum = exam1 + exam2 + exam3;  
    System.out.printf("Average score: %.2f \n",  
        sum / NUM_EXAMS);  
}  
}
```

In contrast, a complete class which implements an interface using a one-dimensional array is shown below. It does the same tasks as the previous class, but the only difference is that an array and loop are used, illustrating the principle of data and procedural abstraction.

```
import java.util.*;  
public class Scores1D implements ScoresAvgADT {  
    private final static int NUM_EXAMS = 3;  
    private Scanner scanner = new Scanner(System.in);  
    private int[] exam;  
  
    public Scores1D() {  
        exam = new int[NUM_EXAMS];  
    }  
  
    public void enterScores() {  
        System.out.print("Enter three scores: ");  
        for(int i=0; i<NUM_EXAMS; i++)  
            exam[i] = scanner.nextInt();  
    }  
  
    public void computeAverage() {  
        double sum;  
        sum = 0.0;  
        for(int i=0; i<NUM_EXAMS; i++)  
            sum = sum + exam[i];  
        System.out.printf("Average score: %.2f \n",  
            sum / NUM_EXAMS);  
    }  
}
```

Further, Fig. 1.2 is a *UML* (Unified Modeling Language) diagram that shows the `ScoresAvgADT` interface and two classes each of which contain two methods. Note that a dashed line from the class to the interface is used to indicate the relationship between the interface and the classes that implement it.

Once a class implements an interface, it can be used to create an instance of a class such as `Scores` using a variable of type `ScoresAvgADT`. For example, if the class `Scores` implements `ScoresAvgADT`, then the following declaration and creation of the object using the variable `scores` is correct:

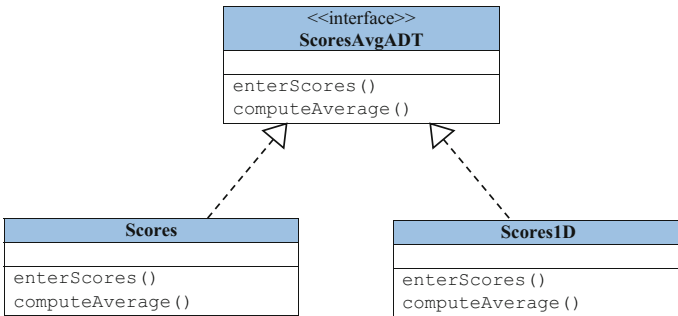
```
ScoresAvgADT scores = new Scores();
```

In general, it is a good idea to declare variables for an object using interface types rather than class types. This allows flexibility because it does not tie the variables to particular implementations. If variables are declared with interface types, then changing to another implementation for that particular reference is usually just a matter of calling a different constructor, such as:

```
ScoresAvgADT scores = new Scores1D();
```

None of the other code in the calling program using the variable `scores` should have to be changed. A simple main program invoking instance of both classes is shown below:

```
public class TestScores {  
    public static void main(String[] args) {  
        ScoresAvgADT scores;  
  
        scores = new Scores();  
        scores.enterScores();  
    }  
}
```



**Fig. 1.2** UML diagram showing the `ScoresAvgADT` and its implementing classes

```
scores.computeAverage();

scores = new Scores1D();
scores.enterScores();
scores.computeAverage();
}
}
```

Interfaces are used to express abstract data types in Java. They define a behavior of the object by listing headings of the methods with comments that specify how each method is to be used without showing the actual code. Thus, encapsulation is enforced by hiding the details of the implementation. These methods can be used only if there is a class which implements an interface that provides the code of all the methods in the interface. Whereas an interface includes only constants and abstract methods, the class implementing the interface can have other public and private methods and data members in addition to the implementation of all the methods from the interface.

---

## 1.5 Java API

Note that there exist in the Java programming language various libraries that contain a number of data structures such as stacks and queues. These libraries are known as the Java Application Program Interface (*API*) and they will be introduced in Chap. 11. The casual reader might think that it would be good enough to just learn how to use these libraries and be done with it. Under certain circumstances, using these libraries might be sufficient “to get the job done.” However, there are certain deficiencies to these libraries and should one want to create their own data structures, then knowledge of how to do so is in order.

Further, being able to just use the libraries would be similar to a mechanical engineer only being able to drive an automobile and not being able to understand how it works or be able to design an automobile. Similarly, a computer scientist must not only know how to use existing data structures, but understand how they work, know their limitations, and be able to design and create his or her own data structures. This can be accomplished by learning how the data structures work, their advantages and disadvantages, and how to design, create, test, and debug them. To that end, Chap. 2 begins with using one of the simplest yet important data structures, the stack.

## 1.6 Complete Program: Implementing Interface

The complete program discussed in Sect. 1.4 is as shown below. Note that each of the classes and interface may need to be placed in separate modules in an *IDE* (Integrated Development Environment) in order to compile and work properly.

```
public class TestScores {
    public static void main(String[] args) {
        ScoresAvgADT scores;

        scores = new Scores();
        scores.enterScores();
        scores.computeAverage();

        scores = new Scores1D();
        scores.enterScores();
        scores.computeAverage();
    }
}

public interface ScoresAvgADT {
    // asks user to enter test scores
    public void enterScores();

    // computes and outputs average of test scores
    public void computeAverage();
}

import java.util.*;

public class Scores implements ScoresAvgADT {
    private final static int NUM_EXAMS = 3;
    private Scanner scanner = new Scanner(System.in);
    private int exam1, exam2, exam3;

    public Scores() {
    }

    public void enterScores() {
        System.out.print("Enter three scores: ");
        exam1 = scanner.nextInt();
        exam2 = scanner.nextInt();
        exam3 = scanner.nextInt();
    }

    public void computeAverage() {
        double sum;
```

```
        sum = exam1 + exam2 + exam3;
        System.out.printf("Average score: %.2f \n",
            sum / NUM_EXAMS);
    }
}

import java.util.*;

public class Scores1D implements ScoresAvgADT {
    private final static int NUM_EXAMS = 3;
    private Scanner scanner = new Scanner(System.in);
    private int[] exam;

    public Scores1D() {
        exam = new int[NUM_EXAMS];
    }

    public void enterScores() {
        System.out.print("Enter three scores: ");
        for(int i=0; i<NUM_EXAMS; i++)
            exam[i] = scanner.nextInt();
    }

    public void computeAverage() {
        double sum;
        sum = 0.0;
        for(int i=0; i<NUM_EXAMS; i++)
            sum = sum + exam[i];
        System.out.printf("Average score: %.2f \n",
            sum / NUM_EXAMS);
    }
}
```

---

## 1.7 Summary

- Big O notation is a valuable way of comparing the efficiency of different algorithms.
- The order of efficiency of algorithms from the fastest to the slowest is:  $O(1)$ ,  $O(\log n)$ ,  $O(n)$ ,  $O(n \log n)$ , and  $O(n^2)$ .
- When there is more than one term in an expression after analysis of an expression, only the most dominate term (slowest) appears without coefficients in Big O notation.
- Data abstraction is the hiding of how the data is stored in a program.



- Procedural abstraction is the hiding of the algorithm.
- An abstract data type (ADT) is a class in which both the data and procedure are hidden or abstracted.
- An interface is a useful Java mechanism to implement ADTs.

---

## 1.8 Exercises (Items Marked with an \* Have Solutions in Appendix B)

1. Evaluate the following code segments in terms of their algorithmic efficiency. Do not use Big O notation, but rather give a specific number for an answer as done in the first part of Sect. 1.3. (Note: Assume that `i++` counts as 2 time units.)

```
*A. x = 0;
    for(i=0; i<3; i++)
        x = x + i * 2;
B. a = 1;
    do {
        a = a + 2;
    } while(a <= 3);
C. i = 1;
    y = 2;
    while(i <= 6) {
        i = i + 2;
        y = y * i;
    }
```

2. Evaluate the following code segments in terms of their algorithmic efficiency. Do not give a specific number, but rather use Big O notation in terms of `n` as done in the last part of Sect. 1.3.

```
*A. i = 1;
    while(i <= n/2) {
        // some statements
        i = i + 1;
    }
B. j = 1;
```

```
while(j <= n) {  
    // some statements  
    j = j * 2;  
}  
C.   for(i=1; i<=n; i++)  
        for(j=1; j<=n; j++)  
            for(k=1; k<=n; k++) {  
                // some statements  
            }
```

3. Given the following expressions from analyzing algorithms, express them in Big O notation.

(Hint: only the dominate term should appear within the parenthesis without any coefficients.)

\*A.  $\log n + 2n + 3$       B.  $n \log n + n^2 + 3n$       C.  $\log n + n \log n + n$

---

## 2.1 Introduction

One of the easier data structures to learn first is the stack. Although it is simpler than other data structures, it still is very important and is used for a variety of purposes. For example, it is very useful in low-level programming such as assembly language, and many processor instruction sets have built-in instructions for stack manipulation [1]. The reason for its use is that it is sometimes faster to use a stack than to transfer data to a labeled memory location. Regardless of the language used, a stack can be used to swap memory locations, reverse data in an array, and evaluate arithmetic expressions. Further, as might have been discussed in a previous course or text, stacks are also used to implement recursion [2].

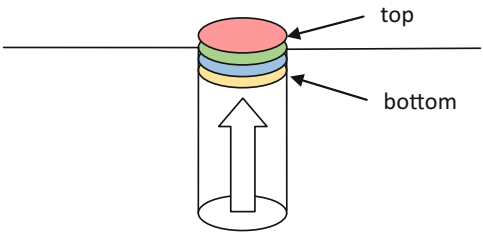
If a stack is so useful, how does it work? A simple explanation is to look at stacks in the real world. For example, if one is eating lunch at cafeteria, one might grab a plate from a stack of dishes. Or when coming back from a class, one might place a textbook on a stack of books on a desk. In both examples, items are only placed on or removed from the top of the stack. Further, it should be noticed that the last item put onto the stack is the first one taken off the stack. The property is known as last-in, first-out or *LIFO* and defines a *stack*. Of course, one might insert or remove an item from the middle or bottom of the stack, but then the structure is no longer has the properties of a stack nor would it be considered a stack. The act of putting something on a stack is known as a *push* operation and the act of removing an item from a stack is known as a *pop* operation.

---

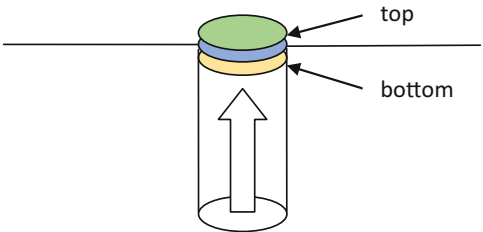
## 2.2 Analysis and Design

How could a stack be implemented in software? The simplest approach is to use an array. (An alternative approach using references will be explored in Chap. 7.) In addition to the stack itself, one needs to determine how to represent the top of the

stack. Looking at a real-world example can help in gaining some insight on how to implement a stack. Using the previous cafeteria plate example where the plates are stored below the surface of the table, consider the following diagram where the horizontal line represent the counter top and the large arrow represents a spring pushing the plates up.



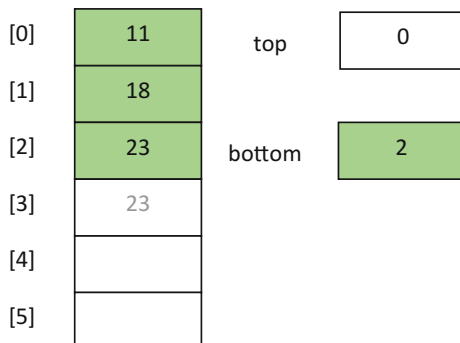
As plates are removed from the top of the stack, the spring below causes the next plate to move to the top and all the plates below move up one position as shown below:



Where there were previously four plates on the stack, now there are only three plates on the stack. Also note that the pointer to the top plate has not moved, but rather only the pointer to the bottom has moved. Although this model looks like a good possibility, how might it translate into an array? Consider the following six-element array containing four integers.

|     |    |        |   |
|-----|----|--------|---|
| [0] | 7  | top    | 0 |
| [1] | 11 |        |   |
| [2] | 18 | bottom | 3 |
| [3] | 23 |        |   |
| [4] |    |        |   |
| [5] |    |        |   |

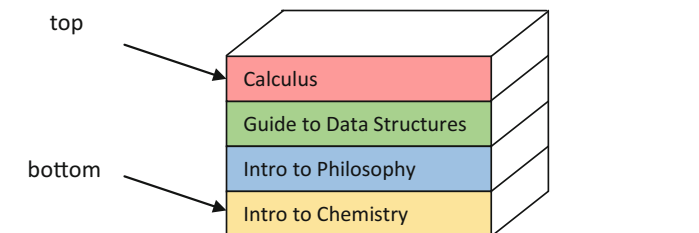
First, recall that a six-element array would have its elements numbered from 0 to 5. Further, do not forget the difference between the index of an array element and the contents of an array, where the zeroth array element above contains a 7. Notice that instead of a pointer, the variables `top` and `bottom` contain the index of the top and bottom elements, respectively. Now if the top element was removed, the integer 11 would have to be copied from the first position to the zeroth position, then the integer 18 could be copied from the second position to the first position, and so on. Lastly, the variable `bottom` would need to change from 3 to 2. All the changes are shown in green as follows:



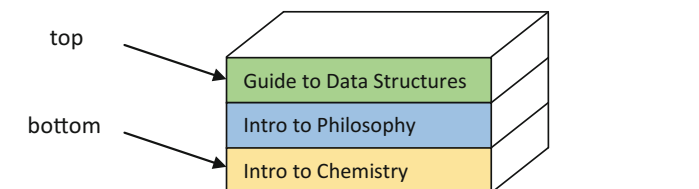
Although shown in light gray in the above figure, in actuality a copy of the integer 23 would still be in position 3 of the array since it was merely copied from one element to another. However, it is the variable `bottom` now at 2 which indicates the bottom of the stack, not the number of items in the array. Note that in removing all the items from the stack, `bottom` would be a  $-1$  and `top` would remain at a 0. This is an important point concerning the distinction between an array and a stack as a data structure as discussed in Chap. 1. The array merely implements the data structure, but it is a combination of all the variables and the algorithm that makes up a data structure.

Although the above representation of a stack seems to work, are there any disadvantages to it? To answer this question, consider if instead of just four items in the stack, what if it was a 100 element array with 100 items in the stack? To remove just one item in the stack, it would need to copy 99 items. What if it was a 1000 element array with a 1000 items in the stack? Then to remove just one item, 999 elements would need to be copied. To extrapolate, if there were  $n$  items in the stack, then  $n - 1$  items would need to be moved. As discussed in Sect. 1.4, this process would be  $O(n)$ . Clearly as the number of items increases, the efficiency of this design is lacking. Even though the above seems to work fairly well for a small number of items, one of the things that must be considered when designing a data structure is how it works with a large number of items.

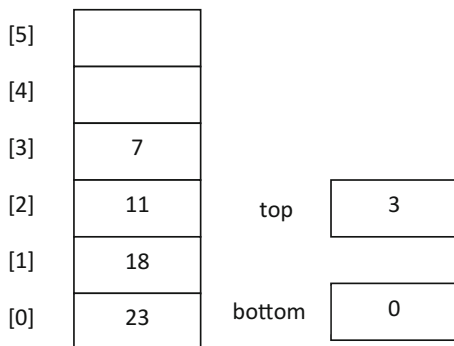
Instead of using the plate example as a model, consider the other example involving a stack of books on a desk in the following diagram where the horizontal line represents the desktop:



In this case when a book is removed from the stack, the rest of the stack does not move as it does in the plate example. As shown below, top moves down one place as opposed to the bottom moving up.

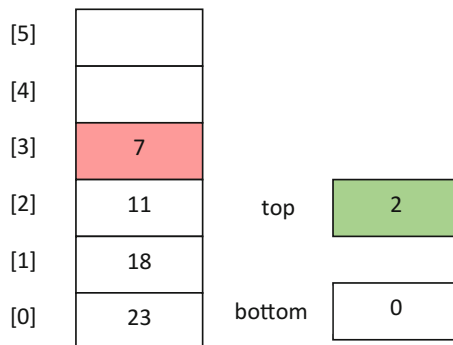


How might this model be represented using an array? Using the same numbers as before, examine the following diagram:



First, note that typically an array would be drawn with the zeroth element on the top and subsequent elements following below as was done in the previous example, but the above array has the zeroth element at the bottom instead. The reason for this is that when using an array to represent a stack it is usually more convenient to have the zeroth element drawn at the bottom, so that when items are added, they are added to the next higher number index and it looks more like a physical stack. Does this affect processing on the computer? No, since how it is drawn has no effect on the processing. Drawing the array this way is only more convenient for viewing by the reader. (Note that this text will typically continue to draw arrays with the zeroth element at the top, except when an array is representing a stack as above.) Also, notice that instead of having top at 0 and bottom at 3, these numbers are also

reversed in this example. When the top number, 7, is removed from the stack, the value of `top` will change from 3 to 2 as shown below.



As shown in red to indicate it has been popped off the stack, the integer 7 is still in element 3. The item was not technically removed from the stack, but rather only `top` was decremented. Since `top` is now at 2, that is what defines the top of the stack data structure, not what is in the array. Further, the advantage of this model is that whether there are four, one hundred, or one thousand items in the stack, none of the items needs to be copied. So, if there were  $n$  items in the stack, the process would not be of  $O(n)$ , but rather a constant. As discussed in Sect. 1.4, if the amount of time is constant, regardless what that number is, it is represented as  $O(1)$ .

---

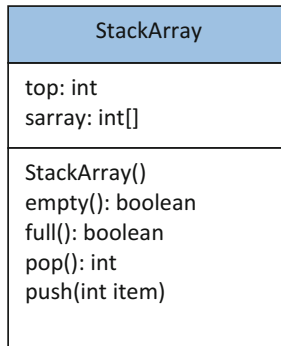
### 2.3 Stack Class: Data Members and Methods

The idea of a stack lends itself well to be implemented as an object. As with any object, one of the considerations is which instance variables should be used. In looking at the variables in the previous section one can see that both the array and the variable `top` are necessary. But what about the variable `bottom`—is it needed? Since it does not appear to change, then it probably is not necessary. Would a count of how many items in the stack be needed? Since the variable `top` seems to indicate how many items are in the stack, a count is not necessary either.

What sort of operations should be allowed on a stack? As mentioned in Sect. 2.1, a stack is a LIFO structure, meaning that the last item pushed onto the stack is the first item popped off the stack and that items are put onto only one end of the stack, the top. In addition to `push` and `pop` methods, since an array is of finite length and it is possible when pushing an item onto the stack that there might not be any room on the stack, it is a good idea to have a method to determine whether the stack is full. Conversely, it is also possible when popping an item off the stack that there might not be any items on the stack, so it is also a good idea to include a method to determine whether the stack is empty. Two other methods that might be useful are a `peek` method, which allows one to see what is on the top of

the stack without removing it, and a `size` method, which indicates how many items are on the stack. Although these two methods might be useful in some situations, they are not used here and are left as exercises at the end of this chapter.

So initially it seems that there should be four methods plus a constructor. The `push` method should put an item on the stack, so it should have a parameter to pass the item to the stack and since it will not be returning a value, it should be a `void` method. The `pop` method will be removing and returning an item from the stack, so it should be a value returning method. Both `empty` and `full` should return a value indicating whether the stack is empty or full respectively, so they too should be value-returning methods that return a `boolean` value of `true` or `false`. Although later in this chapter the concept of generics which allow many different types to be used will be discussed, for now it will be assumed that only integers will be stored on the stack. The result is that the following UML diagram would represent the stack using an array called `sarray`.



When pushing an item onto an empty stack, where should the first item be placed in the array? Looking back at the last figure in the last section, it should be clear that it ought to go in location 0 which could be indicated by the variable `top`. However, should `top` start off at 0 and then be incremented in preparation for the next item to be pushed onto the stack, or should `top` begin at -1 and be incremented by 1 prior to putting the item at location 0? Either technique works well, but this text will use the former as shown below and leave the latter as an exercise at the end of the chapter.

```
sarray[top] = item;
top++;
```

What if `sarray` were full? One solution is to create a larger array and then copy all the items from the old array into the new array. This is a good solution, would allow processing to continue, and is also left as an exercise at the end of the chapter. However, it would take processing time to create a new array and copy the contents, so a simpler solution is used here. Instead, if the stack is full, an error message is output to the user and the item is not placed on the stack. There is yet another solution using exceptions which will also be presented shortly. Lastly, should `push`



be `public` or `private`? Since it will be called from another program, it should be `public`. The complete method for `push` is shown below:

```
public void push(int item) {
    if(full())
        System.out.println("Stack is full");
    else {
        sarray[top] = item;
        top++;
    }
}
```

Should one be concerned that the `full` method has not been written yet and is this a problem? It is if it is never written, but right now it is not a problem. That is the beauty of modular design and UML diagrams. It has already been determined what the method will do, so it does not have to be written immediately. In a larger programming project, the various methods could be written by other team members and as long as the other methods are written according to the specifications, work can progress on all the modules in parallel.

Continuing on to the `pop` method, it should be noticed that `top` is now indicating where the next item will be pushed, so before an item can be popped off the stack, `top` will need to be decremented. The basic code for the `pop` routine is as follows:

```
top--;
item = sarray[top];
```

Notice that it is essentially the same as the `push` routine except in reverse order. As with the `push` method which is concerned with whether the stack is full, the `pop` method should be concerned with whether the stack is empty, so an `if` statement is again used. As with `push`, the `pop` method is declared as `public`.

```
// Caution: Poorly Implemented Code
public int pop() {
    if(empty()) {
        System.out.println("Stack is empty");
        return -1;
    }
    else {
        top--;
        return sarray[top];
    }
}
```

Assuming that the values on the stack will be non-negative, the then section of the `if` statement returns a `-1` so that a value will be returned should the stack be empty, otherwise the else section returns the item from the top of the stack. Although this code segment would “work”, it is not the best solution. Note that using two `return` statements is considered unstructured programming and a method should have only one entry point and one exit point. Although this method has one entry point at the `if` statement, there are two ways it can leave via the two `return` statements. Though not much of a problem here since it is such a small code segment, it is not a good habit to start because in larger code segments it can be a potential source of logic errors and can be very difficult to debug. Alternatively, the following code segment uses only one `return` statement.

```
public int pop() {
    int item = -1;
    if (empty())
        System.out.println("Stack is empty");
    else {
        top--;
        item = sarray[top];
    }
    return item;
}
```

Notice that a local variable `item` is used to hold the `-1` or the value copied from the stack and there is now only one `return` statement. Although this variable takes up one additional memory location, the amount of memory used is negligible and the more structured code is worth the minimal cost. Also, note as mentioned in the previous section that the value returned is still in the array since it was merely copied into `item`. Is this a problem? No, because if another item is pushed onto the array, the prior value will just be overwritten and if another item is popped off the stack, the value of `top` will first be decremented so that the next value in the stack will be returned.

As might have been discussed in a previous course or text, Java provides an extensive set of classes to support exception handling when an execution error occurs. How can one take advantage of these classes during the development of the `StackArray` class? Instead of using a `println` statement, an exception can be thrown. The types of exceptions that may arise during the `push` or `pop` operations are not checked during compile time, but they are detected during runtime and called runtime exceptions. Therefore, an object of `RuntimeException` can be thrown. The result is that the `push` method can be rewritten as shown below:

```
public void push(int item){
    if(full())
        throw new RuntimeException("Stack is full");
    sarray[top] = item;
    top++;
}
```

Note that an `else` statement is no longer necessary because if an exception is thrown, the program will stop and the subsequent code will not be executed. Similarly the `pop` method can be rewritten as follows:

```
public int pop(){
    if(empty())
        throw new RuntimeException("Stack is empty");
    top--;
    return sarray[top];
}
```

In addition to the `else`, note that the variable `item` is also deleted from the code. Again, if an exception is thrown the subsequent code will not be executed and a value is not returned. What about the `full` and `empty` methods? Looking at the `empty` method first, how does one know the stack is empty? As mentioned previously, it was decided that `top` would initially be 0, so that would indicate an empty stack. The code for `empty` could be written as follows:

```
// Caution: Poorly Implemented Code
if(top == 0)
    return true;
else
    return false;
```

Although this code segment would “work”, it suffers from the same problem as the first version of the `pop` method in that it is using two `return` statements and is unstructured. As before, a possible solution is to declare a local variable, assign it a boolean value, and then return the contents of the variable as follows:

```
//Caution: Poorly Implemented Code
boolean flag;
if(top == 0)
    flag = true;
else
    flag = false;
return flag;
```

Though there is one extra variable, the code segment is now structured and has only one exit point. However, there is an even simpler and more efficient solution to this problem. Since the value of `top == 0` is either `true` or `false`, why assign the boolean value to a variable? Why not just return the results of the relation `top == 0` as shown below?

```
return top == 0;
```

If `top` equals 0, the results of `top == 0` will be `true` and this is the boolean value that will be returned. Likewise, if `top` does not equal zero, the results of `top == 0` will be `false` and this is the value that will be returned. The result is that there is no need for an `if` statement and thus no need for an extra variable and no possibility of having two `return` statements.

Lastly, should `empty` be `public` or `private`? Since as will be seen, `empty` is used in the main program, it should be declared as `public`. The complete method is shown below:

```
public boolean empty() {  
    return top == 0;  
}
```

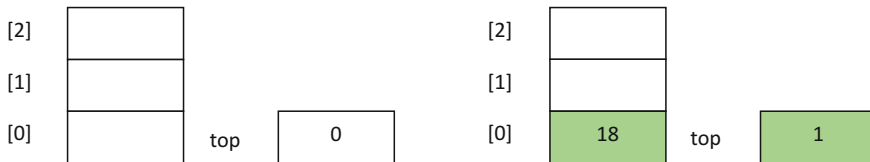
Is there a possibility that `top` will ever be equal to a negative value? If all the modules are written correctly, the answer is no. However, what if the code in `pop` is accidentally modified so that the value in `top` can skip over the value 0 and continue on towards the negative numbers? Of course if access to a negative element of the array is attempted, an execution error would occur. To help ensure that does not happen, the above code segment could be rewritten as:

```
private boolean empty() {  
    return top <= 0;  
}
```

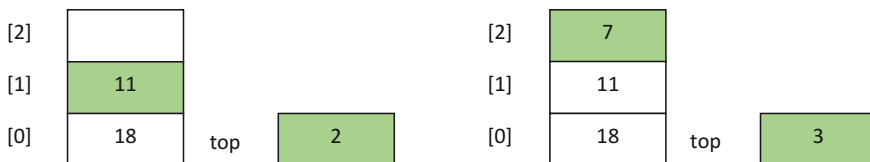
Which one of the two is the better way to code the method? The second one has the advantage that if a negative value is encountered, the code could continue to execute. However, the disadvantage is that it might mask a potential error in the code. If a negative index were to be encountered, then maybe an execution error should occur so that the programmer will be aware of the error and the logic can be corrected. For now, the second approach will be taken, but if one's instructor, place of employment, or an independent reader prefers the first approach, then it should be used.

The `full` method will be very similar to the `empty` method, except it will check the upper bound of the variable `top`. If there is a 3 element array, what is the largest value that `top` will have? Since the elements are numbered 0–2, one might be tempted

to say 2, but although that would work, it is not quite correct. When writing a new code segment it is not the time for guessing, but rather careful analysis of the situation. Instead, drawing a diagram and walking through a small sample size will result in the correct solution. A little time at this stage can save a lot of debugging time later and is well worth the effort. Assuming that the stack is initially empty as shown below to the left below and the numbers 18, 11, and 7 need to be pushed on to the stack, the following diagram to the right indicates the resulting value in `top` after the 18 is pushed onto the stack and `top` is incremented by 1.



Going from left to right in the diagram below, the second number 11 is placed into position 1 and `top` is incremented to 2. Lastly, the number 7 is put into position 2 and `top` is incremented to 3.



As can be seen, the final value of `top` when the stack is full is 3. So, for an `n`-element stack, when the stack is full the value of `top` would be `n`. Given this information, the `full` method would be as follows:

```
public boolean full() {
    return top >= sarray.length;
}
```

Notice the relation `>=` is used, but as mentioned above `==` could alternatively be used if preferred. Also, instead of a user-defined constant indicating the size of the array, the `.length` constant could be used as shown above. The only method remaining is the constructor, where the stack should be set to empty by setting `top` to 0 and the space for the array `sarray` should be allocated using a constant as shown below along with the other data members:

```
public final int N = 3;
private int top, sarray[];

public StackArray() {
    top = 0;
    sarray = new int[N];
}
```

However, what if one wanted to change the size of the array? As a programmer, the constant could be changed prior to compilation, but wouldn't it be more convenient to be able to change it during execution time? This can be accomplished by adding a parameter to the above method and replacing the constant `N` with the parameter `n`. Further, another constructor is created without a parameter and using `this(N)` to call the original constructor as shown below:

```
private static final int N = 3;
private int top, sarray[];

//constructors
public StackArray() {
    this(N);
}
public StackArray(int n) {
    top = 0;
    sarray = new int[n];
}
```

Note that the constant `N` has now been made a class constant using the word `static`, so that it can be used as an argument in the first constructor. Now the users of the class have the choice of either using the default stack of size `N` or they can set the size of the stack through an argument. Putting all the methods together along with the data members, the class definition for the `StackArray` class appears in Fig. 2.1.

Notice that the constructor is first, followed by the value-returning methods, followed by the `void` method, and within each group the methods are in alphabetical order. Alternatively, should one's instructor or place of employment prefer otherwise, all methods regardless of whether they are value-returning or not can be placed in alphabetical order.

```
public class StackArray {
    private static final int N = 3;
    private int top, sarray[];

    // constructors
    public StackArray() {
        this(N);
    }
    public StackArray(int n) {
        top = 0;
        sarray = new int[n];
    }

    // value returning methods
    public boolean empty() {
        return top <= 0;
    }
    public boolean full() {
        return top >= sarray.length;
    }
    public int pop() {
        if(empty())
            throw new RuntimeException("Stack is empty");
        top--;
        return sarray[top];
    }

    // void method
    public void push(int item) {
        if(full())
            throw new RuntimeException("Stack is full");
        sarray[top]=item;
        top++;
    }
}
```

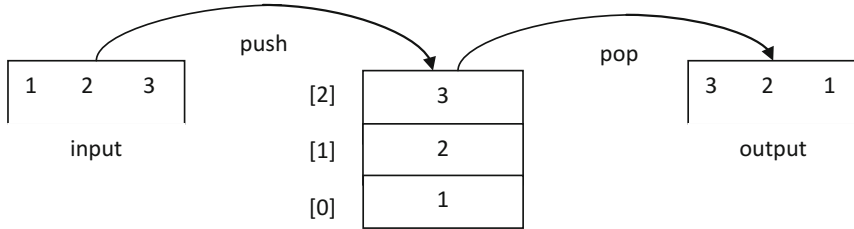
**Fig. 2.1** StackArray class

---

## 2.4 Reversing Integers

Having created a `StackArray` class, how can it be used to solve various problems? As mentioned in the introduction to this chapter, one of the ways in which a stack can be used is to reverse items. As might have been demonstrated in a previous course or text, an array can be used for this task, but a stack is also a convenient way to reverse an array.

Recall that as each item is pushed onto a stack, when an item is popped off the stack, the last one in will be the first one out. For example, first the number 1 could be pushed onto the stack, then the number 2, and lastly the number 3. Then when the numbers are popped off the stack they will be in reverse order. First the number 3 would be popped off, then the number 2, and finally the number 1. If they are output as they are popped off, the output will be in reverse order as shown in the following diagram:



Of course, the first task is to prompt for and input the values to be pushed onto the stack. Although an array and a second loop could be used to push the integers onto the stack, this would be inefficient in terms of space and time, so instead as the values are input, they can be pushed onto the stack. What sort of loop should be used? If there will always be a fixed number, then a fixed iteration `for` loop should be used. However, if there could be a variable number of integers input, then a `while` loop is the best choice. How will the user indicate that all the integers have been input? This would be a good use of a sentinel controlled loop, where assuming only non-negative integers will be entered as data, a negative integer could serve as the sentinel value. In addition to checking for a `-1`, the input loop could also check to see if the stack is full. Then instead of having an exception thrown, it could allow processing to continue. The resulting loop can be seen below:

```
System.out.print("Enter an integer or a -1 to stop: ");
number = scanner.nextInt();
while(number >= 0 && !stack.full()) {
    stack.push(number);
    System.out.print("Enter an integer or a -1 to stop: ");
    number = scanner.nextInt();
}
```

After all the items have been pushed onto the stack, a second loop could be used to pop the items off the stack and output them. What sort of loop would be best in this circumstance? If a count was kept about the number of integers entered, then a fixed iteration loop would be a good choice. However, another alternative is to not use a count and just keep popping integers off the stack until the stack is empty. In this case a `while` loop would be the best choice. The resulting loop can be seen below:



```
System.out.print("The reverse integers are: ");
while(!stack.empty())
    System.out.print(stack.pop() + " ");
System.out.println();
```

Putting all the code together and using the `StackArray` class from Fig. 2.1, the complete program is shown below:

```
import java.util.*;
class Reverse {
    public static void main(String[] args) {

        int number;
        StackArray stack;
        stack = new StackArray();

        Scanner scanner = new Scanner(System.in);

        System.out.println();
        System.out.print("Enter an integer or a -1 to stop: ");
        number = scanner.nextInt();
        while(number >= 0 && !stack.full()) {
            stack.push(number);
            System.out.print("Enter an integer or a -1 to stop: ");
            number = scanner.nextInt();
        }
        System.out.println();
        System.out.print("The reverse integers are: ");
        while(!stack.empty())
            System.out.print(stack.pop() + " ");
        System.out.println();
        System.out.println();
    }
}
```

## 2.5 Generic Types

Given the stack which stores integers and the application that reverses the sequence of integers implemented in the previous section, suppose items of different types were to be reversed. The `StackArray` class would need to be written for the particular type. The new class would be essentially the same as `StackArray` for integers and the only modification that needs to be made would be to change the type of the element stored on the stack. Is there a better way to do this rather than copying and making small changes for different types? It would be nice if the definition of the stack could be written once and used for any type of data. If a parameter could be used to represent the type of an item at the time the abstract data structure is written, then it could be specified when the application is implemented later. The following section describes how to accomplish the above idea using *generic types*.

First, the `StackArray` class presented in Fig. 2.1 could be copied and modified, where the name of the class could be changed to `StackArrayGeneric`, so that if the original class is still needed, there will not be any confusion. In order to define a generic class a *type parameter* in angle brackets is placed at the end of the class name as shown below:

```
public class StackArrayGeneric<T> {
```

Generally single capital letters such as `T` and `E` are used as type parameters in Java. Inside a generic class, the type parameter is used for declarations as if it was a regular type. As can be seen below, `int` is replaced by `T` in the declaration of `sarray`.

```
private T[] sarray;
```

Further, the return type of the `pop` method `int` is also changed to `T`.

```
public T pop() {
```

Note that in `pop`, if the local variable `item` is needed, it could be defined as type `T` and initialized to `null` instead of `-1`. Also, the type of the formal parameter, `item`, in the `push` method needs to be changed to `T`.

```
public void push(T item) {
```

Lastly, one more place the type `T` is needed is in the constructor. In it an array is instantiated, specifically an array of a generic type. At first one might think the following statement could be used to create an array of type `T`:

```
//Caution: Incorrectly Implemented Code  
sarray = new T[n];
```

However, the above code causes a compilation error. In Java an array of a generic type cannot be instantiated. The solution is the use of typecasting as shown below.

```
sarray = (T[]) new Object[n];
```

First, an array of the `Object` class is created and then it is cast into the array of the generic type. The class `Object` is the root of the class hierarchy in Java. In other words, the `Object` class is directly or indirectly a superclass of all the user defined and predefined classes. The above statement will be compiled successfully, but it will produce a warning message during compilation. The message can be ignored and one can proceed to the execution of the program. However, remember that running code which generates a warning is not encouraged unless the meaning of the warning is understood fully and it is not avoidable as it is in this case. Figure 2.2 is the complete `StackArrayGeneric` class definition.

To use a generic class in the main program, a type must be specified for the type parameter. It should be noted that the type has to be a reference type. In other words, it cannot be a primitive data type, such as `int`, `double`, and `char`. To store items of a primitive data type in generic data structures, programmers have to either define a class in order to instantiate an object which contains a field of the primitive data type or they can use a predefined *wrapper class* in the Java API. A wrapper class defines the object which wraps primitive data types and there are eight of them available in Java. They also provide many useful methods dealing with their values. The eight primitive types and their corresponding wrapper classes are shown in Table 2.1.

Before declaring and creating an object of a generic class, recall from the previous section how an object of the `StackArray` class, which was specifically written for the `int` type, was declared and created as shown below:

```
StackArray stack;  
stack = new StackArray();
```

For the generic class, since `int` cannot replace the type parameter, to declare and create a stack which holds integers, the corresponding wrapper class, `Integer`, is used as follows,

```
StackArrayGeneric<Integer> stack;  
stack = new StackArrayGeneric <Integer> ();
```

```

public class StackArrayGeneric<T> {
    private static final int N = 3;
    private int top;
    private T[] sarray;

    // constructors
    public StackArrayGeneric() {
        this(N);
    }

    public StackArrayGeneric(int n) {
        top = 0;
        sarray = (T[]) new Object[n];
    }

    // value returning methods
    public boolean empty() {
        return top <= 0;
    }

    private boolean full() {
        return top >= sarray.length;
    }

    public T pop() {
        if(empty())
            throw new RuntimeException("Stack is empty");
        top--;
        return sarray[top];
    }

    // void method
    public void push(T item) {
        if(full())
            throw new RuntimeException("Stack is full");
        sarray[top]=item;
        top++;
    }
}

```

**Fig. 2.2** StackArrayGeneric<T> class

Instead of using <T> as in the heading of the class definition, <Integer> is used here in the declaration of the variable, `stack`. Once declared, one way to push an `int` onto the stack is by using the following statement to push a 2 onto the stack using the wrapper class `Integer`. First an object of the `Integer` class containing the integer 2 is created by calling the constructor, `new Integer(2)`. Then the reference of the newly created object is used as an argument for the `push` method.

```
stack.push(new Integer(2));
```

**Table 2.1** Wrapper classes

| Primitive type | Wrapper class |
|----------------|---------------|
| byte           | Byte          |
| short          | Short         |
| int            | Integer       |
| long           | Long          |
| float          | Float         |
| double         | Double        |
| char           | Character     |
| boolean        | Boolean       |

The statement below outputs the integer value that is popped off the stack. The `pop` method returns a reference to an `Integer` object, and in order to display the contents of the object, the `intValue` method defined in the `Integer` class is used to convert `Integer` type to `int` type so that it can be output by the `println`.

```
System.out.println(stack.pop().intValue());
```

Combining the above, a simple main program using `StackArrayGeneric` class is implemented below:

```
public class TestStackArrayGeneric {
    public static void main(String[] args) {
        StackArrayGeneric<Integer> stack;
        stack = new StackArrayGeneric <Integer> ();
        stack.push(new Integer(2));
        stack.push(new Integer(3));
        stack.push(new Integer(4));
        while(!stack.empty())
            System.out.println(stack.pop().intValue());
    }
}
```

The wrapper class is necessary to create a stack containing integers when dealing with the generic class. However, it is cumbersome to create an `Integer` object using the constructor in order to push an item onto the stack and extract an `int` value from the `Integer` object using the `intValue` method when popping an item off the stack and outputting it. Although this method works, there is an easier way.

Instead of using a wrapper class explicitly, one can take an advantage of the Java features *autoboxing* and *unboxing*. Autoboxing is an automatic conversion that the Java compiler makes from the primitive types to their corresponding wrapper class, and unboxing is a conversion from the wrapper class to their corresponding primitive types.

Java uses autoboxing if a primitive data type such as `int` is placed where a reference type such as `Integer` is required. Java automatically calls the constructor of the `Integer` class to create an object containing an `int` value. To push a 2 onto the stack, the following statement is legal because Java automatically boxes the `int` value of 2.

```
stack.push(2);
```

Unboxing occurs when, for example, the reference type `Integer` is used in a place where the primitive data type `int` is required. With a stack of type `Integer` and since the return type of the `pop` method is `Integer`, Java would unbox the value returned from `stack.pop()` automatically by calling the `intValue` method so that the content of the `Integer` object is displayed when the following line is executed:

```
System.out.println(stack.pop());
```

Now, relying on Java's autoboxing and unboxing features, the `TestStackArrayGeneric` class can be rewritten as follows:

```
public class TestStackArrayGeneric {
    public static void main(String[] args) {
        StackArrayGeneric<Integer> stack;
        stack = new StackArrayGeneric <Integer> ();

        stack.push(2);
        stack.push(3);
        stack.push(4);

        while(!stack.empty())
            System.out.println(stack.pop());
    }
}
```

The above code is cleaner and expresses the program intentions better without cluttering the code. Both autoboxing and unboxing can occur for all the eight primitive types and their corresponding wrapper classes pairs.

---

## 2.6 Prefix and Postfix Expressions

Typically, an arithmetic expression is in a form known as an infix expression. As a simple example,  $1 + 2$  is an *infix* expression because the arithmetic operator is in-between the two operands, 1 and 2. Alternatively, the arithmetic operator can be placed prior to the operands as in  $+ 1 2$  which is known as a *prefix* expression. This was originally proposed by the Polish mathematician Jan Lukasiewicz in the early 1800s and as a result is sometimes called Polish notation. Further, the arithmetic operator can also be placed after the operands as in  $1 2 +$  and this is known as *postfix* notation or reverse Polish notation (*RPN*).

At first these two alternative forms of expressions might seem a little awkward because one is more familiar with infix notation, but there are various advantages to using prefix or postfix notation and they are a good application for stacks as will be seen shortly. Further, they are very helpful in explaining tree traversals later in Chap. 8.

One important advantage of prefix and postfix notation is their ability to indicate precedence without the use of parentheses. As one should remember from an introductory course in computer science or from mathematics, multiplication and division operators have precedence over addition and subtraction. Further, in infix notation one can always use parentheses to override the precedence of the operators. If there are more than one set of parentheses, then the innermost nested ones are evaluated first. If the parentheses are not nested, then they are evaluated from left to right. In fact, if there is a tie of any sort, such as a subtraction symbol and an addition symbol or a division symbol and a multiplication symbol, the order is also from left to right.

For example, how could the infix expression  $a + b * c$  be converted to prefix? First, one writes down all of the operands in the same order that they appear in the infix expression.

$a \ b \ c$

Note that although one might be able to rearrange the operands and achieve similar results, it is not recommended because it will cause difficulties later. Next, since the multiplication has precedence over addition, it is placed prior to the operands  $b$  and  $c$ .

$a * b \ c$

Then the addition operator is placed prior to the  $a$  and the product of  $* b c$  as follows:

$$+ a * b c$$

However, what if one wanted to have the addition performed first? In infix notation, parentheses would be used as in  $(a + b) * c$ . It should be noted that, in prefix and postfix notation, parentheses are not required to indicate precedence, nor should they be used. Instead the precedence is indicated by the location of the operators. In order to convert the above infix expression to prefix, the process outline above is again followed. First the operands are written in the same order that they appear in the infix expression.

$$a b c$$

Since the addition is done first, the plus sign is placed prior to the  $a$  and  $b$  as follows:

$$+ a b c$$

Next the multiplication is placed prior to  $+ a b$  and the  $c$  as shown below:

$$* + a b c$$

Note that this equation does not contain parentheses and is different than the previous prefix expression indicating the difference in precedence. The same rules follow for postfix except the operators are placed after the operands so that for the infix expression  $a + b * c$ , the multiplication symbol is placed after the  $b$  and  $c$ , and then the plus sign is after the  $a$  and the product  $b c *$  as follows:

$$a b c * +$$

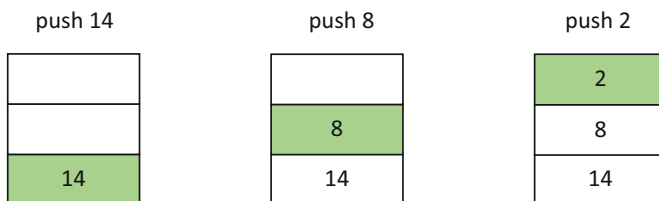
The second infix expression  $(a + b) * c$  would have the plus sign placed after the  $a$  and  $b$ , and the multiplication symbol would be placed after the sum  $a b +$  and the  $c$  as shown below:

$$a b + c *$$

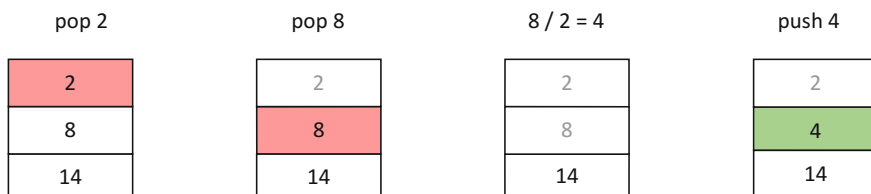
Again note that parentheses are not used and this postfix expression is different from the proceeding one. One of the advantages of prefix and postfix expressions is that they can be evaluated using a stack. Using integers instead of characters, what would the result be for following infix expression  $14 - 8 / 2$ ? An answer of 3 would be incorrect. Recall the order of precedence and note that the correct answer is 10. As demonstrated previously, the corresponding postfix expression would be  $14 8 2 / -$  which can be evaluated using a stack. The rule for evaluating a postfix expression is that when scanning from left to right, every time an operand is encountered it should be pushed onto the stack. Then whenever an operator is encountered, the last two operands should be popped from the stack, the operation performed and the results pushed back onto the stack. Assuming a three-element stack, this is illustrated below



by first pushing the three operands onto the stack with the green indicating the most recent item pushed onto the stack:

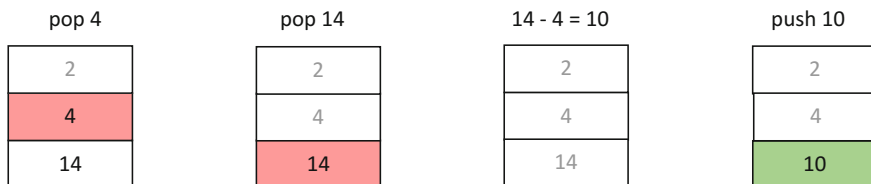


Then when the division sign is encountered, the 2 is popped from the stack, the 8 is popped from the stack, the division is performed, and the result of 4 is pushed back onto the stack as shown below:

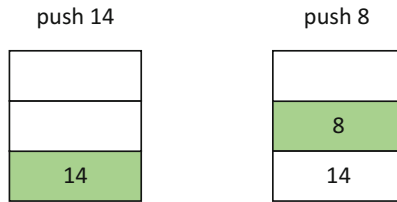


First, notice the order of the operands in the equation as they are popped off the stack when the division is performed. Although not really a concern with addition or multiplication because these operations are commutative, the same does not hold true for division and subtraction. With division, the first operand 2 is used as the divisor and the second operand 8 is used as the dividend. Further, notice that the positions for the 2 and 8 are in red as they are popped off the stack and then the numbers are shown in light gray to indicate that although they are no longer part of the stack, they are still in the array as done previously. Also note that the 8 is overwritten when the 4 is pushed onto the stack.

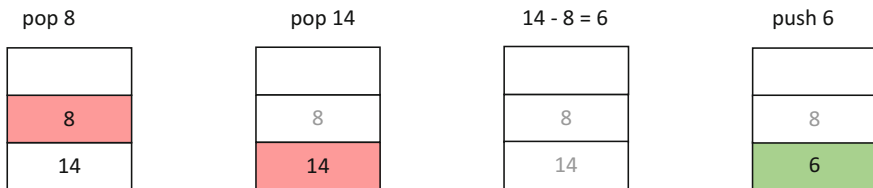
Next when the subtraction is encountered, the 4 and the 14 are popped off the stack, and the 4 is the subtrahend and the 14 is the minuend. The difference is calculated and the result of 10 is then pushed back onto the stack as shown below. If the answer needs to be output or used in another calculation, it can simply be popped off the stack.



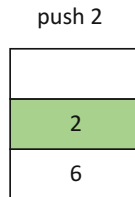
If subtraction is to be performed first, the infix expression would be  $(14 - 8)/2$  and the answer this time would be 3. The corresponding postfix expression is  $14\ 8\ -\ 2\ /$  and its evaluation using a stack is to first



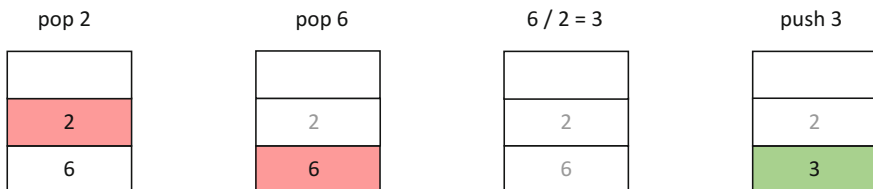
When the minus sign is encountered, the two operands are popped off the stack, the operation is performed, and the results are pushed back onto the stack as follows:



Then the 2 is pushed onto the stack:



When the division symbol is encountered, the 2 and the 6 are popped off the stack, the division is performed and the quotient is pushed back onto the stack. Again, should the answer be needed, it is merely popped off the stack.



As can be seen, both postfix expressions are evaluated correctly using a stack and without the need for parentheses. This is also a good way to check to see if the conversion from infix to postfix has been performed properly. If the operators are not put in the proper order, then the answer might be incorrect. An even worse problem is if an expression is not created properly such as  $a + b c *$  and a program execution error could occur. In this example when the plus sign is evaluated there

would be only one operand on the stack and when the second operand is attempted to be popped from the stack an empty stack exception would be thrown.

Note that prefix expressions can be evaluated similarly but instead of evaluating the expression from left to right, they would need to be evaluated from right to left. The implementation of a program to evaluate postfix expressions is left as an exercise at the end of the chapter, as well the related problem to evaluate prefix expressions.

---

## 2.7 Complete Program: Checking for Palindromes in Strings

This section designs a program that checks if the given word is a palindrome. What is a palindrome? A palindrome is a sequence of symbols such as a word, phrase, verse, or sentence, and that reads the same way from either direction, forward or backward. Examples of single word palindromes include “pop,” “level,” and “radar.” In order to check if the word is a palindrome, the first and the last characters are compared, then the second and the second last characters are compared and so on until all the characters have been compared. During the process, as soon as two symbols do not match, then the word is not a palindrome.

One of the ways to write this application is to use a stack and a stack can be implemented with arrays discussed in this chapter or with objects linked together described in Chap. 7. Therefore, here is the good opportunity to use data abstraction to hide the implementation of these data structures from the person who will be implementing the palindrome application using a stack. As discussed in Sect. 1.3 an interface is used, and generic types and exceptions are incorporated. The `StackGeneric <T>` interface will have headings of all the methods except constructors in the `StackArrayGeneric<T>` class presented in Fig. 2.2 without their code as shown in Fig. 2.3.

One small change needs to be made in the `StackArrayGeneric<T>` class in Fig. 2.2 where the first line is replaced by `public class StackArrayGeneric<T> implements StackGeneric <T>`. Figure 2.4 contains the complete `StackArrayGeneric<T>` class.

In Chap. 7, another class called `StackRefGeneric <T>` class that implements `StackGeneric <T>` interface will be presented. Further, the UML diagram in Fig. 2.5 shows the `StackGeneric <T>` interface and two classes which contains four methods. Note that a dashed line from the class to the interface is used to indicate the relationship between the interface and the classes that implement it.

Now that a stack is implemented using an interface, one can concentrate on how to use a stack to write the palindrome application. First, while scanning a word from left to right, each of the characters are pushed onto a stack. As characters are popped from the stack, they are checked whether they are equal to the characters in the original word. For example, if the characters of the word “level” are pushed onto the stack, the first character popped is “l” and the first character in the original

```
public interface StackGeneric<T> {  
    public boolean empty();  
  
    public boolean full();  
  
    public T pop();  
  
    public void push(T item);  
}
```

**Fig. 2.3** StackGeneric<T> interface

word is “l”. The second character popped and the second character in the original word are both “e”. The third character popped and the third character in the original word are also the same. When the stack is empty and all the characters have been matched, then the word “level” is shown to be a palindrome. Now, if the characters for the word “label” are pushed onto the stack, the first character popped and the first character in the original word are the same. However, the second character popped and the second character in the original word are different, “e” and “a” respectively. Therefore, the word “label” is not a palindrome.

The following program implements the above description. In general it is a good idea to declare variables for an object using interface types rather than class types as discussed in Sect. 1.4. Therefore, the variable `stack` is declared as of type `StackGeneric<Character>`. First, a user will enter a word. Notice the stack of type `StackArrayGeneric<Character>` is created meaning the content of the stack is an object of type `Character`. Also, notice that it is using autoboxing and unboxing. If a `char` type is used where a `Character` object is required, Java will automatically create a `Character` object which contains the `char` value. When a `Character` object is used in a place of where a `char` is required, the `char` value will be extracted from the `Character` object.

```
public class PalindromeVersion1 {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
        int i, j;  
        String word;  
        boolean isPalindrome;  
        StackGeneric<Character> stack;  
  
        System.out.print("Enter a word: ");  
        word = scanner.next();  
        stack = new StackArrayGeneric<Character> (word.length());
```

```

public class StackArrayGeneric<T> implements StackGeneric<T> {
    private static final int N = 3;
    private int top;
    private T[] sarray;

    // constructors
    public StackArrayGeneric() {
        this(N);
    }

    public StackArrayGeneric(int n) {
        top = 0;
        sarray = (T[]) new Object[n];
    }

    // value returning methods
    public boolean empty() {
        return top <= 0;
    }

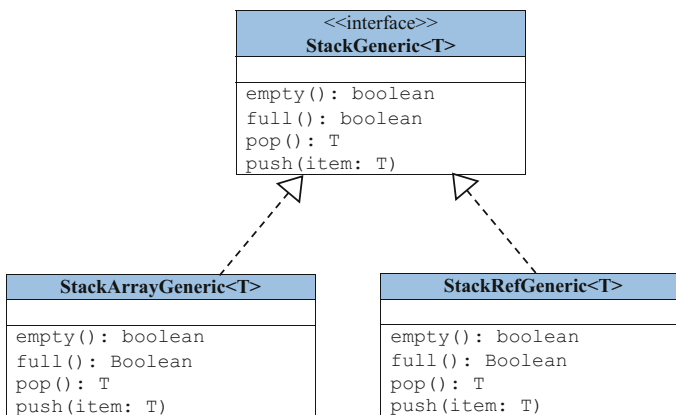
    public boolean full() {
        return top >= sarray.length;
    }

    public T pop() {
        if(empty())
            throw new RuntimeException("Stack is empty");
        top--;
        return sarray[top];
    }

    // void method
    public void push(T item) {
        if(full())
            throw new RuntimeException("Stack is full");
        sarray[top]=item;
        top++;
    }
}

```

**Fig. 2.4** StackArrayGeneric<T> class that implements the StackGeneric<T> interface



**Fig. 2.5** UML Diagram for StackGeneric<T> interface

```
for(i = 0; i < word.length(); i++)
    stack.push(word.charAt(i));
j = 0;
isPalindrome = true;
while(!stack.empty() && isPalindrome) {
    if(stack.pop() != word.charAt(j))
        isPalindrome = false;
    j++;
}
if(isPalindrome)
    System.out.println("The word entered is a palindrome.");
else
    System.out.println("The word entered is NOT a palindrome.");
}
```

Examine the above program carefully and see if there is an improvement that could be made to the code in order to increase efficiency. Instead of storing entire word, what if a half of the word is kept in the stack and characters are checked for equality starting from the middle? The first half of the word from the stack and the last half of the word are compared. The following program is the second version of the palindrome program:

```
import java.util.*;
public class PalindromeVersion2 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int i, j;
        String word;
        boolean isPalindrome;
        StackGeneric <Character> stack;
        System.out.print("Enter a word: ");
        word = scanner.next();
        stack = new StackArrayGeneric<Character> (word.length()/2);
```

```
for(i = 0; i < word.length() / 2; i++)
    stack.push(word.charAt(i));
j = i;
if(word.length()%2 != 0)
    j++;
isPalindrome = true;
while(!stack.empty() && isPalindrome) {
    if(stack.pop() != word.charAt(j))
        isPalindrome = false;
    j++;
}

if(isPalindrome)
    System.out.println("The word entered is a palindrome.");
else
    System.out.println("The word entered is NOT a palindrome.");
}
```

When the above program is compiled and executed using the sample input of “level” and “label,” the output of the program looks as given below:

```
Enter a word: level
The word entered is a palindrome.

Enter a word: label
The word entered is NOT a palindrome.
```

---

## 2.8 Summary

- A stack is a LIFO structure meaning Last-In, First-Out.
- Putting an item on a stack is a push operation, whereas removing an item from a stack is a pop operation.
- Care must be taken to indicate when an array based stack is empty depending on whether `top` is at 0 or -1.
- Depending on the value of `top` used to indicate an empty stack will affect the value used to indicate a full stack.
- Also, care must be taken when creating `push` and `pop` routines to determine whether the increments and decrements occur before or after inserting an item into or removing an item from an array based stack.

- Exceptions are often a convenient way to inform programmers and users when an execution error has occurred.
- Generic types and the wrapper class make data structures reusable with different primitive data types.
- Autoboxing is an automatic conversion that Java makes from the primitive types to their corresponding wrapper classes and unboxing is an automatic conversion that Java makes from wrapper classes to their corresponding primitive types.
- Prefix and postfix expressions should not use parentheses to indicate precedence.

---

## 2.9 Exercises (Items Marked with an \* Have Solutions in Appendix B)

- \*1. Assuming the stack `intStack` of type `StackArray` in Fig. 2.1 was initially empty, draw a stack after the following operations are completed:

```
int x, y, z;
intStack.push(41);
intStack.push(6);
x = intStack.pop();
y = intStack.pop();
intStack.push(78);
intStack.push(5);
intStack.push(79);
z = intStack.pop();
intStack.push(5);
```

2. Assuming the stack `characterStack` of type `StackArrayGeneric<Character>` which is an object of the generic class `StackArrayGeneric<T>` in Fig. 2.2 is initially empty, draw a stack after the following operations are completed:

```
Character x, y, z;
characterStack.push('U');
characterStack.push('V');
x = characterStack.pop();
characterStack.push('W');
y = characterStack.pop();
```



```
characterStack.push('P');  
characterStack.push('Q');  
z = characterStack.pop();  
characterStack.push('C');
```

- \*3. Write a method `peek` that can be added to the `StackArray` class in Fig. 2.1 which returns the item at the top of the stack without removing it.
4. Add the heading of the `peek` method described in the previous question to the `StackGeneric <T>` interface in Fig. 2.3 and implement it in the `StackArrayGeneric<T>` class in Fig. 2.4.
5. Write a method `size` that can be added to the `StackArrayGeneric<T>` class in Fig. 2.2 which returns the number of items in the stack.
6. In Sect. 2.3, the `push` method was implemented by having the variable `top` be 0 initially and incremented by 1 after pushing an item on to the stack. Rewrite the `push` method in Fig. 2.1 where the `top` is initialized to -1 and incremented prior to pushing an item at the location. Also, rewrite the `pop` method so that it works correctly too.
7. Section 2.3 mentioned that there was a possibility an item could not be pushed onto the stack because the stack is full. One solution is to create a larger array and then copy all the items from the old array into the new array. Write a method `expandArray` that creates a new array that is twice the size of the previous one and copies the item over to the new array. The method should be added to the `StackArray` class in Fig. 2.1 and be called from the `push` method.
8. Find a reference on how to convert a decimal number to a binary number such as *Guide to Assembly Language* [1] or other similar text and then write a method to perform the conversion using a stack.
9. What are the time complexities of the methods `full` and `empty` in Fig. 2.1 and also `expandArray` from the question 7? Give your answer in Big O notation.
- \*10. What are the time complexities of the two complete programs in Sect. 2.7? Is there a difference between the two? Give your answer in Big O notation.
11. Write a program to evaluate a postfix expression using an algorithm discussed in Sect. 2.6. Assume that the input contains only single digit numbers as operands and that a valid postfix expression is entered as input.
12. Write a program to evaluate a prefix expression using a stack. A hint for the implementation is given in Sect. 2.6. Assume that the input contains only single digit numbers as operands and that a valid prefix expression is entered as input.

---

## 3.1 Introduction

In addition to stacks discussed in the last chapter, a queue is another simple yet useful data structure. The queue can be found in the real world anytime one observes an assembly line in a factory where a product is being built, waiting to buy tickets for a popular new movie release, or at a drive-thru at a fast food restaurant. The first item on a conveyor belt is the first one to be completely assembled or the first person in a line is the first one to receive service. In other words, a *queue* is a first-in, first-out (*FIFO*) data structure.

A queue is also useful in computers where a queue that is perhaps most often experienced by computer users is the print queue. There are also many other queues that might not seem as obvious to computer users. For example, as one might learn in an operating systems course, that when there are two programs contending for CPU time, the operating system might choose the first one in the queue. Or as might be learned in an interfacing course, a low-level program after servicing a foreground interrupt might put information in a foreground-background queue for subsequent processing in background. There are also a number of other types of queues such as priority queues (discussed in Chap. 10), multilevel queues, and multilevel feedback queues, but this chapter will discuss a simple queue to provide a strong foundation to facilitate learning of these other types of queues in the future. Suffice it to say, queues are a very important data structure in the field of computer science.

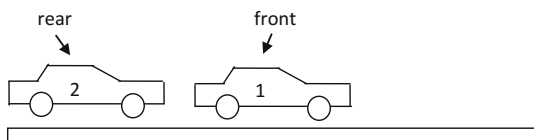
Unlike a stack, where there is only one access point at the top of the stack, there are two access points in a queue, the front and the rear of the queue. Further, whereas data can be pushed or popped from the top of the stack, items can only

enter at the rear of a queue and be removed from the front of the queue. The entering is known as an *enqueue* operation and the removing is known as a *dequeue* operation. Further, just as with a stack, a queue can be full or empty.

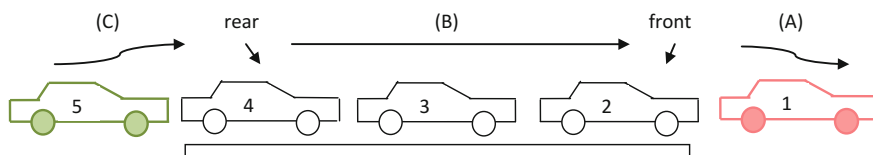
Learning some of the features of a queue can be done by looking at a simulation of an assembly line. Note that the first car in the assembly line will be both at the rear and the front of the line.



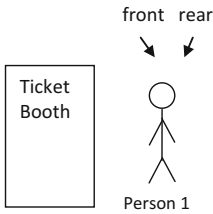
Then as the car at the front moves down the assembly line to have more parts attached to it, there is another car that can be put onto the assembly line or in other words enqueued into the queue as shown below:



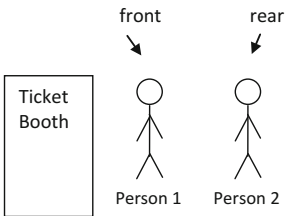
Note that in the above example, the front of the line moves as the rear remains stationary. Once the line or queue is full, (A) an assembled car will need to be dequeued, (B) the remaining cars in the queue will need to move down the line, then (C) another car can be enqueued.



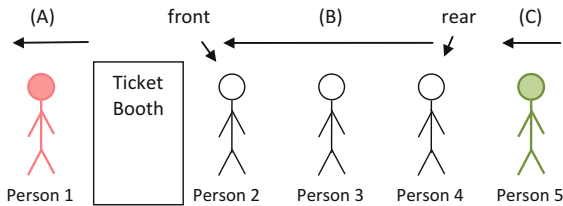
In another example, imagine waiting in line at a movie theater to get tickets for the latest blockbuster movie. As above, the first person in the line is both at the front and rear of the line.



As more people come to stand in line, the rear of the line gets longer, but the person in front stays in the same position.



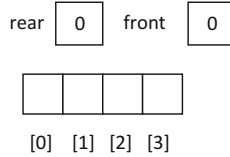
As the person in the front of the line (A) receives their ticket, proceeds to buy popcorn and enter the theatre, (B) then all of the people in the line will need to move up one position so the next person in line can purchase a ticket. Although the queue might not necessarily get full like in the assembly line example, (C) since people can continue the line out into the street, there can be a limit as to how many people will be able to purchase tickets for a seat in the theatre.



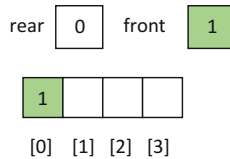
Both of the above are examples of queues, but as will be seen in the next section, having all of the cars move up each time a new one is enqueued or have all the people in line at the movie theatre move up each time a person buys a ticket and is dequeued is inefficient when it comes to analysis of the algorithm. However, there is a way to overcome this inefficiency as will be seen in the next section.

### 3.2 Analysis and Design

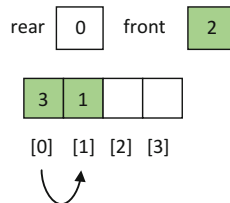
Simulating the assembly line example from the previous section and beginning to think of how to implement a queue using an array, first assume the existence of a four element array, which is small enough to make it easy to manipulate yet large enough to allow patterns to emerge. The queue will initially be empty and there also need to be indexes for the `front` and the `rear` of the queue. Both the `rear` and `front` indexes could be initialized to a 0.



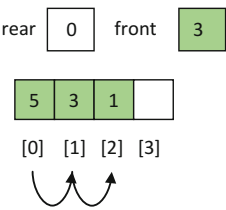
To enqueue an item such as the integer 1 at the rear of the queue, the integer 1 would be placed in position 0 and `front` would be incremented from 0 to 1 to indicate the new front of the queue and with the changes shown in green below:



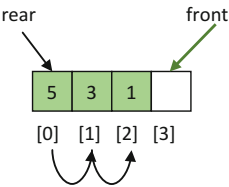
However, what would happen if one wanted to enqueue another item such as the integer 3 into the queue? In this scenario, the integer 1 would be moved to position 1, the integer 3 could then be placed in position 0, and `front` would be incremented to indicate the new front of the queue.



Then if the integer 5 were to be enqueued, the integers 3 and 1 would be moved to the right, the integer 5 placed into position 0 and `front` would again be incremented.



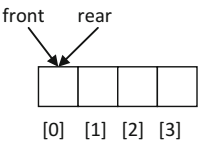
Alternatively, arrows or pointers could be used to graphically show the positions of front and rear as shown below. Either image is acceptable where the previous one is more accurate but the following one might be helpful on occasion to visualize the operations:



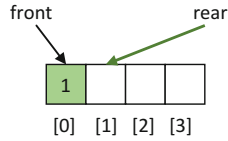
Although the above algorithm appears to work, what is a potential disadvantage to this? Imagine a one-hundred element queue with ninety-nine items in it. Before the one hundredth integer is placed into the queue, the other ninety-nine integers would need to be moved. Further, given an  $n$  element queue, then in the worst-case scenario,  $n-1$  elements would need to be moved which as discussed in Chap. 1 is of order  $n$ , or  $O(n)$ .

However, the queue might not always be filled, so how many items would need to be moved? One could assume that on average one half the items would need to be moved, so in the case of a 100 element queue, 50 would need to be moved and in the case of an  $n$  element queue,  $n/2$  items would need to be moved. Recall from Chap. 1 that  $n/2$  is still a linear function of  $n$ , so it would be on average order  $n$  or  $O(n)$ .

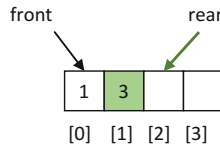
Is there a better solution to this problem and thus a more efficient algorithm? The answer is yes, where instead of moving front and the items in the queue, the rear pointer could be moved instead, similar to the movie theatre example from the previous section. Again starting with an empty queue, note that the use of arrows and that front and rear are reversed to make the drawing a little easier to read.



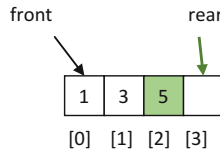
Now when the integer 1 is enqueued, `rear` has moved instead of `front`, in preparation for the next item to be enqueued all as shown in green below:



Now when 3 is enqueued, there is space for it at position 1 and `rear` is again incremented by 1.

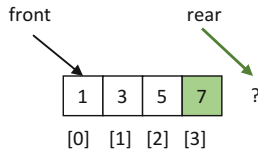


Again, when the integer 5 is enqueued, it is placed in position 2 and `rear` is moved to position 3.

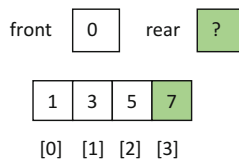


Notice so far that the advantage to this algorithm is that there is no need to move all the items in the queue when a new item is enqueued. Only `rear` needs to be changed, which takes the same constant amount of time regardless of the size of the queue. As discussed in Chap. 1, an operation like this that can be accomplished in constant time is said to be of order 1, or  $O(1)$ .

However, what happens if the integer 7 were to be enqueued? It would go into position 3, but what would happen to `rear`?



Switching back from using arrows to integers to help in the following discussion, the above drawing could be redrawn as follows:



Unlike the movie ticket example in the previous section, the array is of finite length and the queue has become full. Although a new larger array could be created and the contents of this queue could be copied into the new array, this would take time and the problem could occur again when that queue becomes full. An alternative is to implement what is known as a *circular queue* and have `rear` go back to position 0.

The question now is how can `rear` be made to contain a 0? The moving of `front` in the previous example and `rear` in this example up to this point should be seen as a simple increment operation, but that would not work in this case. A possible solution is to employ an `if` statement that when `rear` reaches the number 3, or  $n - 1$  in the case of an  $n$  element queue, it should be set back to 0. Although this works, it would require that the value of `rear` be checked every time a new item is enqueued. The same would eventually have to be done with `front` during a dequeue operation when it reaches the end of the queue. Is there a cleaner way to solve this problem?

If one thinks back to his or her first programming class or textbook, there was an arithmetic operation that might not have seemed to be of much value at the time, but it is here that it can be quite useful. Along with the addition, subtraction, multiplication, and division operators, there was the mod operator `%` that returned the remainder when performing division. As a reminder:

```
1 % 4 = 1
2 % 4 = 2
3 % 4 = 3
```

At first this might not seem too interesting because in all of the above examples the integers divided by 4 returned the original integer. However, in our problem when `rear` is incremented to 4, then the following would cause the value of `rear` to return to 0.

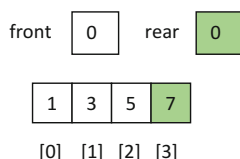
```
4 % 4 = 0
```

In other words, after incrementing either `front` or `rear`, instead of having to use an `if` statement to return the value back to zero, the mod operator could be used. In the cases where the value is 1 through 3, there would be no change, but



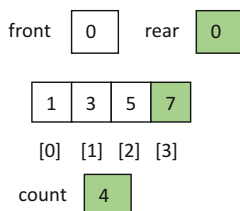
when the value becomes 4 it would revert back to 0 thus achieving the desired effect as will be seen in the code in the next section. This process could be utilized in any queue of size  $n$ .

Now after the value of 7 has been entered into the queue, the value of `rear` would be 0 as shown below:



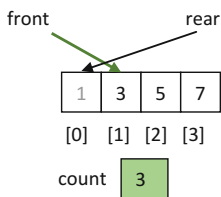
However, this does create a new problem. Note that the values of `front` and `rear` are the same as they were in the first drawing in this section. Although the reader can see that the queue is full, how will program be able to tell if the queue is full or empty? One might be tempted to just check to see if any of the cells contain values, but depending on what was previously stored in the queue this might not be a good indication whether the queue is empty. Another possible solution is to put a special value in a cell to indicate an empty cell, such as 0, but what if 0 is a valid integer that can be stored in the queue?

A solution to this problem is to use a variable `count` to indicate how many items are in the queue. If there are no items in the queue, the value of `count` would be zero. Each time an item is enqueued the value of `count` would be incremented and each time an item is dequeued the value of `count` would be decremented. In other words, when the value of `count` is zero the queue is empty and when the value of `count` equals  $n$ , the size of the array, it is full, where in the current case that value is 4. So the upgraded diagram indicating a full queue would look as follows:



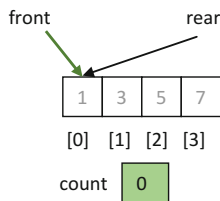
Up to this point, this section has only examined the enqueue operation, but what about the dequeue operation? One solution is to emulate the movie ticket example and move all of the elements in the array back from position 1 through 3 to position 0 through 2 and have `rear` adjusted accordingly. However, this would require the use of a loop and if there were  $n$  items in the array, it would be an  $O(n)$  operation which was what was trying to be avoided at the beginning of this section.

Instead of having all the people in the line move forward one position, a better solution from the movie ticket example is to have the person issuing the tickets move down the line. In an array, instead of moving the items in the array and adjusting `rear`, it is far more efficient just to adjust `front`. In looking at the previous diagram, one should notice that when 1 is dequeued, `front` could be incremented by one to indicate the new front of the queue and `count` decremented by one to indicate one less item in the queue as shown below.

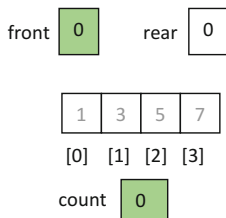


The result is that the dequeuer operation could be  $O(1)$  instead of  $O(n)$ . As with stacks in the last chapter, the integer 1 is not actually removed from the array, but since `front` is no longer at position 0, the 1 is no longer part of the queue. Although in the future the dequeued item might not be shown, it is shaded light gray in this section to help make the distinction between the queue data structure and the array in which it is implemented.

This process of dequeuing would continue until all of the items have been dequeued, `front` and `rear` point to the same location (which does not need to be location 0), and `count` is again 0 indicating an empty queue.

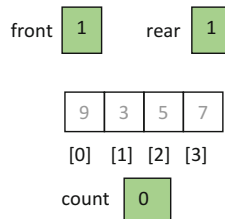


Alternatively, the following drawing indicates the same state.



Again, note that the items in the array are shaded light gray, indicating they are still in the array, but not in the queue. This is why checking the contents of the array to see if it is empty as suggested earlier is not a good indicator that the queue is empty.

Some programmers erroneously think when `front` and `rear` each contain a 0 it indicates an empty queue or possibly a full queue. However, an empty or full queue can occur anywhere in an array. For example, in the above diagram, consider enqueueing the integer 9 which would cause `rear` to become one and `count` would also become one. When it is dequeued, as shown in light gray below, `count` would be decremented to zero and `front` would become one. Both `front` and `rear` would now both equal 1, not 0, and yet the queue is empty. This is yet another reason for the inclusion of the variable `count` as shown below:



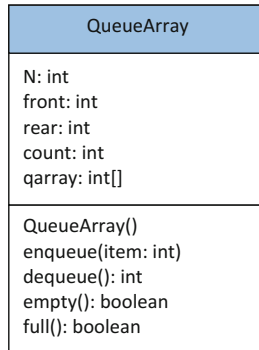
---

### 3.3 Queue Class: Data Members and Methods

Having looked at how a queue functions, the class for the queue can be created. To distinguish it from a queue implemented later in this text, the name of the class could be `QueueArray`. The data members for the queue can be seen in the drawings in the previous section. The queue needs to be implemented as an array which could be called `qarray`, and the three other data members `front`, `rear`, and `count`. Of course keeping with the practices of object-oriented programming, these data members should be declared as `private` and accessible only through `public` methods.

There also need to be four methods as discussed in the previous section plus a constructor. The four methods are `enqueue`, `dequeue`, `full`, and `empty`. The `enqueue` method will need a parameter to send the item to be enqueued and will be a `void` method since it does not return a value. The `dequeue` method will not need a parameter but will be a value-returning method to return the item dequeued. The `full` and `empty` methods will be value-returning methods returning the appropriate boolean value. The `enqueue` and `dequeue` methods should be `public` and the `empty` and `full` methods can be made `public` or `private` as needed.

As with a stack in Sect. 2.3, a UML diagram can be created to illustrate the `QueueArray` class as shown below:



The constructor will need to initialize the three integer variables to 0. The `enqueue` method will place the item into `qarray` at the location indicated by the index `rear`, then `rear` will need to be incremented by 1, and `count` will also need to be incremented by 1 as follows:

```
qarray[rear] = item;
rear = rear + 1;
count = count + 1;
```

Further, if `rear` is equal to the size of the array indicated by the constant `N`, then it will need to wrap around as discussed in the previous section. This can be accomplished by using the mod function after `rear` has been incremented as shown below:

```
qarray[rear] = item;
rear = (rear + 1) % N;
count++;
```

Note that `count` has been changed to the auto increment mode since it does not need to use the mod function. Of course if the array is full then the value in `item` should not be placed in the array and a message should be output.

```
if(full())
    System.out.println("Queue is full: item not enqueued");
else {
    qarray[rear] = item;
    rear = (rear + 1) % N;
    count++;
}
```

Alternatively, an exception could be used to indicate a full queue.

```
if(full())
    throw new RuntimeException("Queue is full: " +
        "item not enqueued");
qarray[rear] = item;
rear = (rear + 1) % N;
count++;
```

As in the previous chapter on stacks, note that the else section is not needed since the program will stop if an exception is thrown. The resulting complete enqueue method would appear as follows:

```
public void enqueue(int item) {
    if(full())
        throw new RuntimeException("Queue is full: " +
            "item not enqueued");
    qarray[rear] = item;
    rear = (rear + 1) % N;
    count++;
}
```

The dequeue operation is similar to the enqueue operation except the value of `item` must be taken from the front of the queue. Since `front` can wrap around the same as `rear` does in `enqueue`, note that the mod function is already included in the code segment. Lastly, `count` will need to be decremented by 1 as shown below:

```
item = qarray[front];
front = (front + 1) % N;
count--;
```

Whereas `enqueue` is concerned with whether the queue is full, `dequeue` must check whether the queue is empty and output the appropriate message.

```
if(empty())
    System.out.println("Queue is empty: item not dequeued");
else {
    item = qarray[front];
    front = (front + 1) % N;
    count--;
}
```

As before, an exception can be used in lieu of a `println` statement.

```
if(empty())
    throw new RuntimeException("Queue is empty: " +
        "item not dequeued");
item = qarray[front];
front = (front + 1) % N;
count--;
```

Once the item is removed from the queue, it will need to be returned to the calling program or method. As with stacks in the previous chapter, when using `if` and `println` statements the method will continue on to the `return` statement and `item` needs to contain a value. As before, the value `-1` is chosen and the complete `dequeue` method is as follows:

```
public int dequeue() {
    int item = -1;
    if(empty())
        System.out.println("Queue is empty: item not dequeued");
    else {
        item = qarray[front];
        front = (front + 1) % N;
        count--;
    }
    return item;
}
```

When using an exception, the program will terminate and the `return` statement will not be executed, so there is no need to include a value for `item`. The complete `dequeue` method using an exception is shown below.

```
public int dequeue() {
    int item;
    if(empty())
        throw new RuntimeException("Queue is empty: " +
            "item not dequeued");
    item = qarray[front];
    front = (front + 1) % N;
    count--;
    return item;
}
```

Lastly, the `full` and `empty` value-returning methods will be needed. As with stacks in the preceding chapter, `full` and `empty` can be written without the use of an `if` statement. Further, recall from the preceding section that when `count` equals `N`, then the queue is full and when `count` equals `0`, the queue is empty.

```
return count == N;           return count == 0;
```

Also, as with stacks previously, the relation can alternatively include checking for values greater than or equal to `N` or less than or equal to `0` as follows:

```
return count >= N;          return count <= 0;
```

The resulting two complete boolean value returning methods are:

```
public boolean full() {      public boolean empty() {
    return count >= N;        return count <= 0;
}                             }
```

Putting the data members and methods together, the complete `QueueArray` class is shown in Fig. 3.1.

Note that the `println` statements are used instead of the exceptions which will help facilitate the development of the main program in the next section. The methods using exceptions can be used instead merely by substituting the methods developed above.

---

### 3.4 Program Using the `QueueArray` Class

To test the `QueueArray` class, a simple program should be developed. This program will merely prompt the users whether they want to enqueue or dequeue an item. The letter `E` is used to indicate the former which is then followed by a prompt and input for the item to be enqueued. The letter `D` is used to indicate the latter and the item dequeued is displayed. To stop the program, the letter `S` is entered which serves as a sentinel value for a sentinel controlled loop.

In order to enter a single character, note that the following code segment would work assuming the appropriate declaration of the `Scanner` class. The character is input into the string variable `tempString` and then the single character is extracted from the string using `charAt(0)` and placed into the `char` variable `selection`.

```
String tempString;
char selection;
System.out.print("Please enter E for Enqueue, " +
```

```

public class QueueArray {
    private final int N = 4;
    private int front, rear, count, qarray[];
    public QueueArray() {
        front = rear = count = 0;
        qarray = new int[N];
    }
    public int dequeue() {
        int item = -1;
        if(empty())
            System.out.println("Queue is empty: " +
                               "item not dequeued");
        else {
            item = qarray[front];
            front = (front + 1) % N;
            count--;
        }
        return item;
    }
    public void enqueue(int item) {
        if(full())
            System.out.println("Queue is full: item not enqueued");
        else {
            qarray[rear] = item;
            rear = (rear + 1) % N;
            count++;
        }
    }
    public boolean empty() {
        return count <= 0;
    }
    public boolean full() {
        return count >= N;
    }
}

```

**Fig. 3.1** QueueArray Class

```

"D for Dequeue, or S for stop: ");
tempString = scanner.next();
selection = tempString.charAt(0);

```

However, this could be shortened some by eliminating the `tempString` variable and invoking the `charAt(0)` method just after the `next` method as shown below:

```

char selection;
System.out.print("Please enter E for Enqueue, " +
"D for Dequeue, or S for stop: ");
selection = scanner.next().charAt(0);

```



This of course could be used in a sentinel controlled loop as follows:

```
System.out.print("Please enter E for Enqueue, " +
"D for Dequeue, or S for stop: ");
selection = scanner.next().charAt(0);
while (selection != 'S' && selection != 's') {
    // body of loop
    System.out.print("Please enter E for Enqueue, " +
"D for Dequeue, or S for stop: ");
    selection = scanner.next().charAt(0);
}
```

Note that there are both the priming read and the second read at the bottom of the loop as is typical for sentinel controlled loops. Also notice that the `while` statement checks for both uppercase `S` and lowercase `s` should a user forget to press the shift key on the keyboard.

Once the loop is created, the logic for the other two possible inputs can be created. As mentioned previously, should the letter `E` be entered, then the user should be prompted for the item to be enqueued and then the `enqueue` method should be invoked. If the queue is full, then the `enqueue` method will output the appropriate message. Should the user enter the letter `D` then the item at the front of the queue should be dequeued and output. If the queue is empty, then the `dequeue` method will also output the appropriate message and return a `-1` which should not be output by the main program. If any other value is entered, a user-friendly message should be output by the main program.

```
if(selection == 'E' || selection == 'e') {
    System.out.print("Please enter an integer to be enqueued: ");
    item = scanner.nextInt();
    queue.enqueue(item);
}
else
    if(selection == 'D' || selection == 'd') {
        item=queue.dequeue();
        if(item != -1)
            System.out.println("The item dequeued is: " + item);
    }
else
    System.out.println("The selection entered " + selection
    + " is not E, D, or S, please try again");
```

Putting the above code inside the body of the loop and including all the necessary declarations, the complete program looks as follows:

```
import java.util.*;
public class TestQueueArray {
    public static void main(String args[]) {
        int item;
        char selection;
        QueueArray queue;
        queue = new QueueArray();
        Scanner scanner = new Scanner(System.in);

        System.out.println();
        System.out.print("Please enter E for Enqueue, " +
            "D for Dequeue, or S for Stop: ");
        selection = scanner.next().charAt(0);
        while(selection != 'S' && selection != 's') {
            if(selection == 'E' || selection == 'e') {
                System.out.print("Please enter an integer to be " +
                    "enqueued: ");
                item = scanner.nextInt();
                queue.enqueue(item);
            }
            else
                if(selection == 'D' || selection == 'd') {
                    item = queue.dequeue();
                    if(item != -1)
                        System.out.println("The item dequeued is: " +
                            item);
                }
            else
                System.out.println("The selection entered " +
                    selection + " is not E, D, or S, please try again");
            System.out.println();
            System.out.print("Please enter E for Enqueue, " +
                "D for Dequeue, or S for Stop: ");
            selection = scanner.next().charAt(0);
        }
        System.out.println();
        System.out.println("End of Program");
        System.out.println();
    }
}
```

Sample output from the program showing all the messages including prompting for a selection, prompting for an item to enqueue, the dequeued output, and the messages that the queue is full, the queue is empty, wrong selection, and the end of program message is as follows:

Please enter E for Enqueue, D for Dequeue, or S for Stop: q  
The selection entered q is not E, D, or S, please try again

Please enter E for Enqueue, D for Dequeue, or S for Stop: e  
Please enter an integer to be enqueued: 1

Please enter E for Enqueue, D for Dequeue, or S for Stop: e  
Please enter an integer to be enqueued: 2

Please enter E for Enqueue, D for Dequeue, or S for Stop: e  
Please enter an integer to be enqueued: 3

Please enter E for Enqueue, D for Dequeue, or S for Stop: e  
Please enter an integer to be enqueued: 4

Please enter E for Enqueue, D for Dequeue, or S for Stop: e  
Please enter an integer to be enqueued: 5  
Queue is full: item not enqueued

Please enter E for Enqueue, D for Dequeue, or S for Stop: d  
The item dequeued is: 1

Please enter E for Enqueue, D for Dequeue, or S for Stop: e  
Please enter an integer to be enqueued: 5

Please enter E for Enqueue, D for Dequeue, or S for Stop: e  
Please enter an integer to be enqueued: 6  
Queue is full: item not enqueued

Please enter E for Enqueue, D for Dequeue, or S for Stop: d  
The item dequeued is: 2

Please enter E for Enqueue, D for Dequeue, or S for Stop: d  
The item dequeued is: 3

Please enter E for Enqueue, D for Dequeue, or S for Stop: d  
The item dequeued is: 4

Please enter E for Enqueue, D for Dequeue, or S for Stop: d  
The item dequeued is: 5

Please enter E for Enqueue, D for Dequeue, or S for Stop: d  
Queue is empty: item not dequeued

Please enter E for Enqueue, D for Dequeue, or S for Stop: s

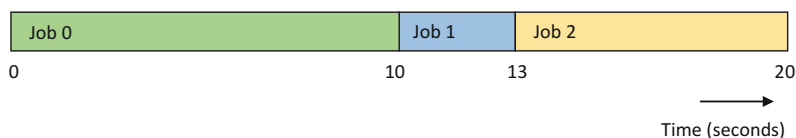
End of Program

### 3.5 Complete Program: Simulating a Scheduling Algorithm

In this section a simple scheduler in an operating system is simulated using an array-based queue. As mentioned in Sect. 3.1, queues are used in many instances in computer software. Often on a computer more than one application, such as a word processor, a spreadsheet, and a web browser, can be opened at the same time. Assuming the machine has one processor, these programs, sometimes called jobs, are submitted to the operating system for execution. A *CPU scheduler* can then decide which program will run next. There are various scheduling algorithms, but one of the simplest uses a First-In, First-Out queue described in this chapter. With a queue, the job that requests the CPU first is allocated the CPU first. Consider the following set of jobs with the length of the CPU time in seconds that is required for finishing the job.

| Job Number | CPU Time Required |
|------------|-------------------|
| 0          | 10                |
| 1          | 3                 |
| 2          | 7                 |

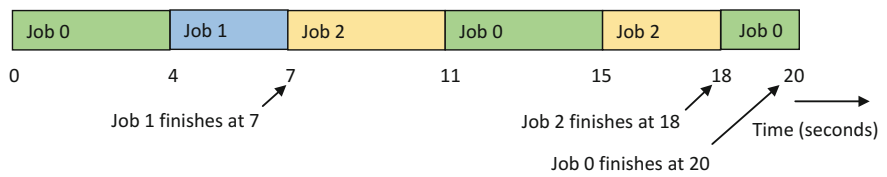
If the jobs arrive in the order Job 0 through Job 2 when the CPU is idle and are served in FIFO fashion, the timeline of how these jobs are executed can be shown in the following chart called a Gantt chart:



To make it simple, all the jobs are submitted at the beginning and no jobs will arrive later. As soon as Job 0 enters the queue, it will be dequeued for execution and finishes after 10 s. After waiting for 10 s for Job 0 to finish, Job 1 is dequeued, the CPU is allocated the job, and it completes after 3 s at the 13 s mark. Job 2 waits for 13 s in the queue while Job 0 and Job 1 are executing. It is then dequeued, enters the CPU, and finishes after 7 s at the 20 s mark. Although all the jobs are processed, the disadvantage of this method is that shorter jobs arriving after the longer jobs have to wait a longer time although they do not require a longer CPU time.

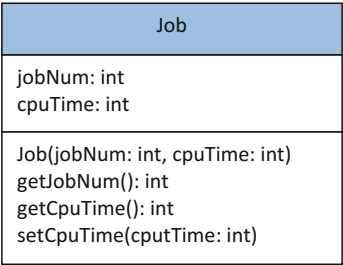
For every job to have a fair waiting time, a *round-robin* (RR) scheduling algorithm can be used. It is basically FIFO scheduling, but each job can only stay in the CPU a certain amount of time. If it does not finish during the time given, it simply gives up the CPU and reenters the queue in order to receive further CPU time later. A small unit of time, called a time quantum, is used. Should a job entering the CPU have a CPU time of less than the time quantum, the job will release the CPU and the scheduler will then proceed to the next job in the queue.

Otherwise, if the CPU time of the currently running job is longer than the time quantum, the job will be put at the rear of the queue. The CPU scheduler will then select the next job in the queue. Using the same example above and a quantum of 4 s the Gantt chart will look like the following:



Job 0 gets the first 4 s which is the size of the quantum. Since it requires another 6 s to complete, it is preempted and placed at the rear of the queue, and then the CPU is given to the next job in the queue, Job 1. Job 1 does not need 4 s, so it finishes before its time quantum expires. The CPU is then given to the next job, Job 2. Once each job has received one time quantum, the CPU is returned to Job 0 at the 11 s mark for an additional time quantum. As can be seen, Job 1 finishes at the 7 s mark, Job 2 at the 18 s mark, and Job 0 at the 20 s mark. The advantage is that the shorter job does not have to wait as long.

Given the preceding brief discussion of CPU scheduling algorithms, a round-robin scheduler will be implemented using an array-based queue to find the time when each job completes. With this application the items stored in the queue are representations of the jobs, rather than simple integers discussed in the previous sections, which can be implemented as objects with data and operations. So, the Job class will need to be defined first. Two values will be kept in a Job object, `jobNum` and `cpuTime`. The `jobNum` is the ID number of each job and the `cpuTime` is the time CPU requires to complete the job. The class will contain one constructor that accepts two parameters to initialize both data members. Two accessors are defined to get the values of the two data members and a mutator to set the value of the `cpuTime`. The following UML diagram represents the Job class.



As can be seen, there is no mutator for the `jobNum` since the value does not change once it is assigned; however, `cpuTime` will need to be updated during the execution using the `setCpuTime` method. The code defining the class for a Job object is shown below:

```
public class Job {
    private int jobNum, cpuTime;

    // constructor
    public Job(int jobNum, int cpuTime) {
        this.jobNum = jobNum;
        this.cpuTime = cpuTime;
    }

    // returns the jobNum
    public int getJobNum() {
        return jobNum;
    }

    // returns the cpuTime
    public int getCpuTime() {
        return cpuTime;
    }

    // sets the cpuTime
    public void setCpuTime(int cpuTime) {
        this.cpuTime = cpuTime;
    }
}
```

Since objects of `Job` class are kept in the queue, the `QueueArray` class shown in Fig. 3.1 needs to be rewritten for the `Job` class. Although it is not difficult to modify the `QueueArray` class for the `Job` object, rewriting a `QueueArray` class for the specific object every time a new application is developed is not very efficient. Remember in Sect. 2.5 where generic types were discussed? The `QueueArray` class that can be used for any objects will be written next. Essentially the `int` in `QueueArray` is replaced by the type parameter `T`. Figure 3.2 shows the complete `QueueArrayGeneric` class definition. Notice that the public method, `getCount` is added in order to get the value of the private data member `count` of the `QueueArrayGeneric` class in the main program. In the `dequeue` method, the local variable `item` is initialized to `null`, indicating that initially no reference to a `Job` object is assigned to the variable. Therefore, instead of returning a `-1`, a value of `null` is returned when the queue is empty.

In the main program, two methods, `setJobs` and `runScheduler` are invoked. The `setJobs` method is to obtain information from a user about the jobs to be executed in the CPU. The `runScheduler` simulates the round-robin scheduler. The main program without the detailed implementation of the two methods is shown below, and the two methods are defined shortly.

```

public class QueueArrayGeneric<T> {
    private final int N = 5;
    private int front, rear, count;
    private T[] qarray;

    public QueueArrayGeneric() {
        front = rear = count = 0;
        qarray = (T[]) new Object[N];
    }

    public T dequeue() {
        T item = null;
        if(empty())
            System.out.println("Queue is empty: item not dequeued");
        else {
            item = qarray[front];
            front = (front + 1) % N;
            count--;
        }
        return item;
    }

    public void enqueue(T item) {
        if(full())
            System.out.println("Queue is full: item not enqueued");
        else {
            qarray[rear] = item;
            rear = (rear + 1) % N;
            count++;
        }
    }

    public boolean empty() {
        return count <= 0;
    }

    public boolean full() {
        return count >= N;
    }

    public int getCount() {
        return count;
    }
}

```

**Fig. 3.2** QueueArrayGeneric < T > class

```

import java.util.*;

public class RoundRobin {
    private static Scanner scanner = new Scanner(System.in);
    private static QueueArrayGeneric<Job> jobQueue
        = new QueueArrayGeneric<Job>();
    private static int quantum;

```

```
public static void main(String[] args) {
    setJobs();
    runScheduler();
}

// gets information about jobs run in CPU
public static void setJobs() {
    // defined shortly
}

// simulates the round-robin scheduler
public static void runScheduler() {
    // defined shortly
}
}
```

In the `setJobs` method, first a user will enter both the number of jobs and size of the time quantum. Then for the number of jobs entered, the user will be prompted for the amount of CPU time needed for each job, a `Job` object will be created, and then placed in the queue. The complete definition of the `setJobs` method is given below:

```
// obtains information from a user about jobs to be run in CPU
public static void setJobs() {
    Job job;
    int jobNum, cpuTime;

    System.out.print("Enter number of jobs: ");
    jobNum = scanner.nextInt();
    System.out.print("Enter size of quantum: ");
    quantum = scanner.nextInt();
    System.out.println();

    for(int i=0; i<jobNum; i++) {
        System.out.print("Enter CPU time for job " + i + ": ");
        cpuTime = scanner.nextInt();
        job = new Job(i, cpuTime);
        jobQueue.enqueue(job);
    }
    System.out.println();
}
```



In the second method, `runScheduler`, the program will continue to execute until the queue becomes empty indicating that all of the jobs are finished. Inside the while loop, after a job in the queue is dequeued to be in the CPU, the program checks if the job will finish before the quantum expires. If it will not, the value of `cpuTime` of the job is updated by subtracting the time quantum from the current value, and then the job reenters the queue. Otherwise, it outputs the message saying the job is completed. Notice that the variable `accumulatedCpuTime` keeps track of the time the CPU spent executing jobs so far. It is updated at the beginning of both the then and the else sections with the value of either quantum or `cpuTime`, respectively. The complete implementation of `runScheduler` is shown below:

```
// simulates the round-robin scheduler
public static void runScheduler() {
    Job job;
    int cpuTime, accumulatedCpuTime;
    accumulatedCpuTime = 0;

    while(!jobQueue.empty()) {
        // front of the queue goes to the CPU
        job = jobQueue.dequeue();
        cpuTime = job.getCpuTime();

        // quantum expires before job finishes
        if(cpuTime - quantum > 0) {
            accumulatedCpuTime = accumulatedCpuTime + quantum;
            // update cpuTime of job and put it back in the queue
            job.setCpuTime(cpuTime - quantum);
            jobQueue.enqueue(job);
        }
        // job finishes before time quantum
        else {
            accumulatedCpuTime = accumulatedCpuTime + cpuTime;
            // job is done, output a message
            System.out.println("Job " + job.getJobNum()
                + " finished at " + accumulatedCpuTime);
        }
    }
}
```

When the above program is compiled and executed using the sample input, the output of the program looks as given below:

```
Enter number of jobs: 3
Enter size of quantum: 4

Enter CPU time for job 0: 10
Enter CPU time for job 1: 3
Enter CPU time for job 2: 7

Job 1 finished at 7
Job 2 finished at 18
Job 0 finished at 20
```

---

### 3.6 Summary

- A queue is a First-In, First-Out (FIFO) data structure.
- Putting an item in a queue is known as an enqueue operation and removing an item from a queue is known as a dequeue operation.
- An enqueue operation places an item at the rear of a queue and a dequeue operation removes an item from the front of a queue.
- It is much faster to move a front or rear index which is  $O(1)$  than to have to move the items in a queue which is  $O(n)$ .
- The mod function % is helpful in implementing the wraparound feature of a queue.
- A count of the number of items in a queue is an easy way of checking for a full or empty queue.
- Generic types make queues reusable with different types of objects.

---

### 3.7 Exercises (Items Marked with an \* Have Solutions in Appendix B)

- \*1. Assuming the queue `intQueue` of type `QueuekArray` in Fig. 3.1 is initially empty, draw a queue after the following operations are completed:

```
int x, y, z;
intQueue.enqueue(41);
intQueue.enqueue(6);
x = intQueue.dequeue();
y = intQueue.dequeue();
intQueue.enqueue(78);
intQueue.enqueue(5);
intQueue.enqueue(79);
```

```
z = intQueue.dequeue();
intQueue.enqueue(5);
```

2. Assuming the queue `characterQueue` of type `QueueArrayGeneric < Character>` which is an object of the generic class `QueueArrayGeneric <T>` in Fig. 3.2 is initially empty, draw a queue after the following operations are completed:

```
Character x, y, z;
characterQueue.enqueue('U');
characterQueue.enqueue('V');
x = characterQueue.dequeue();
characterQueue.enqueue('W');
y = characterQueue.dequeue();
characterQueue.enqueue('P');
characterQueue.enqueue('Q');
z = characterQueue.dequeue();
characterQueue.enqueue('C');
```

3. Assuming the queue `intQueue` of type `QueueArray` in Fig. 3.1 is declared and several items are already enqueued, and also suppose the value of `front` is 3 and the value of `rear` is 0, answer the following questions.
  - \*A. What are the values of `count`, `front` and `rear` after adding an element to `intQueue`?
  - B. What are the values of `count`, `front` and `rear` after removing an element from `intQueue`?
- \*4. Write a method `peek` that can be added to the `QueueArray` class in Fig. 3.1 which returns the item at the beginning of the queue without removing it.
5. In Sect. 2.7 the concept of a palindrome was introduced. A palindrome is a sequence of symbols such as a word, phrase, verse, or sentence, that reads the same way from either direction, forward or backward. Write a program that reads a string, places each character of the string in both a queue and a stack, and then verifies whether the string is a palindrome.
6. Write a method that takes a stack as a parameter and reverse the elements of the stack using a queue whose elements are of the same type.
7. Write a method that takes a queue as a parameter and reverse the elements of the queue using a stack whose elements are of the same type.
8. Some automobiles have the capability to determine the average miles per gallon (MPG) over the last fifty miles driven. In order to simulate this, assume that the MPG can be input once every mile and an average MPG is calculated over the last five miles instead of the last fifty. If fewer than five miles have been input, the average should be over the number of MPGs input. A five-element

array-based queue should be used for this problem, and further, all of the MPGs should not be re-summed each time a new MPG is input. Below is some sample input (current MPG) and output (average MPG) with a negative number that terminates the program.

```
Enter Current MPG: 25
The Average MPG is: 25.0
Enter Current MPG: 30
The Average MPG is: 27.5
Enter Current MPG: 40
The Average MPG is: 31.7
Enter Current MPG: 30
The Average MPG is: 31.3
Enter Current MPG: 40
The Average MPG is: 33.0
Enter Current MPG: 30
The Average MPG is: 34.0
Enter Current MPG: 40
The Average MPG is: 36.0
Enter Current MPG: 40
The Average MPG is: 36.0
Enter Current MPG: 35
The Average MPG is: 37.0
Enter Current MPG: -1
```

9. CPU scheduling algorithms were discussed in Sect. 3.5. Using the complete program as a basis, modify it to find an average waiting time of all the jobs entered in the queue at the 0 second mark, where a waiting time is the total time the job spends in the queue before it finishes executing. Find average waiting times using a FIFO queue and a round-robin scheduler with the same set of jobs.

---

## 4.1 Introduction

Of the data structures looked at so far, the list is the least structured. Unlike stacks where items can only be pushed or popped from the top and queues which can only have items enqueued at the rear or dequeued from the front, a *list* can have items inserted or deleted from the front, rear, or anywhere in between. As such, it might sound as if the list might be easier to work with, but all the possibilities can make the list a little more complicated to implement.

There are two types of lists, unordered and ordered. Unordered lists can have the items stored randomly within them. Although this makes it easy to insert an item into a list, it can make it difficult to find an item. For example, imagine a desk in which all the papers on it are not organized in any particular fashion. If a particular paper needed to be found, then the search would have to start at one end of the papers on the desk and keep on searching until the paper being searched for was found. In the best case, it would be the first paper examined and in the worst case, it would be the last one examined. On average, if there were  $n$  papers on the desk then it would take  $n/2$  times to find the paper being searched for, or more formally the search process would be of order  $n$  or  $O(n)$ .

Clearly it would be nice to shorten this search time, so it would be beneficial to have the papers in some sort of order to reduce the search time. For example, the papers on the desk could be placed in alphabetical order and then a binary search could be used which could cut the time needed significantly. As a reminder, an ordered stack of papers could be cut into two halves, then the paper to be found would be in one of the two halves and the other half would not need to be searched. This process could then be repeated on the half the paper should be in, where that half could be cut in half, and so on. The process of dividing the stack in half repetitively until the paper is found is logarithmic. If there were 64 papers, and each time the pile of papers was cut in half there would be 32, 16, 8, 4, and 2 papers until there was finally 1 paper which means that there would only be six comparisons. As a reminder,  $2^6$  is 64 and conversely the  $\log_2 64$  is 6. So although with a sequential

search the search time is of  $O(n)$ , with a binary search the time is of  $O(\log n)$ . Clearly it is beneficial to use an ordered list over an unordered list should there need to be a lot of searching.

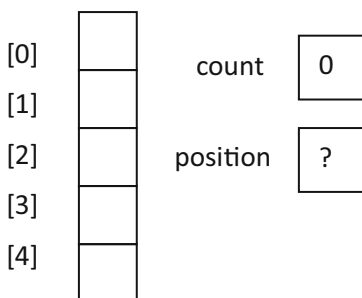
One way of creating a list is to input all the items into the list in a random order and then organize them into the proper order using a sort algorithm. The bubble or selection sort that might have been learned about in a previous course or text could be used, or the insertion or quick sort as discussed in Chap. 9 of this text could be used. However, often all the items for a list are not immediately available and are entered sporadically. Although a new item could be inserted randomly and one of the above sorting algorithms could be used to put the new item in the correct location, the use of a sort algorithm to put one or a few items in the proper location could be considered excessive.

Instead, it is helpful to define the list to be able to put individual items into the proper location as they are entered and not rely on a sort when a simpler algorithm could be used in the first place. This method of inserting an item correctly initially is comparable to students turning in a homework assignment in a professor's office. Instead of students just laying the papers on the desk and the professor going back later and sorting the papers, the professor could insert the homework assignment in the correct position alphabetically in the pile of assignments as they are turned in. That way in case the pile of papers needs to be searched before all the assignments have been turned in, the pile is in the correct order. This method of inserting items in the correct location will be discussed below and it is a variation of the insertion sort which will be discussed in Chap. 9.

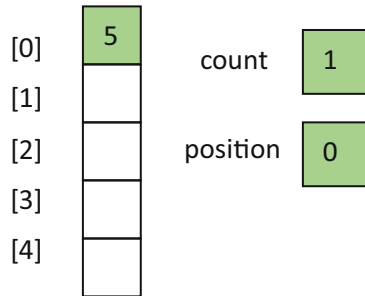
---

## 4.2 Analysis and Design

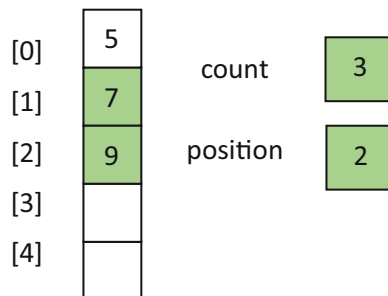
Again using a smaller array so that it is easier to see how a list works, yet large enough so that patterns can be seen, assume the existence of a five-element array, a position that indicates where an item should be inserted or removed, and a count of the number of items in the list. Initially the list would be empty as indicated by the count being 0.



When the first number, for example the integer 5, is inserted into the list, it is placed at position 0 since there are no numbers in the list and the count is incremented to 1 where the changes are shown in green below:

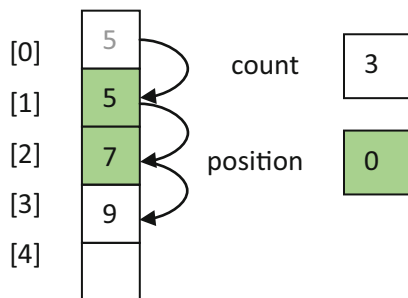


When the next number is inserted into the list, say 7, then the list needs to be searched for the correct position for the number to be inserted, the number is inserted into location 1, and the count is incremented. Likewise with the number 9, where the list is again searched, the number is inserted into the list at position 2, and count is incremented as reflected below:

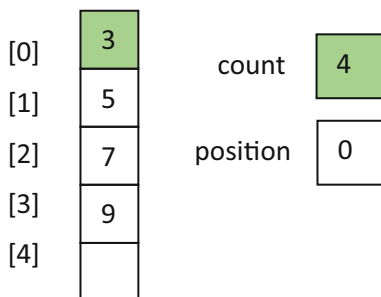


Note that the list can be searched using either a sequential search  $O(n)$  or a binary search  $O(\log n)$ . When the list is small a binary search would not be very helpful, but if the list were very large, then the savings in time would be significant. Since the list here is small, it will be assumed that the sequential search will be used for the time being and a binary search is left as an exercise at the end of the chapter.

So far, the numbers that have been inserted were in order, but what would happen if a number such as 3 were to be inserted? As might be suspected, all the numbers in the list would need to be moved down in the list to make room for the number 3 in position 0.

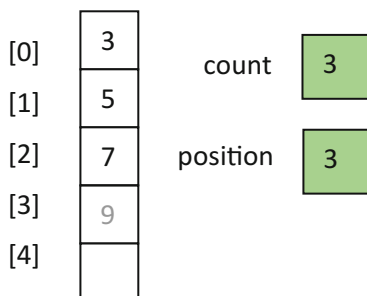


As before, note above that the number 5 is light gray to indicate that it was merely copied from position 0 to 1. When the number 3 is inserted, the number 5 is overlaid and count is incremented by 1 as shown below:



The result is that the moving of  $n$  items in a list would be  $O(n)$  which is something that has been seen in the last two chapters on stacks and queues. Unfortunately, this is a disadvantage of inserting an item into an ordered list that has been implemented using a simple array, and will be discussed more in detail later.

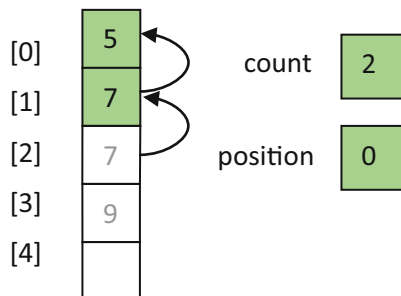
So far numbers have only been inserted into a list. What if the number 9 were to be deleted from the list? Again, the list would need to be searched, the number removed, and the count decremented by one.





As with stacks and queues previously, note that the number 9 above is now gray indicating that it is no longer part of the list but still present in the array. Again, this illustrates that the list is distinct from the array in which it is implemented.

Removing a number from the end of the list does not seem to be a problem, but what if a number is to be removed from the middle or beginning of the list? For example, what if the number 3 were to be removed? Again, the problem of having to move all the numbers in the list occurs but in reverse from when the number was inserted and the count is decremented.



Again note above that the number 7 in grey is still in the array, since it was only copied from position 2 to 1. However, as before the count has been decremented to indicate that the 7 in position 2 is no longer part of the list. Of course, when another item is inserted, the number will be overwritten.

The problem of moving the numbers occurs when inserting or deleting items from the middle or beginning of the list and is difficult to avoid when implementing an ordered list in a simple array. One solution when deleting an item is to just put in a number that is not part of the potential data set, so that if the list is searched or output it is just ignored. However, this could cause problems in trying to find a unique number to use. An alternative is to include another dimension in the array or a field in an object in the array that indicates when an item has been deleted. However, this can waste space to include an extra field and might pose a problem when a large number of items have been deleted where the deleted items might actually take up more space in the array than there are actual items in the list.

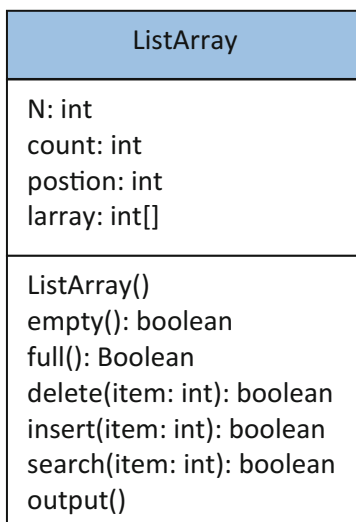
As will be seen in the next chapter, there is a solution to the problem of having to move items around in an array which will speed up the insertion and deletion of items. The solution is that of a linked list which can be implemented using references and objects. However, linked lists are not without their own problems concerning searching and memory utilization as will be seen later. For now, the inefficiency of inserting and deleting will just be an inconvenience, since there is much that can be learned from implementing a list using an array.

### 4.3 ListArray Class: Data Members and Methods

Similar to the data members in previous two chapters on stacks and queues, the data members needed for the `ListArray` class can be seen in the previous section. An array will be needed which can be called `larray` and it can be of size `N`. Another variable needed is `position` which indicates where an item will be inserted or deleted. The last variable needed is `count` which indicates how many items are in the list and as before can be helpful when determining whether the list is full or empty.

Of course there will need to be public methods for `insert` and `delete`, where `insert` will have a parameter for the number to be inserted called `item` and will return a `boolean` value to indicate whether the number in `item` was inserted. The `delete` method will also have a parameter called `item` containing the number to be deleted, and there will also be a `boolean` value returning method to indicate whether the number was deleted. As before, there will need to the `boolean` value returning `full` and `empty` methods which could be `public` or `private` as necessary. Alternatively, as with stacks and queues, a method can be included for increasing the size of the array should the list become full. Further, as mentioned in the previous section, there also needs to be a `search` method. It should have a parameter `item` containing the number to be searched for, would return a `boolean` to indicate whether the `item` was found, `update position` where the number was found, and it could be `public` or `private` as needed. Lastly, a method to output the contents of the list called `output` is also helpful.

The following UML diagram summarizes what is necessary:



### 4.3.1 The Insert Method

It is probably easiest to start with the `insert` method because it will be used to create a list. As indicated in the UML diagram, it accepts an `item` as a parameter to be inserted and returns a `boolean` value to indicate whether the `item` was inserted or not. In the basic skeleton for the method, the `boolean` variable `inserted` is declared locally and initialized for the default case of `false`.

```
public boolean insert(int item) {  
    boolean inserted=false;  
    //code for insert  
    return inserted;  
}
```

To fill in the skeleton, it can be assumed that if the list is full, then a number would not be able to be inserted and the default value of `false` is returned. Since it would seem that the list would usually not be full, the message for a full list is included in the `else` section of the `if` statement as shown below:

```
if(!full())  
    //code for inserting item in list  
else  
    System.out.println("List is Full");
```

As before, the `println` could be replaced with an exception, but since the code for the test program in the next section will need to continue processing when the list is full, the `println` statement is used. Further, since the `full` and `empty` methods from the `QueueArray` class used in Sect. 3.3 utilize `count`, they can be copied into the current `ListArray` class as will be seen later in Fig. 4.1.

Before writing the code to insert an item in the list, it should be determined whether duplicate values will be allowed. For example, in the case where there is more than one person with the same last name in a classroom, duplicates could be allowed. However, in the current case duplicate numbers will not be allowed and code for duplicates is left as an exercise at the end of the chapter.

The result is that the list should be searched to see if the item is already in the list before it is inserted. As shown previously in the UML diagram, the `search` method is sent an `item` and returns a `boolean` indicating whether the item has been found or not. If the number in `item` is not found, it should be inserted; otherwise it should not be inserted and again the default value is returned.

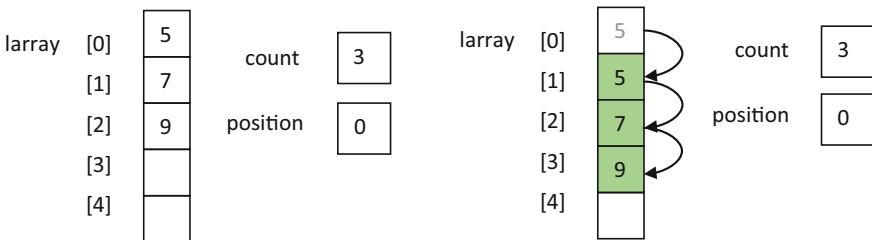
```

if(!full())
    if(!search(item)) {
        // code for insert
    }
else
    System.out.println("List is Empty");

```

If the number in `item` is not found and it is to be inserted in the list, the location or position in the list needs to be known. Since the `search` method can only return one piece of information, in this case whether the `item` was found or not, the location in the array where the `item` should be inserted can be stored in the data member `position` which will be accessible by both the `search` and `insert` methods. There are other ways to accomplish this, but for now this technique allows the code to be less complicated.

However, unless an item is being inserted into the last location in the list, it will be necessary to move all the items following the `position` further down in the list. For example, given the following scenario from the previous section shown on the left below, assume that the number 1 is to be inserted into `larray[0]`, then all the numbers would need to be copied down further in the array. The number in `larray[2]` needs to be copied to `larray[3]`, then the number in `larray[1]` can be copied to `larray[2]` and then the number in `larray[0]` can be copied to `larray[1]` as shown on the right below:



In trying to see the pattern it sometimes helps to list the indexes in a table as follows:

```

larray[2] → larray[3]
larray[1] → larray[2]
larray[0] → larray[1]

```

Notice that the locations of the data being copied are from `larray[2]` through `larray[0]` to `larray[3]` through `larray[1]`. Although a loop could be written with an index, say `j`, to go from 3 to 1, that would not be very useful under differing circumstances since there might not always be exactly three items to copy.

Instead, note that `count` also contains a 3 and that would be a good starting point. Although in this instance the value decreases until it hits 1, this constant would not be very useful depending on where in the list a number were to be inserted. Instead note that the `position` to be inserted is at 0, so the value of `j` could be decremented as long as it is greater than the `position` to be inserted. Lastly, for each iteration of the loop the numbers in the list need to be moved from one less than the `j`th position into the `j`th position. In other words, from `larray[j-1]` to `larray[j]` as shown in the following segment:

```
for(int j = count; j > position; j--)
    larray[j] = larray[j-1];
```

In the case of inserting an item at the beginning or end of the list, and after the location in `larray` is available, then `item` should be copied into that `position`, the variable `count` incremented by 1, and the `inserted` flag should be set to `true` as follows:

```
larray[position] = item;
count++;
inserted = true;
```

What a number was to be inserted at the end of the list? In this case, none of the numbers would need to be moved. Does this mean that a separate `if` statement is needed prior to the `for` loop to accommodate this special case? For example, should an `if` statement such as the following be inserted prior to the `for` statement?

```
if(position == count)
```

Although it might seem that this is necessary, recall from a previous class or text, what type of iteration structure is the `for` loop? Yes, it is a fixed iteration loop, but it is also a pretest loop which means its test is at the beginning of the loop and the body of the loop is executed zero to many times. If the `for` loop is written correctly, the body of the loop should never execute in this case. For example in the previous drawing, if the number `item` 11 is to be inserted, then `search` should return the value 3 for `position` where the number 11 should be inserted. The initial value of `j` in the `for` loop would then take on the value 3 from `count` and when `j` is compared to `position`, the value 3 in `j` will not be greater than the value 3 in `position`, so the loop will not iterate, and thus the `if` statement is not needed. Then as before, the value in `item` will be put into `larray[3]` indicated by `position`, the variable `count` will be incremented by 1, and the `inserted` flag would be set to `true`.

Putting all of the above code together results in the following method:

```
public boolean insert(int item) {
    boolean inserted=false;
    if(!full())
        if(!search(item)) {
            for(int j=count; j>position; j--)
                larray[j] = larray[j-1];
            larray[position] = item;
            count++;
            inserted = true;
        }
    else
        System.out.println("List is Full");
    return inserted;
}
```

As an aside, notice in the above code segment that there is a set of braces in the then section of the outer `if` statement. The question is, should the braces be included? At first glance, one might answer no, because there is only one statement, the nested `if` statement, and the outer braces would not appear to be necessary. If the outer `if` was only an if-then structure and not an if-then-else structure, this would be correct. However, this is an example of a nested if-then-if structure. Thinking back to one's first Java course or text, what is one of the problems with a nested if-then-if structure? Recall that they are susceptible to the "dangling else" problem, where an `else` always matches up with the closest `if` and without the enclosing braces, the `else` above would match up with the inner `if` instead of the outer `if` which is not what is intended and results in a logic error. The alternative is to rewrite the if-then-if as an if-then-else-if structure, but as mentioned previously the list will probably tend not to be full, so the error message is placed further down in the method in the `else` section instead.

### 4.3.2 The Delete Method

Having created the `insert` method, it is convenient to examine the `delete` method before designing the `search` method, since the `search` method will be used by both methods. As with `insert`, examining the UML diagram provides the skeleton for the `delete` method.

```
public boolean delete(int item) {  
    boolean deleted = false;  
    //code for delete  
    return deleted;  
}
```

Again, the number to be deleted is sent in the parameter `item` and the method returns a boolean value indicating whether the number was deleted or not, where the boolean variable `deleted` is set to `false` as the default.

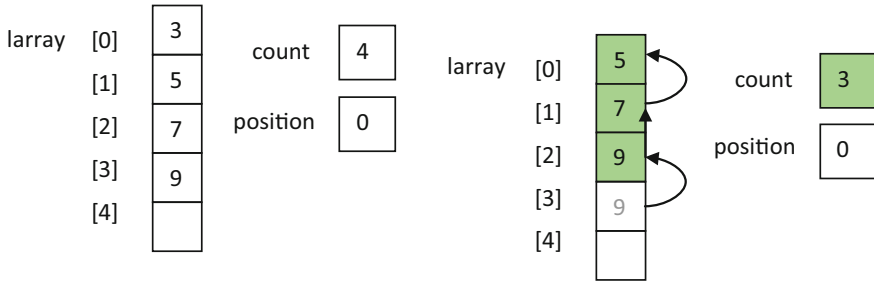
Opposite to `insert`, which is concerned with whether the list is full, the `delete` method checks to ensure that the list is not empty before attempting to delete a number. If the list is empty, then a message is output (or alternatively an exception can be thrown) and the default value of `false` will be returned.

```
if(!empty())  
    // code for delete  
else  
    System.out.println("List is Empty");
```

If the list is not empty, then the list will need to be searched for the number to be deleted and if the number is found then it can be deleted.

```
if(!empty())  
    if(search(item)) {  
        // code for delete  
    }  
else  
    System.out.println("List is Empty");
```

The act of deleting is merely moving the other numbers in the list to take up the spot once occupied by the number in `item` as shown in these figures modified from Sect. 3.2.



Again, following the type of analysis used in Sect. 4.3.1, note that if the number 3 were to be deleted, then `search` would return the `position` as 0. Then all of the other numbers in the list would need to be copied down in the list to take up the space vacated by the number 3. The number in `larray[1]` will be copied to `larray[0]`, the number in `larray[2]` will be copied to `larray[1]`, and the number in `larray[3]` will move to `larray[2]` as shown below:

```
larray[1]→larray[0]
larray[2]→larray[1]
larray[3]→larray[2]
```

A variable, again say `j`, could start at `larray[0]`, but what if the number to be deleted was at a location other than 0? As before, note that the `search` method would return the location of the number in the data member `position`. In this case it would be 0, and is a good place to start the loop control variable `j`. Then the number at location `j + 1` would need to be moved into location `j` for each iteration of the loop. Since `count` is originally 4 as shown above, the loop would iterate while `j` is less than or equal to `count-2`. Alternatively, the `for` could be rewritten as `j<count-1` as follows:

```
for(int j=position; j<count-1; j++)
    larray[j] = larray[j+1];
```

Once all the numbers have been copied, then `count` would need to be decremented by 1 and the `deleted` flag would be set to `true`:

```
count--;
deleted = true;
```

Putting all the above pieces together for the complete `delete` method, the result would be:



```
public boolean delete(int item) {
    boolean deleted=false;
    if(!empty())
        if(search(item)) {
            for(int j=position; j<count-1; j++)
                larray[j] = larray[j+1];
            count--;
            deleted = true;
        }
    else
        System.out.println("List is Empty");
    return deleted;
}
```

### 4.3.3 The Search Method

Having created both the `insert` and `delete` methods, it is now time to create the `search` method. Starting again with the UML diagram, the skeleton of the method can be written

```
private boolean search(int item) {
    boolean found = false;
    // code for search
    return found;
}
```

As with `insert` and `delete`, the `search` method is sent a number in `item` to be searched for, and will return a `boolean` value whether it was found or not with the initial value as `false`. As mentioned earlier, to keep it simple, a sequential search will be used. The `position` in which the number might be found can be initialized to 0 and while `position` is less than the `count` of the number of items in the list, the loop will continue to iterate by incrementing the `position` counter. Further, the `item` being searched for can be compared to each location in the list. If the number in `item` is found, the `found` flag can be set to `true` as shown below:

```
position = 0;
while(position < count) {
    if(larray[position] == item)
        found = true;
    position++;
}
```

The above code would work for either an unordered or an ordered list. However, once a number is found, is there a reason to continue looking at the other numbers in the list? No, there isn't, and although stopping the loop in the average case will not make it an order of magnitude faster since  $n/2$  is still  $O(n)$ , it would help speed the algorithm up some and it is worth adding the code to stop the loop once the item is found as shown below:

```
position = 0;
while(position < count && !found) {
    if(larray[position] == item)
        found = true;
    position++;
}
```

So in order to delete a found item the above code would work. However, what if the item was not in the list? In an unordered list the loop would need to continue and check all of the items in the list just as with an unordered pile of papers on a desk. In an ordered list once the loop had gone past the spot where the item should have been, there is no reason to continue searching for the item. Again, if the pile of papers on the desk are in alphabetical order, and the name being searched for began with the letter R, then once one reached a last name beginning with the letter S, there would be no reason to continue looking for the paper.

Unfortunately, the above code only stops when the number in `item` is found and not when one has gone past where the number should be located in an ordered list. So instead of stopping the loop only when the number is found, it should also stop when the number has been found or when it has gone past the location where the number should have been. Since the `found` flag is used to indicate only when a number is found, it should not be used to control the `while` loop. Instead another `boolean` variable could be used to indicate the above two conditions when the loop should stop. As a result, the `boolean` variable `stop` is added to the local variables in the method and is also initialized to the default value of `false` as well. The resulting code is as follows:

```
position = 0;
while(position < count && !stop) {
    if(larray[position] >= item)
        stop = true;
    position++;
}
```

Note the use of the variable `stop` instead of the variable `found` and that the `if` statement has changed from `==` to `>=`. This will now stop the loop when either the location of the number is found or when it goes past where the number should have been found. Although this helps for the case when it is not found, the code segment needs to be further modified to distinguish between these two possibilities. This can be accomplished by adding a second `if` statement inside the existing `if` statement. Once it is known that either the number has been found or has gone past where it should have been found, the second `if` statement checks if it is equal and sets the `found` flag to `true`.

```
position = 0;
while(position < count && !stop)
    if(larray[position] >= item) {
        stop = true;
        if(larray[position] == item)
            found = true;
        position++;
    }
```

In other words if the number is not in the list, `stop` will be set because the number is greater than or equal to one in the list. Further, since it is not equal, the `found` flag will not be set. On the other hand if it is equal, `stop` will still be set to terminate the loop and `found` will also be set to indicate it was in the list. However, notice that `position` is incremented in both cases. In the case where it is found, `position` will be pointing to the element after where it was found and if it is not in the list, `position` will be one beyond where it should be. How can this be solved?

Unfortunately, `position` is being incremented in both cases when it shouldn't be incremented. It should not be incremented when the loop has ceased iterating when `stop` has been set. The solution is that an `else` statement can be paired with the outer `if` statement so that it only increments when the loop continues to search for the correct location as shown below:

```
position = 0;
while(position < count && !stop)
    if(larray[position] >= item) {
        stop = true;
        if(larray[position] == item)
```

```
        found = true;
    }
    else
        position++;
```

As in previous methods in this section, note the careful use of the braces to ensure that the `else` is paired with the outer `if` and not accidentally creating a dangling `else` problem by being paired with the inner `if` statement. Putting all the code together and including the declaration and default initialization of the boolean variable `stop`, the complete search method is as follows:

```
private boolean search(int item) {
    boolean found=false, stop=false;

    position = 0;
    while(position < count && !stop)
        if(lArray[position] >= item) {
            stop = true;
            if(lArray[position] == item)
                found = true;
        }
        else
            position++;
    return found;
}
```

#### 4.3.4 Complete ListArray Class

In addition to the constructor, and the `empty` and `full` methods, the only remaining method is `output` which outputs the list and is fairly simple to implement. All four of these are listed without explanation in the complete `ListArray` class in Fig.4.1.

---

## 4.4 Simple Test Program

As in the previous chapter a simple test program should be created. Similar to the queue program in the previous chapter, this program will continue prompting from a sentinel controlled loop until a command of S for Stop is entered by the user. If the user enters an I for Insert, the user is prompted for a number to be inserted, and upon return from the `insert` method a message is output indicating whether or not the number was inserted. Similarly, should the user enter a D for Delete, the

```
public class ListArray {

    static private final int N=5;
    private int position, count, larray[];

    public ListArray() {
        count=0;
        larray = new int[N];
    }

    private boolean empty() {
        return count<=0;
    }

    private boolean full() {
        return count>=N;
    }

    public boolean delete(int item) {
        boolean deleted=false;
        if(!empty())
            if(search(item)) {
                for(int j=position; j<count-1; j++)
                    larray[j] = larray[j+1];
                count--;
                deleted = true;
            }

        else
            System.out.println("List is Empty");
        return deleted;
    }

    public boolean insert(int item) {
        boolean inserted=false;
        if(!full())
            if(!search(item)) {
                for(int j=count; j>position; j--)
                    larray[j] = larray[j-1];
                larray[position] = item;
                count++;
                inserted = true;
            }

        else
            System.out.println("List is Full");
        return inserted;
    }

    private boolean search(int item) {
        boolean found=false, stop=false;
        position = 0;
        while(position < count && !stop)
            if(larray[position] >= item) {
                stop = true;
                if(larray[position] == item)
                    found = true;
            }
    }
}
```

**Fig. 4.1** ListArray Class

```

        }
    else
        position++;
    return found;
}

public void output() {
    System.out.print("List: ");
    int j = 0;
    while(j != count) {
        System.out.print(larray[j]+" ");
        j++;
    }
    System.out.println();
}
}

```

**Fig. 4.1** (continued)

user is then prompted for the number to be deleted, and upon return from the `delete` method a message is output indicating whether the number was deleted or not. A command of O for Output invokes the `output` method, where the contents of the list are output to the screen. Lastly, should the user enter some other letter, a message is output to indicate to try again. As before, both upper and lower case letters are allowed for the commands.

```

import java.util.*;
class Ch3Test {
    public static void main(String[] args) {

        Scanner scanner;
        scanner = new Scanner(System.in);

        char response;
        int number;

        ListArray list;
        list=new ListArray();

        System.out.println();
        System.out.print("Do you wish to I = Insert, D = Delete, " +
            "O = Output, or S = Stop: ");
        response = scanner.next().charAt(0);
        while(response!='S' && response!='s') {
            if (response=='I' || response=='i') {
                System.out.print("Enter a number to insert into " +
                    "the list: ");
                number = scanner.nextInt();
                if(list.insert(number))

```

```

        System.out.println("The number was inserted");
    else
        System.out.println("The number was not inserted");
    }
    else
        if(response=='D' || response=='d') {
            System.out.print("Enter a number to delete from " +
                "the list: ");
            number = scanner.nextInt();
            if(list.delete(number))
                System.out.println("The number was deleted");
            else
                System.out.println("The number was not deleted");
        }
        else
            if(response=='O' || response=='o')
                list.output();
            else
                System.out.println("The command entered is not "
                    + "I, D, O, or S, please try again.");
        System.out.println();
        System.out.print("Do you wish to I = Insert, D = Delete, "
            + "O = Output, or S = Stop: ");
        response = scanner.next().charAt(0);
    }
    System.out.println();
    System.out.println("End of Program");
    System.out.println();
}
}

```

Sample output is shown below indicating all the prompts and many of the messages generated by the main program as well as some of the messages indicating whether the list is full or empty from the `ListArray` class:

```

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: I
Enter a number to insert into the list: 1
The number was inserted

```

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: o  
List: 1

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: i  
Enter a number to insert into the list: 3  
The number was inserted

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: i  
Enter a number to insert into the list: 5  
The number was inserted

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: i  
Enter a number to insert into the list: 7  
The number was inserted

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: i  
Enter a number to insert into the list: 9  
The number was inserted

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: o  
List: 1 3 5 7 9

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: i  
Enter a number to insert into the list: 11  
List is Full  
The number was not inserted

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: d  
Enter a number to delete from the list: 2  
The number was not deleted

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: d  
Enter a number to delete from the list: 9  
The number was deleted

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: o  
List: 1 3 5 7

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: d  
Enter a number to delete from the list: 1  
The number was deleted

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: d  
Enter a number to delete from the list: 3  
The number was deleted



```
Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: d
Enter a number to delete from the list: 5
The number was deleted

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: d
Enter a number to delete from the list: 7
The number was deleted

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: o
List:

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: i
Enter a number to insert into the list: 11
The number was inserted

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: o
List: 11

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: d
Enter a number to delete from the list: 11
The number was deleted

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: d
Enter a number to delete from the list: 11
List is Empty
The number was not deleted

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: q
The command entered is not I, D, O, or S, please try again.

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: s

End of Program
```

---

## 4.5 Complete Program: Checking Opcodes in Assembly Language

In this section a program that simulates an assembler is developed to determine whether all the opcodes in an assembly language program are valid. Assembly language is a low-level language that consists of mnemonics instead of ones and zeros used in machine language programs. In order for a program to be executed on a machine, programs written in a high-level language can be converted to an

**Fig. 4.2** Assembly code

| Symbol    | Opcode | Operand   |        |
|-----------|--------|-----------|--------|
| -----     |        |           |        |
| SAMPLE    | START  | FIRST     | Line 1 |
| THOMAS    | WORD   | 3         | Line 2 |
| SYLVESTER | WORD   | 7         | Line 3 |
| CATS      | WORD   | ?         | Line 4 |
| FIRST     | LOAD   | THOMAS    | Line 5 |
|           | ADD    | SYLVESTER | Line 6 |
|           | STORE  | CATS      | Line 7 |
|           | STOP   |           | Line 8 |
|           | END    |           | Line 9 |

assembly language program by a compiler and then converted to a machine language program by an assembler. A program written in an assembly language that implements the following Java code might look like the one in Fig. 4.2.

```
int thomas = 3, sylvester = 7, cats;
cats = thomas + sylvester;
```

Essentially, the program adds two numbers 3 and 7 that are in the variables THOMAS and SYLVESTER respectively, and stores the result 10 in the variable CATS. The column labels, the line below the column labels, and line numbers are not part of the program but included to show how the words line up. In order from left to right, the three fields of an instruction are the symbol, the operation code (opcode), and the operand. The first field is typically reserved for the names of variables or labels for instructions. The second field is typically used for opcodes that represent executable instructions and also assembler directives which tell the assembler what to do. The third field is used for operands.

The first line indicates the start of the program. The symbol SAMPLE is the name of the program, the opcode START tells the computer when the program starts, and operand FIRST indicates the location of the first instruction. The second through fourth lines are directives that tell the assembler to reserve the memory locations. Line 2 reserves a space for the symbol THOMAS which is initialized to 3. Similar to the second line, the third line initializes the variable SYLVESTER to 7. The fourth line reserves an uninitialized memory location called CATS. Line 5 fetches the contents from the memory location THOMAS and stores it into a register. A register is a memory location in the CPU where among other things arithmetic can be performed. Then the ADD operation in Line 6 gets the value from the variable SYLVESTER and adds it to the contents of the register. The next line stores the value in the register into the memory location CATS. The STOP instruction halts the execution of the program. In the last line, the END directive indicates to the assembler the end of the program.

When the assembly code is translated into machine code, an assembler first checks if the program consists of valid opcodes. If there is a syntax error, the assembler will output an error message. This is the part of the assembler that will be developed. First, the list of valid opcodes called OpTable is created. A list of the

valid opcodes shown below are stored in a text file, `OpCode.txt`. They are listed in unordered manner in the file and will be inserted in alphabetical order in the list.

```
STORE
ADD
LOAD
WORD
DIV
STOP
SUB
END
START
MUL
```

With this application the items stored in the list are `String` objects. Therefore, the `ListArray` class shown in Fig. 4.1 would need to be modified for the `String` class. As discussed in Chap. 3, since rewriting the `ListArray` class for the specific type of object every time a new application is developed is not very efficient, instead the `ListArrayGeneric` class that can be used for any objects will be defined by replacing the `int` in `ListArray` by the type parameter `T`. Therefore the array, `larray` is of type `T` and item that will be inserted to the list is also of type `T`. Unlike the `StackArrayGeneric` and `QueueArrayGeneric` classes, `ListArrayGeneric` needs to compare items to determine where the new item should be inserted in the array. Unfortunately, comparing two items using the following statement does not work in the generic class.

```
larray[position] >= item
```

Since `larray[position]` and `item` are both of type `T` which are references to an object, instead of comparing the contents of the objects, the above statement compares the addresses of two objects. What is the solution to this problem? The solution is the `compareTo` method that is included in the `Comparable` interface could be used to compare two objects. The `Comparable` interface is generic and is used to order objects. It is found in the `Java.lang` package and contains only one method named `compareTo`. The method is used to compare contents of the object that calls the method and the contents of the specified object in the parameter. The `compareTo` method takes an object of any type and returns an integer. Specifically, it returns a positive integer, zero, or a negative integer if the object that is calls the method is greater than, equal to, or less than the specified object respectively. Now the above statement can be rewritten using the `compareTo` method as shown below, which indicates whether contents of the object that `larray[position]` references is greater than or equal to contents of the object that `item` references.

```
larray[position].compareTo(item) >= 0
```

First, the type parameter in the `ListArrayGeneric` class must support comparison with other instances of its own type. To accomplish this the class declaration will be:

```
public class ListArrayGeneric<T extends Comparable<T>>
```

indicating the type `T` inherits a `compareTo` method from the `Comparable` interface and the `compareTo` method will be implemented in the `ListArrayGeneric` class.

Instead of an array of the `Object` typecast into the generic type in the `StackArrayGeneric<T>` class as in Fig. 2.2 and the `QueueArrayGeneric<T>` class as in Fig. 3.2, an array of the `Comparable` type is created and then cast into the array of the generic type in the constructor of the `ListArrayGeneric` class as shown below. This is done because every `T` is a `Comparable` type as in the class declaration of the `ListArrayGeneric` class.

```
larray = (T[]) new Comparable[N];
```

The implementation of the `compareTo` method, which is the only method the `Comparable` interface contains, is added at the end of the class. This method will be used in the search method.

```
public int compareTo(T item) {
    int result;
    if (reference.compareTo(item) > 0)
        result = 1;
    else
        if (reference.compareTo(item) < 0)
            result = -1;
        else
            result = 0;
    return result;
}
```

Figure 4.3 shows the complete `ListArrayGeneric` class definition.

In the main program, two methods, `createOpTable` and `checkOpCodes` are called. The `createOpTable` method creates a list of opcodes. An unordered list of opcodes is read from a file and placed in alphabetical order in the list. The `checkOpCodes` method simulates an assembler. The main program without detailed implementation of the two methods is shown below and the two methods are defined shortly.

```

public class ListArrayGeneric<T extends Comparable<T>> {

    private final int N=20;
    private int position, count;
    private T[] lArray;
    T reference;

    public ListArrayGeneric() {
        count=0;
        larray = (T[]) new Comparable[N];
    }

    private boolean empty() {
        return count <= 0;
    }

    private boolean full() {
        return count >= N;
    }

    public boolean insert(T item) {
        boolean inserted=false;

        if(!full())
            if (!search(item)){
                for(int j=count; j>position; j--)
                    larray[j] = larray[j-1];
                larray[position] = item;
                count++;
                inserted = true;
            }

        else
            System.out.println("List is Full");
        return inserted;
    }

    public boolean delete(T item) {
        boolean deleted=false;
        if(!empty())
            if(search(item)){
                for(int j=position; j<count-1; j++)
                    larray[j] = larray[j+1];
                count--;
                deleted = true;
            }

        else
            System.out.println("List is Empty");
        return deleted;
    }

    public boolean search(T item) {
        boolean found, stop;
        found = false;
        stop = false;
        position = 0;
    }
}

```

**Fig. 4.3** ListArrayGeneric<T> class

```

        while(position != count && !stop)
            if(larray[position].compareTo(item) >= 0){
                stop = true;
                if(larray[position].compareTo(item) == 0)
                    found = true;
            }
            else
                position++;
        return found;
    }

    public void output() {
        System.out.print("List: ");
        int j = 0;
        while(j != count) {
            System.out.print(larray[j]+" ");
            j++;
        }
        System.out.println();
    }

    public int compareTo(T item) {
        int result;
        if(reference.compareTo(item) > 0)
            result = 1;
        else
            if(reference.compareTo(item) < 0)
                result = -1;
            else
                result = 0;
        return result;
    }
}

```

**Fig. 4.3** (continued)

```

import java.util.*;
import java.io.*;

class OpCodes {
    private static ListArrayGeneric<String> opTable
        = new ListArrayGeneric<String>();

    public static void main(String[] args) throws IOException {

        // create op table
        createOpTable();
        // check op codes in assembly code
        checkOpCodes();
    }
}

```

```
// create op table
public static void createOpTable() throws IOException {
    // defined shortly
}

// check op codes in assembly code
public static void checkOpCodes() throws IOException {
    // defined shortly
}
}
```

In the `createOpTable` methods, first the file `opCodes.txt` is opened for reading. It reads the contents of the file line by line until it reaches the end of the file. For each line it reads, it inserts the opcode into the list called `opTable` in alphabetical order. Notice that because it deals with a file which may not exist in the system, `throws IOException` is added for the system to handle such a situation. The complete definition of `createOpTable` method is given below:

```
public static void createOpTable() throws IOException {
    String inStr;
    Scanner inFile = new Scanner(new File("opCodes.txt"));
    while(inFile.hasNextLine()) {
        inStr = inFile.nextLine();
        opTable.insert(inStr);
    }
    inFile.close();
}
```

In the second method, `checkOpCodes`, after the program prints the column labels for the output, it opens the file `assemblyCode.txt` for reading. As it reads a line of assembly code inside the `while` loop, it outputs what it read and checks if the opcode of the instruction is valid. Although the opcode lies between column 11 and 20 of the line, the line without the operand ends before column 20. Therefore, two cases are examined; a line with an operand and a line without an operand. For the line with an operand, a substring between columns 11 and 20 is extracted from the original line. Because the actual opcode may not occupy the entire 10 character long space, the substring most likely contains trailing whitespace. That has to be removed before it is compared to the opcodes in the list, `opTable`. How can trailing whitespace in the string be removed? Among many useful methods already defined in the `String` class, the `trim` method is the one that can be used here. It returns a copy of the string with the whitespace from the beginning and end of the string removed. After checking the opcode, if the line does

not contain a valid opcode an error message will be output. Note that depending on whether or not there is an operand, the error message will need to be tabbed differently.

```
// check op codes in assembly code
public static void checkOpCodes() throws IOException {
    String inStr;
    int len;

    System.out.println("Symbol   Opcode   Operand   Error");
    System.out.println("-----");
    + ("-----");

    Scanner inFile = new Scanner(new File("assemblyCode.txt"));

    while(inFile.hasNextLine()) {
        inStr = inFile.nextLine();
        System.out.print(inStr);
        len = inStr.length();

        // line with operand
        if(len >= 20) {
            inStr = inStr.substring(10, 20);
            if(!opTable.search(inStr.trim()))
                System.out.print("\t\tInvalid Opcode "
                    + inStr.trim());
        }
        // line without operand
        else {
            inStr = inStr.substring(10, len);
            if(!opTable.search(inStr.trim()))
                System.out.print("\t\t\tInvalid Opcode "
                    + inStr.trim());
        }
        System.out.println();
    }
    inFile.close();
}
```

Using the assembly code shown in Fig. 4.2, which does not have any syntax errors, a slightly altered assembly language program is created as shown in Fig. 4.4. Note that WORD in the third line and END in the last line are changed to WARD and EMD respectively. It is saved in `assemblyCode.txt` in simple text format.

When the main program is compiled and executed using the new sample assembly code in Fig. 4.4, the output of the program looks as given below:



```
SAMPLE    START    FIRST
THOMAS    WORD     3
SYLVESTER WARD     7
CATS      WORD     ?
FIRST     LOAD     THOMAS
          ADD      SYLVESTER
          STORE    CATS
          STOP
          EMD
```

**Fig. 4.4** Assembly code containing syntax errors, a variation of the code in Fig. 3.2

| Symbol    | Opcode | Operand   | Error                |
|-----------|--------|-----------|----------------------|
| -----     |        |           |                      |
| SAMPLE    | START  | FIRST     |                      |
| THOMAS    | WORD   | 3         |                      |
| SYLVESTER | WARD   | 7         | Invalid Opcode: WARD |
| CATS      | WORD   | ?         |                      |
| FIRST     | LOAD   | THOMAS    |                      |
|           | ADD    | SYLVESTER |                      |
|           | STORE  | CATS      |                      |
|           | STOP   |           |                      |
|           | EMD    |           | Invalid Opcode: EMD  |

The output is a listing of the assembly language program and syntax error messages. Using the code in `assemblyCode.txt` shown in Fig. 4.4 there are two opcodes that are misspelled, WARD and EMD. Therefore, output indicates two syntax errors.

### 4.6 Summary

- Lists can be ordered or unordered.
- Although lists are easier to use because there are few restrictions as to where an item can be inserted or deleted, they can be complicated to implement.
- A sequential search can be used on both ordered and unordered list, whereas a binary search can be used only on an ordered list.
- Using a sequential search on an ordered list, once the search has gone past the location where an item should have been located, the search can stop. Although not an order of magnitude faster, it is faster than letting the search continue through to the end of the list.
- A count can be used to indicate whether a list is full or empty as was done with queues.
- Generic types make lists reusable with different types of objects.

## 4.7 Exercises (Items Marked with an \* Have Solutions in Appendix B)

- \*1. Assuming the list `intList` of type `ListArray` in Fig. 4.1 is initially empty, draw a list after walking through the following code segment:

```
intList.insert(41);
intList.insert(6);
intList.delete(5);
intList.delete(6);
intList.insert(78);
intList.insert(5);
intList.insert(79);
intList.delete(41);
intList.insert(5);
```

2. Assuming the list `characterList` of type `ListArray Generic<Character>` which is an object of the generic class `ListArrayGeneric<T>` in Fig. 4.3 is initially empty, draw a list after walking through the following code segment:

```
characterList.insert('U');
characterList.insert('V');
characterList.delete('U');
characterList.insert('W');
characterList.delete('A');
characterList.insert('P');
characterList.insert('Q');
characterList.delete('P');
characterList.insert('C');
```

- \*3. Write a method `deleteAt` that can be added to the `ListArray` class in Fig. 4.1 which removes the item at the specific position from the list. The method accepts a position as a parameter.
4. Write a method `positionOf` that can be added to the `ListArray` class in Fig. 4.1 which returns the position of the specific item in the list. The method accepts an item as a parameter.
5. Write a method `search` that performs a binary search and that can replace the existing search method in Fig. 4.1 `ListArray` class.
6. Implement a `ListArray` class which allows duplicates in an ordered list. The method `search` should indicate the number of the occurrences of the item

being searched for, and all the duplicates should be deleted in the `delete` method.

7. Write a program to merge two lists of strings using `ListArrayGeneric<T>` in Fig. 4.3.

The resulting list should not include duplicates.

8. Write a program to find an intersection of two given sets of integers using `ListArrayGeneric<T>` in Fig. 4.3. For example, if two integer sets are  $P = \{4, 7, 1, 10\}$  and  $Q = \{4, 2, 5, 1, 3\}$ , the intersection of these two sets is  $P \cap Q = \{1, 4\}$ .

---

## 5.1 Introduction

In the previous chapter it was seen that lists can be created using arrays. However, there are some apparent disadvantages when inserting or deleting an item when the contents of the list must be moved to make way for the new item or filling in an empty space respectively. There is also a problem when an array becomes full where either an item cannot be inserted or the array must be increased in size. In all of these cases, it takes time to accomplish the task at hand.

A solution is to use what is known as a linked list. A *linked list* can be implemented using an array or using references and objects. Although a linked list using an array can be implemented in any language, it can be somewhat complicated and does not eliminate all of the problems of non-linked lists mentioned above. For languages that support references, such as Java, it is easier to understand linked list concepts and they do not require the writing of as many methods.

Although linked lists are faster for insertion or deletion, and do not suffer the problem of trying to decide how large of an array should be declared, they do have their disadvantages. Although in many cases linked lists are faster than non-linked lists implemented in arrays, they use more memory for the increase in speed. In other words, the major disadvantage is that they sacrifice memory for speed. Although generally faster, another disadvantage is that they cannot be searched as quickly even when an ordered list is used, since a binary search cannot be used.

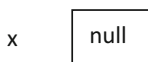
In spite of these limitations, linked lists are very useful and are frequently employed in computers. Some of these uses are storing data on disks or in memory, and are utilized extensively in operating systems. Before actually implementing a linked list in the next chapter, there are a number of concepts that must be discussed.

## 5.2 References

Before looking at a linked list, it is important to review the concept of a reference. Recall from a previous course or text on Java, when creating an array, a variable is declared that can refer to an array.

```
int[] x;
```

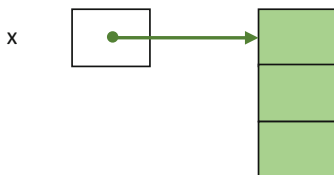
Initially `x` contains a `null` reference as shown below:



Then when the following statement is executed,

```
x = new int[3];
```

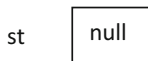
a three element array of integers is created and a reference (represented by the arrow) is assigned to the variable `x`.



Similarly, when a variable of type `String` is declared as follows:

```
String st;
```

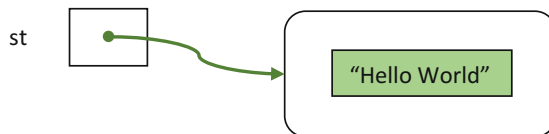
The variable `st` initially contains a `null` reference:



Then when the following instruction is executed:

```
st = new String("Hello World");
```

a `String` object is created by the new instruction and a reference to the string is placed into the variable `st` as follows:



As should be familiar to the readers of the previous text, *Guide to Java* [2], the outside rounded rectangle is a contour which represents an object of type `String` and within the contour is the text data.

---

## 5.3 Objects

As with strings and as a review, the same thing occurs when one creates an object. The class below is called `Node` because as will be seen in the next section, each object in a linked list is typically called a *node* and it is helpful to have the class have the same name. Likewise, the data member is called `data`, so again it helps to use that name as well. The class also contains two constructors: one which accepts a parameter to initialize `data` and the other that invokes the previous constructor to initialize `data` to 0. The class also contains the usual `getData` and `setData` methods.

```
public class Node {  
    private int data;  
  
    public Node() {  
        this(0);  
    }  
}
```

```

public Node(int data) {
    this.data = data;
}

public int getData() {
    return data;
}

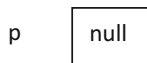
public void setData(int data) {
    this.data = data;
}
}

```

As in the previous section, the following declaration of the variable `p` is of type `Node`:

```
Node p;
```

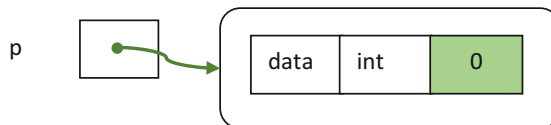
Again the variable `p` would contain a `null` reference inside the main method. Although a complete contour for the main method could be drawn, only a simple labeled box will be drawn for the variable `p` in the main method as done with both `x` for an array and `st` for a string previously, and as shown below:



Then when the following instruction is executed,

```
p = new Node();
```

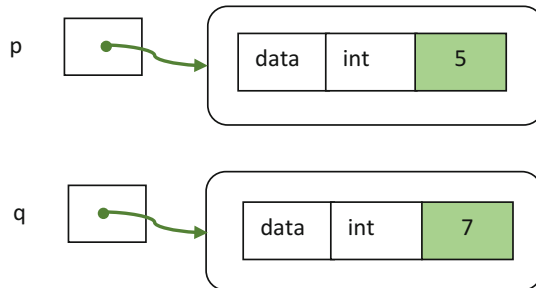
a new object of type `Node` is created and a reference to the object is assigned to the variable `p`:



The above contour represents the object containing the data member `data`. As before the arrow represents the reference to the instance of the `Node` class. To create a second instance of the `Node` class, a second variable can be declared and another object assigned to it as given in the following code segment:

```
Node p, q;  
p = new Node(5);  
q = new Node(7);
```

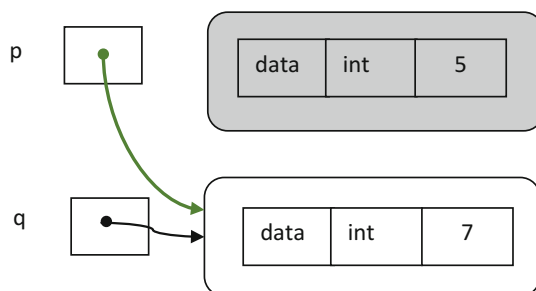
which would create the following contour diagrams:



However, what would happen if the following instruction was executed?

```
p = q;
```

Would the variable `data` inside `p` contain the number 7? The answer is no, because the contents of `data` in the object referenced by `q` are not copied into `data` in the object `p`, but rather the contents of `q`, which is a reference to the object containing the 7, would be copied into `p`. In other words, it is not the contents of the object that are copied, but rather the contents of the variable `q` that are copied. Since `q` contains a reference to an object, it is a copy of the reference to that object that is copied, not the object itself. The result is that since `q` refers to the object with the 7, `p` would also refer to the object with the 7 as indicated by the green arrow in the contour diagram below:



What would happen to the object containing the 5? Is it still accessible? The answer is no and the object will be returned to the heap by the garbage collector as indicated by the contour now being gray in color. The reason why many beginning



programmers sometimes think that the 7 was copied is that when they execute the following `println` statement, a 7 is output causing them to mistakenly think that 7 was copied.

```
System.out.println("p = " + p.get());
```

However, to prove that it was not copied, assume that the following statement is executed:

```
q.setData(9);
```

What would be output if the preceding `println` statement was executed again? Would the number 7 still be output? It might be thought that since the data in `q` was changed to 9 that the `p` would not change. However, the answer would be no, because `p` and `q` are still referring to the same object and the output would be 9.

This is why it is important to have a good mental model of objects and references by using drawings such as contour diagrams, because without them one will have difficulties understanding how references work and subsequently understanding linked lists. For more information on or for a review of contours, see Chaps. 3 and 6 in *Guide to Java* [2]. Although exploring references with simple nodes like these are interesting in and of themselves, they are just a step in learning how to create a linked list.

---

## 5.4 The Node Class

In order to link nodes together, the `Node` class will need to be modified to contain another data member to hold a reference such as the following:

```
Node next;
```

It is called `next` because it will be used to reference the next node in the linked list. But what might be confusing is that it is declared to be of type `Node`. Is it legal to have a data member be of the same type as the class that it is declared in? The answer is yes, because that is how one will be able to link the objects together as will be seen in the next section.

In addition to the above data member, there will need to be the appropriate modification to the initializing constructor to set the `next` data member to `null`.

```
public Node(int data) {  
    this.data = data;  
    next = null;  
}
```

In addition there will need to be two new methods. One to get the contents of `next` as shown below:

```
public Node getNext() {  
    return next;  
}
```

and there should also be the ability to set the `next` data member to refer to another node as follows:

```
public void setNext(Node next) {  
    this.next = next;  
}
```

Lastly, adding two constructors along with all of the above to the previous `Node` class results in the new `Node` class in Fig. 5.1.

Given the new `Node` class it is now possible to create links between instances of the `Node` class in the following section.

**Fig. 5.1** Node class

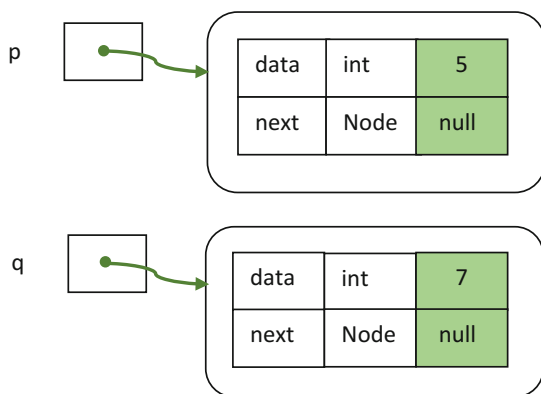
```
public class Node {  
    private int data;  
    private Node next;  
  
    public Node() {  
        this(0);  
    }  
  
    public Node(int data) {  
        this.data = data;  
        next = null;  
    }  
  
    public int getData() {  
        return data;  
    }  
  
    public void setData(int data) {  
        this.data = data;  
    }  
  
    public Node getNext() {  
        return next;  
    }  
  
    public void setNext(Node next) {  
        this.next = next;  
    }  
}
```

## 5.5 Creating Links

Starting again with the code segment from Sect. 5.3,

```
Node p,q;  
p = new Node(5);  
q = new Node(7);
```

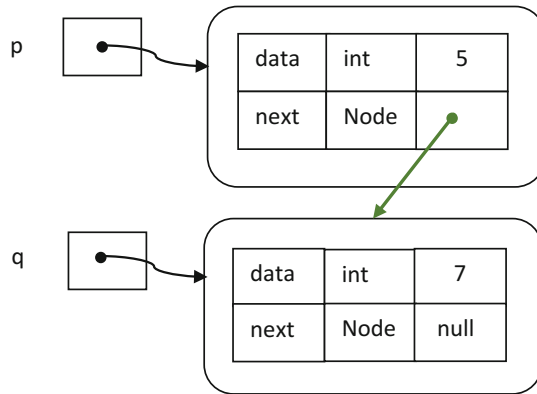
the following would be created using the new `Node` class. However, this time note the inclusion of the data member `next` in each object.



Further, notice that the `next` data member is of type `Node` which means it can refer to objects of type `Node`. The result is that the object referred to by `p` can now refer to the object referred to by `q`. How can this be accomplished? The intent is to have `next` in the object referred to by `p` refer to the object that is referred to by `q`. So, the `setNext` method must be used as follows:

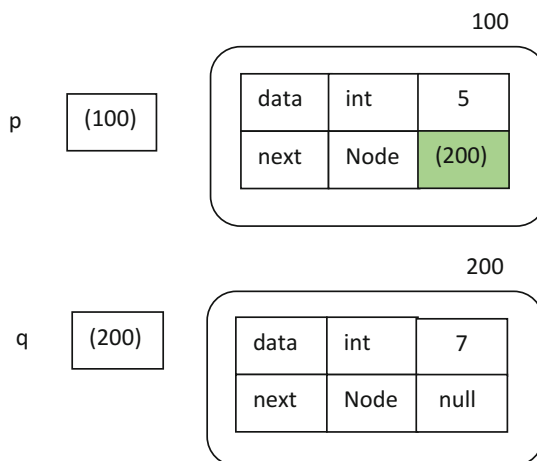
```
p.setNext(q);
```

This causes the `next` data member in the object referred to by `p` to be set the same as `q` as follows:

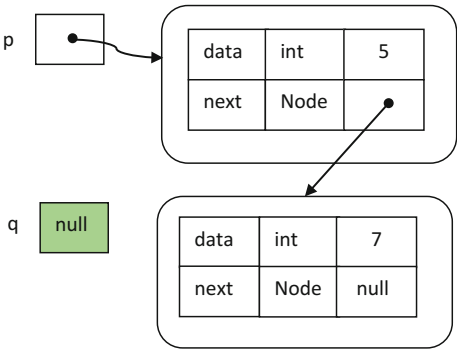


In other words, since *q* references the node with the 7 in it, so too does the *next* field in the object referred to by *p*. It sometimes confuses programmers learning about links as to what is in fact being copied. For example, if a person in a room is pointing at the door and you are going to copy what they are doing, you would be pointing to the door also.

Instead of using arrows, if *q* is hypothetically referring to memory location 200, the *next* field would refer to memory location 200 also. In the following diagram, the top contour is at memory location 100 and the bottom one is at memory location 200. The variable *p* is referencing 100 as indicated by the 100 in parentheses and *q* is referencing 200 as indicated by the 200 in parentheses. When *p.setNext(q)* is executed, then the (200) in *q* is copied into *next* in the contour referenced by *p* as shown below:

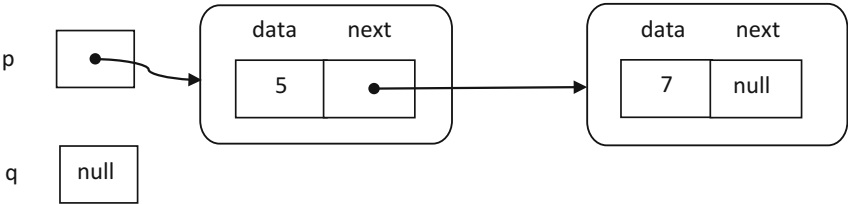


**Fig. 5.2** List using references and objects

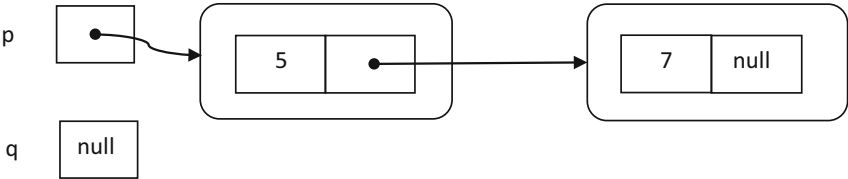


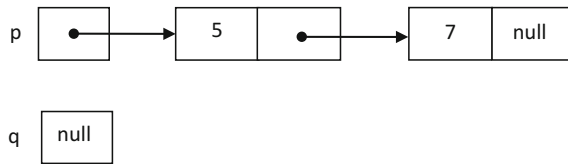
Either addresses or arrows can be used, but one will find the arrows are much easier to work with in subsequent examples. Returning back to the arrows instead of addresses, the result is that it is the base of the pointer that is copied into the new memory location. Since the reference in `q` is no longer needed, it can be set to `null` and a small linked list has been created as shown in Fig. 5.2.

Although contours are extremely helpful with understanding objects, when learning about linked lists and trying to understand more complicated combinations of nodes, the use of contours can become somewhat cumbersome. To that end, the following will be a series of drawings that remove some of these details a step at a time. First, note that the types could be removed, the boxes reorganized, and the names of the cells placed above the cells as shown below:



So that each memory cell does not need to be labeled, it could be assumed that the left cell will always be the `data` memory location and the right cell will always be the `next` memory location as shown below:



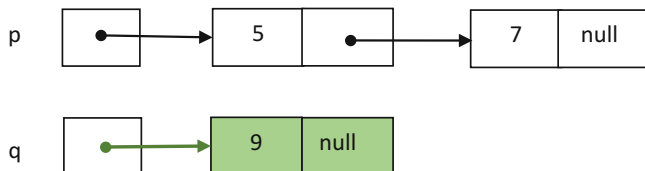


**Fig. 5.3** A Simple linked list

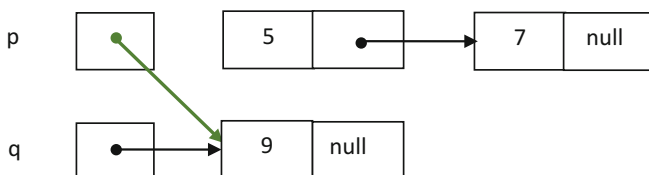
Of course, if needed the labels could always be reintroduced. Notice that the references in this drawing and in each of the past three drawings are not pointing at the data members but rather at the contours representing the nodes. This is an important distinction, because references do not point at the individual data members, but rather at the entire node. However, just as before when the contours for the main method were not drawn, it can be helpful not to draw the contours for the nodes to help simplify the linked list drawings as in Fig. 5.3.

The above drawing at first might seem to contradict what was just said in the last paragraph. It appears that the references are no longer pointing at a node, but rather an individual data member within the node. Is the above drawing correct? As before, sometimes the use of contours can complicate an already complex drawing. As a result, it is sometimes helpful to remove them. However, the problem is that the references can no longer point to the edge of the contour representing the node, but rather only to one of the cells within the node. Does this mean that it is now pointing at an individual data member? The answer is no. Instead what one has to do is imagine that there is an invisible contour surrounding the entire node and pointing at one data member is no different than pointing to any other data member within the node. Although typically the arrows will point at the data cell, one must remember that when a reference is pointing at a particular cell it is not just pointing at a cell but rather the entire node. In confusing instances and if needed, all the contours can be included, just as the labels and types can also be included as shown previously in Fig. 5.2. Again, when the pictures get more complicated, it helps remove some of the other aspects of the drawing to help see the "big picture" as in Fig. 5.3.

To continue, what if one wanted to insert a node containing a 9 at the beginning of the above list? Although it might appear that the 9 should belong at the end of the list, assume this is an unordered linked list so where it is placed is irrelevant and the beginning of the list is probably the easiest place to insert it. To accomplish this task, first a new node containing the number would need to be created using `q = new Node(9)`; which would look as follows:

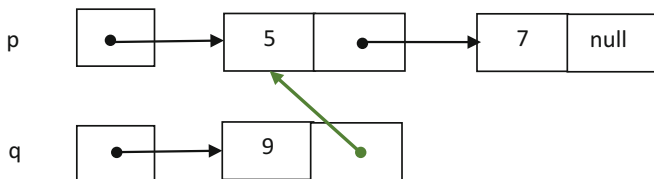


Then, in examining the picture, what needs to be done to insert the node containing the 9 at the beginning of the list? At first, one might think that the reference in  $p$  would need to reference the node containing the 9, using  $p = q$ ; as shown in the following diagram. But what is wrong with this scenario?



Notice that although  $p$  is now referencing the node with the 9, there is no longer a reference to the node containing the 5 and the rest of the list. What would happen to the rest of the list? If one responded that it is now no longer accessible, one would be correct. Technically, since nothing is referencing the list, the garbage collector would return those nodes to the heap to be reused. Although  $p$  eventually needs to refer to the same node as  $q$ , this illustrates that the order in which operations are performed is just as important as which operations need to be performed.

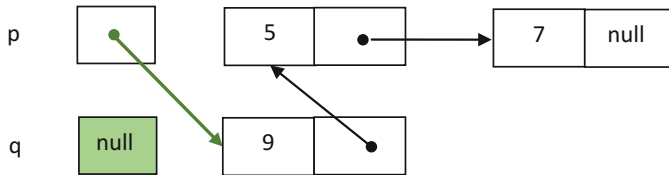
Instead, a general rule of thumb is to establish new links before breaking old links. In the situation above, the node containing the 9 first needs to be linked to the node containing the 5 before  $p$  refers to the node containing the 9. The way this is accomplished is to use  $q.setNext(p)$ ; which would look as follows:



Now that the link connecting the node with the 9 to the node with the 5 has been established, it is now safe to have  $p$  reference the same node as  $q$  and have  $q$  set back to `null` in the following code segment:

```
p = q;
q = null;
```

And as shown below:



Although not necessary, the drawing can be redrawn so that the list is on one horizontal line:

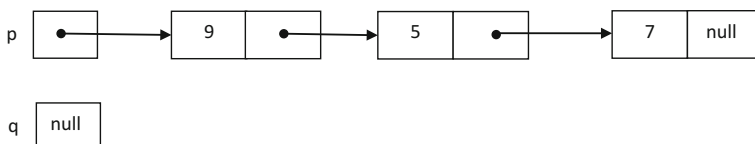
Other possible manipulations of this list, such as removing the node containing the number 5 or adding a node containing the number 3 between the 9 and 5, are left as exercises at the end of this chapter.

## 5.6 Output

Given the linked list in Fig. 5.4, it would be helpful to be able to output the contents of the list. Since there are only three elements in the list, it would be simple to just output the first item in the list as follows:

```
System.out.println(p.getData());
```

However, in the most recent drawing of the list, `q` had been set to `null`, so how could the contents of the second element be output? One method would be to set `q` to reference the second node in the list. But how could this be done? The only route to the second node is to go through `p`. The next cell in the node referenced by `p` is referencing the node with the 5, therefore `p` would have a copy of that reference as well. So to have `q` reference the second node the following would work:



**Fig. 5.4** A Singly linked list



```
q = p.getNext();
```

And the following `println` would output the contents of data 5.

```
System.out.println(q.getData());
```

However, this could be shortened where the variable `q` would not need to be used by combining the two `get` methods in the `println` statement as follows:

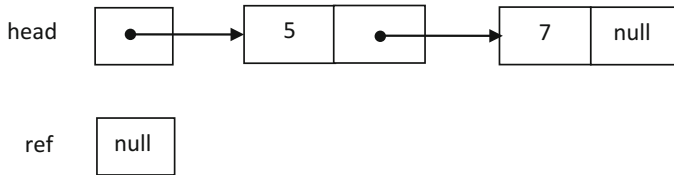
```
System.out.println(p.getNext().getData());
```

In other words by following the reference out of `p`, one can get the contents of `next`, which is a reference to the second node, and `getData` would get the number 5 to be output by the `println`.

Although this way is a little cleaner because it does not need to use another variable, such as `q`, and it only takes up one line, what about the third node in the list? The `println` statement for each node would need to have another `getNext` and each successive `println` statement could become rather unwieldy. Further, if the number of nodes were unknown, it would be difficult to create a program to output `n` nodes.

As one might suspect, the solution to such a problem involving `n` things to be processed is a loop. Although a `for` loop could be used, it is a little easier in this case to initially use a `while` loop. However, instead of iterating `n` times requiring the use of a count as done previously with the list using an array, the linked list above has a built in terminator with the last node containing a `null` in the `next` cell. Although a count could be kept if needed, it is not necessary in this instance. Further, the loop will not need a variable such as `i` as a loop control variable to count the number of items (unless a count of the number of nodes is needed), nor will it be needed as an index since there is obviously no array.

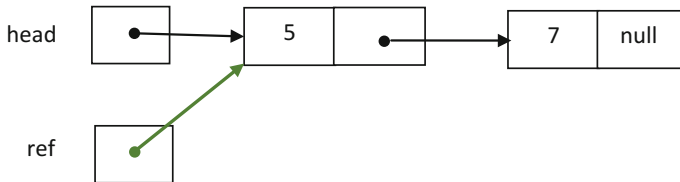
Instead a variable capable of referencing nodes is needed. Renaming some of the variables from the previous figure, often the variable referencing the first node in a linked list is called `first` or `head` indicating the first node or the head of the linked list. Then another variable is needed for moving down or in other words traversing the length of the list. In other languages which use pointers, this variable has been called `ptr` for pointer, but since Java uses references this text will use the abbreviation `ref` for reference or alternatively `curr` as an abbreviation for current. Replacing `p` and `q` from the two-element simple linked list in Fig. 5.3 with `head` and `ref`, the new drawing is as follows:



Recall from a first course or text in programming the three parts of most loops are initialize, test, and change. The first thing that needs to be done is to initialize the loop control variable (LCV) which in this case is `ref`. It will need to be initialized to the first node in the list, which is referenced by `head`.

```
ref = head;
```

Thus the drawing would be as follows:



Then for each time through the loop there needs to be a test to terminate the loop. In the case of an empty list, such as at the beginning of this chapter, the value for `head` would be `null` and when the above instruction is executed, `ref` would also contain a `null`, and the loop would terminate. So, the loop should continue iterating while `ref` is not equal to `null` as follows:

```
while(ref != null)
```

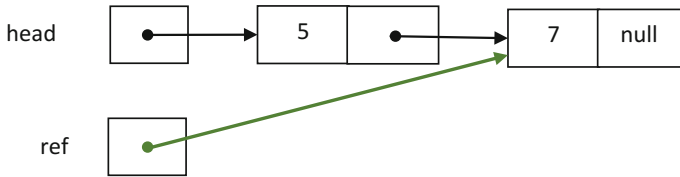
In the previous drawing, `ref` is not equal to `null` so the contents of the node could be output as shown below:

```
System.out.println(ref.getData());
```

Now the only thing remaining is the change portion of the loop which requires the value of `ref` to move to the next node. In other words the value of `ref` needs to take on the value of `next` in the node that `ref` is referring to. Putting this into code results in the following:

```
ref = ref.getNext();
```

Again, the value of `next` in the node that `ref` is currently referencing is copied into `ref`, thus causing the old value of `ref` to be updated with a reference to the second node in the list as follows:

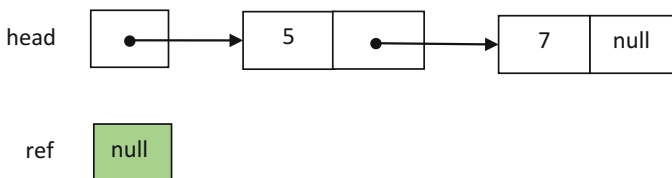


Putting all of the code together results in the following segment:

```

ref = head;
while(ref != null) {
    System.out.println(ref.getData());
    ref = ref.getNext();
}
  
```

After the first iteration of the loop, control returns back to the `while` statement, where `ref` is not equal to `null`, so the body of the loop executes again. The 7 from the data cell of the node is output by the `println` statement and the `next` instruction is executed. After the second iteration, the value in `next` in the node referenced by `ref` contains a `null` and this is copied into `ref` as shown below:



This time when the flow of control returns back to the `while` statement, it is `false` since `ref` is now equal to `null`, the loop ceases to iterate, and the contents of the list have been output.

---

## 5.7 Test Program

Instead of creating each individual node as they are needed, as done previously, wouldn't it be nice to be able to use the power of an iteration structure to create a list similar to the way an iteration structure was used to output a list? The program could utilize a sentinel control loop to prompt for and input numbers that could then be placed in a linked list. Although an ordered linked list will be discussed in the

next chapter, this does not preclude creating an unordered linked list here. Further, a `LinkedList` class will also be introduced in the next chapter, so for now just a main program will be used to create this test linked list.

Since only integers are being used here, it might be helpful to assume that only non-negative numbers will be entered into the list, so if a negative number is entered, the loop will stop and the program can continue on to output the list. While non-negative numbers are entered, a new node is created and a reference to the node is placed in a variable called `newp`. Then the `next` field of the new node is made to reference the beginning of the list and `head` is made to reference the new first node in the list, similar to the process in Sect. 5.5. Once a negative number is entered, the loop stops iterating, the column heading is output, and the list is output as done in Sect. 5.6. The following code segment implements this algorithm:

```
int number;
Node head, newp, ref;

head = null;
System.out.println();
System.out.print("Enter a negative integer to stop: ");
number = scanner.nextInt();
while(number >= 0) {
    newp = new Node(number);
    newp.setNext(head);
    head = newp;
    System.out.print("Enter a negative integer to stop: ");
    number = scanner.nextInt();
}
System.out.println();
System.out.println(" Unordered");
System.out.println("Linked List");
System.out.println();
ref = head;
while(ref != null) {
    System.out.println("      " + ref.getData());
    ref = ref.getNext();
}
```

Implementing and executing the above program with some sample data yields the following output:

```
Enter a negative integer to stop: 9
Enter a negative integer to stop: 5
Enter a negative integer to stop: 7
Enter a negative integer to stop: -1
```

```
Unordered
Linked List

7
5
9
```

In examining the above output, is there a noticeable pattern? Yes, it should be noticed that the data that was input is output in reverse order. Further, is there a particular data structure that behaves in this manner? Yes, as one recalls from Chap. 2, the stack can be useful when one needs to output data in reverse order, where here a linked list is used instead of an array. Although the above is not a complete implementation of a stack, it illustrates that linked lists can be used to implement a variety of data structures, and the stack will be revisited in Chap. 7.

---

## 5.8 Output Using Recursion

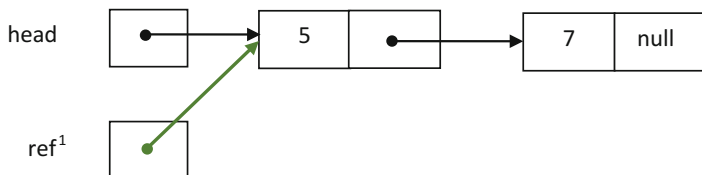
The preceding section illustrated how the contents of the list can be output using iteration, but as illustrated in *Guide to Java* [2] and other texts, many problems that can be solved using iteration can be solved using recursion. Instead of just calculating a number such as  $x^n$  or  $n!$ , recursion can be extremely useful in processing or outputting data from linked structures. Using the same list from the preceding section, how could recursion be used to traverse and output the contents of each node? As with trying to solve problems utilizing just numbers, such as  $n!$ , it helps to first recognize a pattern. Using the two-element list in Fig. 5.3, notice that if the first node in the list is processed or output, what is remaining? The answer is a list, but this time with only one node in it. And if that node is processed or output, what is remaining? Still another list, but this time it is an empty list or `null`. This last case is known as the base case or terminal case because should the reference be `null` then the recursion ought to stop. Instead of using a loop to test for the terminal case, an `if` statement can be used, where if the reference is not `null`, then the contents of the node would be output and the reference would move on to the next node in the list. Including the rest of the code needed for the method, the method would look as shown in Fig. 5.5.

```
public void printRecursive(Node ref) {
    if(ref != null) {
        System.out.println(ref.getData());
        printRecursive(p.getNext());
    }
}
```

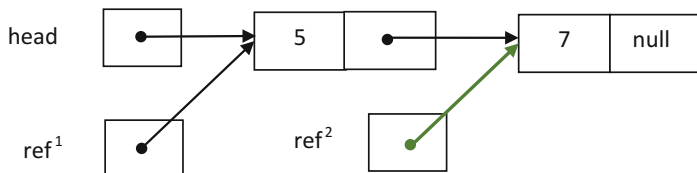
**Fig. 5.5** `printRecursive` method

Although complete contours could be drawn to show each instance of the method as it is invoked, since the only variable (besides the return location) would be `ref`, it makes sense not to include the entire contour as discussed earlier in this chapter. Instead, only the variable `ref` will be drawn with a superscript to indicate which invocation is currently being used.

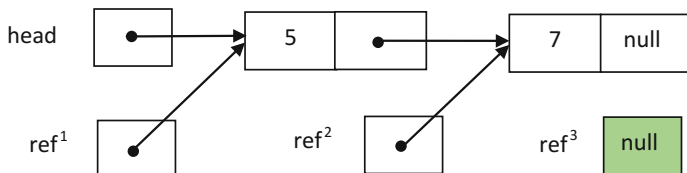
To invoke the above method, the invocation `printRecursive(head)` can be used. Upon entering the `printRecursive` method, `ref` will refer to the same node as `head`:



Notice that `ref` has a superscript of 1 indicating that it is in the first call to the `printRecursive` method. Since `ref` is not equal to `null`, the contents of the node are output and the method is invoked a second time. However, instead of `head` being sent as the argument, the contents of the next cell that `ref` is referring to is sent to the second invocation of `printRecursive`. The result is that `ref` in the second invocation will be referring to the node that contains the 7 as shown in the following diagram:



Notice that the second occurrence of `ref` has a superscript of 2 indicating that it is in the second call to the `printRecursive` method. Since `ref2` is not equal to `null`, the contents of the node are output and the method is invoked a third time. Again, the contents of the next cell that `ref2` is referring to are sent as an argument to the third invocation of `printRecursive`. The result is that `ref3` will contain a `null` as shown in the following diagram:



Since `ref`<sup>3</sup> contains a `null`, which is the terminal case, no more recursion will take place. Instead, the third call to `printRecursive` will return to the second invocation, the second will return to the first, and the first will return back to the main program. Further, each of the variables for `ref` will be deallocated and the list will have been successfully output. Note that the implementation of recursion will be demonstrated in the complete program in the following section.

---

## 5.9 Complete Program: Outputting Items in the List Using Recursion

As in Chaps. 2–4, the Complete Program section will include generic classes. The concept of outputting the data in the list using recursion described in Sect. 4.8 will be implemented with a generic class so that the list of strings can be printed recursively.

First, the `Node` class shown in Fig. 5.1 will be rewritten so the `NodeGeneric` class can be used for any objects. As before, type `int` for the data in the node will be replaced by the type parameter `T`. Then, `NodeGeneric < T >` is going to be the reference type for the node instead of `Node`. The complete `NodeGeneric < T >` class is shown in Fig. 5.6.

```
public class NodeGeneric<T> {
    private T data;
    private NodeGeneric<T> next;

    public NodeGeneric() {
        this(null);
    }

    public NodeGeneric(T data) {
        this.data = data;
        next = null;
    }

    public T getData() {
        return data;
    }

    public void setData(T data) {
        this.data = data;
    }

    public NodeGeneric getNext() {
        return next;
    }

    public void setNext(NodeGeneric<T> next) {
        this.next = next;
    }
}
```

**Fig. 5.6** `NodeGeneric < T >` class

Since strings stored in the list will be printed recursively, small changes will be made to the `printRecursive` routine described in Sect. 5.8. All that is needed to be done is to change the data type of the parameter `ref` from `Node` to `NodeGeneric < String >` shown below:

```
public static void printRecursive(NodeGeneric < String > ref) {
    if(ref != null) {
        System.out.println(ref.getData());
        printRecursive(ref.getNext());
    }
}
```

Next, a main program which calls the `printRecursive` method is implemented. The test program shown in Sect. 5.7 is rewritten to accept strings from the user and print the items using the `printRecursive` routine instead of using a `for` loop. The following is the complete program including the `printRecursive` method located in the same class.

```
import java.util.*;

class PrintRecStr {
    public static void main(String[] args) {
        Scanner scanner;
        scanner = new Scanner(System.in);

        String str;
        NodeGeneric < String > head, newN;

        head = null;
        System.out.println();
        System.out.print("Enter \"stop\" to stop: ");
        str = new String(scanner.next());
        while(!str.equals("stop")) {
            newN = new NodeGeneric < String > (str);
            newN.setNext(head);
            head = newN;
            System.out.print("Enter \"stop\" to stop: ");
            str = scanner.next();
        }
        System.out.println();
        printRecursive(head);
    }
}
```



```
public static void printRecursive(NodeGeneric < String > ref) {  
    if(ref != null) {  
        System.out.println(ref.getData());  
        printRecursive(ref.getNext());  
    }  
}  
}
```

When the main program is compiled and executed, it will prompt for various words. Whenever the user types the word "stop", it will output all the string values entered in reverse order.

```
Enter a word, "stop" to end: node  
Enter a word, "stop" to end: link  
Enter a word, "stop" to end: recursive  
Enter a word, "stop" to end: list  
Enter a word, "stop" to end: stop  
  
list  
recursive  
link  
node
```

So far in this chapter, the concepts of references and nodes have been introduced, and a simple linked list has been created and output both iteratively and recursively. However, there are far more complicated processes that can be performed with linked lists as will be explored in subsequent chapters. But before proceeding, it is important that the reader have a firm foundation in the fundamentals, so if there is any concern be sure to review this chapter again before proceeding.

---

## 5.10 Summary

- References can be used to implement linked lists.
- Linked lists are made up of objects that typically contain a data section and a reference section, which are often called `data` and `next`.
- If contours are not drawn to represent objects in a link list, one should note that the arrows drawn are not pointing to the individual data members, but rather to the object or node in its entirety.
- A reference that does not refer to anything is known as a `null` reference.
- Linked lists can be unordered or ordered.

- When manipulating references, it is usually better to create new references before breaking old references.
- A reference variable of the same type as `head`, such as `ref`, should be used to traverse the list.

---

## 5.11 Exercises (Items Marked with an \* Have Solutions in Appendix B)

1. Suppose a linked list was created using the `Node` class described in Fig. 5.1 and contains three nodes with data 34, 26, and 78. Further, `head` references the first element.

\*A. Draw a diagram of the linked list described above.

\*B. Describe the effect of the following code segment and draw a diagram of the linked list.

```
Node curr;  
curr = head.getNext();  
curr.setData(9);
```

C. Starting from the original list above containing 34, 26, and 78, describe the effect of the following code segment and draw a diagram of the linked list.

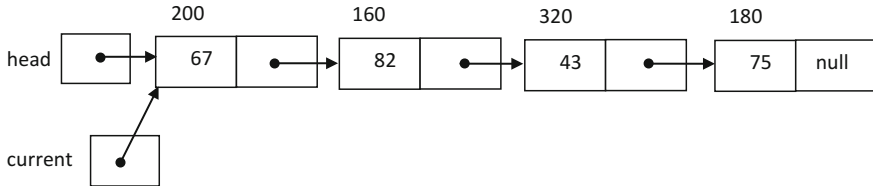
```
Node nodeRef;  
nodeRef = head.getNext();  
head.setNext(nodeRef.getNext());
```

D. Starting from the original list above containing 34, 26, and 78, describe the effect of the following code segment and draw a diagram of the linked list.

```
Node temp;  
temp = new Node(51);
```

```
temp.setNext(head);
head = temp;
```

Using the linked list created with the `Node` class given in Fig. 5.1 and shown below, answer the following questions, 2 through 4. The values above each node represent the address of the nodes and for questions that have an answer as an address, represent the address using parentheses, such as (300).



2. State if the syntax of the following statements is correct. If incorrect, explain why.

A. `current.getNext().getNext() = head;`

\*B. `head.setNext(current.getData().getNext());`

C. `int ptr = 45;`  
`head.setNext() = ptr;`

D. `head.getNext().getNext().getNext().getNext()`

3. Find the values of following expressions:

A. `current`

\*B. `current.getNext().getNext()`

C. `current.getData()`

D. `head.getNext().getNext().getNext().getNext()`

4. Write Java statement(s) to do the following. For all the problems, start using the original list:
  - A. Make `current` reference the next node.
  - \*B. Make `head` reference the node containing data 43.
  - C. Make `current` reference the last node.
  - D. Make `head` reference an empty list.
  - E. Change the value of the node containing 82 to 29.
  - F. Create and insert the node with data 10 after the node referenced by `current`.
  - G. Delete the node containing the number 82.
5. Write a code segment to remove the node containing the number 5 from the list shown in Fig. 5.4.
6. Write a code segment to add the node containing the number 3 between 9 and 5 in the list shown in Fig. 5.4.

The previous chapter introduced some of the basic concepts of linked lists using *unordered linked lists*. Those concepts form the basis for this chapter and many of the chapters that follow. In particular, this chapter will examine *ordered linked lists* and includes the development of a linked list class which utilizes the `Node` class as shown in Fig. 5.1 in the previous chapter. The `insert` routine will be developed first, followed by the `delete` routine, and a driver main program utilizing the `LinkedList` class.

---

## 6.1 Creating the Insert Method

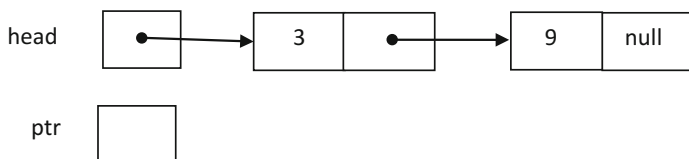
This section will explore the development of an `insert` method. Although in the last chapter the test program in Sect. 5.5 inserted nodes, they were restricted to being inserted only at the beginning of the list and it was done from the main program instead of from a method. Here, a method will be developed as part of a `LinkedList` class that will be capable of inserting nodes in the proper location at the beginning of a list, middle of the list, the end of a list, or in an empty list. Further, it will not allow for duplicate items in the list, but this is left as an exercise at the end of the chapter. Since this method will need to be sent the `item` to be inserted it will have one parameter. Also, since it should be able to inform the invoking program whether the item was inserted or not, it will return a `boolean` value.

### 6.1.1 Inserting in the Middle of the List

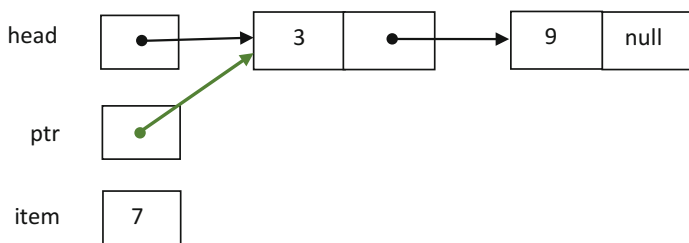
When developing code utilizing references, it is tempting to begin with the first item on a list. Unfortunately this can sometimes lead to algorithms that work well for a specialized case such as inserting an item at the beginning of the list but can lead to

code that is not as efficient for cases such as inserting items in the middle of the list, or other special cases such as at the end of the list or in an empty list. Instead, it is often helpful to create the algorithm for the more general case that might occur more frequently, such as inserting an item somewhere in the middle of the list, and then work out the special cases including the beginning and end of the list, or an empty list.

Given this advice, consider the existence of the ordered list below. Note that `head` references the first item in the list, and that the list is an ordered linked list which means that each node in the list follows one another in a prescribed order, and in this case it is in ascending order. Other names for `head` can be `first`, `list`, and `front`, but this text will chose `head` for the beginning of the list and if necessary use `tail` for the end of the list. Here, `ptr` is used as a shortened name for pointer and although technically the arrows are references, it is convenient to use `ptr` or `p` as is done in many other languages.



At first it might be frustrating to assume the existence of a list and not know how it was created in the first place. However, again when trying to create an algorithm, it can be easier to start with a generic case first. Assuming the above list, what if one wanted to insert the number 7? First, the list would need to be searched to determine where the item should be inserted. In order to search the list, a loop can be employed. The variable `ptr` would need to be initialized to the same value as `head` as shown below:



While `ptr` is not equal to `null` the loop can continue to iterate and each time through the loop the value of `ptr` would need to be updated to reference the next node in the list by taking on the value of the `next` field in the node referenced by

`ptr`. The basic loop structure to accomplish the above should look familiar and similar to the loop that was used in the iterative method to output the contents of a linked list presented in Sect. 5.6.

```
ptr = head;
while(ptr != null) {
    //body of loop
    ptr = ptr.getNext();
}
```

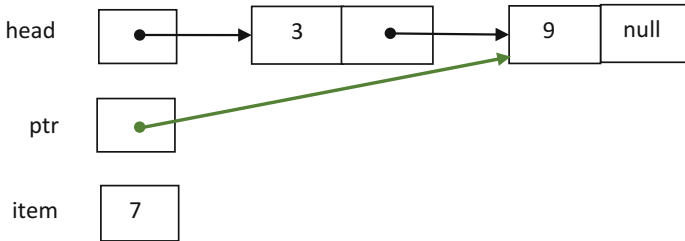
In the case where the list is empty, `head` is `null` as will be `ptr`, so the loop will not iterate as discussed later. Should the list not be empty, the loop would continue to iterate until `ptr` takes on the value of `null` from the last node in the list. However, is there any need to continue on to the end of the linked list? As was the case with array-based lists, when an item being searched for is found, there is no need to continue searching whether the linked list is ordered or unordered. However, if an item is not in the list then the answer would be yes in an unordered list since all the items would need to be searched. The answer is no for an ordered list since once `ptr` has gone beyond where the item should be located, there is no need to continue searching the list. This is an advantage of ordered linked lists over unordered lists and this can be included in the code as follows:

```
ptr = head;
while(ptr != null && ptr.getData() < item) {
    //body of loop
    ptr = ptr.getNext();
}
```

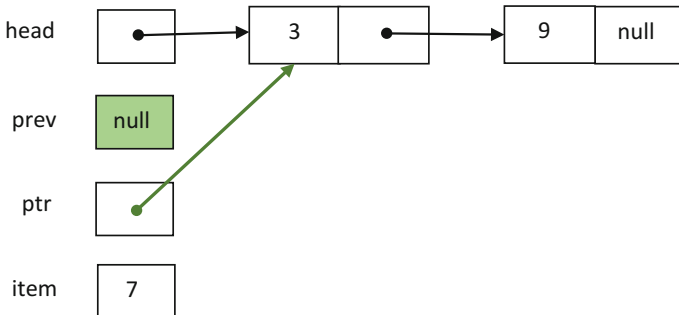
While the data in the current node is less than the item being searched for, the loop can continue to iterate. The test for `null` at the end of the list should be included for the cases where all the items in the list are less than the item being searched and `ptr` eventually would take on the value of `null`. Note that the order of operations in the conditional part of the `while` loop is critical. Should the order of the operands be reversed, it is possible that `ptr` could take on the value of `null`, and when `ptr.getData()` is invoked there could be a `null` pointer exception. By keeping the order above, should `ptr` be `null`, the first part of the `&&` statement would be `false`, and the second part of the `&&` would not be executed thus avoiding the execution error. Recall from a first course or text in programming such as *Guide to Java* [2] that this is known as a short circuit.

In looking at the previous diagram, it should seem obvious to the reader that the number 7 should be inserted immediately after the node containing the 3. However,

this cannot be determined by the algorithm since the contents of the next node have not been inspected. For all the algorithm knows, the subsequent node could contain a 4, not a 9, and the number 7 should not be inserted. Instead, the `ptr` needs to reference the next node to see what number is in it. So, since `ptr` is not equal to `null` and the number in the current node referenced by `ptr` is less than the value 7 in `item`, `ptr` is updated to reference the next node in the linked list as shown below:



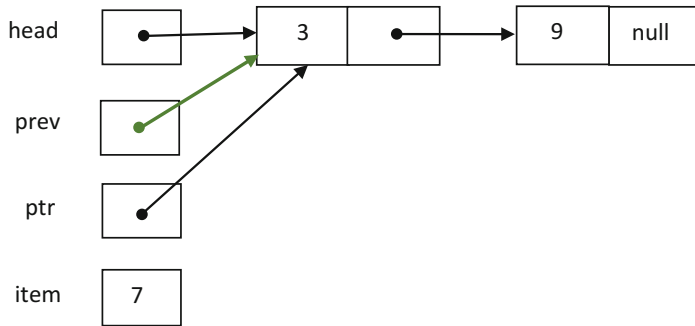
The `ptr` variable now references the node containing the 9, and it can be checked to determine that the loop should quit iterating since 9 is not less than the 7 in `item`. A new node should be created containing the 7 and it should be inserted after the node containing the 3. Unfortunately, `ptr` is no longer referencing the node containing the 3. Could `head` be used? Yes, but only in this particular instance. What if there were many more nodes prior to the node containing the 9? The result is that `head` is not a viable choice due to the number of nodes that could exist prior to the location for insertion. Instead a second variable is needed to reference the previous node which will be called `prev` in this text. Initially when `ptr` references the first node in the list, there is no previous node, so `prev` is initialized to `null` as shown in Fig. 6.1.



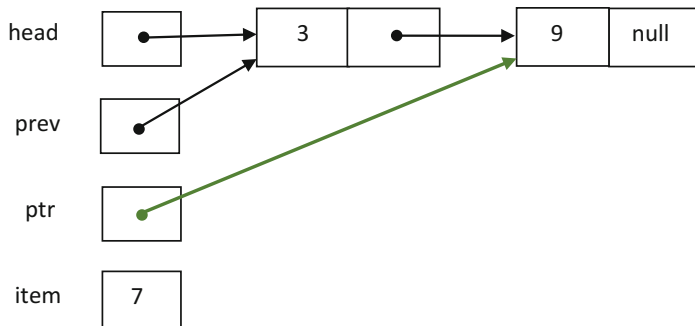
**Fig. 6.1** Linked list with initial values for `prev` and `ptr`



Then during each iteration, `prev` needs to be set to the current node prior to moving `ptr` to the next node in the list. This is accomplished by copying the reference in `ptr` to `prev` as shown below:



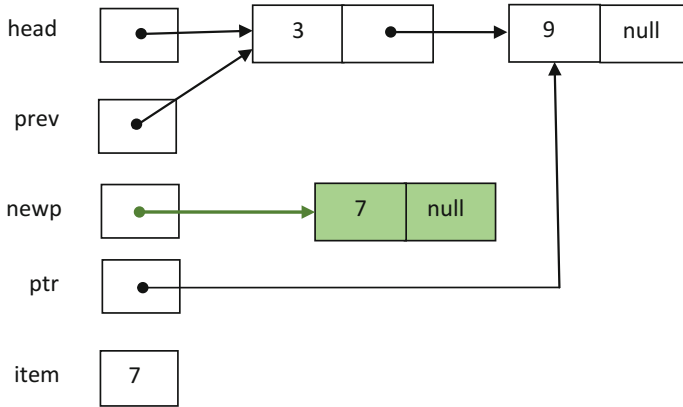
Then `ptr` can be move to reference the next node in the list as before.



The updated code to accomplish the above is shown below:

```
ptr = head;
prev = null;
while(ptr != null && ptr.getData() < item) {
    prev = ptr;
    ptr = ptr.getNext();
}
```

Now a new node can be created to include the 7 after the 3 referenced by `prev` and prior to the 9 referenced by `ptr`. A variable of type `Node` called `newp`, which stands for new pointer, is used. Then a new node needs to be created with the next field of the node set to null by the constructor in the `Node` class, and the data field of the node needs to be set to 7 as shown below:



The code to accomplish this is as follows:

```
Node newp = new Node();
newp.setData(item);
```

Or alternatively, the following can be used:

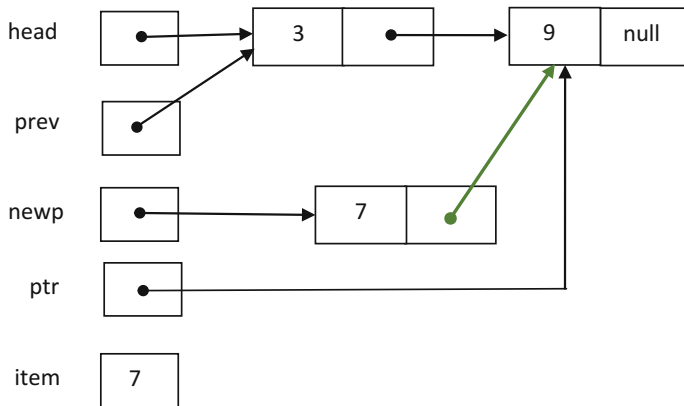
```
Node newp = new Node(item);
```

The new node then needs to be inserted into the linked list, and there are two references that need to be created: one from the node containing the 3 to the new node containing the 7, and one from the node containing the 7 to the node containing the 9. But which reference should be created first? In one sense it doesn't matter, but a general rule of thumb, or heuristic, is to create new links before removing old links. The reason for this is that on occasion by removing an old link, a link needed to get to a particular node might be lost. In this particular case, it might be better to create a new reference from the node containing the 7 before changing the reference in the node that contains the 3.

To create this new reference, the `next` portion of the node containing the 3 needs to reference the node containing the 9 and there are multiple ways this can be accomplished. However, another heuristic is to generally take the shortest route. In this case it is easier to use the reference from the variable `ptr` as illustrated in the following line of code,

```
newp.setNext(ptr);
```

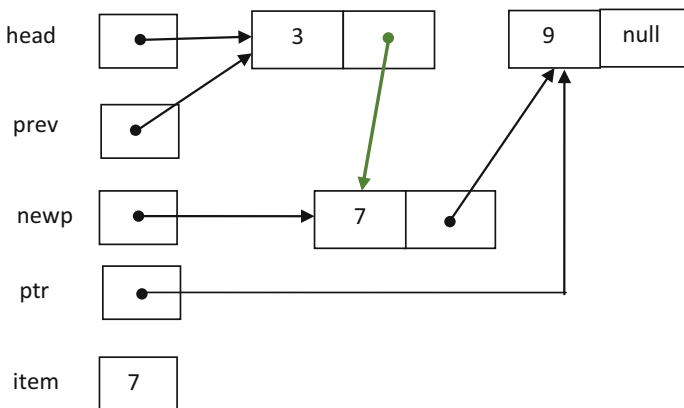
and the following drawing:



Then after the new node has been linked to the remaining part of the linked list, the connection from the node containing the 3 that references the node containing the 9 can be changed to refer to the new node containing the 7. However, which node should be used to access the next section of the node containing the 3: `head` or `prev`? Again, there could be many nodes between the first node referenced by `head` and the one referenced by `prev`. This illustrates that one must be careful when writing code since there are often multiple links that could be used. Instead of choosing the one that seems convenient for the particular case at hand, a programmer must consider the other situations and choose the one that works in many of the other possibilities that might exist. So, it is better to use the reference from `prev` rather than `head` as shown in the following instruction:

```
prev.setNext(newp);
```

and the following figure:



Putting the four lines of code together results in the following segment:

```
Node newp = new Node();
newp.setData(item);
newp.setNext(ptr);
prev.setNext(newp);
```

But what if the item is already in the linked list? The while loop as written stops when the data in the current node referenced by `ptr` is less than the number in `item`. As mentioned at the beginning of this chapter, duplicates will not be allowed in this linked list. What should be done is to check the `data` field of the current node that `ptr` is referencing to see if `item` is equal to it. If it is not, then the item should be inserted as shown in the following modified code segment:

```
if(ptr.getData() != item) {
    Node newp = new Node();
    newp.setData(item);
    newp.setNext(ptr);
    prev.setNext(newp);
}
```

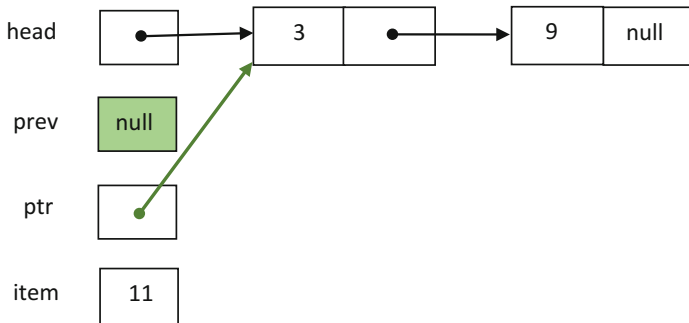
Combining the above code with the previous while loop results in the following code:

```
ptr = head;
prev = null;
while(ptr != null && ptr.getData() < item) {
    prev = ptr;
    ptr = ptr.getNext();
}
if(ptr.getData() != item) {
    Node newp = new Node();
    newp.setData(item);
    newp.setNext(ptr);
    prev.setNext(newp);
}
```

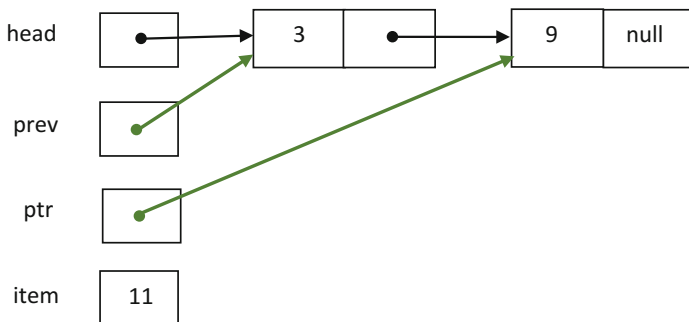
### 6.1.2 Inserting at the End of the List

Having written an algorithm for inserting an item in the middle of a linked list, how can an item be inserted at the end of the list? Although it is tempting to instead look at inserting an item at the beginning of the list, the insertion of a node at the

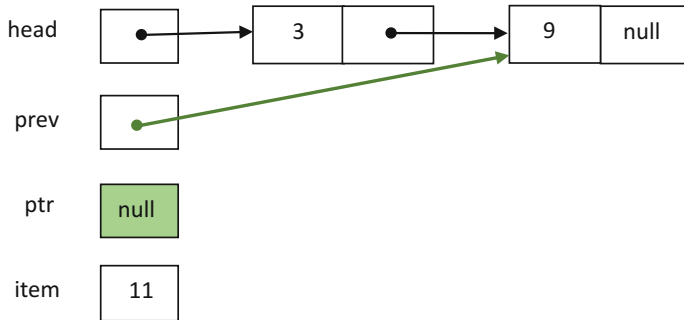
beginning requires that `head` be altered whereas inserting a node at the end does not. Since inserting at the end shares some similarities with inserting in the middle, it is helpful to look at this situation next. Using the above code segment one can walk through the code to see if it works for this scenario. Starting with the previous initial list in Fig. 6.1, what if one wanted to insert an 11? Again, the variable `ptr` would be initialized with the same reference as `head` and `prev` would be initialized to `null` as follows:



Since `ptr` is not equal to `null` and 3 is less than the 11 in `item`, the value in `prev` would reference the same node as `ptr` and then `ptr` would reference the next node in the linked list as shown below:



Again, `ptr` would not be equal to `null`, the 9 in the node referenced by `ptr` is less than the 11 in `item`, and another iteration of the loop occurs. The `prev` variable would then reference the same node as `ptr` and since there are no more nodes in the list, `ptr` would not point to a subsequent node, but rather take on a value of `null` from the next portion of the last node in the linked list as follows:



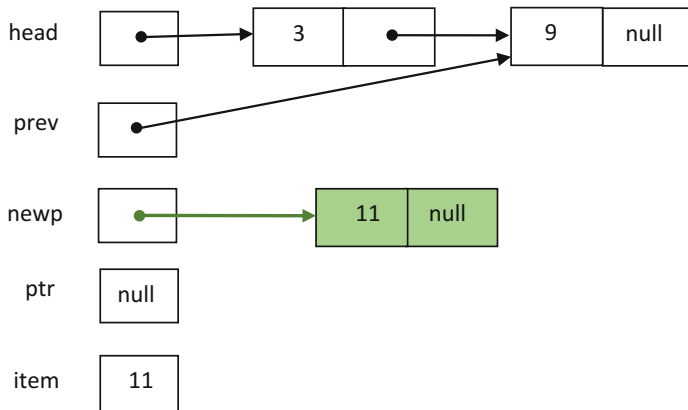
As the loop iterates back to the condition in the while loop, the value in `ptr` is now `null` so the loop stops iterating and continues on to the following `if` statement. But there is a problem with this `if` statement that is similar to the previous problem with the `while` statement. Since `ptr` is `null`, then `ptr.getData()` will cause an exception. So a check to see whether `ptr == null` needs to be added to the `if` statement and it should be prior to the `ptr.getData()` to avoid a short circuit as before. However, should the logical relation be `&&` or `||`? In the previous `while` statement it was an `&&` because both conditions needed to be `true` prior to moving on to the next node in the list. Here the situation is different because if `ptr` is equal to `null`, it is an indication that the end of the list has been reached, and `item` needs to be inserted. The result is that if this condition is `true`, there is no need to execute `ptr.getData()` and the then section of the `if` statement needs to be executed, so `||` is the proper choice.

```

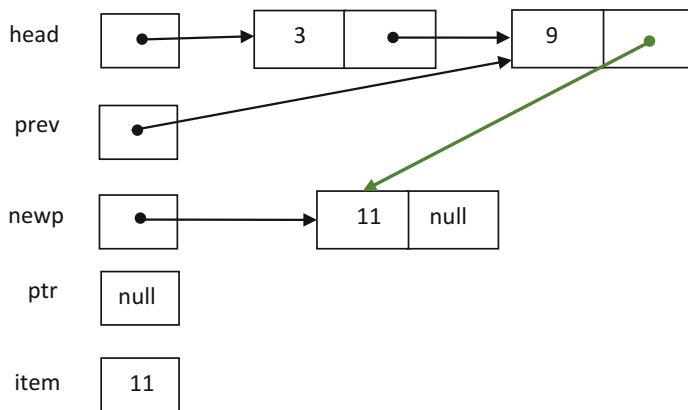
ptr = head;
prev = null;
while(ptr != null && ptr.getData() < item) {
    prev = ptr;
    ptr = ptr.getNext();
}
if(ptr == null || ptr.getData() != item) {
    Node newp = new Node();
    newp.setData(item);
    newp.setNext(ptr);
    prev.setNext(newp);
}

```

Again, it is imperative that `ptr == null` come first to avoid an exception. Since `ptr` is `null`, the code in the then section of the `if` statement is executed and a new node is created as follows:



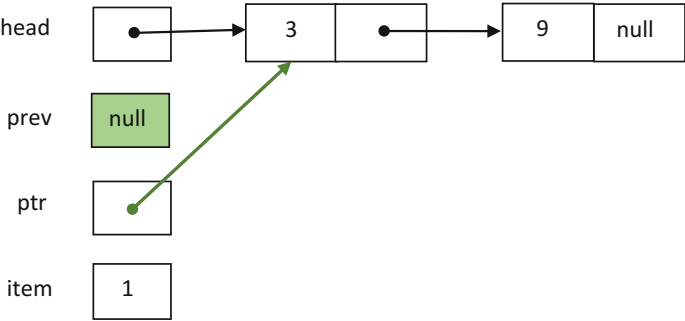
The next section of the new node is set to the same value as `ptr`. Even though the next section of the new node is already set to `null` by the constructor, since `ptr` is `null`, the next section is also `null` indicating the new end of the linked list. Lastly, the next section of the node referenced by `prev` is updated to reference the new node as follows:



Thus the number 11 is added to its proper place at the end of the linked list. What is interesting to notice is that the only change needed to the code is the `ptr == null` in the `if` statement to accommodate inserting at the end of the list. This is the reason why working on the more generic case first helps when implementing specific cases such as this.

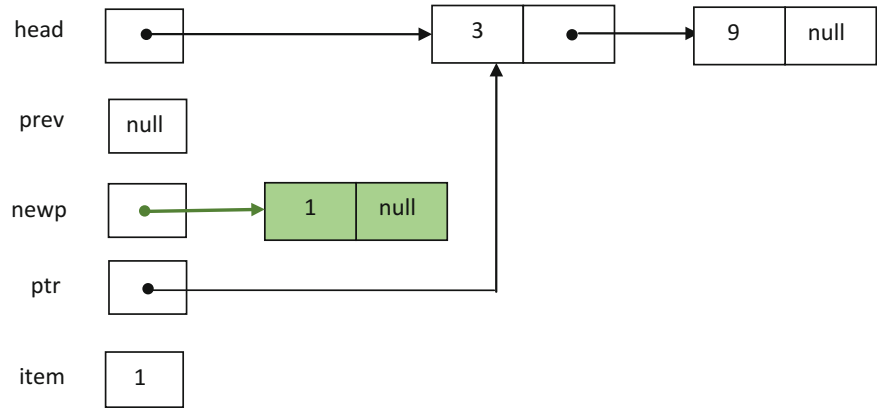
6.1.3 Inserting at the Beginning of a List

Having developed code to insert in the middle and end of a list, it is time to turn to the task of inserting at the beginning of a list. As alluded to in the previous subsection, inserting at the beginning of the list requires altering head, which will require some alterations to the previous code segment. Again using the initial drawing in Fig. 6.1, what if a 1 were to be inserted at the beginning of the list? As shown below, ptr is referencing the first node in the list, and prev has been set to null.



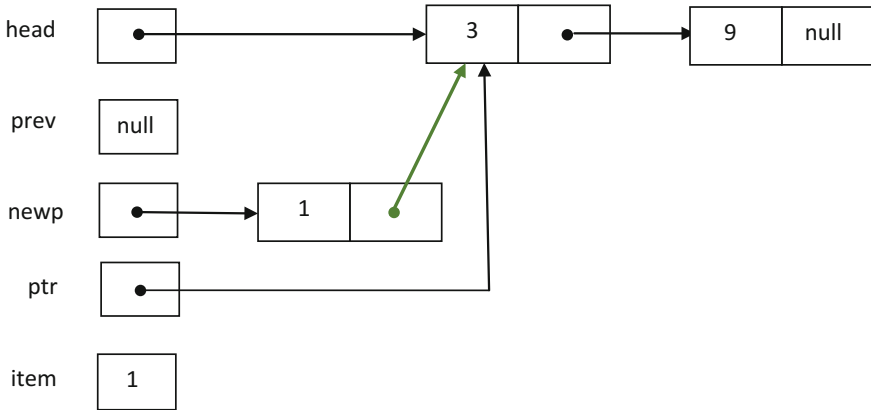
Upon reaching the while statement, ptr is not equal to null, but the 3 in the node referenced by ptr is not less than the 1 in item. The result is that the loop does not iterate because the correct spot in the list has been found. The execution of the segment jumps to the if statement where ptr is not equal to null, so the second part of the condition is checked where 1 is not equal to 3 indicating that it is not a duplicate, and execution can proceed into the then section of the if statement.

As before, a new node is created that contains the number 1 as shown below, where the linked list has been stretched out to make room for the new node at the beginning of the list as shown below:





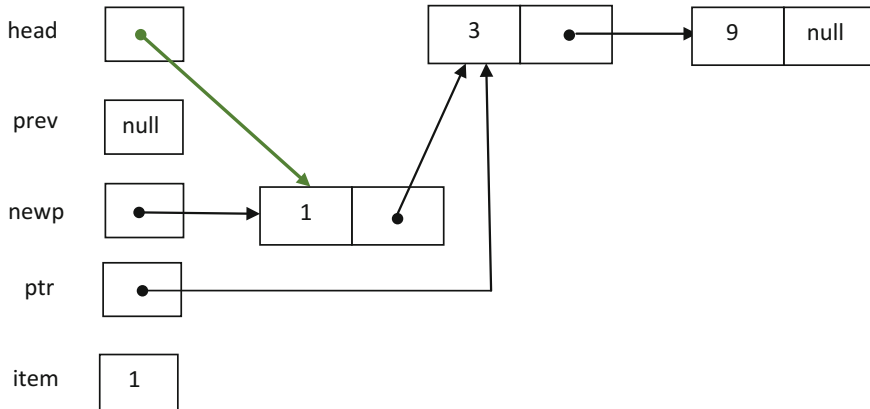
Then the reference in `ptr` is copied to the `next` section of the new node containing the 1 to reference the node containing the 3.



So far all of the code in the segment has worked properly. The problem is that the `prev.setNext(newp)` will cause an exception because `prev` is equal to `null`. How can this problem be solved? Since `prev` is `null`, it is indicating that the current node referenced by `ptr` is the first node in the list. Further, the fact that `prev` is `null` can be used to indicate that the value in `head` should be altered instead of a previous node. This takes the form of an `if` statement as follow:

```
if (prev == null)
    head = newp;
else
    prev.setNext(newp);
```

The results of executing the above code is the following diagram showing that the node containing the number 1 has been added to the beginning of the linked list:

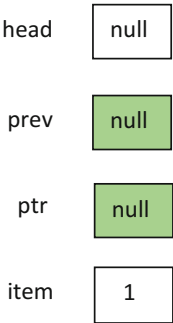


Adding the new `if` statement to the prior code segment results in the following code segment:

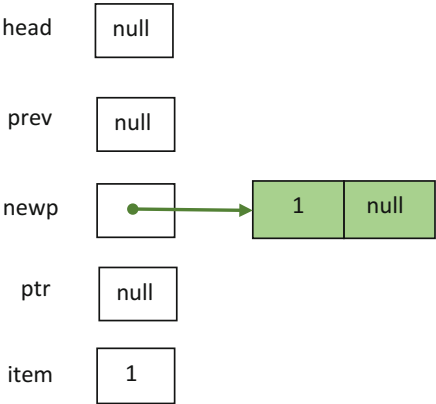
```
ptr = head;
prev = null;
while(ptr != null && ptr.getData() < item) {
    prev = ptr;    ;
    ptr = ptr.getNext();
}
if(ptr == null || ptr.getData() != item) {
    Node newp = new Node();
    newp.setData(item);
    newp.setNext(ptr);
    if(prev == null)
        head = newp;
    else
        prev.setNext(newp);
}
```

#### 6.1.4 Inserting into an Empty List

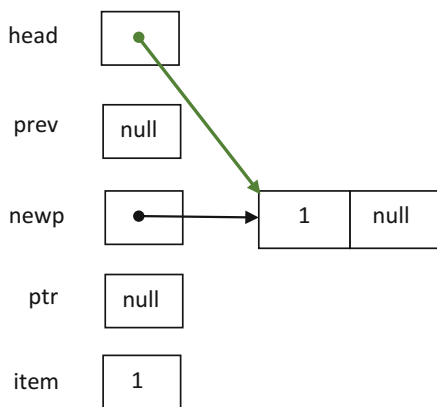
Three out of the four possibilities have been accounted for, so now it is time to consider the case of an empty list. If a list is empty, then `head` would be `null`. Walking through the code segment at the end of the last subsection, `ptr` is initialized to the value in `head` which is `null` and `prev` is initialized to `null` as shown below:



Since `ptr` is equal to `null` the condition in the `while` loop is short circuited, the body of the loop is not executed, and the execution continues on to the `if` statement. In the then section of the `if` statement, since `ptr` is equal to `null`, a new node is created containing the 1 as shown below:



Although the next section of the new node already contains a `null` by the constructor, the `null` of `ptr` is copied into the new node indicating it is the end of the list and since `ptr == null`, the reference in `newp` is copied into `head` indicating it is the beginning of the list as shown below:



This is an interesting feature of creating the first node in a linked list in that it is both the first and last node in the list and the code created for both these scenarios is utilized. Further, note that there were no changes needed to the existing code to make this scenario work. Again, it is helpful to not start writing code for the first node in the list, because often the code previously written for the other scenarios works without many or any alterations.

### 6.1.5 The Insert Method

Before completing the `insert` method, it is important to check to ensure that all four scenarios work with the algorithm before implementing it. The reason is that when code is added to accommodate one of the new scenarios, it is possible that those changes might introduce new errors to the previous existing scenarios. Once one is sure that no new errors were introduced, the only thing left to add to the algorithm is a `boolean` flag called `inserted` to indicate to the invoking program whether the item has indeed been inserted into the list or not. Also, the declaration of local variables needs to be included along with the method heading that includes the parameter and a `return` statement at the end. The completed `insert` method is as shown in Fig. 6.2.

```
public boolean insert(int item) {
    boolean inserted;
    Node ptr, prev;
    inserted = false;
    ptr = head;
    prev = null;
    while(ptr != null && ptr.getData() < item) {
        prev = ptr;
        ptr = ptr.getNext();
    }
    if (ptr == null || ptr.getData() != item) {
        inserted = true;
        Node newp = new Node();
        newp.setData(item);
        newp.setNext(ptr);
        if(prev == null)
            head = newp;
        else
            prev.setNext(newp);
    }
    return inserted;
}
```

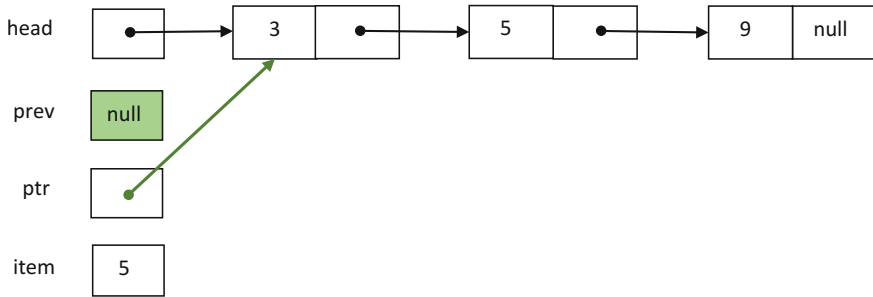
**Fig. 6.2** insert method

---

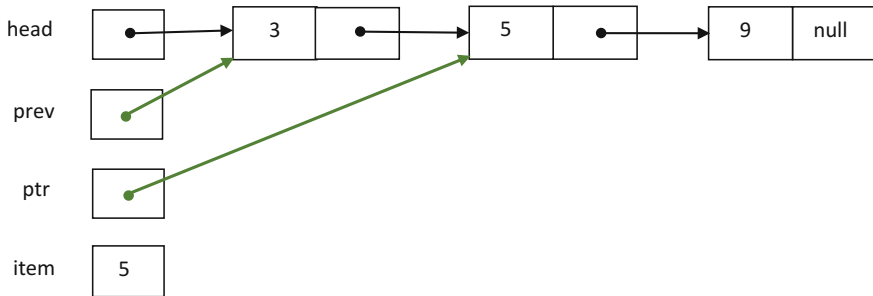
## 6.2 Creating the Delete Method

Having created the complete `insert` method in the previous section, it is now time to examine what is needed to create the `delete` method. However, instead of actually developing the code for the `delete` method, this section will look at what is needed to create the method. One can then use the same steps outlined in the previous section to write the necessary code as an exercise. Of course there will be differences between the two segments since they perform different tasks, but the techniques involved are the same.

As always, it helps to draw pictures to visualize what needs to be done and also as suggested in the previous section, it helps to start with the generic case before proceeding to the more specialized cases. Again, assuming that the list already exists and that the node containing the number 5 needs to be deleted, the list will need to be searched. Initially `ptr` will point to the same node as `head`, and `prev` will be pointing to `null`, as shown below:

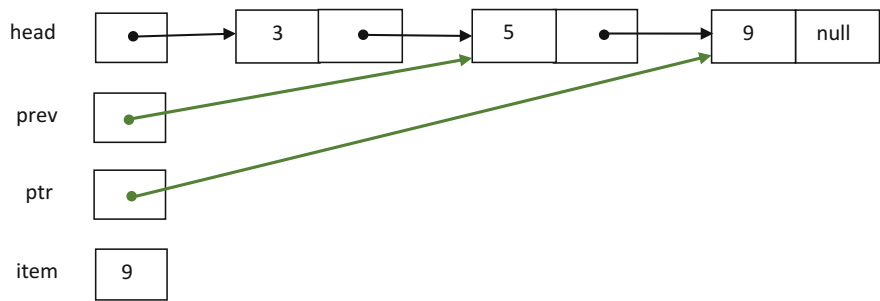


Once the node is found, `ptr` should be referencing the node to be deleted, and `prev` is referencing the node prior to the node to be deleted. Whereas the `insert` method would not insert a number that was already in the list, the `delete` method must be careful not to delete a number that is not in the list. As before, if the node is not found, the loop should not iterate to the end of the list but rather stop where the number would have been located. Also as with the `insert` method, one must make sure that conditional statements short circuit properly.

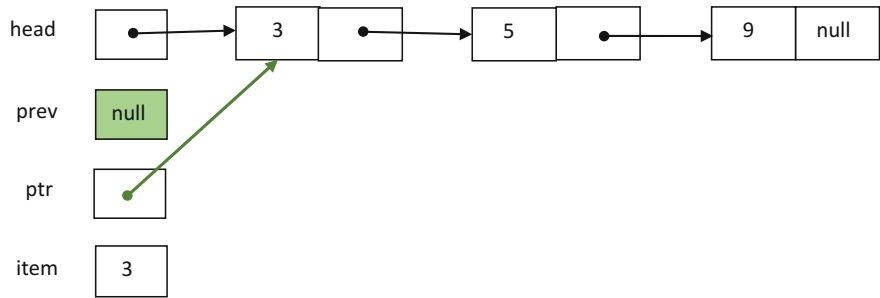


When writing the code for the actual deletion of the node, an `if` statement is probably needed to insure that `ptr` contains a reference and that the node containing the actual number to be deleted exists. Also, be sure to follow the heuristics given in the last section, such as the shortest route is often the best one and that it is usually better to create new links before destroying old links. Further, when there are two routes, make sure that the more general route is followed that can be adapted to other situations, such as using `prev` instead of `head` when trying to delete in the middle of a short list.

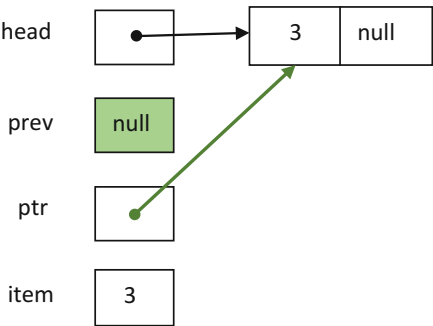
As before, it is a good idea to then work on deleting from the end of the list so as to defer having to deal with `head`. In the following figure it is the 9 that should be deleted:



Only after the middle and end have been taken care of is it a good idea to consider removing the first node at the beginning of a list:



And then to consider a list with only one item in it as follows:



As before, after adding code for the subsequent possibilities, it is important to go back and ensure that any subsequent additions and changes to the code have not altered the functionality of the previously written code. Lastly, the item to be deleted needs to be sent to the method as a parameter, and it is helpful to return a `boolean` value (with a default value of `false`) indicating whether the item has actually been deleted as shown in the following skeleton:

```
public boolean delete(int item) {  
    boolean deleted = false;  
    Node ptr, prev;  
  
    //Body of method  
  
    return deleted;  
}
```

---

### 6.3 The `LinkedList` Class

Using the `Node` class from Fig. 5.1, and creating a constructor, including the `insert` method from Fig. 6.2, the `delete` method skeleton from the previous section, the recursive print method from Fig. 5.5 renamed `printR`, along with a print driver called `printRecursive` below, results in the `LinkedList` class as shown in Fig. 6.3.

---

### 6.4 Test Program

Once the `LinkedList` class and the associated `Node` class are ready, they need to be tested. One wants to make sure that the `insert` and `delete` methods work flawlessly. This means that a node can be inserted in the middle, end, beginning, and into an empty list. The same goes for deleting in the middle, end, beginning, or from a list with only one node. Of course if there are not to be duplicates, then if an item is already in the list it should not be inserted, and of course if an item is not in the list, it should not be deleted. The following test program prompts for and inputs a command to either insert, delete, print, and stop using a sentinel control loop. For insertion or deletion, the program prompts the user for the number to be inserted or deleted, respectively. If the item is successfully inserted, not inserted, deleted, or not deleted, the appropriate message is output. The print command outputs the contents of the list to the screen, and a stop command causes the program to stop.



```
public class LinkedList {

    private Node head;

    public LinkedList() {
        head = null;
    }

    public boolean insert(int item) {
        boolean inserted;
        Node ptr, prev;
        inserted = false;
        ptr = head;
        prev = null;
        while(ptr != null && ptr.getData() < item) {
            prev = ptr;
            ptr = ptr.getNext();
        }
        if(ptr == null || ptr.getData() != item) {
            inserted = true;
            Node newp = new Node();
            newp.setData(item);
            newp.setNext(ptr);
            if(prev == null)
                head = newp;
            else
                prev.setNext(newp);
        }
        return inserted;
    }

    public boolean delete(int item) {
        boolean deleted=false;
        Node ptr, prev;
        // Body of method
        return deleted;
    }

    public void printRecursive() {
        System.out.print("List Recursive: ");
        printR(head);
        System.out.println();
    }

    private void printR(Node p) {
        if(p != null) {
            System.out.print(p.getData()+" ");
            printR(p.getNext());
        }
    }
}
```

**Fig. 6.3** LinkedList Class

```
import java.util.*;
class Ch5 {
    public static void main(String[] args) {

        Scanner scanner;
        scanner = new Scanner(System.in);

        String response;
        int number;

        LinkedList list;
        list = new LinkedList();

        System.out.println();
        System.out.print("Do you wish to i = insert, d = delete, ");
        System.out.print("p = print, or s = stop: ");
        response = scanner.next();
        while(!response.equals("s")) {
            if(response.equals("i")) {
                System.out.print("Enter a number to insert into the list: ");
                number = scanner.nextInt();
                System.out.print("The number " + number);
                if (list.insert(number))
                    System.out.println(" was inserted");
                else
                    System.out.println(" was not inserted");
            }
            else
                if(response.equals("d")) {
                    System.out.print("Enter a number to delete from the list: ");
                    number = scanner.nextInt();
                    System.out.print("The number " + number);
                    if(list.delete(number))
                        System.out.println(" was removed");
                    else
                        System.out.println(" was not removed");
                }
            }
            else
                if(response.equals("p"))
                    list.printRecursive();
                else {
                    System.out.print("The command entered is not i, d, p, ");
                    System.out.println("or s, please try again.");
                }
        }
        System.out.println();
    }
}
```

```
        System.out.print("Do you wish to i = insert, d = delete, ");
        System.out.print("p = print, or s = stop: ");
        response = scanner.next();
    }

    System.out.println();
    System.out.println("End of Program");
    System.out.println();

}
}
```

---

## 6.5 Doubly Linked Lists

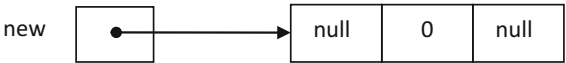
*Doubly linked lists* are like singly linked lists, except that in addition to a next reference they also have a back reference. The partial Node class for this type of list would appear as follows:

```
public class Node {
    private Node back;
    private int data;
    private Node next;
}
```

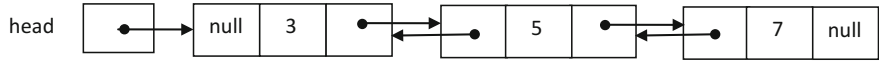
Or alternatively as follows:

```
public class Node {
    private int data;
    private Node back, next;
}
```

The order of `back` and `next` do not matter in memory, but the former one reflects more how a Node is drawn, and the second one reduces the number of keystrokes. Given the former, and the creation of a new node, the appearance would be as follows:

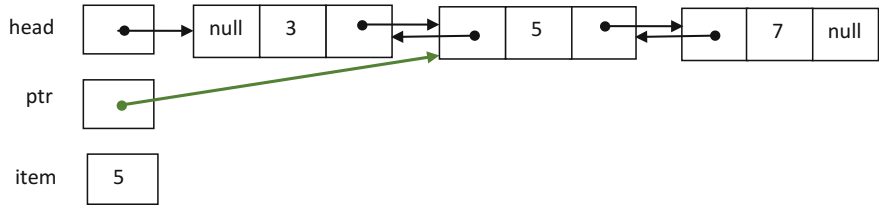


Further, the constructor in Fig. 5.1 would need to be altered to set back to null and both a setBack and getBack method would be needed to be added to the class. Assuming the existence of a doubly linked list, it could look as follows:



In looking at the doubly linked list above, what would be the advantage of a doubly linked list over a singly linked list? Given the existence of a back pointer for each node, it would be that a prev reference is not necessary and there could be fewer lines of code in the insert and delete methods. However, with every advantage there often seems to be a corresponding disadvantage. What appears to be the disadvantage of a doubly linked list over a singly linked list? In looking at the diagram it should be apparent that doubly linked lists require more memory than singly linked lists.

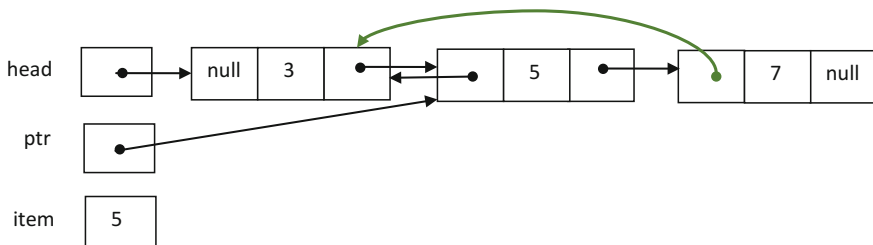
Reusing the preceding diagram to demonstrate how code might be written for a doubly linked list, assume that the number 5 is to be deleted. In this case ptr would refer to the node containing the 5 as shown below:



Given the drawing above, which references need to be altered? Only the back reference of the node containing the 7 needs to refer to the node containing the 3, and the next reference of the node containing the 3 needs to reference the node with the 7. Which reference should be altered first: the next reference of the node containing the 3, or the back reference of the node containing the 7? Although in this case either can be done first, one should always be careful not to delete a reference that might be needed later to complete the task at hand. In choosing one, the back reference of the node containing the 7 needs to refer to the node containing the 3, so the following instruction can accomplish this task:

```
ptr.getNext().setBack(ptr.getBack());
```

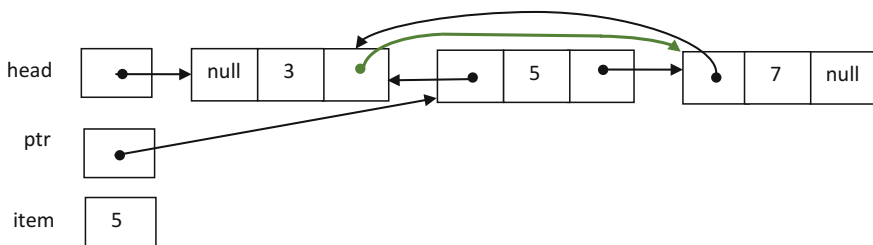
Through `ptr`, the above line of code first gets from `next`, in the node containing the 5, the reference to the node containing the 7. It then sets the `back` reference in the node containing the 7 to the same as the `back` reference of the node containing the 5 which is referencing the node containing the 3 as follows:



Similarly, the following instruction causes the `next` section of the node containing the 3 to reference the node containing the 7:

```
ptr.getBack().setNext(ptr.getNext());
```

Again through `ptr`, it first gets from `back`, in the node containing the 5, the reference to the node containing the 3. It then sets the `next` reference in the node containing the 3 to the same as the `next` reference of the node containing the 5 which is referencing the node containing the 7 as follows:



Do the `next` and `back` references for the node with the 5 need to be altered? No, because when `ptr` is `null`, the garbage collector will return the node with the 5 along with its two references back to the heap.

Given that each line of code might be more complex than with singly linked nodes, not having to deal with a `prev` reference makes for fewer lines of code. Further, even though only two lines of code are provided in the section for deleting from the middle of the list, the exercises at the end of this chapter provide further practice with doubly linked lists.

Although doubly linked lists provide an alternative way to handle lists, the use of two references in a node foreshadow the construction of more sophisticated data structures called binary trees as will be discussed in Chap. 8.

---

## 6.6 The Node Class as an Inner Class

The `LinkedList` class discussed in previous sections depends on an external `Node` class. However, when dealing with linked lists, there is no reason to have access to an individual node in the list from the main program. The result is that the `Node` class can be made to be a private inner class in the `LinkedList` class, so that linked list structures are contained in themselves. This would eliminate the need for accessor and mutator methods in the `Node` class, and programs introduced in later chapters implemented with nodes, such as binary trees, can also be simpler compared to ones using the external `Node` class.

The following `Node` class without an accessor or mutator can be added to the `LinkedList` class which will be renamed to `LinkedListInnerNode`.

```
private class Node {  
  
    private int data;  
    private Node next;  
  
    public Node() {  
        this(0);  
    }  
    public Node(int data) {  
        this.data = data;  
        next = null;  
    }  
}
```

When the `Node` class is an inner class, the `LinkedListInnerNode` class has direct access to the private data members of the `Node` class, `data` and `next`, without using public accessor and mutator methods. Therefore, the statements that include get methods such as `ptr.getData()` and `ptr.getNext()` can be replaced by `ptr.data` and `ptr.next` respectively using dot notation. And `newp.setData(item)` and `newp.setNext(ptr)` are changed to `newp.data = item` and `newp.next = ptr`. The complete `LinkedListInnerNode` class with the inner `Node` class is shown below:

```
public class LinkedListInnerNode {

    private class Node {

        private int data;
        private Node next;

        public Node() {
            this(0);
        }

        public Node(int data) {
            this.data = data;
            next = null;
        }
    }

    private Node head;

    public LinkedListInnerNode() {
        head = null;
    }

    public boolean insert(int item) {
        boolean inserted;
        Node ptr, prev;
        inserted = false;
        ptr = head;
        prev = null;
        while(ptr != null && ptr.data < item) {
            prev = ptr;
            ptr = ptr.next;
        }
        if (ptr == null || ptr.data == item) {
            inserted = true;
            Node newp = new Node();
            newp.data = item;
            newp.next = ptr;
            if (prev == null)
                head = newp;
            else
                prev.next = newp;
        }
        return inserted;
    }
}
```

```
public boolean delete(int item) {
    boolean deleted = false;
    Node ptr, prev;
    //Body of method
    return deleted;
}

public void printRecursive() {
    System.out.print("List Recursive: ");
    printR(head);
    System.out.println();
}

private void printR(Node p) {
    if(p != null) {
        System.out.print(data + " ");
        printR(p.next);
    }
}
}
```

The only change that needs to be made in the main program, which uses the `LinkedList` class, is to replace the declaration and creation of a linked list,

```
LinkedList list;
list = new LinkedList();
```

with the following code:

```
LinkedListInnerNode list;
list = new LinkedListInnerNode();
```

---

## 6.7 Complete Programs: List of User Defined Objects

In this section, an application which keeps track of students' information is considered. What is the best way to implement this application? Some of the common operations applied on the data include addition, deletion, and search. Keeping the data in an organized fashion is good to help facilitate retrieval. Therefore, the



answer to the question is to keep the information in an ordered list. As discussed before, a list can be implemented using arrays or linked lists. Here a linked list is used. Since the items stored in the list are not simple primitive data types, a `LinkedList` class which accepts generic types is developed so that the list can store any type of items. The class that defines the node containing an object could be a separate class from the `LinkedList` class, which will be described in Sect. 6.7.1, or an inner class of the `LinkedList` class discussed in Sect. 6.7.2.

### 6.7.1 Generic `LinkedList` Class with External `NodeGeneric` Class

To define linked lists which keep any user defined objects, the type of data in the node of the list should be a generic type so that a new `LinkedList` class does not have to be written every time items of different types are stored. The `NodeGeneric<T>` class has already been implemented in Fig. 5.6, which will be used with a generic linked list to be developed next.

As discussed in Sect. 4.5, the type `int` in the `LinkedList` class shown in Fig. 6.3 will be replaced by the type parameter `T`. The reference type `Node` will be changed to the generic type `NodeGeneric<T>`. Similar to the `ListArrayGeneric` class, the `LinkedListGeneric` class needs to compare items to determine where the new item should be inserted in the list, and the comparison `ptr.getData() < item` in Fig. 6.3 does not work.

In the generic class, this will compare the references to the objects, not the contents. Just as in the `ListArrayGeneric` class, the `compareTo` method that is included in the `Comparable` interface is used to compare two objects. How to compare two objects differs from one object to an other, therefore actual implementation of the `compareTo` method would be in the class that defines the objects. Also, since the condition `ptr.getData() != item` in Fig. 6.3 does not work with objects, the `equals` method will be implemented. The complete `LinkedListGeneric` class which uses `NodeGeneric<T>` class is shown in Fig. 6.4, and the definitions of the methods `compareTo` and `equals` are discussed next.

Since the items stored in the list for this application consist of student information such as name, ID number, and major, a class which defines student objects will be implemented. There will be three data members describing a student: name and major of type `String`, and id of type `int`. For each data member, an accessor and a mutator are included. Because the class will implement the `compareTo` method which is included in the `Comparable` interface, the class will be named as `StudentComparable`. Because students are ordered by their id the `compareTo` method compares the id number of two students, and returns an integer. Specifically, it returns a positive integer, zero, or a negative integer if the id

```

public class LinkedListGeneric<T extends Comparable<T>> {

    private NodeGeneric<T> head;

    public LinkedListGeneric() {
        head = null;
    }

    public boolean insert(T item) {
        boolean inserted;
        NodeGeneric<T> ptr, prev;
        inserted = false;
        ptr = head;
        prev = null;
        while(ptr != null && ptr.getData().compareTo(item) < 0) {
            prev = ptr;
            ptr = ptr.getNext();
        }
        if(ptr == null || !(ptr.getData().equals(item))) {
            inserted = true;
            NodeGeneric<T> newp = new NodeGeneric();
            newp.setData(item);
            newp.setNext(ptr);
            if (prev == null)
                head = newp;
            else
                prev.setNext(newp);
        }
        return inserted;
    }

    public boolean delete(int item) {
        boolean deleted=false;
        Node ptr, prev;
        // Body of method
        return deleted;
    }

    public void printRecursive() {
        System.out.print("List Recursive: ");
        printR(head);
        System.out.println();
    }

    private void printR(Node p){
        if(p != null) {
            System.out.print(p.getData()+" ");
            printR(p.getNext());
        }
    }
}

```

**Fig. 6.4** LinkedListGeneric class

of the object that is calling the method is greater than, equal to, or less than the one of the specified object, respectively. This method will be used in the `insert` and `delete` methods. The code below is the `compareTo` method:

```
public int compareTo(StudentComparable otherStudent) {
    int result;
    if(id < otherStudent.getId())
        result = -1;
    else
        if(id > otherStudent.getId())
            result = 1;
        else
            result = 0;
    return result;
}
```

The `equals` method, which can be used in the `insert` and `delete` methods, is also implemented in the `StudentComparable` class. The `equals` method takes a reference to the object of type `Object` which means an object of any type to maintain the generic nature of the code. The object will be typecast into the `StudentComparable` type so that the `id` number of the object calling the method can be compared to the `id` of the object passed to the method as a parameter. As discussed in Chap. 2, the class `Object` is the root of the class hierarchy in Java, meaning the `Object` class is directly or indirectly a superclass of all the user defined and predefined classes. The definition of the `equals` method is shown below:

```
public boolean equals(Object otherStudent) {
    StudentComparable otherStudentObject
        = (StudentComparable)otherStudent;
    return (id == otherStudentObject.id);
}
```

Lastly, the `toString` method shown below is added at the end to return a string representation of the contents of the data members of an object. The method would return the `name`, `id`, and `major` of the student and would be written as follows:

```
public String toString() {  
    return ("Student named " + name + " with id " + id +  
           " and major " + major);  
}
```

Putting everything discussed above together, the complete `StudentComparable` class is shown below:

```
public class StudentComparable implements Comparable<StudentComparable> {  
  
    private String name, major;  
    private int id;  
  
    public StudentComparable(String name, String major, int id) {  
        this.name = name;  
        this.major = major;  
        this.id = id;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public String getMajor() {  
        return major;  
    }  
  
  
    public int getId() {  
        return id;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    public void setMajor(String major) {  
        this.major = major;  
    }  
  
    public void setId(int id) {  
        this.id = id;  
    }  
}
```

```

public int compareTo(StudentComparable otherStudent) {
    int result;
    if(id < otherStudent.getId())
        result = -1;
    else
        if(id > otherStudent.getId())
            result = 1;
        else
            result = 0;
    return result;
}

public boolean equals(Object otherStudent) {
    StudentComparable otherStudentObject
        = (StudentComparable)otherStudent;
    return (id == otherStudentObject.id);
}

public String toString() {
    return ("Student named " + name + " with id " + id +
        " and major " + major);
}
}

```

Now it is time to test three classes discussed above. A main program which creates an ordered linked list with 3 students is shown below.

Sample output from the above program is:

```

List:
Student named George with id 1212 and major CS
Student named Maya with id 1234 and major MIS
Student named Joffrey with id 3456 and major CS

```

where students are sorted by their id number.

### 6.7.2 Generic LinkedList Class with Internal NodeGeneric Class

In this section, a linked list with an inner node class is used to perform the same task discussed in the previous section. More specifically, the `NodeGeneric<T>` class in Fig. 5.6 without accessors and mutators is included in the `LinkedListGeneric<T>` class in Fig. 6.4. And it will be called the `LinkedListGenericInnerNode` class as shown below.

```
public class LinkedListGenericInnerNode<T extends Comparable<T>> {

    private class NodeGeneric<T>{

        private T data;
        private NodeGeneric<T> next;

        public NodeGeneric() {
            this(null);
        }

        public NodeGeneric(T data) {
            this.data = data;
            next = null;
        }
    }

    private NodeGeneric<T>head;

    public LinkedListGenericInnerNode() {
        head = null;
    }

    public boolean insert(T item) {
        boolean inserted;
        NodeGeneric<T>ptr,prev;
        inserted = false;
        ptr = head;
        prev = null;
        while(ptr != null && ptr.data.compareTo(item) < 0) {
            prev = ptr;
            ptr = ptr.next;
        }
        if(ptr == null || !(ptr.data.equals(item))) {
            inserted = true;
            NodeGeneric<T>newp = new NodeGeneric();
            newp.data = item;
            newp.next = ptr;
            if(prev == null)
                head = newp;
            else
                prev.next = newp;
        }
    }
}
```

```

        return inserted;
    }

    public boolean delete(T item) {
        boolean deleted;
        NodeGeneric<T>ptr, prev;
        return deleted;
        //Body of method
    }

    public void printRecursive() {
        System.out.print("List Recursive: ");
        printR(head);
        System.out.println();
    }

    private void printR(NodeGeneric<T> p) {
        if (p != null) {
            System.out.print(p.data + " ");
            printR(p.next);
        }
    }
}

```

The `studentComparable` class discussed in the previous section can be used here. The only change that needs to be made in the main program in Fig. 6.5 is to modify the declaration and creation of a linked list. Their type is now `LinkedListGenericInnerNode` instead of `LinkedListGeneric` shown below:

```

public class TestLinkedListGenericStudentComparable {

    public static void main(String[] args) {

        LinkedListGenericInnerNode<StudentComparable>list
            = new LinkedListGenericInnerNode<StudentComparable>();
        LinkedListGenericInnerNode<StudentComparable>ptr;

        StudentComparable student1
            = new StudentComparable("George", "MIS", 1234);
        StudentComparable student2
            = new StudentComparable("Maya", "CS", 1212);
        StudentComparable student3
            = new StudentComparable("Joffrey", "CS", 3456);
    }
}

```

```
list.insert(student1);
list.insert(student2);
list.insert(student3);

list.printRecursive();
}
}
```

---

## 6.8 Summary

- Drawing pictures can be extremely helpful when trying to write or debug code for linked lists.
- Ordered linked lists have each node in a prescribed order such as ascending order for integers and alphabetical order for strings.
- It is often helpful to write code for the generic case first and then proceed to write code for the other cases, such as inserting or deleting from the beginning or end of a list.
- Doubly linked lists have the advantage that they do not need an extra variable referencing the previous node in the list, but the disadvantage is that each node takes up more memory referencing the previous node.

```
public class TestLinkedListGenericStudentComparable {

    public static void main(String[] args) {

        LinkedListGeneric<StudentComparable> list
            = new LinkedListGeneric<StudentComparable>();
        LinkedListGeneric<StudentComparable> ptr;

        StudentComparable student1
            = new StudentComparable("George", "MIS", 1234);
        StudentComparable student2
            = new StudentComparable("Maya", "CS", 1212);
        StudentComparable student3
            = new StudentComparable("Joffrey", "CS", 3456);

        list.insert(student1);
        list.insert(student2);
        list.insert(student3);

        list.printRecursive();
    }
}
```

**Fig. 6.5** Main program using the generic `LinkedList` class



- Doubly linked lists require fewer lines of code, but each line of code tends to be more complicated.
- The use of inner class can eliminate the need for accessor and mutator methods, especially when dealing with more complex data structures.

---

## 6.9 Exercises (Items Marked with an \* Have Solutions in Appendix B)

- \*1. Assuming the list `intList` of type `LinkedList` in Fig. 6.3 is initially empty, draw a list after walking through the following code segment:

```
intList.insert(41);
intList.insert(6);
intList.delete(5);
intList.delete(6);
intList.insert(78);
intList.insert(5);
intList.insert(79);
intList.delete(41);
intList.insert(5);
```

2. Assuming a list `characterList` of type `LinkedListGeneric<Character>` which is an object of the generic class `LinkedListGeneric<T>` in Fig. 6.4 is initially empty, draw a list after walking through the following code segment:

```
characterList.insert('U');
characterList.insert('V');
characterList.delete('U');
characterList.insert('W');
characterList.delete('A');
characterList.insert('P');
characterList.insert('Q');
characterList.delete('P');
characterList.insert('C');
```

3. Write a method `addToTail` that can be added to the `LinkedList` class in Fig. 6.3 which adds a node at the end of the list. Assume that the existence of a variable `tail` which reference the last node in a linked list.
- \*4. Write a method `deleteTail` that can be added to the `LinkedList` class in Fig. 6.3 which removes the last node from the list. The method does not accept anything and returns `true` if the list contained at least one node and `false` if the list was empty.
5. Write a method `delete` discussed in Sect. 6.2 that can be added to the `LinkedList` class in Fig. 6.3.
6. Write a method `size` that can be added to the `LinkedList` class in Fig. 6.3 which returns the number of nodes in the list. What other changes need to be made to the other methods in order to make this method work better?
- \*7. Write a method `isEmpty` that can be added to the `LinkedList` class in Fig. 6.3 which returns `true` if the list is empty.
8. Write a method `clear` that can be added to the `LinkedList` class in Fig. 6.3 which clears an existing list by making it an empty list.
9. Develop a class `LinkedListDuplicate` derived from the `LinkedList` class in Fig. 6.3 which accepts duplicates. The new class should contain all the methods in the `LinkedList` class. The `delete` method should delete all occurrences of an item from the list.
10. Draw a list after the complete program in Fig. 6.5 is executed.
11. Rewrite the `LinkedList` class in Fig. 6.3 for a doubly linked list.
12. Write a method `copy` that can be added to the class implemented for question 11. The method accepts a doubly linked list and makes a duplicate of it.
13. Write a program to find a difference of two given sets of integers stored in a linked list using `LinkedListGeneric<T>` in Fig. 6.4. For example, if two integer sets are  $P = \{4, 7, 1, 10\}$  and  $Q = \{4, 2, 5, 1, 3\}$ , the differences of these two sets are  $P - Q = \{7, 10\}$  and  $Q - P = \{2, 5, 3\}$ .
14. Implement an unordered `LinkList` class.

As discussed previously, the concept that programs can be written using different algorithms to accomplish the same task, such as using iteration or recursion, is known as procedural abstraction. Similarly with data structures, the concept that when two different techniques can be used to represent a structure, such as using an array or references, this is known as data abstraction. In addition to lists, this concept of data abstraction can be extended to stacks and queues which were implemented using arrays in Chaps. 2 and 3, and will be implemented using references in this chapter. Since the basic concepts of stacks and queues were discussed previously, this chapter will concentrate on the implementation using references.

---

## 7.1 Stack

As should be recalled from Chap. 2, a stack is a LIFO (last-in, first-out) structure. Further, there were four methods developed for the class: `push`, `pop`, `empty`, and `full` plus a constructor. In this current section the constructor along with the `push` method will be developed leaving the `pop` and `empty` methods as exercises. Note that the `full` method is not included as will be discussed below. Assuming the existence of the `Node` class from Fig. 5.1, the basic skeleton for the class `StackRef` would look as follows:

```
public class StackRef {  
    private Node top;
```

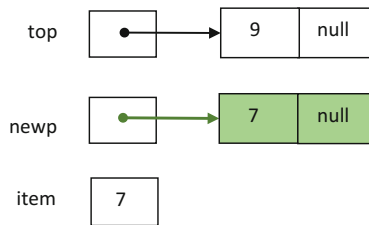
```
public StackRef() {  
    top = null;  
}  
}
```

This should look familiar in that it is similar to the basic structure of the `LinkedList` class except that instead of the private variable `head`, the variable `top` is used. Further, note that the constructor initializes `top` to `null` indicating an empty stack.

As in the last chapter, it might be helpful to start again with the generic case when pushing an item onto the stack. Assume that the stack already contains the number 9 and the number 7 will be pushed onto the stack. The first thing that needs to be done is to create a new node using the statement:

```
newp = new Node(item);
```

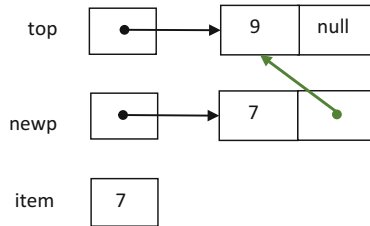
and as shown in the following drawing:



Then before altering the `top` reference, recall the heuristic from the previous chapter that it is usually better to create a new reference before altering any existing references. So the `next` part of the node referenced by `newp` needs to reference the node referenced by `top` using the following statement:

```
newp.setNext(top);
```

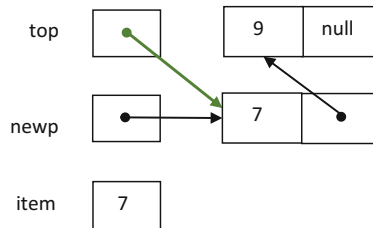
as shown in the following drawing:



Now that the new node is linked to the rest of the items in the stack, `top` can be changed to reference the new node as the top of the stack using the following:

```
top = newp;
```

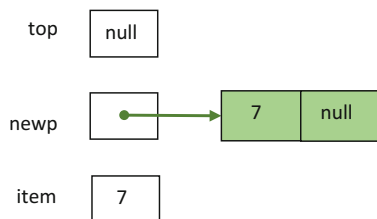
and as illustrated below:



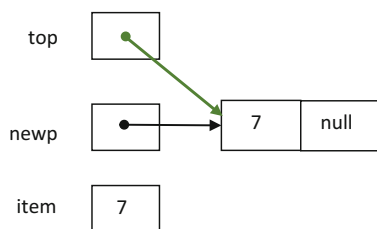
Putting the three lines together in the method `push` results in the following which can be added to `StackRef` class:

```
public void push(int item) {
    newp = new Node(item);
    newp.setNext(top);
    top = newp;
}
```

Will the above method work with an empty stack? Executing the first instruction in the method would result in the following:



Then executing the second instruction would copy the `null` reference in `top` to the next portion of the node referenced by `newp` which would look the same as the previous drawing because the constructor had already initialized the next portion to `null`. Lastly the third statement would cause `top` to refer to the same node as `newp` resulting in the following:



The result is correct which indicates the same code segment could be used for both an empty stack or a stack already containing items. Although the method could have probably been written for the empty stack first, this reinforces the technique used in the previous chapter.

Does one need to worry about whether the stack is full as is done with a stack implemented with arrays? Technically, it is possible that there are no more nodes available on the heap. However, since the stack here is only going to contain a small number of items, this should not be a concern. However, though the `full` method is not used for the `push` method, the `empty` method is most certainly needed when creating the `pop` method.

The partially complete class for `StackRef` is given in Fig. 7.1, Note that only the skeletons for the `pop` and `empty` methods are given, and they are left as exercises.

Again as mentioned earlier, the `StackRef` class in Fig. 7.1 assumes the existence of the `Node` class in Fig. 5.1.

```
public class StackRef {  
  
    private Node top;  
  
    public StackRef() {  
        top = null;  
    }  
  
    public void push(int item) {  
        Node newp = new Node(item);  
        newp.setNext(top);  
        top = newp;  
    }  
  
    public int pop() {  
        int item=-1;  
        // insert code  
    }  
  
    public boolean empty() {  
        // insert code  
    }  
}
```

**Fig. 7.1** StackRef class

---

## 7.2 Stack Test Program: Reversing Integers

The beauty of data abstraction is that data structures can be implemented in classes using more than one way of storing the data and when using the class, one does not need to know the details. As a result, one should be able to use the program from Sect. 2.4 with very few changes. In fact, there are only three changes required to the main program. Two of these are that the references to the `StackArray` class need to be changed to `StackRef` as shown in the fifth and sixth lines below. Also, since there is theoretically no upper bound to the size of the stack, it is not necessary to check and the `&& !stack.full()` can be removed from the first while statement as shown below.

```
import java.util.*;  
class Reverse {  
    public static void main (String [] args){  
  
        int number;  
        StackRef stack;  
        stack = new StackRef();
```

```
Scanner scanner = new Scanner(System.in);

System.out.println();
System.out.print("Enter an integer or a -1 to stop: ");
number = scanner.nextInt();
while(number >= 0) {
    stack.push(number);
    System.out.print("Enter an integer or a -1 to stop: ");
    number = scanner.nextInt();
}

System.out.println();
System.out.print("The reverse integers are: ");
while(!stack.empty())
    System.out.print(stack.pop() + " ");
System.out.println();
System.out.println();
}
}
```

When running the program above with the new `StackRef` class, one will get similar results as before, without the limitation of the stack size as shown below:

```
Enter an integer or a -1 to stop: 1
Enter an integer or a -1 to stop: 2
Enter an integer or a -1 to stop: 3
Enter an integer or a -1 to stop: 4
Enter an integer or a -1 to stop: -1

The reverse integers are: 4 3 2 1
```

---

## 7.3 Queues

As with the `StackRef` class, the `QueueRef` class can implement the queue data structure using references. Recall that a queue is a FIFO (First In First Out) structure. Further, instead of a single variable `top`, two variables are needed, `front` and `rear`, which will be initialized to `null` in the constructor. In this



section, the enqueue method will be developed, and the dequeue and empty methods will be left as exercises. The basic skeleton class for the QueueRef class is as follows:

```
public class QueueRef {

    private Node front, rear;

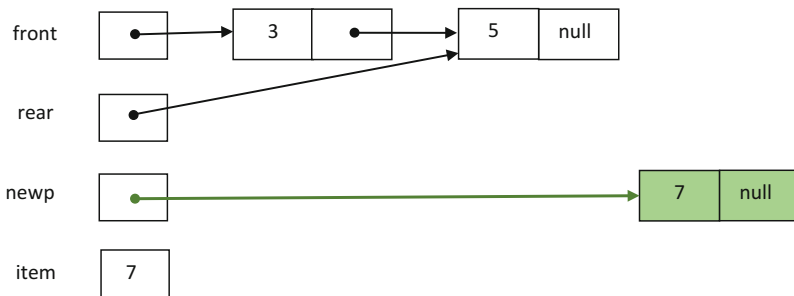
    public QueueRef() {
        front = null;
        rear = null;
    }

}
```

Starting again with a generic case, with 3 and 5 in the queue and ready to enqueue a 7, one would need to create a new node containing the item to be enqueued:

```
newp = new Node(item);
```

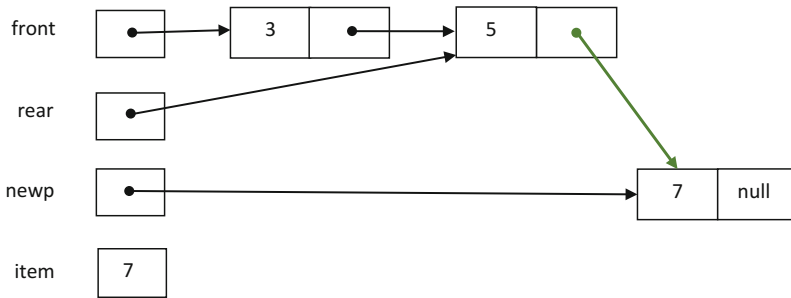
as illustrate below:



Note that the new node is drawn to the right to help facilitate the drawing in subsequent steps. Then the new item would be placed at the end of the queue by altering the next portion of the node referenced by `rear` to reference the same node referenced by `newp`:

```
rear.setNext(newp);
```

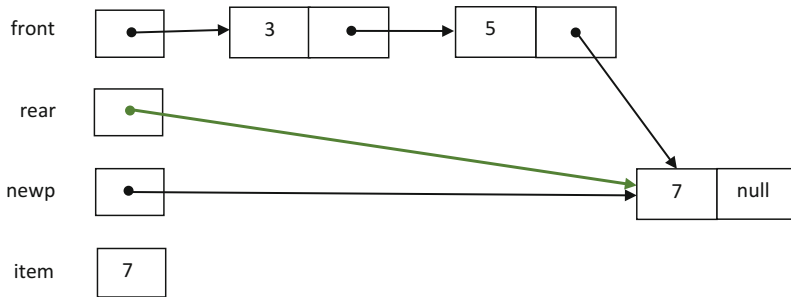
as shown below:



As before, note that the new reference is created before moving any other references. Now, the **rear** reference can be changed to reference the same node as **newp**:

```
rear = newp;
```

as illustrated below:

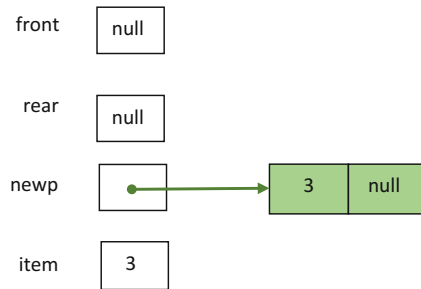


Putting these individual lines of code together creates the beginning of the **enqueue** method:

```
public void enqueue(int item) {
    newp = new Node(item);
    rear.setNext(newp);
    rear = newp;
}
```

As before, is a **full** method necessary? In this case there is no plan to enqueue very many items, so it is unlikely that the heap will run out of nodes. Similar to the stack discussed previously, does this code work with an empty queue? After

executing the first line of the method above, the following represents the state of execution:



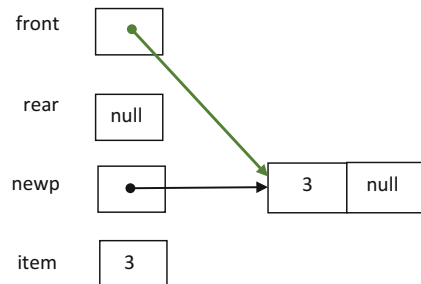
However, when trying to execute the second line of code, there would be an exception because `rear` is equal to `null`. As a result, an `if` statement needs to be inserted to ensure that `rear` is not equal to `null` before attempting to modify the node it is referencing.

```
if(rear != null)
    rear.setNext(newp);
```

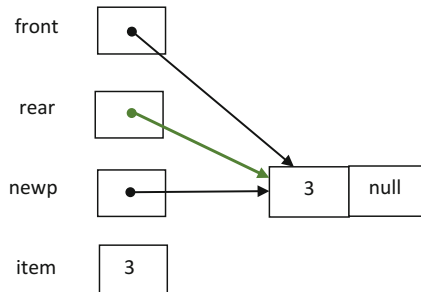
But if `rear` is equal to `null`, what should be modified? Instead of modifying the last node in the queue, `front` will need to reference the new node using the following code:

```
if(rear != null)
    rear.setNext(newp);
else
    front = newp;
```

and as illustrated below:



Then the last line in the method can be executed with `rear` also referencing the new node as shown below:



The result is that when inserting a node into an empty queue, it will be both the first and last node in the queue as can be seen with both `front` and `rear` referencing the same node. The new modified method for `insert` is given below:

```
public void enqueue(int item) {
    newp = new Node(item);
    if(rear != null)
        rear.setNext(newp);
    else
        front = newp;
    rear = newp;
}
```

The partially complete class for `QueueRef` is given in Fig. 7.2. Note that only the skeletons for the `dequeue` and `empty` methods are given and they are left as exercises.

As mentioned earlier with the `StackRef` class, the class in Fig. 7.2 assumes the existence of the `Node` class from Fig. 5.1.

```
public class QueueRef {  
  
    private Node front,rear;  
  
    public QueueRef(){  
        front = null;  
        rear = null;  
    }  
  
    public void enqueue(int item) {  
        Node newp = new Node(item);  
        if(rear != null)  
            rear.setNext(newp);  
        else  
            front = newp;  
        rear = newp;  
    }  
  
    public int dequeue() {  
        int item=-1;  
        // insert code  
    }  
  
    public boolean empty() {  
        // insert code  
    }  
  
}
```

**Fig. 7.2** QueueRef class

---

## 7.4 Queue Test Program

As with stacks, queues can exhibit data abstraction, and the QueueArray class can be replaced with the QueueRef class. Using the program from Sect. 3.4 and assuming the existence of the Node class from Sect. 5.1, only two changes need to be made to the program. These are changing the class name QueueArray to QueueRef in the sixth and seventh lines as shown in the code below:

```
import java.util.*;  
public class TestQueueRef {  
    public static void main(String args[]) {  
        int item;  
        char selection;  
        QueueRef queue;  
        queue = new QueueRef();  
        Scanner scanner = new Scanner(System.in);
```

```

System.out.println();
System.out.print("Please enter E for Enqueue, " +
"D for Dequeue, or S for Stop: ");
selection = scanner.next().charAt(0);
while(selection != 'S' && selection != 's') {
    if(selection == 'E' || selection == 'e') {
        System.out.print("Please enter an integer to be " +
"enqueued: ");
        item = scanner.nextInt();
        queue.enqueue(item);
    }
    else
        if(selection == 'D' || selection == 'd') {
            item = queue.dequeue();
            if(item != -1)
                System.out.println("The item dequeued is: " + item);
        }
    else
        System.out.println("The selection entered " + selection
+ " is not E, D, or S, please try again");
System.out.println();
System.out.print("Please enter E for Enqueue, " +
"D for Dequeue, or S for Stop: ");
selection = scanner.next().charAt(0);
}
System.out.println();
System.out.println("End of Program");
System.out.println();
}
}

```

When running the program above with the new `QueueRef` class, one will get similar results as before, without the limitation of the queue size. Looking back at program in Sect. 3.4 which used the `QueueArray` class, note that the number 6 could not be enqueued without increasing the size of `qarray`; however using the `QueueRef` class there is no such restriction and note that number 6 is enqueued and dequeued below:

```

Please enter E for Enqueue, D for Dequeue, or S for Stop: d
Queue is empty, item not removed

```

```

Please enter E for Enqueue, D for Dequeue, or S for Stop: e
Please enter an integer to be enqueued: 1

```

Please enter E for Enqueue, D for Dequeue, or S for Stop: e  
Please enter an integer to be enqueued: 2

Please enter E for Enqueue, D for Dequeue, or S for Stop: e  
Please enter an integer to be enqueued: 3

Please enter E for Enqueue, D for Dequeue, or S for Stop: e  
Please enter an integer to be enqueued: 4

Please enter E for Enqueue, D for Dequeue, or S for Stop: d  
The item dequeued is: 1

Please enter E for Enqueue, D for Dequeue, or S for Stop: e  
Please enter an integer to be enqueued: 5

Please enter E for Enqueue, D for Dequeue, or S for Stop: e  
Please enter an integer to be enqueued: 6

Please enter E for Enqueue, D for Dequeue, or S for Stop: d  
The item dequeued is: 2

Please enter E for Enqueue, D for Dequeue, or S for Stop: d  
The item dequeued is: 3

Please enter E for Enqueue, D for Dequeue, or S for Stop: d  
The item dequeued is: 4

Please enter E for Enqueue, D for Dequeue, or S for Stop: d  
The item dequeued is: 5

Please enter E for Enqueue, D for Dequeue, or S for Stop: d  
The item dequeued is: 6

Please enter E for Enqueue, D for Dequeue, or S for Stop: d  
Queue is empty, item not removed

Please enter E for Enqueue, D for Dequeue, or S for Stop: s

End of Program

## 7.5 Comparison of Arrays Verses References

Which should be used for stacks and queues: arrays or references? The answer depends on the application. Given an equal number of items in a stack, which would take up more memory? The answer is the one with references because for each item in the stack, there needs to be an associated memory location referencing the next item in the stack. The same applies to queues. If this is the case, why would one want a stack or queue implemented using references? Recall from Chap. 2 that when using an array, it needs to be declared as a fixed size and there is always the possibility that the stack or queue could be declared too small and there could be an out of bounds exception. So this is an advantage to using a stack or queue implemented with references. However, one might also recall that code could be implemented that when a stack became full, a new larger array could be allocated and all the items in the old array could be copied into the new array and processing could continue. However, the problem with this solution is that the allocation and copying of the contents of an array would take time and slow down the algorithm.

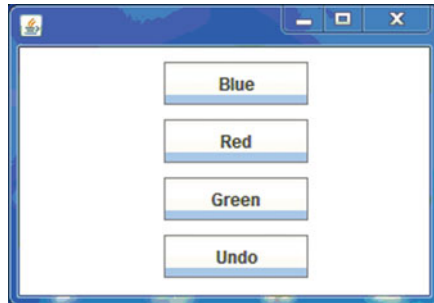
The result is that if the maximum number of items that might be stored in a stack or queue is known in advance, then using an array based data structure would probably be the best choice due to the savings in memory. But if the number of items is not known in advance and could vary by quite a bit during each execution of the program, then a data structure implemented using references might be the better choice. So although data abstraction might hide the details of how a data structure is implemented, it is sometimes advantageous for a user of a class to understand how it is implemented to utilize the best one for the task at hand.

---

## 7.6 Complete Program: Undo Button

Consider the common feature “undo” which is a part of most applications software. Using the undo command, previous actions can be retrieved. It is very useful when an unwanted deletion or an unexpected modification has occurred while using software such as text editors. How can the undo feature be implemented in a program? To retrieve the actions already taken, they have to be stored in some way. Which data structure is appropriate to keep track of the history? When the undo is applied, the last action will be brought back. If the undo is applied again consecutively, the second last action needs to be recovered. So, as can be seen in the pattern, the answer to the question is a stack. In this section, buttons which change the background color of a small window and an undo button that retrieves the previous colors will be implemented with a stack using references as shown below.





### 7.6.1 Graphical User Interface

The programs developed so far use standard input and output. The interface with `System.in` and `System.out` is convenient and sufficient for basic programs. However, if a more user friendly interface is required a Graphical User Interface (GUI) can be employed. In Java, GUI is implemented using the standard classes defined in the `javax.swing` and `java.awt` packages. In order to build an effective GUI with objects from these predefined classes, a different style of program called event-driven programming is introduced. An event occurs when the user interacts with a GUI object. For example, when a user clicks a button or selects a menu option, an event occurs. In event-driven programs, objects that respond to these events are defined in an event-handling method. Next the basics of event-driven programming, which implements buttons and events when buttons are clicked, are discussed. For more information about the `javax.swing` and `java.awt` packages, refer to the References and Useful Websites section at the end of the text which has the link to the Java API specification document at the Oracle website.

### 7.6.2 User Defined Frame

To create an interactive user interface, a window is used where buttons are placed. A window is an instance of the `JFrame` class and the `JFrame` class contains minimum functionalities to support features found in any frames. Therefore, in order to design a window which performs customized meaningful tasks, a subclass of the `JFrame` class is declared where methods and data members are added to implement the necessary functionalities.

For example, using the `setTitle` method, the frame's title can be set. The `setSize` method can be used to set the frame's size, the `setLocation` method to position the frame's top left corner to coordinate (x, y), and `setBounds` to specify the size and the location of the frame. To terminate the program when the

frame is closed, the `setDefaultCloseOperation` method with the class constant `EXIT_ON_CLOSE` as an argument can be used. In this section only the `setSize` and the `setDefaultCloseOperation` methods are used as shown below:

```
setSize(300, 210);
setDefaultCloseOperation(EXIT_ON_CLOSE);
```

In the above, the size of the frame is 300 pixels wide and 210 pixels high and the program terminates when the Close box located at the top right corner of the window is clicked. These properties are set inside the default constructor of the user defined class which will be called `UndoButton`. A stack with objects and references, which is one of the main topics in this chapter, is used to implement an undo button. A main method will be included in this class so that a separate main class will not have to be defined. Here is the `UndoButton` class:

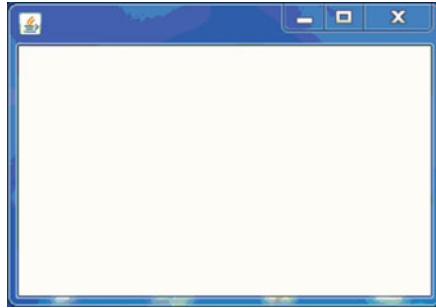
```
import javax.swing.*;

class UndoButton extends JFrame {
    public UndoButton() {
        // set the frame size
        setSize(300, 210);

        // exit upon closing the frame
        setDefaultCloseOperation(EXIT_ON_CLOSE);
    }

    public static void main(String[] args) {
        UndoButton frame = new UndoButton();
        frame.setVisible(true);
    }
}
```

In the main method, an instance of `UndoButton` class is declared and created. An argument `true` is sent to the `setVisible` method to have the frame appear on the screen. When the above program is executed, the following window appears at the top left corner of the screen. If the different location is desired, the `setLocation` method can be used to specify the placement of the window.



### 7.6.3 Placing Buttons

There are two key aspects involved in GUI programming. One is the placement of GUI objects on the content pane of a frame, and the other is the handling of events generated by these GUI objects. In this section, a customized user interface is defined to show how four buttons labeled Blue, Red, Green and Undo are placed on the content pane of the frame. Then in the next section, how the button events are processed to change the background color and to undo the background color will be explained.

To display buttons and change the background color, the content pane of a frame needs to be accessed. A content pane is the area of the frame where the content such as buttons and images are displayed. It excludes the title, a menu bar, and the border. The content pane of a frame is accessed by calling the frame's `getContentPane()` method as shown below:

```
Container contentPane = getContentPane();
```

To deal with buttons, instances of the `javax.swing.JButton` class are created. The button with a label "Blue" can be declared and created in the following manner:

```
JButton blue;  
blue = new JButton("Blue");
```

The text passed as an argument to the constructor is the label of a button. In the complete program four buttons are placed by specifying their position and size explicitly on the content pane. This approach is called absolute positioning. Since a

layout manager for a container, which is an object that controls the placement of the GUI objects, is not used here, an argument `null` is sent to the method `setLayout`.

```
contentPane.setLayout(null);
```

The position and the size of the buttons are set using the `setBounds` method. For example, the Blue button is located 100 pixels to the right and 10 pixels down from the top left corner and the size is 100 pixels wide by 30 pixels high as follows:

```
blue.setBounds(100, 10, 100, 30);
```

To be able to see a button in the window, it has to be added to the content pane as in:

```
getContentPane().add(blue);
```

Here is the program to display four buttons in a frame.

```
import javax.swing.*;
import java.awt.*;

class UndoButton extends JFrame {
    private JButton blue, red, green, undo;

    public static void main(String[] args) {
        UndoButton frame = new UndoButton();
        frame.setVisible(true);
    }

    public UndoButton() {
        Container contentPane = getContentPane();

        // set the frame size
        setSize(300, 210);

        // set the content pane properties
        contentPane.setLayout(null);

        // create and place four buttons
        // on the frame's content pane
        blue = new JButton("Blue");
        blue.setBounds(100, 10, 100, 30);
        getContentPane().add(blue);
```

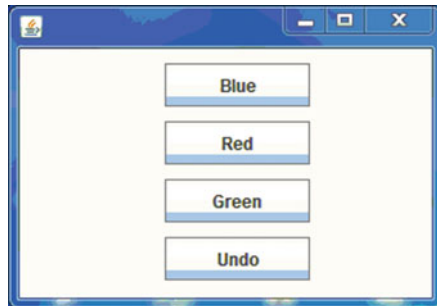
```
red = new JButton("Red");
red.setBounds(100, 50, 100, 30);
getContentPane().add(red);

green = new JButton("Green");
green.setBounds(100, 90, 100, 30);
getContentPane().add(green);

undo = new JButton("Undo");
undo.setBounds(100, 130, 100, 30);
getContentPane().add(undo);

// exit upon closing the frame
setDefaultCloseOperation(EXIT_ON_CLOSE);
}
}
```

When the program is executed, four buttons appear on the screen as shown below. These buttons can be clicked, but nothing happens because the code to handle the button clicks is not yet added to the class.



By default, the background is gray in color. To change it to white, for example, `setBackground` method can be used as in:

```
contentPane.setBackground(Color.white);
```

### 7.6.4 Handling Button Events

An action such as clicking a button is called an event and the mechanism to process the event is called event handling. Event handling is implemented by two types of objects: event source objects and event listener objects.

A GUI object, such as a button, where the event occurs is called an event source. When the user clicks a button, the corresponding  `JButton`  object will generate an event. When an event is generated, the system notifies the relevant event listener object that responds to the events by executing a method.

Among the many different types of events when a button is clicked, an event source will generate an action event. For the event to be processed, an event listener needs to be associated to the event sources. Otherwise, generated events are simply ignored. That is what happened in Sect. 7.6.3, where clicking buttons did not do anything. There are different kinds of listeners for each type of event: an action listener for action events, a window listener for window events, a mouse listener for mouse events, and so on.

To process an action event, an action listener class needs to include an  `actionPerformed`  method. To ensure this, the class must implement the  `ActionListener`  interface. Therefore, the  `UndoButton`  class is declared in the following way, a subclass of  `JFrame`  that implements the  `ActionListener`  interface.

```
class UndoButton extends JFrame implements ActionListener {  
    // implementation of UndoButton class  
}
```

Depending on which button is clicked, the background of the window will change. This is done inside the  `actionPerformed`  method. An argument to the  `actionPerformed`  method is an  `ActionEvent`  object that represents an action event, where the  `ActionEvent`  class contains methods to access the properties of a generated event. The general idea of the method is as follows:

```
public void actionPerformed(ActionEvent event) {                               // Line 1  
    String buttonText, color;                                                  // Line 2  
    JButton clickedButton = (JButton)event.getSource();                      // Line 3  
    buttonText = clickedButton.getText();                                       // Line 4  
  
    if(buttonText.equals("Undo"))                                              // Line 5  
        // process undo button  
    else                                                                        // Line 6  
        // process blue, red, and green buttons  
}
```

The text of the clicked button can be retrieved as in Line 3 and 4. The text of the blue button is the string “Blue”, the text of the red button is the string “Red”, and so on. Notice the typecasting of an object returned by the `getSource` method to `JButton`. This is because an object returned by the `getSource` method can be an instance of any class.

### 7.6.5 Generic Linked Stack

In order to keep the history of the command, the background color will be stored in the stack. Since the color of the background is of type `String`, the `StackRef` class which can be used only for integers is modified to accommodate items of any type. The `StackRefGeneric<T>` class that implements the `StackGeneric<T>` interface discussed in Sect. 2.7 is defined as shown in Fig. 7.3.

Remember that the class which implements an interface must have definitions for any methods listed in the interface. Although the list of objects linked together will never become full, and there is no use for the `full` method since it is included in the `StackGeneric<T>` interface for an array implementation of a stack, the code for the `full` method is added here to simply return the `false`. As before, the `pop` and `empty` methods are left as an exercise for the reader at the end of the chapter.

```
public class StackRefGeneric<T> implements StackGeneric<T> {
    private NodeGeneric<T> top;

    public StackRefGeneric(){
        top = null;
    }

    public boolean empty() {
        // insert code
    }

    public boolean full() {
        return false;
    }

    public T pop() {
        T item = null;
        // insert code
    }

    public void push(T item) {
        NodeGeneric<T> newp = new NodeGeneric<T>(item);
        newp.setNext(top);
        top = newp;
    }
}
```

**Fig. 7.3** `StackRefGeneric < T >` class

### 7.6.6 The UndoButton Class

Because the `UndoButton` class is a subclass of `JFrame` that implements the `ActionListener` interface, a frame object is also an action event listener. Therefore, in the constructor the `addActionListener` method of the button is called by passing `this` as an argument. An example for the blue button is shown below:

```
blue.addActionListener(this);
```

If a user clicks either the Blue, Red, or Green button, the background color changes and the color is pushed onto the stack. This is done in the `setBackground` method. Inside the `actionPerformed` method, when a button for the different color is clicked the `setBackground` method is called. If the undo button is clicked, first it checks whether the stack is empty. The stack could be empty if the action was the very first one or if the undo operation emptied the stack previously. Then the program outputs a message. If the stack is not empty, it pops the current background color from the stack. After the top item is removed, the stack could be empty, since the user could have clicked one of the background color buttons only once or the undo button has been clicked and there was only the current color on the stack. If the stack is empty, the program will output a message. If it is not, the undo operation is performed to display the last background color in the window. The portion of the program which deals with the undo button is shown below:

```
if (buttonText.equals("Undo"))
    if (undoStack.empty())
        JOptionPane.showMessageDialog(null, "cannot undo");
    else {
        undoStack.pop();
        if (undoStack.empty())
            JOptionPane.showMessageDialog(null, "cannot undo");
        else {
            color = undoStack.pop();
            setBackground(color);
        }
    }
}
```

The complete listing of the `UndoButton` class which is a subclass of `JFrame` that implements the `ActionListener` interface discussed in Sect. 7.6.4 uses the `NodeGeneric<T>` class in Fig. 5.6 and the `StackRefGeneric<T>` class from Sect. 7.6.5 is shown in Fig. 7.4. An instance of the `StackRefGeneric<String>` class is created and used to store the color of the background every time the user changes it.



```

import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class UndoButton extends JFrame implements ActionListener {
    private JButton blue, red, green, undo;
    private StackRefGeneric<String> undoStack
        = new StackRefGeneric<String>();

    public static void main(String[] args) {
        UndoButton frame = new UndoButton();
        frame.setVisible(true);
    }

    public UndoButton() {
        Container contentPane = getContentPane();

        // set the frame size
        setSize(300, 200);

        // set the content pane properties
        contentPane.setLayout(null);
        contentPane.setBackground(Color.white);

        // create and place four buttons on the frame's content pane
        blue = new JButton("Blue");
        blue.setBounds(100, 10, 100, 30);
        getContentPane().add(blue);

        red = new JButton("Red");
        red.setBounds(100, 50, 100, 30);
        getContentPane().add(red);

        green = new JButton("Green");
        green.setBounds(100, 90, 100, 30);
        getContentPane().add(green);

        undo = new JButton("Undo");
        undo.setBounds(100, 130, 100, 30);
        getContentPane().add(undo);

        // associate an action listener to the four buttons
        blue.addActionListener(this);
        red.addActionListener(this);
        green.addActionListener(this);
        undo.addActionListener(this);

        // exit upon closing the frame
        setDefaultCloseOperation(EXIT_ON_CLOSE);
    }

    public void actionPerformed(ActionEvent event) {

```

**Fig. 7.4** UndoButton class

```

String buttonText, color;
JButton clickedButton = (JButton) event.getSource();
buttonText = clickedButton.getText();

if(buttonText.equals("Undo"))
    if(undoStack.empty())
        JOptionPane.showMessageDialog(null, "cannot undo");
    else {
        undoStack.pop();
        if(undoStack.empty())
            JOptionPane.showMessageDialog(null, "cannot undo");
        else {
            color = undoStack.pop();
            setBackgroundColor(color);
        }
    }
else
    setBackgroundColor(buttonText);
}

private void setBackgroundColor(String color) {
    if(color.equals("Blue")) {
        getContentPane().setBackground(Color.blue);
        undoStack.push("Blue");
    }
    else
        if(color.equals("Red")) {
            getContentPane().setBackground(Color.red);
            undoStack.push("Red");
        }
        else
            if(color.equals("Green")) {
                getContentPane().setBackground(Color.green);
                undoStack.push("Green");
            }
    }
}

```

**Fig. 7.4** (continued)

---

## 7.7 Summary

- Hiding how a data structure is implemented, whether using arrays or references, is known as data abstraction.
- Using references to implement stacks and queues can take up more memory, but the data structure can grow or shrink as needed.
- Although full is not necessarily needed with stacks and queues implemented using references, an empty method is definitely needed.
- In order to build an effective graphical user interface (GUI), event-driven programming is used.

- In order to process the action events generated by a button, an action listener needs to be associated to the button.

---

## 7.8 Exercises (Items Marked with an \* Have Solutions in Appendix B)

1. Assuming the stack `intStack` of type `StackRef` in Fig. 7.1 was initially empty, draw a stack after the following operations are completed:

```
int x, y, z;
intStack.push(41);
intStack.push(6);
x = intStack.pop();
y = intStack.pop();
intStack.push(78);
intStack.push(5);
intStack.push(79);
z = intStack.pop();
intStack.push(5);
```

- \*2. Assuming the stack `characterStack` of type `StackRefGeneric<Character>` which is an object of the generic class `StackRefGeneric<T>` in Fig. 7.3 is initially empty, draw a stack after the following operations are completed:

```
Character x, y, z;
characterStack.push('U');
characterStack.push('V');
x = characterStack.pop();
characterStack.push('W');
y = characterStack.pop();
characterStack.push('P');
characterStack.push('Q');
z = characterStack.pop();
characterStack.push('C');
```

3. Assuming the queue `intQueue` of type `QueueRef` in Fig. 7.2 is initially empty, draw a queue after the following operations are completed:

```

int x, y, z;
intQueue.enqueue(41);
intQueue.enqueue(6);
x = intQueue.dequeue();
y = intQueue.dequeue();
intQueue.enqueue(78);
intQueue.enqueue(5);
intQueue.enqueue(79);
z = intQueue.dequeue();
intQueue.enqueue(5);

```

4. Implement a `QueueRefGeneric<T>` class including all the data members and methods from the `QueueRef` class in Fig. 7.2.
- \*5. Assuming the queue `characterQueue` of type `QueueRefGeneric<Character>` which is an object of the generic class `QueueRefGeneric<T>` created in problem 4 is initially empty, draw a queue after the following operations are completed:

```

Character x, y, z;
characterQueue.enqueue('U');
characterQueue.enqueue('V');
x = characterQueue.dequeue();
characterQueue.enqueue('W');
y = characterQueue.dequeue();
characterQueue.enqueue('P');
characterQueue.enqueue('Q');
z = characterQueue.dequeue();
characterQueue.enqueue('C');

```

6. Write a method `pop` discussed in Sect. 7.1 that can be added to the `StackRef` class in Fig. 7.1.
7. Write a method `empty` that can be added to the `StackRef` class in Fig. 7.1 which returns `true` if the list is empty.
8. Write a method `pop` discussed in Sect. 7.1 that can be added to the `StackRefGeneric<T>` class in Fig. 7.3.
9. Write a method `empty` that can be added to the `StackRefGeneric<T>` class in Fig. 7.3 which returns `true` if the list is empty.
10. Write a method `dequeue` discussed in Sect. 7.3 that can be added to the `QueueRef` class in Fig. 7.2.
11. Write a method `empty` that can be added to the `QueueRef` class in Fig. 7.2 which returns `true` if the list is empty.

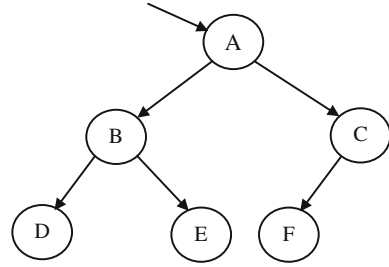
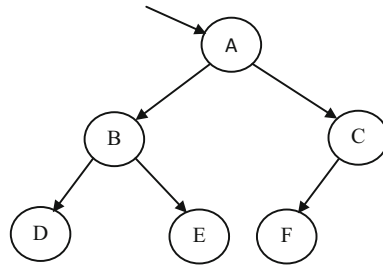
## 8.1 Introduction

All the data structures discussed in the previous chapters, lists, stacks, and queues, whether implemented using arrays or references could be described as linear. In this chapter, nonlinear structures called *trees* will be explored. Trees can have any number of branches, but this chapter will restrict the study to *binary trees* with no more than two branches. Just like the previous data structures, binary trees can also be implemented with arrays or references, but typically it is easier to implement them using references. However, as discussed in Chap. 10 another structure known as a heap which is a version of a binary tree can be implemented more easily with arrays.

Before proceeding with the implementation of binary trees it is useful to briefly explain some of the terminology which will help with subsequent discussions. Further, as with the previous data structures, it helps to draw a picture to visualize a binary tree as shown in Fig. 8.1.

First, it might appear that the tree is drawn upside down, but it is helpful to draw them this way when trying to create various algorithms. Each of the circles represent nodes in the tree, not unlike the node in a doubly linked list as discussed in Sect. 6.5. Each of the lines are called *edges* or branches specifically *directed edges* as indicated by the arrows. The node labeled A is known as the *root* of the tree or root node as indicated by the arrow pointing to it. Nodes that have branches below them are called *nonterminal nodes* (nodes A and B) and the ones without branches (nodes C, D, and E) are called *terminal nodes*, leaf nodes, or leaves. Also, note that B, D, and E form a subtree and B can be thought of as a subroot.

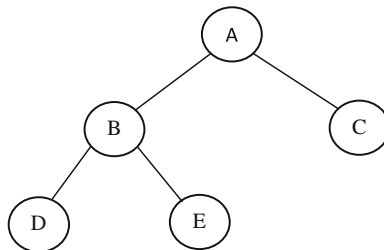
The *depth* of a node is how far down it is from the root of the tree. For example, node A has a depth of 0, B and C are 1, and D and E are 2. The *height* of a node is from the bottom up, so that C, D and E have a height of 0, B has a height of 1, and A has a height of 2. Also, the height of the tree is 2. Further, the tree in Fig. 8.1 is a *complete tree*, where the nodes are filled in from top to bottom and from left to right. This last concept will be explored further in the discussion on heaps in

**Fig. 8.1** Complete tree**Fig. 8.2** Full tree

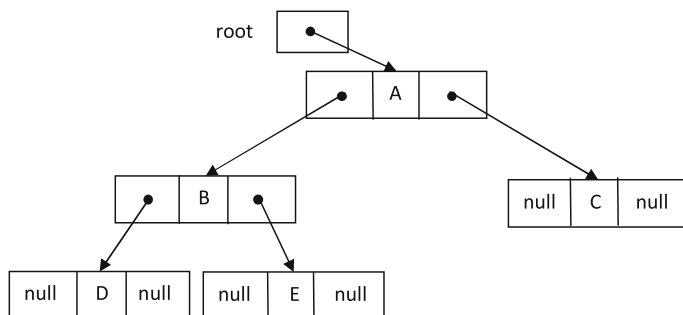
Chap. 10. In contrast, a *full tree* is one that every non-terminal node has two branches as shown in Fig. 8.2:

Any node that is on a branch is known as a *child*. For example, B and C are children of A. Further, A is the *parent* of nodes B and C. Note that a child can have only one parent. This ancestry terminology can be continued where D and E are grandchildren of A and as one might suspect, A is the grandparent of the same nodes. Further, B and C are siblings and they can be distinguished from one another where B is the left child of A and C is the right child of A.

To make the drawing in Fig. 8.1 a little simpler, the arrow pointing at the root node can be left off and it can be understood that the top node is the root node. In addition, the arrows can be drawn as just lines since it can also be understood that the nodes below node A are all children as follows:



Although the above illustration is a generic way of drawing a tree, it is sometimes helpful to draw the tree more as it is represented in memory which helps when trying to write the code necessary to implement various tree structures. Remembering the doubly linked list from Sect. 6.5, recall that in addition to the data section of a node, there were *back* and *next* references, which in a binary tree are replaced with *left* and *right* references, respectively. This same node structure can be used to implement binary trees as shown below:



Note the variable `root` which references the root node. Also, notice that the left and right portion of each nonterminal node references a child node, whereas the left and right section of each terminal or leaf node contains a `null`. In fact, this feature will become important in subsequent sections when writing code for binary trees to determine whether a node is a leaf node or a nonterminal node. Further, notice that unlike a doubly linked list, there are no references pointing back up to the parent nodes. If a reference does not refer back to a parent node, how does one return back to it? As will be seen in Sect. 8.2.2, recursion will provide this capability.

Which type of drawing is better? It depends upon what one is trying to do. If one is trying to understand general concepts, then the first and second drawings are probably better, since they have fewer details. However, if one is trying to write or debug code, then the latter one might be better. Throughout the remainder of this text, the drawing that helps the most in a particular situation will be used, but one always has the option to redraw the tree in an alternative scheme if it proves helpful. Although there are a variety of different types of binary trees, the above three drawings will help in understanding and writing the necessary algorithms. The following sections will look at binary expression trees and binary search trees.

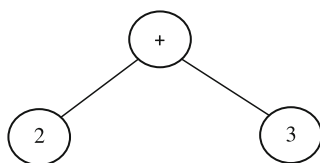
## 8.2 Binary Expression Trees

### 8.2.1 General Concepts

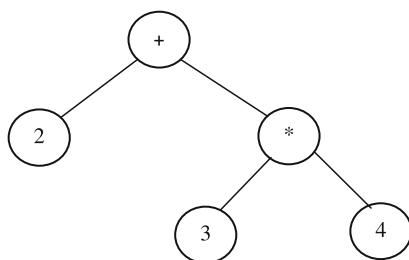
Binary expression trees are a natural way to begin a discussion of binary trees because they build on the material presented in Chap. 2. Recall that a stack could be

used to convert infix expressions to postfix and evaluate prefix or postfix expressions (see Sects. 2.6 and 2.9). Given this knowledge of prefix, infix, and postfix expressions, *binary trees* can be used to store arithmetic expressions and output them in prefix, infix, and postfix order. Before writing code to create and output these types of trees, it helps to first understand conceptually how an expression can be stored in a tree. The concept of binary expression trees can help in the future understanding of parse trees that are found in programming languages and compiler courses.

First, it helps to start with a very simple infix expression such as  $2 + 3$ . The way the expression is stored in a tree is that the operator, the addition symbol, forms the root of the tree and the two operands, the 2 and the 3, form the leaves of the tree as follows:

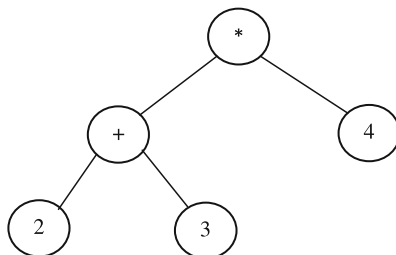


Although this seems fairly straight forward, what about a more complex arithmetic expression such as  $2 + 3 * 4$ ? Which operator should be at the root? It is here that one's knowledge of the priority of arithmetic operations is needed from a first course or textbook in computer science and/or Sect. 2.6 of this text. The rule for creating a binary expression tree is to place the operator with the lowest priority at the root of the tree. In the expression under consideration, that would be the addition symbol. The number 2 would be the left child and the subexpression  $3 * 4$  would then form a subtree of the right branch of the addition symbol as shown below:



What if the order of the operations was changed? For example, consider the inclusion of parentheses around the addition operator and its two operands as in  $(2 + 3) * 4$ . Now, the multiplication operator is the lowest priority and it would be at the root as follows:



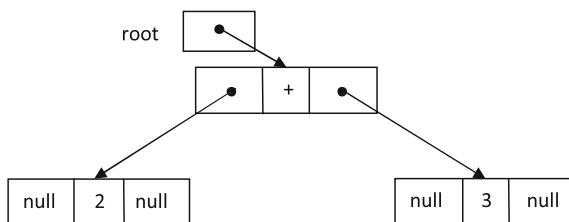


Notice that the parentheses do not appear in the binary expression tree, and that this tree is different from the previous tree where the addition operator is now in the left subtree of the root node. The significance of these differences will become apparent in the next section.

### 8.2.2 Output in Prefix Form

The beauty of this form of representation of an arithmetic expression is that the tree can be traversed and the contents of the tree can be output in either prefix, infix, or postfix order which corresponds to a preorder, inorder, or postorder traversal. As in the past, it is helpful to first assume the existence of a tree, output the tree, and defer the code for creating the tree until later. The reason for this is that it is easier to understand how to write the code for an existing tree and then apply the knowledge gained to the creation of a tree.

Assuming that each node is visited starting at the root, then traversing down the left branch first, then returning to the root node second, followed by the right child, and lastly returning to the root, one can perform various operations such as output when visiting each node. This type of traversal is known as a depth first traversal. Also, it is interesting to note that the root is visited three times: once prior to traversing the left branch, a second time after returning from the left branch and prior to traversing the right branch, and a third time after traversing the right branch. This can prove to be a helpful observation in subsequent discussions. Now, consider the first binary expression tree for  $2 + 3$  from the previous section which is redrawn below showing the internal contents of each node.



Using the strategy just outlined, if the contents of the data section were output when visiting the root node, the arithmetic operator would be output. Then when the left child is visited, the 2 would be output. Lastly the contents of the right child, the 3, would be output. Putting the output together the result would be +23. Thinking back to Sect. 2.6, one should recognize this as a prefix expression. But how could code be written for this? Although it could be hardcoded for this example only, the algorithm would not work for the other two trees in the previous subsection. Instead a solution needs to be found that would be capable of outputting all three trees.

Recalling from Sect. 5.8, it was possible to output the contents of a linked list not only using iteration but recursion as well, and it is the recursive solution that will help output the contents of a binary expression tree. First, it is best to create a skeleton for the `BinaryExpressionTree` class and `Node` class as follows:

```
import java.util.*;

public class BinaryExpressionTree {

    private Node root;

    public BinaryExpressionTree() {
        root = null;
    }

    private class Node {
        private Node left;
        private char data;
        private Node right;

        public Node() {
            this('');
        }
        public Node(char data) {
            left = null;
            this.data = data;
            right = null;
        }
    }
}
```

The local data member `root` and the constructor which initializes it to `null` form the outer class and the `Node` class is implemented as an inner class which will make the coding of the algorithms easier. However, it could be implemented as an outer class with the appropriate `get` and `set` methods and this is left as an exercise at the end of the chapter. Note that this inner class should look similar to the `Node`

class in Sect. 6.6 except that instead of just `next` there is now both `left` and `right` fields. Also, instead of `data` as type `int` it is of type `char` to allow it to contain both the arithmetic symbols and the character representations of the digits.

As done previously in Sects. 6.3, and 6.6 it is helpful to have a driver routine that can be called from a main program. The word "Prefix" is output, the recursive method is invoked, and upon return the cursor is moved down to the next line as shown below:

```
public void preFix() {  
    System.out.print("Prefix:");  
    preOrder(root);  
    System.out.println();  
}
```

The driver is a simple part, so how can the recursive method `preOrder` be written? When `preOrder` is invoked from the driver routine, the argument `root` will be sent to the parameter `p` as shown in the following skeleton:

```
private void preOrder(Node p) {  
  
}
```

Although this is also relatively simple, what should the body contain? First, when the node is not a terminal or leaf node, the contents of the `data` field should be output. How can one determine whether or not the current node is a leaf node? Recall from the previous section, it was noted that leaf nodes have both the `left` and `right` references equal to `null`. So in order to determine whether they are not equal to `null`, an `if` statement should be used.

A second question is whether an `&&` or an `||` should be used in the `if` statement? In looking at the drawing of the binary expression tree, notice that either both `left` and `right` contain references or both contain a `null`. Should both contain a `null`, do both `left` and `right` need to be checked for `null`, or would it be sufficient to check just one or the other? If there was never a case where one of the two contained a reference and the other contained a `null`, then checking just one or the other would be sufficient. However, what if there was an error and the scenario mentioned above occurred? Then it could be possible that the method might try to traverse and output a node where there is a `null` reference and an exception would occur.

Some think it is best that if this situation occurs, there should be an exception so that the error is brought to the attention of the programmer and it can be fixed. However, others think that having an exception halting the execution of the program is not a good alternative either. The ideal solution is to allow the exception to occur and output the appropriate message but allow processing to continue.

Of these three choices, the second is pursued in this section. This would dictate that an `&&` operation should be used in the `if` statement as follows:

```
private void preOrder(Node p) {  
    if(p.left != null && p.right != null) {  
  
    }  
}
```

Could the above code segment be rewritten using an `||`? The answer would appear to be yes given the above discussion, but could it be rewritten using `||` while still checking both `left` and `right`? If one remembers from a first class or textbook, the answer is yes. The answer involves the use of DeMorgan's rules, where the `!` symbol is moved outside and the `&&` is changed to `||` as follows:

```
private void preOrder(Node p) {  
    if(!(p.left == null || p.right == null)) {  
  
    }  
}
```

Which of the two methods is better? It is really a matter of choice, but one must be very careful when creating Boolean expressions because logic errors can be some of the most difficult problems to debug. That is the reason for the detailed analysis above for writing a mere `if` statement because time spent analyzing an algorithm reduces potentially long debugging sessions later.

Continuing, what should be placed within the then portion of the `if` statement? Given that the prefix representation of the tree is to be output and assuming it is the first time to the root node, the contents of the `data` field should be output prior to going on to the left branch:

```
System.out.print(p.data);
```

After it has been output, then the left branch should be traversed. The code for this is not much different than recursively traversing a linked list back in Sect. 5.8 except that the `next` link is replaced with `left` as shown below:

```
preOrder(p.left);
```

Once the left branch has been traversed, then the right branch should be traversed where the `left` is replaced with `right` as shown below:

```
preOrder(p.right);
```

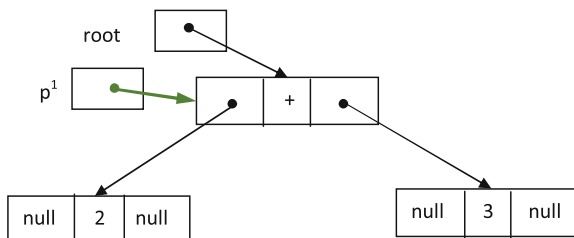
But what should happen if a terminal or leaf node is encountered? Both the `left` and `right` would contain `null` and the `then` section would not be executed. Recursion should not take place and the contents of the `data` field should be output which could be placed in the `else` section of the `if` statement. Putting all of the above together in the method, a possible solution would look as follows:

```
// possible preOrder method
private void preOrder(Node p) {
    if(!(p.left == null || p.right == null)) {
        System.out.print(p.data);
        preOrder(p.left);
        preOrder(p.right);
    }
    else
        System.out.print(p.data);
}
```

Although the above method would work, is there a way to make the code cleaner? If one recalls when learning about if-then-else structures, should the same code appear at the beginning or end of both the `then` and `else` sections, then it can be moved prior to or after the `if` structure, respectively. This also applies when writing recursive algorithms. In examining the method above, notice that the output of the `data` field appears as the first line of code in both sections of the `if` statement, so it could be moved prior to the `if` statement and still have the same functionality. The cleaner method appears below:

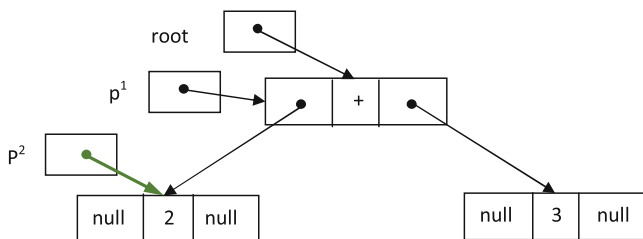
```
// cleaner preOrder method
private void preOrder(Node p) {
    System.out.print(p.data);
    if(!(p.left == null || p.right == null)) {
        preOrder(p.left);
        preOrder(p.right);
    }
}
```

Clearly, the above method is simpler than the previous method and without having duplicate code it is less prone to logic errors when the code is modified. Before compiling and executing the method, does it work? It can be desk checked prior to it being implemented which will also help in understanding how the method works. First, it is easiest to check it out on a simple binary expression tree below:



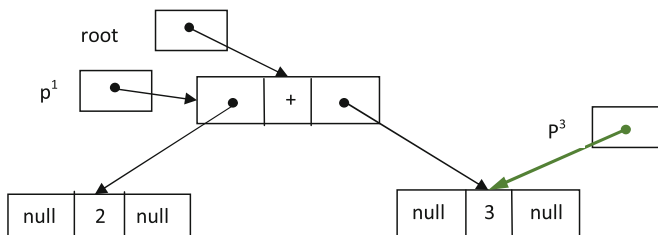
In the first invocation of the `preOrder` method, `p` would take on the value of `root` from the invoking method which is indicated by the  $p^1$  in the above drawing. The first line of code would output the `+` sign and proceed on to the `if` statement. It would check to see that the `p.left==null` is `false` and `p.right==null` is `false`, where `false||false` is `false`. The result is that `!false` is `true` so that the `then` section of the `if` statement would be executed.

The first line of the `then` section is a recursive call down the left branch of the tree. The parameter `p` in the second invocation of the `preOrder` method would take on the value of `p.left` from the first invocation as indicated by  $p^2$  in the following drawing:



The first thing that happens in the second invocation of the `preOrder` method is that the `2` in the `.data` section of the node is output and execution proceeds on to the `if` statement. It would check to see that the `p.left==null` is `true` and `p.right==null` is `true`, where `true||true` is `true`. In this case `!true` is `false` so that the `then` section of the `if` statement is not executed. Since that is all there is in the method, execution would return back to the first invocation of the method.

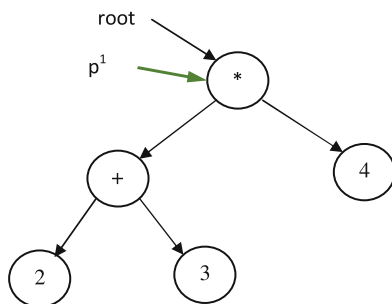
The next line of code in the `then` section of the first invocation is the call to traverse down the right branch of the tree as shown below:



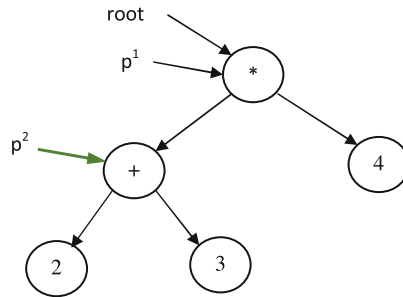
As with the left branch previously, the 3 in the data field would be output and control would return back to the first invocation. The then section of the if statement would be complete and execution would return back to the driver routine `preFix` which then would return back to the calling program. The result would be the following output:

`Prefix: + 2 3`

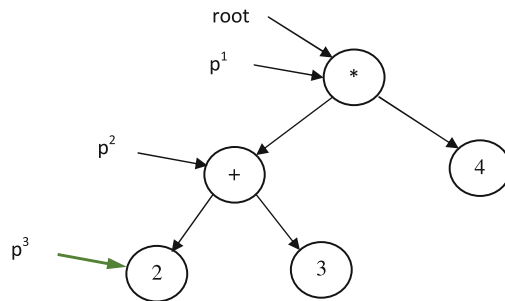
So far so good, but what about a more complicated tree such as the one for  $(2 + 3) * 4$ ? The principle is the same except there would be more recursive calls. Again the calling program would call the driver routine which would in turn call the first invocation of `preOrder` as shown using the simpler drawing below:



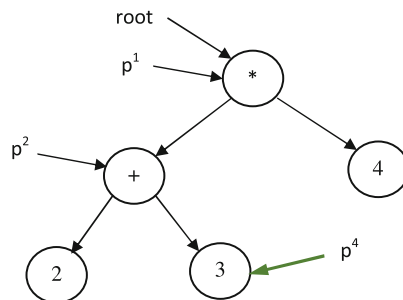
In the first invocation,  $p^1$  references the root node and the multiplication symbol is output. Since the root node is not a leaf node, a recursive call is made on the left branch and  $p^2$  references the node containing the addition symbol as shown below and it is output.



Again since this node is not a leaf node, the third invocation of `preOrder` occurs, where  $p^3$  references the `p.left` reference of the second invocation as shown below and the 2 is output.



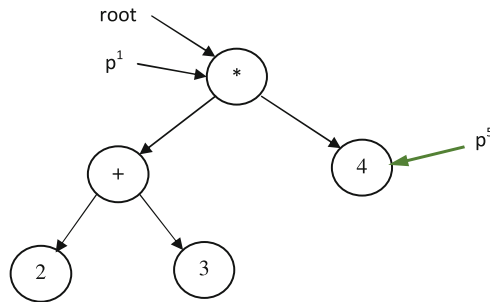
Since the node referred to by  $p^3$  is a leaf node, control is returned back to the second invocation. Having traversed the left branch, the right branch is sent to the fourth invocation of `preOrder` as given below:



Again, since the node referenced by  $p^4$  is a terminal node, the 3 is output, control returns back to the second invocation, and since there are no more instructions in the `then` portion of the `if` statement, control is returned back to first



invocation of `preOrder`. Having traversed the entire left branch of the tree, the fifth invocation of `preOrder` is down the right branch of the tree as follows:



The 4 is output and since this is also a terminal node, control is returned back to the first invocation of `preOrder`, and from there back to the driver routine `preFix` and the calling program. The result is that the output appears as follows:

Prefix: \* + 2 3 4

Walking through the other expression from the previous subsection,  $2 + 3 * 4$ , is left as an exercise at the end of the chapter.

### 8.2.3 Output in Infix Form

Having examined the prefix traversal of a binary expression tree, what about an infix traversal? Again a driver routine similar to `preFix` is needed and called `inFix` as shown below:

```

public void inFix(Node root) {
    System.out.print("Infix: ");
    inOrder(root);
    System.out.println();
}

```

and the skeleton for `inOrder` is as follows:

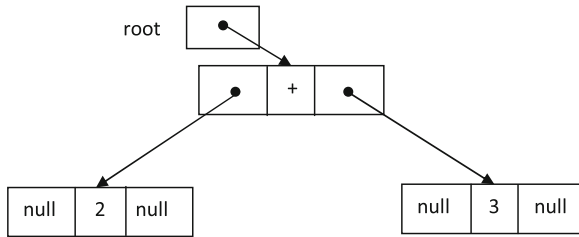
```

private void inOrder(Node p) {

}

```

Also, it is helpful to start with a simple expression  $2 + 3$  as shown below:



So far everything is similar to the previous example, but now an analysis of when things are to be output is needed. As before, it is necessary to determine whether a node is a terminal or nonterminal node, so the same `if` statement can be used as shown below:

```
private void inOrder(Node p) {
    if(!(p.left == null || p.right == null)) {

    }
}
```

On the first visit to the root node, which is not a leaf node, should the addition symbol be output? The answer appears to be no. Instead, what should be done? It seems that instead of outputting the contents of the node, the left branch should be taken:

```
inOrder(p.left);
```

Since the left branch is a leaf node, the `else` section of the `if` statement is executed and the 2 in the node should be output. So far the partial `if` statement would look as follows:

```
if(!(p.left == null || p.right == null)) {
    inOrder(p.left);
}
else
    System.out.print(p.data);
```

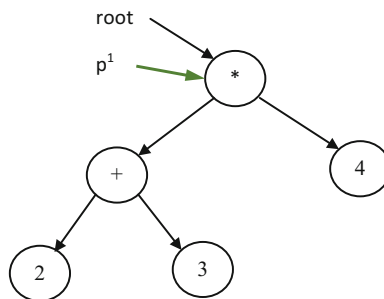
Of course once the contents of the leaf node are output, control would need to return back to the nonterminal node. But what should be done next? In the prefix example, the right recursive call occurred immediately after the left recursive call. Should that occur here? If it did, the number 3 would be output immediately after the number 2 without the addition operator between the two operands. Instead, before calling for the traversal of the right branch, the contents of the nonterminal

node should be output. Only after that should the call to the right be invoked. Once invoked, the contents of the leaf node would be output, control would return to the nonterminal node, and then back to the driver method. The final version of the `inOrder` method can be seen below:

```
// possible inOrder method
private void inOrder(Node p) {
    if(!(p.left == null || p.right == null)) {
        inOrder(p.left);
        System.out.print(p.data);
        inOrder(p.right);
    }
    else
        System.out.print(p.data);
}
```

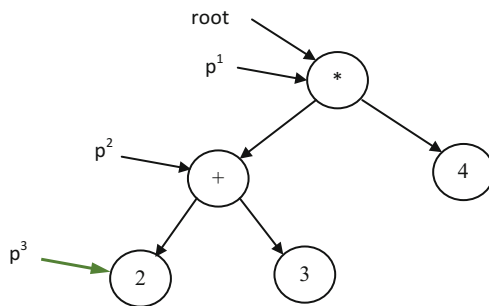
Can this code be made cleaner as was done with the `preOrder` method? Since the same code does not appear either at the beginning or end of the then and else sections, the answer is unfortunately no. In the then section the left recursive call must come before the output and the right recursive call must come after the output.

Although one can walk through the simple example  $2 + 3$  on his or her own as was done in the previous section, it is helpful here to start with the example  $(2 + 3) * 4$ . Also, since all of the steps were illustrated in the previous section, some of them might be combined together below in an effort to save space, but this does not preclude the reader from drawing any intermediate steps if needed. As before, the root is passed from the calling program to the driver method `inFix`, which is then sent on to the first invocation as  $p^1$  as shown using the simpler drawing below:

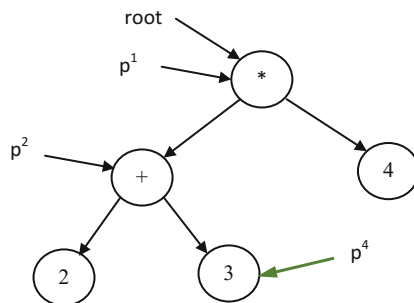


In the first invocation since the root node is not a terminal node, nothing is output and the left branch is sent to the second invocation. In the second invocation,

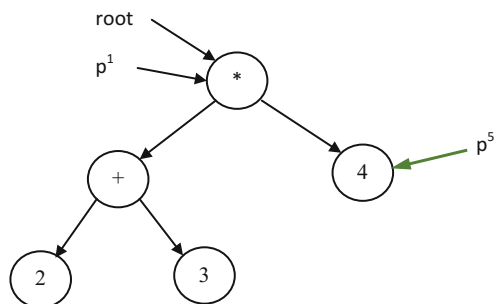
the node is also not a leaf node so the left branch is sent to the third invocation as shown below:



Here the 2 is output and control is returned back to the second invocation where the addition symbol is output prior to sending the right branch to the fourth invocation.



The 3 is then output in the output and control is returned to the second invocation which completes the then portion of the `if` statement and control is returned to the first invocation where the multiplication symbol is output and the right branch is sent to the fifth invocation.



Then the 4 is output returning control back to the first invocation which then returns back to the driver routine and the calling program. The output from the driver and recursive `inOrder` method is as follows:

```
Infix: 2 + 3 * 4
```

However, does one see a problem with this output? At first it might seem to be okay but what was the original expression that was stored in the tree? It was  $(2 + 3) * 4$ , not  $2 + 3 * 4$ . The result is that the user of the program would assume that the multiplication had higher priority than the addition, which is not the expression stored in the tree. What is needed are parentheses. Ideally parentheses need only be placed around the  $2 + 3$ , but writing that program is a little more complicated, and is left as an exercise at the end of the chapter.

Instead, a simpler way to solve the problem is to just output all parentheses, even if they are redundant. The resulting expression would look like  $((2 + 3) * 4)$ . Although the outer parentheses are not needed, it would still correctly reflect the expression stored in the tree. The question is then how would this be accomplished? Looking back at the original `inOrder` method, a left parenthesis needs to be output prior to outputting the 2. A possible solution is to place a `print` statement to output a left parenthesis just prior to the output of the leaf node and the `print` for the right parenthesis could be placed just after the output of the leaf node in the `else` section as follows:

```
// possible else section
else {
    System.out.print('(');
    System.out.print(p.data);
    System.out.print('');
}
```

But what is wrong with this possible solution? It would certainly output a parenthesis prior to the 2, but after walking through the algorithm one should see the output would look as shown below which is clearly not what is needed:

```
Infix: (2)+(3)*(4)
```

Not only are the parentheses unnecessary but the expression still does not correctly represent the correct order of operations. Instead, since each pair of parentheses should be associated with the arithmetic operators, they ought to be in the `then` section. Specifically, what is needed is to have the left parenthesis output prior to making the recursive call down the left subtree and the right parenthesis be output after making the recursive call down the right subtree as shown below:

```
// correct inOrder method
private void inOrder(Node p) {
    if(!(p.left == null || p.right == null)) {
        System.out.print('(');
        inOrder(p.left);
        System.out.print(p.data);
        inOrder(p.right);
        System.out.print(')');
    }
    else
        System.out.print(p.data);
}
```

The reader is encouraged to walk through the algorithm and the previous binary expression to ensure that it works correctly. Having shown how to output both the prefix and infix version of a binary expression tree, what about outputting the tree in postfix form? Instead of answering this question here, it is left as an exercise at the end of the chapter where the techniques used in this and the proceeding subsection can be used to create a clean version of the method. However, before one tackles a `postOrder` method, it might first be helpful to examine how a binary expression tree can be created in the next section and also summarize the characteristics of many recursive algorithms the section after that.

---

### 8.3 Creating a Binary Expression Tree from a Prefix Expression

Just as there are different ways of outputting a binary expression tree, there are different ways of creating a binary expression tree. In this section a tree will be created from a prefix expression and the other ways such as infix and postfix are left as exercises at the end of the chapter. Further, to make things simple, it is helpful to assume that only well-formed prefix expressions are entered. Starting with a simple expression such as `+23`, how could the algorithm be created?

First, each character will need to be processed one at a time. An entire string could be read from the keyboard and then each individual character from the string could be placed in a tree or alternatively each character could be read in one at a time from the keyboard. It is this latter approach that will be explored here.

Just as trees can be output using recursion, trees can be created using recursion. As before, it helps to have a driver program that includes a heading and also includes the initial call to the recursive method. Notice that unlike the previous calls to the recursive method which only output the contents of the trees and were `void` methods, here the recursive method needs to create a tree so it needs to be a value

returning method that returns the reference to the newly created tree that is assigned to `root` as shown below:

```
public void createTree() {  
    System.out.println();  
    System.out.println("Enter a prefix arithmetic expression.");  
    System.out.println();  
    root = inputData();  
}
```

Having created the driver routine, it is now time to look at the recursive method. Since the method will be prompting for and inputting the data for each node, a scanner object will be declared and created here. Also, both a local variable `ch` of type `char` for the characters being input and a variable `p` of type `Node` to hold the reference for the node that holds the character are needed. Lastly, the reference for the new node needs to be returned to the invoking method as shown in the skeleton below:

```
private Node inputData() {  
    Scanner scanner;  
    scanner = new Scanner(System.in);  
    System.out.print("Enter an operator or operand: ");  
    char ch = scanner.next().charAt(0);  
    Node p = new Node(ch);  
    return p;  
}
```

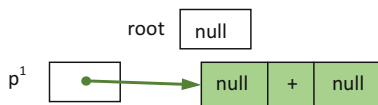
Taking a few hints from the previous `preOrder` routine for output, where should the prompt and input statements be placed? Recall that the `print` statement was prior to the `if` statement so that the first time to either a terminal or nonterminal node the contents of the node were output. Similarly, the prompt and input statement should appear prior to the `if` statement along with the creation of a new node.

But how will one determine whether the node is terminal or nonterminal? In fact, when a new node is created, the constructor initially places a `null` in both `.left` and `.right` making them leaf nodes initially. Instead of checking the left and right branches one needs to check the character being input. In the case of an arithmetic operator, recursion must occur to create the left and right branches. As the branch is created, a reference to the node created will be returned and assigned to the appropriate branch. If it is not an operator, one can assume that it is a leaf node. The

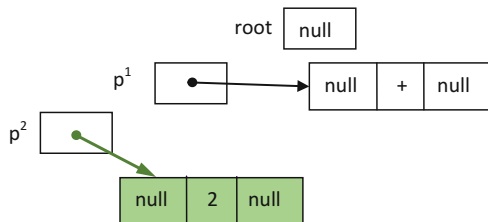
result is the following if statement which would be placed just prior to the return statement in the above method:

```
if(ch == '+' || ch == '-' || ch == '*' || ch == '/') {
    p.left = inputData();
    p.right = inputData();
}
```

Having created the method, it helps to walk through the code to ensure that a tree is created properly. Assuming the simple prefix expression `+23`, the call from the calling program to the driver method `createTree` outputs a heading and invokes the first invocation of the recursive `inputData` method which prompts for and inputs the `+` sign, creates a node, and assigns the `+` sign to the node as shown below:

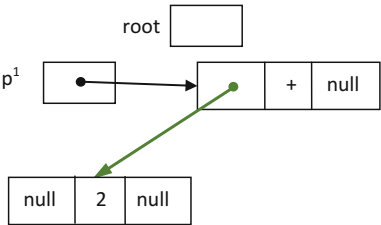


Notice that `root` does not yet contain a reference to the new node. Since the character input is an operator, the first recursive call in the then section of the `if` statement is executed. This is the second invocation to `inputData` where the digit `2` is prompted for and input, and another node is created with the `2` within it as shown below:

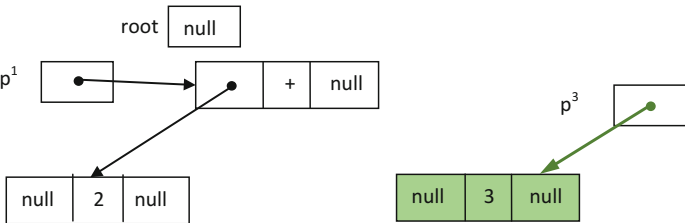


Again notice that the reference to this node has yet to be placed into the tree. Since this is not an operator but rather a digit, the then portion of the `if` statement in the second invocation is not executed. Instead, the reference in `p` is returned to the first recursive call in the first invocation and assigned to `p.left` as shown below:

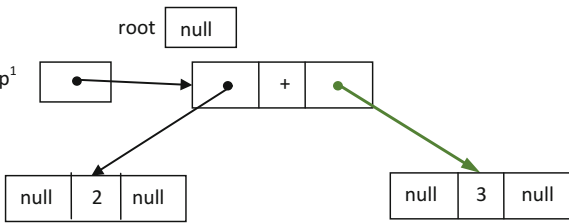




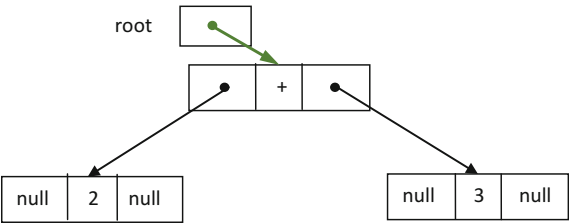
Then the second recursive call in the first invocation is executed. This will be the third invocation to the `inputData` method. There the digit 3 will be prompted for and input, and a node created with the digit as follows:



Again notice that this new node is not yet part of the tree, but since this is not an operator, control returns back to the first invocation and the value in  $p^3$  from the third invocation is assigned to `p.right` in the first invocation as shown below:



Now that both recursive calls are completed in the first invocation, the reference in  $p^1$  is returned to the driver routine and assigned to `root` as shown below:



This same process works for creating more complicated trees. The reader is encouraged to walk through the code for other expressions and these are left as exercises at the end of the chapter.

---

## 8.4 Generic if Statement for Recursive Algorithms

In looking at the previous prefix and infix recursive methods, is there a pattern? Yes, they all contain an `if` statement and two recursive calls for nonterminal nodes in the `then` section.

```
if(nonterminalNode) {  
    call(p.left);  
    call(p.right);  
}
```

In the case of the prefix methods, one should notice they could be written with or without the `else` statement. If they perform the same action such as input or output the first time to any node (terminal or nonterminal) it could be done prior to the `if` statement. Likewise, it could be assumed that performing the same action the last time visiting any node could be done after the `if` statement. In these cases, an `else` section was not needed and the basic structure would look something like this:

```
// first time to any node  
if(nonterminalNode) {  
    call(p.left);  
    call(p.right);  
}  
// last time to any node
```

As seen previously in the case of `infix`, if something needed to be done the first time visiting a nonterminal node such as outputting a left parenthesis, it was before the first recursive call. If something such as outputting the contents of the nonterminal node needed to be done it was done the second time visiting the node between the two recursive calls. The outputting of the right parenthesis was done the third time visiting the nonterminal node after the last recursive call. Further, outputting the contents of the terminal or leaf node was done in the `else` section. The `if-then` structure from above is modified to indicate the location of these actions as shown in the `if-then-else` structure below:

```
// first time to any node
if(nonterminalNode) {
    // first time to nonterminal node
    call(p.left);
    // second time to nonterminal node
    call(p.right);
    // third time to nonterminal node
}
else {
    // terminal node
}
// last time to any node
```

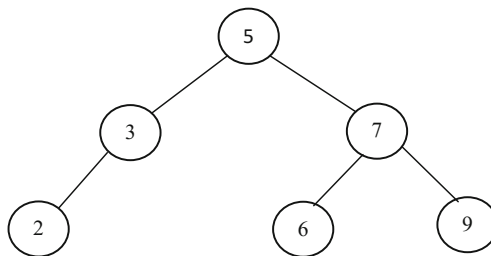
Instead of haphazardly trying to get a recursive method to work on binary trees by trial and error, the above template can serve as a starting point when trying to craft a recursive method and should help reduce the development and debugging time.

---

## 8.5 Binary Search Trees

### 8.5.1 General Concepts

Another form of binary tree stores data to facilitate rapid searching. The advantage of a *binary search tree (BST)* is that instead of having search each node to find an item, every time a comparison is made, it cuts the search space in half. The result is that instead of having an  $O(n)$  algorithm to find an item, it only takes  $O(\log n)$  which is extremely fast. As in the past, it is helpful to look at an existing binary search tree as shown below:



The first thing to notice is that not every nonterminal node has two children, as was the case with binary expression trees. Further, notice that every number to the left of the root node is less than the contents of the root node, in this case less than 5. Also, every number to the right of the root node is greater than 5. Not only does

this principle apply to the root node of the tree, but to every subtree. For example, in the left branch of the right subtree the 6 is less than 7 and the 9 in the right branch is greater than 7. This is the reason the binary search tree can be searched quickly.

The `Node` class from the previous section can be used here as well. If characters are stored in the tree, then no change needs to be made. However, integers are used here, so the data section needs to be changed to type `int` instead of type `char`. Of course, if generic types are used, there would be no need to change the `Node` class.

### 8.5.2 Inorder Traversal

Before looking at how to search the tree, it is helpful to be able to output the contents of the search tree to determine whether it is in proper order. What sort of traversal would accomplish this task? Remember that a prefix, infix, or postfix traversal corresponds to a preorder, inorder, and postorder traversal, respectively as discussed in the last section. If one is not sure, then a walk through of a tree will help in seeing the pattern.

When starting the walk through of the tree in the previous section, does one want to output the 5 in the root node? No, then what about the 3 after traversing further down the left subtree? The answer is again, no. Continuing to traverse down the left subtree to the node containing the 2, should it be output? Yes, and then returning back to the node containing the 3, should it be output? Again, yes and continue on to traverse down the right subtree. But since there is no right subtree, one would return back to the root node containing the 5 which should be output. Then down the right subtree past the node containing the 7 where the left subtree is traversed and the 6 is output. Then it is back to the 7 which is output before going down the right subtree where the 9 is output before returning past the node containing the 7 and back to the root node. When the tree is traversed in the fashion, the output is in the correct order.

Does this seem similar to a pattern and traversal from the previous section? It is similar to outputting a binary expression tree in infix order, so this would be an inorder traversal. Before writing the recursive method, it helps to have a driver method that is similar to previous ones. Further, if `root` is null, it is nice to output a message indicating that the tree is empty as shown below:

```
public void traverseBST() {  
    System.out.print("The tree is:");  
    if (root!=null)  
        traverse(root);  
    else  
        System.out.print(" Empty");  
    System.out.println();  
}
```

Continuing with the `traverse` method, although there are some changes it is the same basic structure of the generic `if` statement presented in the last subsection. The contents of the node should be output both when visiting a nonterminal node a second time and each time a terminal node is visited. Since it is possible that there might not be a left or right subtree, one way to solve this problem is to traverse down each subtree and make recursive calls until `p` is `null` as shown in the following possible method:

```
//possible inorder traversal method
private void traverse(Node p) {
    if(p != null) {
        traverse(p.left);
        System.out.print(" " + p.data);
        traverse(p.right);
    }
}
```

This is a compact routine, and note that there is no `else` section. This is because each terminal node is treated like a nonterminal node and it will traverse down each `null` branch. This is especially helpful in cases where there is only one branch such as in the previous binary search tree when trying to traverse the right subtree of the node containing the 3. When it is found that it is a `null`, it can return back to the node containing the 3.

Although this method would work, is there a potential problem? For example, when traversing the left subtree, instead of stopping at the node containing the 2, the `null` from the `p.left` is sent to `p` when the method is called recursively and then since `p` equals `null` it does nothing and simply returns back. In other words, for each `null` the method makes an additional recursive call. So given the six nodes in the example tree being used, there would be a total of twelve recursive calls instead of five. In another example, a tree of depth 3 with eight leaf nodes would contain a total of fifteen nodes and there would be thirty one calls instead of just fifteen. The result is that there would be more than twice as many calls as needed because each of the eight leaf nodes would require sixteen unnecessary recursive calls. To see this, draw the tree and count the number of calls that would be made. As should be recalled, recursion can be expensive in terms of how much memory is used, and anything that can be done to lessen the number of calls is helpful. So even though this method is fairly compact, it is not very efficient.

How can the number of recursive calls be lessened? It requires that the recursion on `null` branches be avoided. In the previous section concerning binary expression trees both branches of terminal nodes were `null` so that a single `if` statement could be used to check for this condition. But given that some nonterminal nodes only have one branch, how can this be accomplished? Instead of grouping both

recursive calls in the then section of an `if` statement, it appears that two `if` statements might be necessary, one to check the left branch and one to check the right. Only when a branch is not `null` should it be traversed. Would the following work?

```
// potential if structure
System.out.print(" ");
if(p.left != null) {
    traverse(p.left);
    System.out.print(" " + p.data);
}
else
    if(p.right != null) {
        System.out.print(" " + p.data);
        traverse(p.right);
    }
    else
        System.out.print(" " + p.data);
```

At first it seems that it might when traversing the previous tree. The left branch of the root node containing the 5 is not `null`, so it traverses the left branch to the node containing the 3. Its left branch is not `null` so it traverses to the node containing the 2. The left branch of the node containing the 2 is equal to `null` so the last `else` is taken and the 2 is output. So far it appears to be good and there was also not an extra recursive call to determine whether the left branch was `null`. It then returns to the node that contains the 3 and it is output. Since there was no right branch there appears to be no harm done by not checking it and it returns to the root node containing the 5. However, upon the return to the root node, the 5 is output and the right branch is not checked so the recursion and output is stopped.

What caused this problem? By nesting the check for a right branch, the right branch is not checked when there is a left branch. It is only when there is no left branch that the right branch is checked. Remember that in a binary search tree a nonterminal node could not only have a left or a right branch but both possibilities must be taken under consideration. Yes, there need to be two `if` statements, but they should not be nested. Given this, consider the following method:

```
// inorder traversal method
private void traverse(Node p) {
    if(p.left != null)
        traverse(p.left);
    if(p.right != null){
        System.out.print(" " + p.data);
    }
```

```
        traverse(p.right);
    }
    else
        System.out.print(" " + p.data);
}
```

If there is a left branch, the method will traverse it. If there is no left branch, then it will check for a right branch and if there is none it will output the contents of the terminal node in the else section of the second `if` statement. If there is a right branch, it will output the contents of the nonterminal node and traverse the right branch. This appears to work correctly and avoids the unnecessary recursion, but can the code be cleaner? As in the past, notice that the first thing in both the then and the else of the second `if` statement is the same and can be moved prior to the second `if` as follows:

```
// better inorder traversal method
private void traverse(Node p) {
    if(p.left != null)
        traverse(p.left);
    System.out.print(" " + p.data);
    if(p.right != null)
        traverse(p.right);
}
```

Note that the output of the contents of a nonterminal node still appears between the two recursive calls. But what if there is just a left or right branch? Remember that both branches must be `null` for a node to be a terminal node and so if a node has only one branch, it is a nonterminal node. The contents of the node will be output either after traversing down the left branch or before traversing down the right branch. If it is a terminal node, will the contents be output? Yes, because both branches will be `null`, both `if` statements will be `false`, there will be no recursion, and the `System.out.print` statement will still be executed.

A quick walk through of the algorithm using the previous tree will help one see that the algorithm does indeed work. Visiting the root node the first time the left branch is not `null`, so it is traversed. The node with the 3 also has a left branch and it is traversed. Upon reaching the 2, `p.left` is `null` so it is not traversed and the 2 is output. Further, `p.right` is also `null` and control returns back to the node containing the 3 and it is output. Unlike one of the previous versions, which ignored the right branch, this version checks to see if there is a right branch. Because there is not one, it returns back to the root node and outputs the 5. Since there is a right branch at the root node, it is traversed. At the node containing the 7, it traverses the left branch and outputs the 6. It returns to the node containing the 7, outputs it, then traverses the right branch and outputs the 9. It returns back to the

node containing the 7, followed by the root node, then the driver method, and lastly back to the calling program.

By outputting the contents of the tree, this method is useful to determine whether the tree has been created correctly. Although maybe not seen as often, the tree could also be traversed in preorder or postorder and these two methods are left as exercises at the end of the chapter.

### 8.5.3 Creating a Binary Search Tree

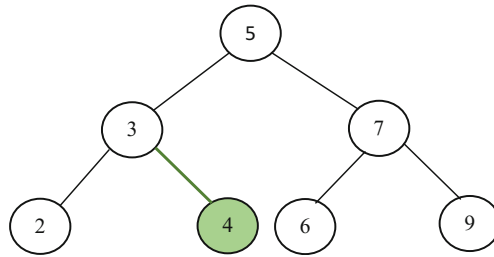
Having a method to output a binary search tree, it is now time to create a method to insert data into the tree. This will be accomplished somewhat similarly to creating a binary expression tree, but with some modifications. As done previously, it is nice to have a driver routine which will accept the integer to be inserted from the calling program and send it along with the root of the tree as an argument as shown below. Optionally, the `traverseBST` method could be called to output the contents of the tree or this could be done in the main program as shown in the test program in Sect. 8.7.

```
public void insertBST() {  
    root = insert(num, root);  
}
```

For each integer input, the `insert` method needs to determine where in the tree it should be placed. Again assuming the existence of the previous tree in the preceeding subsection and that the user inputs the number 4, what would need to occur? First it would need to check to see if 4 was less than or greater than the integer at the root. If less than it would traverse the left branch and if greater than it would traverse the right branch. In this case 4 is less than 5 so it would traverse the left branch. Upon reaching the node containing the 3, the same set of tests would need to be performed. In this case the 4 is greater than 3, so it could traverse the right branch. Of course, in this case the right branch is `null`, so when calling `insert` and then testing whether it is less than or greater than would cause an execution error.

As a result, before making these two tests, the method should first determine whether or not the current value of `p` is `null`. If it is, what should occur? Since it is neither less than or greater than, one should assume that this is the place where the value should be inserted into the tree. A new node needs to be created and the reference to the node returned back to the invoking routine so that the reference can be placed in the `left` or `right` field of the parent node, which in this case is the right branch as shown in green below:





So far, the `if` statement would look something like this:

```
if(p == null)
    p = new Node(num);
else
    if(num < p.data)
        p.left=insert(num,p.left);
    else
        if(num > p.data)
            p.right=insert(num,p.right);
```

However, what would happen if the value in `num` is equal to the number in the current node? It depends on the needs of the user. One option is instead of trying to store the same number more than once in the tree is to include a count within the node to keep track of the number of occurrences of each number, and this is left as an exercise at the end of the chapter. Another is to not allow duplicates because if, for example, the numbers represented account numbers, one would not want to have two accounts with the same account number and an error message could be output. Alternatively, instead of calling it an error, a user-friendly message could be output indicating that the number has already been entered into the tree and the reference sent is returned unaltered. Since the number is neither less than or greater than, it must be equal and the message can be included in the `else` at the end of the nested `if-then-else-if` structure. This, along with the two parameters and `return` statement, are shown in the following complete method:

```
private Node insert(int num, Node p) {
    if(p == null)
        p = new Node(num);
    else
        if(num < p.data)
            p.left = insert(num,p.left);
        else
            if(num > p.data)
```

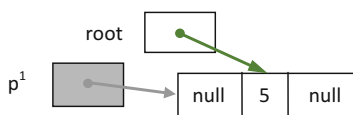
```

        p.right=insert(num,p.right);
    else
        System.out.println("Item in tree and not inserted.");
    return p;
}

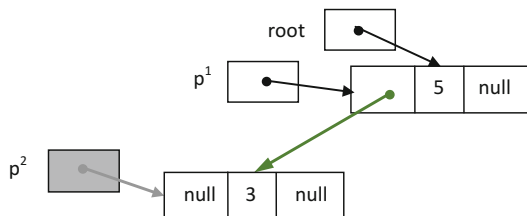
```

Notice that value of `p` is either set to reference a new `Node`, `p.left`, or `p.right`, and is replaced with the reference returned from the `insert` method. In the case of a duplicate number a message is output and the original reference in `p` is returned.

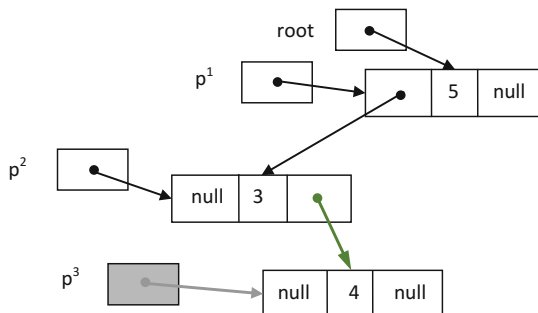
Although previously code was often written or walked through assuming the presence of an existing tree, it helps in this case to begin with an empty tree. What if the tree is initially empty and `root` contains a `null`? In this case assume that `num` contains a 5 and when `root` from the driver is sent to `p`, the first `if` statement is `true`, a new node is created with the number 5 in it. Then the reference to this in `p` will be returned back to the invoking driver routine and assigned to `root` as shown in green below. Thus a simple tree of only one element has been created. Note that `p1` is shaded in gray because technically it will have been deallocated once control has been returned to the driver.



Now assume that another number, say a 3, is sent from the driver along with `root` to first call to `insert`. In this case, `p` is not equal to `null`, so a new node is not created. Instead, `num` is compared to `p.data` and the 3 in `num` is less than 5 in `p.data`. So the 3 in `num` along with the `null` in `p.left` is sent to the next invocation of `insert`. This second invocation tests `p`, which contains a `null`, and a new node containing the 3 is created. Then this is returned to the first invocation and the reference to the new node shown in green below is placed in `p.left`. Again, note that `p2` is gray indicating it has been deallocated upon the return to the parent node `p1` and prior to control returning to the driver.

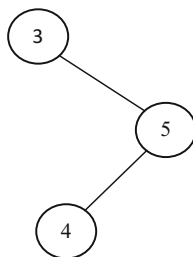


Assume that another number, 4, is to be inserted. Starting at the root,  $p$  is not equal to null, so  $\text{num}$  is compared to  $p.\text{data}$  and the 4 in  $\text{num}$  is less than 5 in  $p.\text{data}$ . So the 4 in  $\text{num}$  along with the reference in  $p.\text{left}$  is sent to the next invocation of `insert`. This second invocation tests  $p$  which is also not equal to null. It then tests that 4 is not less than 3, so it then checks to see if it is greater than 3. It is, so a third invocation of `insert` is called with the value 4 and  $p.\text{right}$ . In this third call,  $p$  is equal to null and new node containing the 4 is created. The reference to the new node is returned to the second invocation and the reference to the new node placed in  $p.\text{right}$  as shown below:

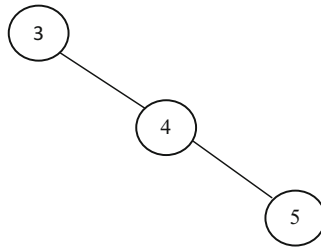


Then the second invocation returns  $p$  back to the first invocation which is assigned to  $p.\text{left}$ . Lastly  $p$  in this last invocation is returned to  $\text{root}$ . This process of creating the tree can continue for additional numbers that are to be inserted into the tree at the appropriate locations.

It is interesting to note that when the numbers are input in a different order, the tree would look different. In the previous example, the order was 5 3 4, but what if the numbers were input in the order 3 5 4? The result is that the tree would look as follows:



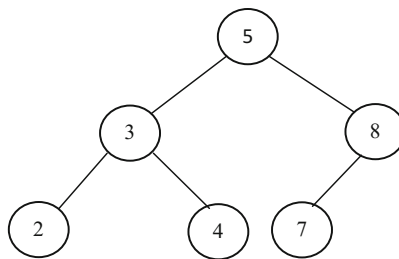
Does the order in which numbers are input affect the binary search tree order? No it does not. In looking at the above two trees, note that all the numbers in the left subtrees are less than the root or subroots and the numbers in the right subtrees are greater than the root or subroots. The order in which numbers are entered can cause some trees to appear more like linked lists as with the following sequence 3 4 5:



These types of trees still hold the binary search tree property, but their ability to be searched quickly is diminished. Instead of taking  $O(\log n)$  to find a value, they are of  $O(n)$  just like searching a linked list. As a result, when entering data into a binary search tree it is best to have the data in random order instead of a predefined order such as ascending or descending order. The reader is encouraged to try other combinations of data, and some are exercises at the end of this chapter. Also, other types of trees attempt to alleviate the above problem, such as AVL trees, but they are beyond the scope of this text, and can be further researched by the reader.

#### 8.5.4 Finding the Minimum and Maximum

Finding the minimum and maximum in a binary search tree is relatively easy. In examining the following tree, what are the minimum and maximum values?



They are 2 and 8. But notice where they are located. The minimum is in the leftmost branch and the maximum is in the rightmost branch. Are they hard to find? Not really. If one looks carefully at the tree, the leftmost and rightmost branches are linear and almost looks like a linked list. In fact, a simple traversal of either leftmost or rightmost branches will get to the end of the branch and access to its data. The code for this is not much different than an iterative traversal of a linked list as found in Sect. 5.6.

To find the minimum item in the tree, the left branch will be traversed. Instead of traversing until `p` is equal to `null`, it is better to instead stop on the last node, so the traversing will continue while `p.left` is not equal to `null`.

```
while(p.left != null)
    p = p.left;
```

However, what if the tree is empty? Testing `p.left` in the `while` statement would result in an exception, so the value of `p` should be tested prior to traversing the branch as shown in the following complete method:

```
private Node findMin(Node p) {
    if(p != null)
        while(p.left != null)
            p = p.left;
    return p;
}
```

Can the maximum value be found? Yes, using similar code, but by traversing the right branch instead of the left branch, and this is an exercise at the end of the chapter. Also, recall that not only can linked lists be traversed using iteration, but using recursion as well as discussed in Sect. 5.7. Can recursion be used instead of iteration to find the minimum and maximum in a binary search tree? Yes, and again both of these routines are also left as exercises at the end of the chapter.

### 8.5.5 Removing an Item from a BST

Having inserted and traversed a BST, it is now time to examine how an item can be removed from a tree. The technique for this is a little more complicated, but given the previous methods, the use of careful analysis and recursion, an algorithm can be developed. Again, it helps to start with a driver program which will accept the number to be removed and call the recursive method as shown below. As before, optionally a call to `traverseBST` could be included here to verify whether the item was removed correctly from the tree, or it could be invoked from the calling program as done in Sect. 8.7.

```
public void removeBST(int num){
    root=remove(num,root);
}
```

The skeleton of the recursive `remove` method would have two parameters: the number to be removed, `num`, and the reference to the tree or subtree, `p`. It will also need to return the modified tree or subtree back to the invoking method:

```
private Node remove(int num, Node p) {
    //body of method
    return p;
}
```

To analyze what should be done and as with the `insert` method, it helps again to start with an empty tree. Should a number need to be removed and the tree is empty, obviously it cannot be removed. Also, to avoid any exceptions, it helps to first check whether `p` is not `null`. Should it be `null`, then a message can be output indicating that the number is not in the tree and it was not removed.

```
if(p != null)

else
    System.out.println("Item not in tree and not removed");
```

Could this test and message be in the driver instead as with the `traverseBST` method? No, because this code is not just checking for an empty tree, but for empty subtrees as well and whether the item is not found. If the tree is not empty, then there are nodes that need to be searched in the then section of the `if` statement. As one might suspect, this part of the code is very similar to the `insert` method. If the item is less than the current node, the left branch is traversed, and if the item is greater than the current node, the right branch is traversed as shown below:

```
if(num < p.data)
    p.left=remove(num,p.left);
else
    if(num > p.data)
        p.right=remove(num,p.right);
```

However, when an item is found, the code is much different. In the `insert` method a simple message statement was output indicating the item was already in the list. In the `remove` method the item must be deleted.

Consider a tree with just one item as follows and the 5 needs to be removed:

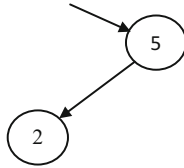


In calling the `remove` method, `num` would contain the 5 and `p` would have a reference from the root node. Since `num` is neither less than or greater than 5, an `else` section would be executed. If both the left and right branches are `null`, then it

is a leaf node. The result is that a null should be returned back to the invoking method (in this case the driver) and root would be set to null.

```
if(p.left == null && p.right == null)
    p = null;
```

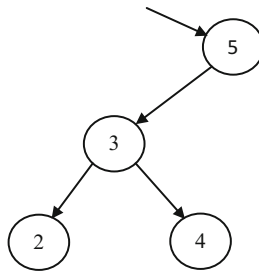
But what if the tree (or subtree) had only a single left branch and the right branch was null as in the following:



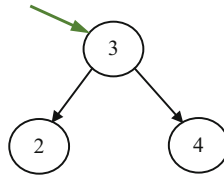
Further, what if the 5 was to be removed from the tree? Assuming p is referencing the node containing the 5, then p should no longer refer to the node containing the 5, but rather it should reference the node containing the 2 as follows:

```
p = p.left;
```

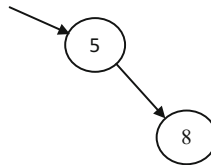
Further, would it matter how large that left branch was? For example, consider the following tree:



No, it would not, because the node containing the 3 would now become the new root of the tree as follows:



But what if there was only a single right branch and the left branch was `null` as follows:



Again, it would not matter how large that right branch would be, and `p` would take on the value of the right branch as follows:

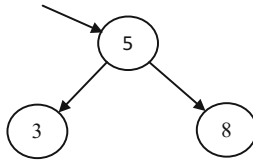
```
p = p.right;
```

Assuming that both branches are not equal to `null`, then one or the other must not be `null`, where the case for both not being `null` will be examined shortly. So if `p.left` is `null`, then `p.right` must contain a reference. Otherwise, if `p.right` is `null`, then `p.left` must contain a reference. So the following `if` statement would work for these two cases:

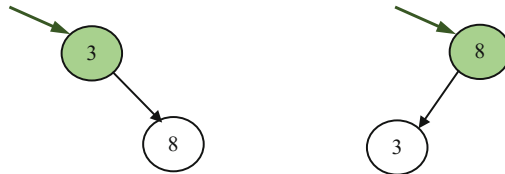
```
if(p.left==null)
    p=p.right;
else
    if(p.right==null)
        p = p.left
```

But what if there were two branches, where both the left and right branches are not `null`? Does this need to be checked? Since both branches are not `null` and further neither one nor the other are `null`, then there must be two branches and this would form the last `else` section of the nested `if-then-else-if` structure. Consider the following tree:

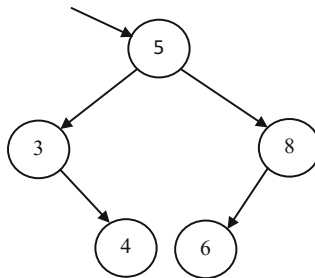




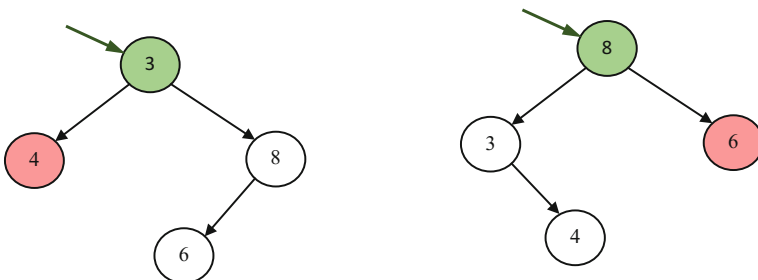
If the 5 is to be removed, which branch should be made the new root node? In one sense, it does not matter and either one of the following is acceptable.



In the left diagram the 3 was brought up to the root node and the 8 remains as the right child and in the right diagram the 8 was brought up and the 3 remained the left child. However, consider the following tree:

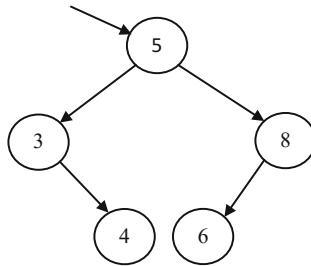


If the 3 is brought up to the root node, the tree might look as shown below on the left. And if the 8 is brought up to the root node, it might look like the one on the right:

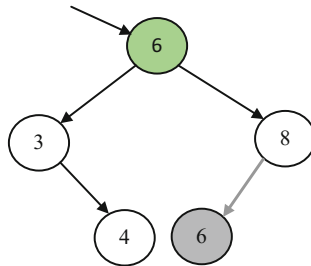


However, what is wrong with both of these trees? Note that in the tree on the left, the 4 is in the left branch and it is greater than 3. In the tree on the right, the 6 is in the right branch and it is less than 8. How can this problem be corrected? Instead of just moving up the left or right branch, the largest number from the left branch can become the root thus guaranteeing that all numbers to the left will be smaller or the smallest number from the right branch can be used thus guaranteeing that all numbers to the right will be larger.

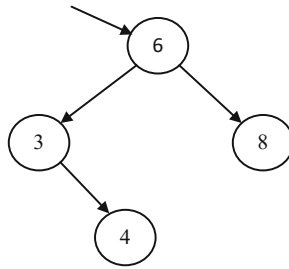
Although either can be used, for ease of programming it would be nice to be consistent. Here the smallest number from the right branch will be used, and the alternative is left as an exercise at the end of the chapter. Returning back to the original tree, what is the smallest number in the right branch?



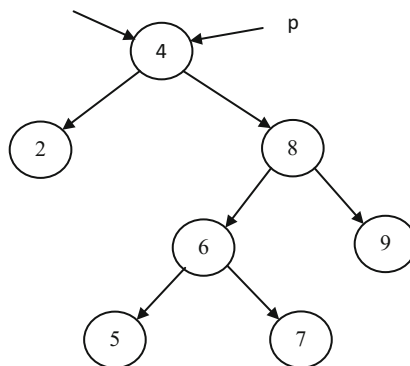
The smallest number in the left branch of the right subtree, is the number 6. It can be moved up to root to replace the 5 as shown below:



But now there are two occurrences of the number 6 in the tree. What needs to be done to correct the problem? The 6 in the left subtree of the right branch must be removed as follows:



Notice that all the numbers to the right of the root are larger and all the numbers to the left are smaller, thus still maintaining the binary search tree property. Having seen how to accomplish this conceptually, what is required to write the necessary code? First the minimum number in the right subtree needs to be found. In this case it is easy, but what about a more complicated tree such as the following where the 4 referenced by `p` would be removed?



The smallest number, 5, is still the one farthest down the left branch of the right subtree. If one recalls from the last section, there was a method developed to find the minimum item in a tree. All that is needed is to call the method from the root of the right subtree as follows:

```
Node t = findmin(p.right);
```

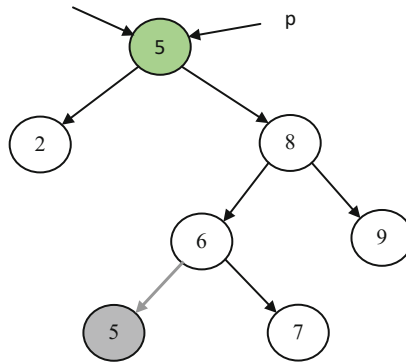
Notice that a temporary variable `t` of type `Node` is used to store the reference to the smallest node found in `p.right`. Then the data from `t.data` is transferred to `p.data` which in this case is where the 4 was at the root of the tree.

```
p.data = t.data;
```

Lastly, the right branch is searched to find and remove the number 5, just like calling for any other number to be removed, except instead of hunting for num, the number that was copied to `p.data` needs to be removed.

```
p.right=remove(p.data,p.right);
```

All of which results in the following:



Putting these three lines together results in the following which is in the last else of the nested if statements:

```

else {
    Node t = findmin(p.right);
    p.data = t.data;
    p.right = remove(p.data,p.right);
}

```

Instead of removing the smallest item from the right subtree, could the code be made to remove the largest from the left subtree? Yes, and again this along with the writing of the `findMax` method is left as an exercise at the end of the chapter. Putting all the above pieces of code together results in the following complete method:

```

private Node remove(int num, Node p) {
    if(p!=null)
        if(num < p.data)
            p.left=remove(num,p.left);
        else
            if(num > p.data)
                p.right = remove(num,p.right);

```

```

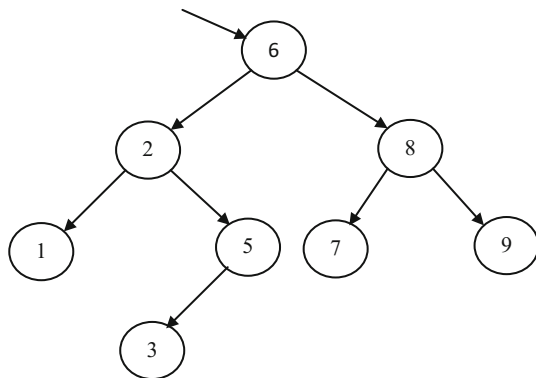
else
    if(p.left==null && p.right ==null)
        p = null;
    else
        if(p.left ==null)
            p = p.right;
        else
            if(p.right ==null)
                p = p.left;
            else {
                Node t = findMin(p.right);
                p.data = t.data;
                p.right = remove(p.data,p.right);
            }
else
    System.out.println("Item not in tree and not removed");
return p;
}

```

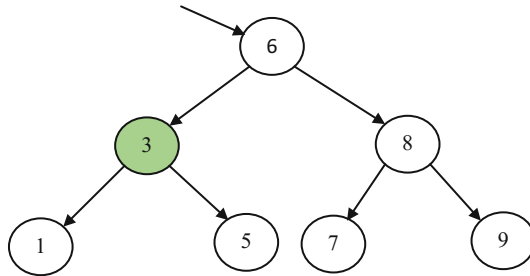
The code above is fairly good, but there are a few modifications that can be made. Note that in the `else` section for dealing with two branches in the tree, the first two lines can be rewritten as one line without the use of the temporary variable `t`. Also, the last three nested `if` statements can be reduced to just two `if` statements by carefully analyzing the logic. To make the code a little cleaner, both of these possible modifications are left as exercises at the end of the chapter.

Does the above code work for other parts of the tree besides just the root? Yes, because, any subtree has a subroot and the code works the same just at a deeper level of recursion. For example, given the tree in Fig. 8.3, what it would look like if one wanted to remove the 2?

**Fig. 8.3** Binary search tree



The result would be that the 3 would be found to be the smallest item in the right subtree of the node with the 2. The 3 would be copied into the spot previously occupied by the 2, and the duplicate 3 would need to be removed as shown below:



Practice walking through the code can help better understand the algorithm and exercises can be found at the end of the chapter.

---

## 8.6 The Complete BinarySearchTree Class

Putting all the pieces together from above results in the complete `BinarySearchTree` class which includes the constructor, `insertBST` driver, recursive `insert` method, `removeBST` driver, `remove` method, `findMin` method, `traverseBST` driver, recursive `traverse` method, and inner `Node` class as shown below:

```
import java.util.*;

public class BinarySearchTree {
    private Node root;

    public BinarySearchTree() {
        root = null;
    }

    public void insertBST(int num) {
        root = insert(num, root);
    }

    private Node insert(int num, Node p) {
        if (p == null)
            p = new Node(num);
        else
            if (num < p.data)
                p.left = insert(num, p.left);
    }
}
```

```
        else
            if(num > p.data)
                p.right = insert(num,p.right);
            else
                System.out.println("Item in tree and not inserted.");
        return p;
    }

    public void removeBST(int num){
        root = remove(num,root);
    }

    private Node remove(int num, Node p){
        if(p != null)
            if(num < p.data)
                p.left = remove(num,p.left);
            else
                if(num > p.data)
                    p.right = remove(num,p.right);
                else
                    if(p.left == null && p.right == null)
                        p = null;
                    else
                        if(p.left == null)
                            p = p.right;
                        else
                            if(p.right == null)
                                p = p.left;
                            else {
                                Node t = findMin(p.right);
                                p.data = t.data;
                                p.right = remove(p.data,p.right);
                            }
            else
                System.out.println("Item not in tree and not removed");
        return p;
    }

    private Node findMin(Node p) {
        if(p != null)
            while(p.left != null)
                p = p.left;
        return p;
    }
}
```

```
public void traverseBST() {
    System.out.print("The tree is:");
    if(root != null)
        traverse(root);
    else
        System.out.print(" " + "Empty");
    System.out.println();
}

private void traverse(Node ptr) {
    if(ptr.left != null)
        traverse(ptr.left);
    System.out.print(" " + ptr.data);
    if(ptr.right != null)
        traverse(ptr.right);
}

//Inner Class: Node
private class Node {
    private Node left;
    private int data;
    private Node right;

    public Node(){
        this(0);
    }

    public Node(int data) {
        left = null;
        this.data = data;
        right = null;
    }
}
```

---

## 8.7 Test Program

The following test program prompts for and inputs integers to be inserted into the BST until a negative integer is input. It then prompts for and inputs integers to be removed from the BST until a negative integer is input. As mentioned earlier, after an integer has been inserted or removed, the tree is traversed to help verify that the integer was inserted into the proper location.



```
import java.util.*;
class Ch72 {
    public static void main(String [] args) {

        Scanner scanner;
        scanner = new Scanner(System.in);

        int number;

        BinarySearchTree tree;
        tree = new BinarySearchTree();

        System.out.println();
        System.out.print("Enter a positive integer to insert ");
        System.out.print("or a negative integer to stop: ");
        number = scanner.nextInt();
        while(number >= 0) {
            tree.insertBST(number);
            tree.traverseBST();
            System.out.println();
            System.out.print("Enter a positive integer to insert ");
            System.out.print("or a negative integer to stop: ");
            number = scanner.nextInt();
        }

        System.out.println();
        System.out.print("Enter a positive integer to remove ");
        System.out.print("or a negative integer to quit: ");
        number = scanner.nextInt();
        while(number >= 0) {
            tree.removeBST(number);
            tree.traverseBST();
            System.out.println();
            System.out.print("Enter a positive integer to remove ");
            System.out.print("or a negative integer to quit: ");
            number = scanner.nextInt();
        }

        System.out.println();
        System.out.println("End of Program");
        System.out.println();
    }
}
```

Sample input and output follows:

```
Enter a positive integer to insert or a negative integer to stop: 5
The tree is: 5
Enter a positive integer to insert or a negative integer to stop: 4
The tree is: 4 5
Enter a positive integer to insert or a negative integer to stop: 8
The tree is: 4 5 8
Enter a positive integer to insert or a negative integer to stop: 6
The tree is: 4 5 6 8
Enter a positive integer to insert or a negative integer to stop: 9
The tree is: 4 5 6 8 9
Enter a positive integer to insert or a negative integer to stop: -1
Enter a positive integer to remove or a negative integer to quit: 6
The tree is: 4 5 8 9
Enter a positive integer to remove or a negative integer to quit: -1
End of Program
```

---

## 8.8 Complete Program: Implementing a Binary Expression Tree

In this Complete Program section, taking pieces from Sects. 8.2 and 8.3, a program which accepts a prefix expression from a user, creates a binary expression tree, and outputs the contents of the tree in infix order will be developed. The application also deals with any type of data using a generic class.

The name of the `BinaryExpressionTree` class from Sect. 8.2.2 is changed to `BinaryExpressionTreeGeneric <T>` and the inner `Node` class becomes `NodeGeneric <T>` class that can accommodate items of generic type. A public driver method, `createTree`, and a private `inputData` method which is a recursive method to create a tree from a prefix expression discussed in Sect. 8.3 are included. To output a public driver method, `inFix`, and the recursive method, `inOrder`, described in Sect. 8.2.3 are also included in the `BinaryExpressionTreeGeneric <T>` class.

```
import java.util.*;

public class BinaryExpressionTreeGeneric < T > {
    private NodeGeneric root;

    public BinaryExpressionTreeGeneric( ) {
        root = null;
    }

    public void createTree(){
        System.out.println();
        System.out.println("Enter a prefix arithmetic expression.");
        System.out.println();
        root = inputData();
    }

    private NodeGeneric inputData() {
        Scanner scanner;
        scanner = new Scanner(System.in);

        System.out.print("Enter an operator or operand: ");
        char ch = scanner.next().charAt(0);
        NodeGeneric p = new NodeGeneric(ch);
        if(ch == '+' || ch == '-' || ch == '*' || ch == '/') {
            p.left = inputData();
            p.right = inputData();
        }
        return p;
    }

    public void inFix(){
        System.out.print("Infix: ");
        inOrder(root);
        System.out.println();
    }

    private void inOrder(NodeGeneric ptr) {
        if(ptr.left != null && ptr.right != null){
            System.out.print("(");
            inOrder(ptr.left);
            System.out.print(" " + ptr.data + " ");
            inOrder(ptr.right);
            System.out.print(")");
        }
        else
            System.out.print(ptr.data);
    }
}
```

```
//Inner Class: NodeGeneric
class NodeGeneric < T > {
    private NodeGeneric < T > left;
    private T data;
    private NodeGeneric < T > right;

    public NodeGeneric() {
        this(null);
    }

    public NodeGeneric(T data) {
        left = null;
        this.data = data;
        right = null;
    }
}
}
```

The main method which uses the `BinaryExpressionTreeGeneric < T >` class is shown below. It asks a user to enter a prefix expression, creates a binary expression tree, and outputs the contents of the tree in infix order.

```
import java.util.*;
class PrefixToInfix {
    public static void main(String [] args){
        Scanner scanner;
        scanner = new Scanner(System.in);

        BinaryExpressionTreeGeneric tree;
        tree = new BinaryExpressionTreeGeneric();

        tree.createTree();
        System.out.println();
        tree.inFix();
        System.out.println();
    }
}
```

When the above program is compiled and executed using a sample input of `* + 234`, the output of the program looks like this:

```
Enter a prefix arithmetic expression.
Enter an operator or operand: *
Enter an operator or operand: +
Enter an operator or operand: 2
```

```
Enter an operator or operand: 3
Enter an operator or operand: 4
Infix: ( (2 + 3) * 4)
```

---

## 8.9 Summary

- In a binary expression tree, the lowest priority operator in an expression forms the root, the next lowest a subroot, and so on.
- Any parentheses in an infix expression are not stored in a binary expression tree.
- A template such as the one found in Sect. 8.4 can help in creating recursive algorithms for trees.
- In a binary search tree (BST) all the items to the left are less than the root or subroot and all of the items to the right are greater than the root or subroot.
- A binary search tree that has had its items entered in sequential order can appear as a linked list thus having its search efficiency reduced from  $O(\log n)$  to  $O(n)$ .

---

## 8.10 Exercises (Items Marked with an \* Have Solutions in Appendix B)

1. In Sect. 8.2.2, the skeleton of the `BinaryExpressionTree` class was implemented having the `Node` class as an inner class. Rewrite the `BinaryExpressionTree` class using an external `Node` class.
2. Walk through the following expressions and draw a binary expression tree similar to the one for  $(2 + 3) * 4$  in Sect. 8.2.1.  
A.  $2 + 3 * 4$                       B.  $5 / 2 - 4 / 3$   
C.  $2 * ((3 - 4) * 5) * D. 5 - (4 + 3) * (2 - 6)$
3. In Sect. 8.2.3, an algorithm that outputs an expression in infix form with parentheses was discussed. Rewrite the `inOrder` method so that it puts parentheses in the appropriate places without any redundant parentheses.
4. Algorithms to output a binary expression tree in prefix form and infix form were introduced in Sect. 8.2. Develop an algorithm outputting the tree in postfix form and call the method `postOrder`.
5. Develop an algorithm that creates a binary expression tree from an infix expression.
6. Develop an algorithm that creates a binary expression tree from a postfix expression.

7. Walk through the algorithm which creates a binary expression tree from a prefix expression for each of the following and draw series of diagrams similar to the ones for +23 in Sect. 8.3.  
A.  $* + 2 3 4$       B.  $+ 2 * 3 - 4 5$   
C.  $- + * 2 3 4 5$     D.  $- * 2 + 3 4 5$
- \*8. Develop an algorithm to traverse a binary search tree in preorder.
9. Develop an algorithm to traverse a binary search tree in postorder.
10. Rewrite the `insert` method in Sect. 8.5.3 to accept duplicate numbers and keep track of the number of occurrences of each number by including a count within the node object.
11. Write a `findMax` method to find the largest number in the binary search tree. It can be found by traversing the right branch.
- \*12. Write a method using a recursion to find the minimum value in a binary search tree.
13. Write a method using a recursion to find the maximum value in a binary search tree.
14. Develop an algorithm to remove a node having two children in a binary search tree by replacing it with the largest number from its left branch.
15. Make modifications suggested in Sect. 8.5.5 to the `remove` method.
16. Draw all possible binary search trees for the three numbers: 3, 4, and 5.
17. Draw a binary search tree after removing the node with the 6 in Fig. 8.3.
- \*18. Draw a binary search tree after removing the node with the 8 in Fig. 8.3.

---

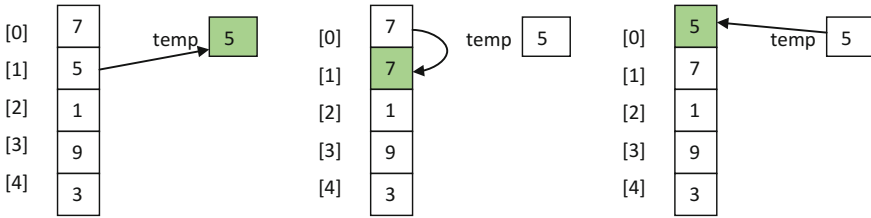
## 9.1 Introduction

Some sorts are slower than others but are easy to create and work well on small sets of data. Many of these sorts are known as “in place” sorts because they swap items within an array and have a speed of  $O(n^2)$ . Examples of these types of sorts are the bubble sort and the selection sort which can be found in *Guide to Java* [2] and *Guide to Assembly Language* [1], respectively. Another sort in this category is the *insertion sort* which will be covered in Sect. 9.2. In contrast, one of the fastest sorts at  $O(n \log n)$  is the *quick sort* and it will be covered in Sect. 9.3. Another  $O(n \log n)$  sort is the *heap sort* which will be discussed in Chap. 10 on Heaps. Lastly, an unusual sort that is used to physically sort items such as checks, but can be adapted to sort data in memory, is the *radix sort* as discussed in Sect. 9.4.

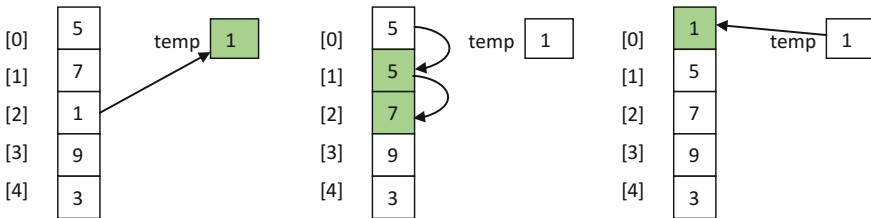
---

## 9.2 The Insertion Sort

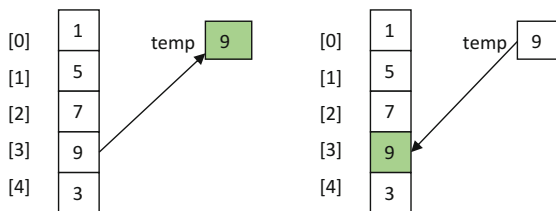
As mentioned above, the insertion sort is a relatively easy sort to write and is good for use with smaller sets of data. The basic idea behind this sort is to take an item from the unsorted part of the list and insert it into the proper order in the ordered part of the list. To demonstrate, consider the following array called `array` containing unordered data. Going from left to right in the diagrams below, initially the 5 in `array[1]` is copied into `temp` and compared with the 7 in `array[0]`. Since the 5 in `temp` is smaller than the 7 in `array[0]`, the 7 is copied down to `array[1]`. Then the 5 in `temp` is copied into `array[0]`. At present, the first two items in the array are sorted.



Then the 1 in array[2] is copied into temp and compared to the sorted items above starting with array[1]. Since 1 is less than 7, the 7 is copied to array[2]. The 1 is then compared with 5 and the 5 is copied to array[1]. Then the 1 in temp is copied into array[0]. At this point the top three items in array are sorted as shown below:



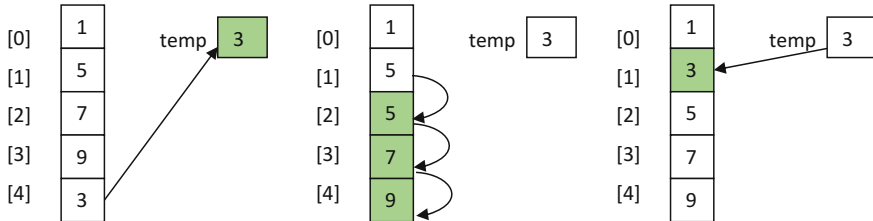
Now the 9 in array[3] is copied into temp and it is compared to the 7 in array[2]. Since 9 is not less than 7, it is copied back to array[3] as shown below. Is it necessary to keep checking further up the array? No, because the items in the top part of the array are in order, and since all the numbers before the 7 are smaller, there is no need to continue comparing. Also, is the copying of 9 back necessary? Technically no, but it is easier to merely copy the item back rather than take the time to see if it needs to be copied.



Lastly, the 3 in array[4] is copied into temp. It is compared with the 9 in array[3] and since 3 is less than 9, the 9 is copied into array[4]. Then the 3 is compared with the 7 in array[2] and since 3 is less than 7, the 7 is copied



into `array[3]`. The 3 is then compared with the 5 in `array[1]` and again, since 3 is less than 5, the 5 is copied into `array[2]`. Lastly, the 3 is compared to the 1 in `array[0]` and since 3 is not less than 1, the 3 is copied from `temp` into `array[1]` and the sort is complete as shown below:



So how can an algorithm be written for this sort? As before, one needs to search for patterns in the above description and figures to determine what kind of iterations, comparisons, and copying need to be done. First, how many passes need to occur through the array? Note that there are five items in the array and the first item to be compared was the item in `array[1]` and the last one was the item in `array[4]`. Assuming that `n` contains the number of items, in this case 5, the outer while loop could be written to loop 4 times as shown below:

```
i=1;
while(i < n) {
    i++;
}
```

During each pass, the value in `array[i]` is placed into `temp`.

```
temp=array[i];
```

Then each item above the `i`th position is compared to `temp`. This would require another loop which would loop from the `i-1` position until the 0th position. The result is a nested loop as follows:

```
j=i-1;
while(j >= 0) {
    j--;
}
```

However, recall that since all the items prior to the `i`th position are in order, there is no need to continue once a smaller number is found, such as with the case with 9 and 3 in the example above. So, while the number `temp` is less than the number in `array[j]`, the loop should continue which results in the following modification to the previous loop:

```
j=i-1;
while(j >= 0 && temp < array[j]) {
    j--;
}
```

Then each number in the array that is larger than the number in `temp` needs to be moved down one slot in the array as follows:

```
array[j+1] = array[j];
```

Once a smaller number is found, then the inner loop terminates and the item in `temp` is copied into its proper location in the array as follows:

```
array[j+1] = temp;
```

The reason for `j+1` is that `j` was decremented by 1 prior to returning to the `while` statement and terminating the loop. Putting all of the above pieces together results in the following method enclosed in the `sortClass`:

```
public class SortClass {
    public SortClass() {
    }
    public void insertionSort(int[] array, int n) {
        int i,j,temp;
        for(i=1; i<n; i++) {
            temp = array[i];
            j = i-1;
```

```
        while(j >= 0 && temp < array[j]) {
            array[j+1] = array[j];
            j--;
        }
        array[j+1] = temp;
    }
}
```

The outer loop iterates  $n-1$  times which is  $O(n)$ . The inner loop iterates from 0 to  $n-i$  times. On average this would be  $n/2$  times and is also  $O(n)$ . Since one loop is nested inside the other, this would be the equivalent of  $n*n$  which is  $O(n^2)$ . However, what if all the data was in ascending order? The outer loop will still need to loop  $n-1$  times, but the inner loop would never iterate. So for presorted data, the insertion sort is one of the fastest sorts at  $O(n)$ , like the bubble sort. Although this should not happen very often, if one wanted to ensure that a data set is indeed already in order, this sort would be a good choice. However, generally data would not be in order and this sort is a relatively simple sort for use on smaller sets of data.

The main program for invoking the `insertionSort` is shown in Fig. 9.1 which with minor modifications can be reused to invoke some of the subsequent sorts:

Sample input and output are given below:

```
Enter an integer or a -1 to stop: 9
Enter an integer or a -1 to stop: 8
Enter an integer or a -1 to stop: 1
Enter an integer or a -1 to stop: 2
Enter an integer or a -1 to stop: 7
Enter an integer or a -1 to stop: 6
Enter an integer or a -1 to stop: 4
Enter an integer or a -1 to stop: 5
Enter an integer or a -1 to stop: 3
Enter an integer or a -1 to stop: -1
```

```
import java.util.*;
class Ch8InsertionSort {
    public static void main(String[] args){
        Scanner scanner;
        scanner = new Scanner(System.in);
        int i,n,number;
        int[] array = new int[20];
        SortClass sortClass = new SortClass();
        i=0;
        System.out.println();
        System.out.print("Enter an integer or a -1 to stop: ");
        number = scanner.nextInt();
        while (number > -1) {
            array[i] = number;
            i++;
            System.out.print("Enter an integer or a -1 to stop: ");
            number=scanner.nextInt();
        }
        n = i;
        sortClass.insertionSort(array,n);
        System.out.println();
        System.out.println("Insertion Sort");
        System.out.println();
        for(i=0; i<n; i++)
            System.out.println("    " + array[i]);
        System.out.println();
    }
}
```

**Fig. 9.1** Main program for invoking the insertion sort

Insertion Sort

1  
2  
3  
4  
5  
6  
7  
8  
9

## 9.3 The Quick Sort

One of the fastest sorts on random data is the quick sort, thus its name. However, with increased speed comes more complexity. First, this sort essentially tries to select a number that would appear approximately in the middle of the range of the numbers, or in other words the median. It then splits the list of numbers into two sections and moves all the items smaller than the median into the top portion of the list and puts all the items larger than the median in the bottom portion of the list. This process continues on each of the sections and subsections until the list is sorted.

### 9.3.1 General Concepts

For example, consider the following list drawn horizontally for convenience:

5 9 4 8 1 2 3 7 6

The median of this range of numbers from 1 to 9 is the number 5. This number would be swapped with a number near the middle of the array with the swapped numbers in bold and underlined as follows:

1 9 4 8 **5** 2 3 7 6

Then all the numbers less than 5 would be placed in the left or upper part of the array and all the numbers larger would be placed in the bottom or right part of the array. First the 9 and 3 would be swapped as shown below:

1 **3** 4 8 5 2 **9** 7 6

And then the 8 and 2 would be swapped:

1 3 4 **2** 5 **8** 9 7 6

Note that all the numbers less than 5 are on the left and all the numbers larger than 5 are on the right. This process continues again on each of the two sections, where the median in each section is found and moved to approximately the middle of each section. Looking at the section left of the 5 first, since a fraction cannot be

used, a truncated integer could be used. So the median of the left section is 2 and is swapped with the 3 to be in approximately the middle of the section as follows:

1 2 4 3 5 8 9 7 6

Then the numbers are moved before and after the middle number as before, but in this case, they are on the correct side. Then the left subsection is sorted, where the 1 and the 2 are already in order and then the right subsection is sorted as shown below:

1 2 3 4 5 8 9 7 6

The median of the section to the right of the 5 is found and the 7 is swapped to be approximately in the middle as follows:

1 2 3 4 5 8 7 9 6

Then the items on the left that are greater than the median of the section are swapped with the items less than the median on the right:

1 2 3 4 5 6 7 9 8

Then the two subsections are sorted where the 6 and 7 in the left subsection are already in order and the 9 and 8 in the right subsection are sorted, with the sorted list as follows:

1 2 3 4 5 6 7 8 9

### 9.3.2 Potential Problems

If this sounds like a job for recursion, the answer is yes. However, there are a number of details that can cause problems with the quick sort that can make it a slower sort. First is the difficulty trying to find a number that is or is approximately the median. A program could go through all the numbers, but this would take time thus slowing down the sort. Assuming that the numbers are in random order, then any number could be chosen and there might be a chance that this number is somewhere near the middle of the range of numbers. If this is the case, the first item in the list could be chosen which in the original list at the beginning of this section happened to be the number 5.

But what if the items happened to already be sorted? The first number in the list would be the lowest number, in this case 1, and would be nowhere near the middle of the list. Then one section would have a length of zero and the other section would be the rest of the numbers. The advantage of having two approximately equal sections would be lost and this would slow the sort down considerably. This same problem would occur if the last number of the list was chosen, where the length of the right section would be zero.

Another possibility is to choose a number from the middle of the list. In the case of an ordered list, this would be the number 5 and the sort could continue on as intended. But unlike the insertion sort, the sorting of a list that is already in order is not one of the strengths of the quick sort. Further, consider the original list at the beginning of this section where the number 1 happened to be in the middle. Again, the length of the left section would be zero and the right section would contain the rest of the numbers.

A possible solution to this problem is to choose three numbers: one from the beginning, one from the end, and one from the middle of the list and find the median of the three numbers. If the list happened to be in order, then the middle number would be chosen. If the numbers are indeed in random order, this would increase the chance of picking a number that is near the median. In the original list the three numbers are 5, 1, and 6, where the middle number is 5. Although this is the same as before, if the number 5 had not been in the first position and some other larger number was there, then the number 6 would have been chosen. This technique does not guarantee that a number near the middle of the range of numbers would be chosen, but it certainly helps increase the odds.

Another problem with the quick sort is that it is not very efficient with small sets of numbers. Notice in the above example, when the subsections of two numbers were being sorted, the use of a recursive algorithm to attempt to sort lists of one, two, or three elements is rather inefficient. To help with this type of situation, the quick sort can be written to call a different sort if a section is less than 10–20 elements. For example, with a larger array of one thousand elements and assuming each subsection is approximately half of the previous section, then as the quick sort recursively calls each subsection the sizes would start at 1000 and then be approximately 500, 250, 125, 62, 31, and 15. Instead of continuing on to subsections of size 7, 3 and then 1, the quick sort could call another sort such as the insertion sort for the small subsections of size 15 or smaller, because the insertion sort would be more efficient than the quick sort.

### 9.3.3 The Sort3 and SwapElements Methods

The technique for finding the middle of three elements involves a method that has three parameters. These three elements are then compared to each other and the median would be returned. But since the three numbers need to be compared anyway, might it be helpful to not only find the median, but sort these three numbers as well since they will need to be moved to either side of the median later

anyway? This can be accomplished as follows: Initially, the first and second elements are compared and if the second is smaller than the first, they are swapped. Then the first and third elements compared and if the third one is smaller, then they are swapped. Lastly, since there is no guarantee that the last two are in order, the second and third items are compared and if the third one is less than the second, they are swapped. The result is that the first, second, and third items are in order.

The three positions in the array that need to be compared are sent along with the array as shown in the following method:

```
public void sort3(int[] a, int first, int second, int third) {
    if(a[second] < a[first])
        swapElements(a, second, first);
    if(a[third] < a[first])
        swapElements(a, third, first);
    if(a[third] < a[second])
        swapElements(a, third, second);
}
```

The ability to swap elements of an array is discussed in many introductory texts such as *Guide to Java* [2]. Further, it can be made into a method which needs to be sent parameters for the array *a*, and the indexes *i* and *j* of the elements in the array to be swapped. The `swapElements` method is given below:

```
public void swapElements(int[] a, int i, int j) {
    int temp;
    temp = a[i];
    a[i] = a[j];
    a[j] = temp;
}
```

### 9.3.4 The quickSort Method

Since the `quickSort` routine will be recursive, it is helpful to have a driver routine as with other recursive routines. Given an *n* element array, the leftmost location in the array is 0 and the rightmost position is element *n*-1. If there were no data to be sorted, *n* would be 0, so *n*-1 would be -1 and would cause an exception of an under-indexed array. The solution is to include an `if` statement and the following would work:



```
public void quickSort(int[] array, int n) {  
    if(n>0)  
        quickSort(array,0,n-1);  
}
```

The skeleton for the overloaded recursive `quickSort` method would be sent the `left` and `right` locations of the entire array, or later sections of the array, that need to be sorted as follows:

```
private void quickSort(int[] array, int left, int right) {  
    // rest of the quick sort code  
}
```

The previous two methods, `sort3` and `swapElements`, can be used to find the median of three numbers and to sort three items in an array. If one knows the location of both the left element and the right element of an array, then the location of the element in the middle of the array can be found by adding the `left` and `right` indexes and dividing the sum by 2.

```
int middle = (left+right)/2;
```

For example, given a 9 element array sent from the driver routine, `left` would be set to 0 and `right` would be set to 8. The sum of the two would be 8, which divided by 2 would be 4 and assigned to `middle`. In the case of an even number of elements such as 8, `left` would again be 0, `right` would be 7, and when divided by 2 the `middle` would be set to 3. Remember when dealing with integers, the result of integer division is an integer. A question here might be since in these examples `left` has been 0, why bother adding the 0? Remember that not only an entire array will be sent to the method, but sections and subsections can be sent which can begin anywhere within the array. For example, if a section is from location 4 to 8, then  $4 + 8$  is 12 and when divided by 2 the result in `middle` would be 6.

Having determined `middle`, these three positions in the array could be sent to the `sort3` method which would place the three numbers in order. Further, the one in the middle of the three is the one that will be used to approximate the median.

```
sort3(a, left, middle, right);
```

Note that the sizes of the arrays used for examples in this text will not be as large as a one-thousand element array, but rather will more likely be ten to twenty elements long. If small arrays are used, the quick sort might just call the insertion

sort and the algorithm for the quick sort would not be illustrated. As a result, the use of the insertion sort will not be demonstrated here but is left as an exercise at the end of the chapter.

Instead, a unique solution to this problem will be shown that utilizes the above `sort3` method to sort the smaller sections that are inefficient for the quick sort. If any section has three or fewer elements, the `sort3` method can sort them. Although it might end up trying to sort only two elements, this is better than having the quick sort trying to do it recursively which would be very inefficient. So, in addition to approximating the median when there are more than three elements, `sort3` can be used to sort when there are three or fewer elements.

The number of elements can be determined by subtracting `left` from `right` and if the number is greater than or equal to 3, this indicates that there are more than three elements in the array. For example, if an array is four elements long, then `left` would be 0 and `right` would be 3, then `right-left` would equal 3 indicating more than three elements.

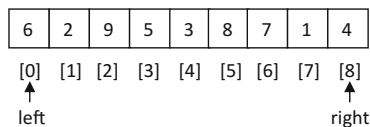
Putting the above ideas together results in the following code segment:

```
private void quickSort(int[] array, int left, int right) {
    int middle = (left+right)/2;
    if(right-left >= 3) {
        sort3(array, left, middle, right);
        // rest of the quick sort code
    }
    else
        sort3(array, left, middle, right);
}
```

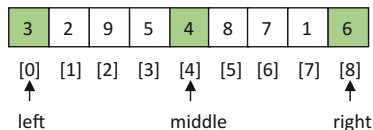
However, as has happened in the past, the above code can be cleaned up. Whether `sort3` is used to find a midpoint for the `quickSort` method or there are three or fewer items that need to be sorted, the `sort3` method is the first line of code in both the then or else sections of the `if` statement. As a result, it can be moved prior to the `if` statement and instead of using an if-then-else structure, a simpler if-then structure can suffice as follows:

```
private void quickSort(int[] array, int left, int right) {
    int middle=(left+right)/2;
    sort3(array, left, middle, right);
    if(right-left >= 3) {
        // rest of the quick sort code
    }
}
```

To illustrate how this code works, consider the list below of length 9, where `left` would contain a 0 and `right` would contain an 8 as indicated by the arrows below:



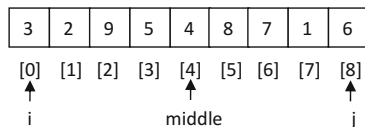
After calculating `middle` as 4 and calling `sort3`, the array would look as follows:



Since `right-left` is greater than or equal to 3, there are more than three items in the list. To continue developing the rest of the code, since `left` and `right` will be needed later when making recursive calls they can be assigned to temporary variables `i` and `j`, respectively. As will be seen, `i` will proceed from the left toward the right and `j` will proceed from the right to the left to allow swapping of items to their correct side of the array or section. While `i` is less than `j`, they will continue on until they cross.

```
int i = left;
int j = right;
while(i < j) {
    // rest of the quick sort code
}
```

Note that initially `i` is set to `left` which is 0 and `j` is set to `right` which is 8 as indicated by the arrows below:



But since these two elements have already been sorted by the `sort3` method, there is no need to check to see if these two numbers should be swapped. Instead `i` and `j` should look at the next items in the list. In particular, `i` should move to the next number to the right and see if it less than the number stored in array `[middle]`. If it is, then `i` should move on to the next item in the array, and so on.

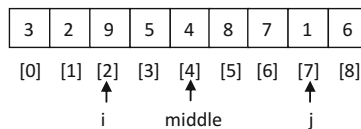
However, if the number is larger than the one in `array[middle]`, then `i` should quit moving so that the current number can be moved to the other side of the array. Similarly, `j` needs to move to the left off the last element in the array and check whether the next item is greater than the number in `array[middle]`. If it is, then it is on the correct side of the array and `j` needs to move to next item and so on until it finds a number smaller than `array[middle]`.

This process can be done by a pair of non-nested loops. Although a `while` loop could be used, since `i` and `j` both need to be incremented to move off the already sorted first and last elements, respectively, a `do-while` loop is well suited for this task.

```
do {
    i++;
} while(array[i] <= array[middle]);
do {
    j--;
} while(array[j] >= array[middle]);
```

Notice that `<=` and `>=` are used to allow `i` and `j` to get past the number in `array[middle]` to continuing hunting for numbers as will be explained shortly. Further, note that the above code would not work for duplicate items in the array because if all the items were identical, then `i` or `j` could go off the end of the array and cause an out of bounds exception. This does not happen with nonduplicate numbers because the `sort3` method initially sorts numbers to the correct side on both ends of the array (or subsection of an array) to cause both of the above loops to stop iterating. In this discussion, the code presented will only consider non-duplicate items and the modification of the above loops to allow for duplicate items is left as an exercise at the end of the chapter.

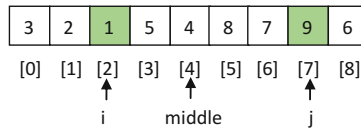
Continuing, the first `do-while` loop is started and `i` is incremented by 1. Since the 2 in `array[1]` is less than the 4 in `array[middle]`, the loop iterates and `i` is incremented by 1. But this time the 9 in `array[2]` is not less than `array[middle]` so the iteration of the first `do-while` stops. Then the next `do-while` loop is executed and `j` is decremented. Since the 1 in `array[7]` is less than `array[middle]`, this loops stops iterating as well as shown below:



If `i` and `j` have not crossed, indicated by `i` being less than `j`, the two values should be swapped using the same `swapElements` method that is utilized in the `sort3` method.

```
if(i<j)
    swapElements(array,i,j);
```

In this particular case  $i$  is less than  $j$ , and the swap occurs as follows:

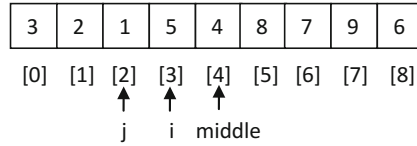


Putting all of the above together so far with the previous `if` and `while` statements results in the following partial code segment:

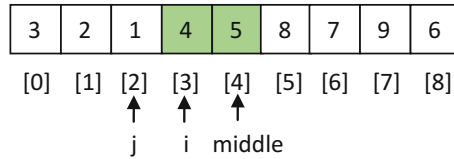
```
int middle = (left+right)/2;
sort3(array, left, middle, right);
if(right-left >= 3) {
    int i=left, j=right;
    while(i<j) {
        do {
            i++;
        } while(array[i] <= array[middle]);
        do {
            j--;
        } while(array[j] >= array[middle]);
        if(i<j)
            swapElements(array,i,j);
    }
    // rest of the quick sort code
}
```

Continuing with the walk-through of the code, control is returned back to the beginning of the `while` loop and since  $i$  is less than  $j$ , the outer loop continues. In the first `do-while` loop  $i$  is incremented and the 5 in `array[3]` is greater than or equal to the 4 in `array[middle]` so the loop stops. In the second `do-while` loop  $j$  is decremented and the 7 in `array[6]` is greater than the 4 in `array[middle]`. Again,  $j$  is decremented and the 8 in `array[5]` is greater than 4. Continuing,  $j$  is decremented again, but now  $j$  is equal to `middle` and is referring to the same array element as `array[middle]`.

Although some versions of the quick sort move the middle element to either the first or last element in the array to keep it out of the way, this current method keeps the middle element in the middle of the array. Although it results in an extra comparison, it also avoids an unnecessary swap from either end of the array. Again, it is why the comparison in the two `do-while` loops includes a test for equality as



**Fig. 9.2** The indexes  $i$  and  $j$  have crossed left of middle

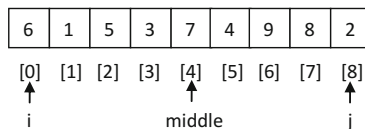


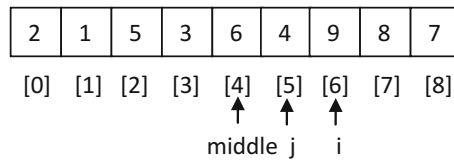
**Fig. 9.3** Swap of  $\text{array}[i]$  and  $\text{array}[\text{middle}]$

$\leq$  and  $\geq$  instead of  $<$  and  $>$ , respectively. In this case since 4 equals 4,  $j$  is decremented again. The 5 in  $\text{array}[3]$  is greater than 4 and  $j$  is again decremented. However, the 1 in  $\text{array}[2]$  is not greater than or equal to 4, so the second `do-while` loop stops iterating. But note that  $i$  is no longer less than  $j$ , so the `then` section of the `if` statement is skipped and the outer `while` loop stops iterating as shown in Fig. 9.2.

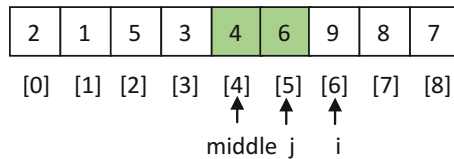
Before continuing on after the `while` loop and making recursive calls to sort the two sections, notice above that the larger number 5 is to the left of the 4 in the array. Further, the 4 cannot stay in the middle of the array because it is not a true median. Remember that it was just the median of three numbers, used to approximate the median of all the numbers. The result is that it needs to move from the middle towards the left. Its proper place in the list is so that all the numbers to the left are smaller and all the numbers to the right are larger. The result is that the 5 and 4 need to be swapped or in other words  $\text{array}[i]$  and  $\text{array}[\text{middle}]$  need to be swapped as shown in Fig. 9.3.

Now all of the numbers to the left of the 4 in  $\text{array}[i]$  are smaller and all of the numbers to the right are larger. But what if the median of three numbers appears to the right of the middle, couldn't the opposite occur as well? The answer is yes as in the following simplified scenario and new list of numbers:



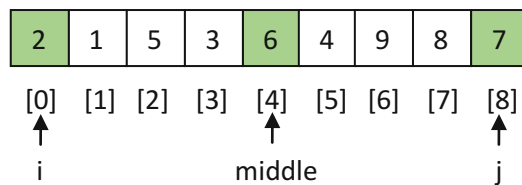


**Fig. 9.4** The indexes  $i$  and  $j$  have crossed right of middle



**Fig. 9.5** Swap of array[middle] and array[j]

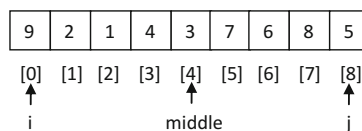
First the three items are sorted as shown below:



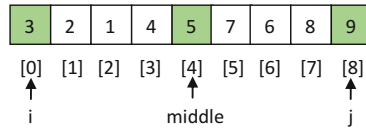
In the first `do-while` loop `i` would continue until it reached the 9 in array `[6]` and in the second `do-while` loop `j` would continue to the left until it reached the 4 in array `[5]`. Since `i` and `j` have crossed, then the outer `while` loop would stop as shown in Fig. 9.4.

In this case the 4 is to the right of the 6 indicating that `array[j]` should be swapped with `array[middle]` as shown in Fig. 9.5.

So how can one tell whether `array[i]` or `array[j]` should be swapped with `array[middle]`? Before answering this question, consider one more scenario where the median of the three selected numbers happens to also be the median of all the numbers as in the following simplified example and new list of numbers:



As before, `sort3` is invoked and the array looks as follows:



As `i` and `j` go left and right, respectively, they will cross, and `i` will end up on the 7 in `array[5]` and `j` will end up on the 4 in `array[3]`. As shown in Fig. 9.6, the 5 is already in the correct position and no swap needs to occur.

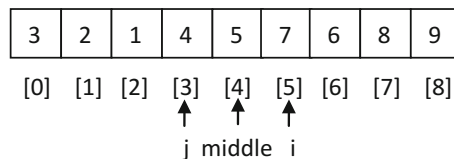
Given these three scenarios, what can be used to indicate whether a swap with `middle` should occur prior to making the recursive calls to sort the two sections? As before, the pattern needs to be discovered. Notice that in the first scenario in Fig. 9.2, both `i` and `j` were left of `middle`, in which case `array[middle]` needed to be swapped with `array[i]`. In the second scenario in Fig. 9.4, both `i` and `j` were to the right of `middle` indicating that `array[middle]` needed to be swapped with `array[j]`. And in the last case in Fig. 9.6, `i` and `j` were on opposite sides indicating that no swap was necessary. The following code implements this pattern:

```

if(i<middle && j<middle) {
    swapElements(array,i,middle);           // Fig. 9.2
    // other code
}
else
    if(i>middle && j>middle) {
        swapElements(array,j,middle);       // Fig. 9.4
        // other code
    }
    else {
        // other code                       // Fig. 9.6
    }

```

In each of the above cases, the left and right sections need to be called recursively. However, in each case the median will be in a different location and this



**Fig. 9.6** Swap not necessary since median is already in the middle



needs to be reflected in the subsequent recursive calls. In the first case, the calls should be on either side of  $i$  with  $i-1$  and  $i+1$ . Similarly in the second case, the calls should be on either side of  $j$ , and the third case even though no swap occurs, the calls should be on either side of `middle`. This is reflected in the following possible code with the differences highlighted in bold:

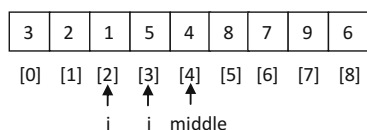
```
// possible solution
if(i<middle && j<middle) {
    swapElements(array, i, middle);
    quickSort(array, left, i-1);
    quickSort(array, i+1, right);
}
else
    if(i>middle && j>middle) {
        swapElements(array, j, middle);
        quickSort(array, left, j-1);
        quickSort(array, j+1, right);
    }
    else {
        quickSort(array, left, middle-1);
        quickSort(array, middle+1, right);
    }
}
```

Again, note that in the third case there is no swapping and the recursive calls are to the left and right of `middle`. However, having three different sets of recursive calls to `quickSort` could be confusing when trying to walk through the code. Can the number of calls to `quickSort` be reduced? Instead of six separate calls, the values of  $i$ ,  $j$ , or `middle` can be stored in a temporary variable such as `temp` that could be used as a parameter in the subsequent recursive invocations as given below:

```
int temp;
if(i<middle && j<middle)
    temp = i;
else
    if(i>middle && j>middle)
        temp = j;
    else
        temp = middle;
swapElements(array, temp, middle);
quickSort(array, left, temp-1);
quickSort(array, temp+1, right);
```

This code appears much cleaner than the previous code segment where instead of two calls to `swapElements` there is now only one, and instead of six calls to `quickSort` there are only two. But what are the disadvantages of the above code segment? First there is the inclusion of the temporary variable `temp` which in and of itself does not seem bad, but remember this variable is appearing in a recursively called method and each time `quickSort` is invoked another instance of this variable is declared. The other disadvantage with this is that if `temp` is set to `middle`, then `array[middle]` will be swapping with itself. But the odds that the median of the three is the same as the median of the entire section is fairly small, especially with very large arrays.

Continuing with the example where `i` and `j` are both left of `middle` in Fig. 9.2 and redrawn below:



Using the cleaner code developed above, the value of `i` is placed into `temp` and the contents of `array[middle]` and `array[temp]` are swapped by `swapElements` as shown below:

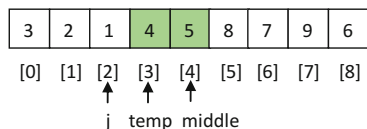
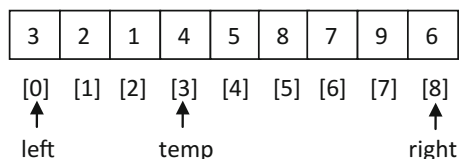


Figure 9.7 shows the array and variables in preparation for the recursive calls by excluding `j` and `middle`, and instead showing `left`, `right`, and `temp`:

At this point the 4 is in the correct position at `array[temp]` with all the numbers to the left smaller and all the numbers to the right larger. Now it is necessary to sort to the left of `temp` and to the right of `temp`. Looking at the left first, what numbers need to be sorted? Although all of them are less than 4, they are not necessarily in the correct order. So, the following call to `quickSort` is invoked from `left` to one just left of the 4 at `temp-1`:



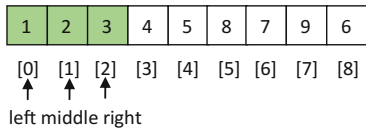
**Fig. 9.7** Showing `left`, `temp`, and `right`

```
quickSort(array, left, temp-1);
```

When quicksort is called recursively the above arguments will be sent to the following parameters:

```
private void quickSort(int[] array, int left, int right) {
```

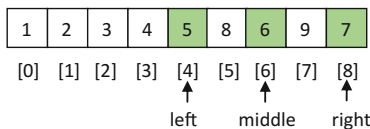
The parameter left will contain the 0 from the argument left and the parameter right will contain the contents of the argument temp-1 which in this case will be 2. Upon entry to the quickSort routine, the sort3 method will be invoked and the three elements will be sorted. Since the difference between right and left is less than 3, this is a terminal case and there is no recursion. Having sorted the three elements to the left, the array would look as follows:



Now the right section of Fig. 9.7 needs to be sorted from one to the right of the 4 at temp + 1 to right:

```
quickSort(array, temp+1, right);
```

The ensuing recursive call to quickSort will have the parameter left set to temp + 1 which is 4 and right which is 8. The middle is calculated as 6 and the sort3 method is called which results in the following:



Since the difference between right and left is greater than or equal to 3, i and j take on the values of left and right, respectively. The index i is incremented to 5 and it stops at the 8 and j is decremented and ends up passing by the 9, 6, and 8 because they are greater than or equal to the 6 in array[middle] and stops at array[4] which contains a 5. Since i is now greater than j there is no swap and the outer while loop stops.

|     |     |     |     |     |     |        |     |     |
|-----|-----|-----|-----|-----|-----|--------|-----|-----|
| 1   | 2   | 3   | 4   | 5   | 8   | 6      | 9   | 7   |
| [0] | [1] | [2] | [3] | [4] | [5] | [6]    | [7] | [8] |
|     |     |     |     | ↑   | ↑   | ↑      |     |     |
|     |     |     |     | j   | i   | middle |     |     |

Since both  $i$  and  $j$  are less than  $middle$ ,  $temp$  is set to  $i$ , and the 8 and the 6 are swapped as shown below:

|     |     |     |     |     |      |        |     |     |
|-----|-----|-----|-----|-----|------|--------|-----|-----|
| 1   | 2   | 3   | 4   | 5   | 6    | 8      | 9   | 7   |
| [0] | [1] | [2] | [3] | [4] | [5]  | [6]    | [7] | [8] |
|     |     |     |     | ↑   | ↑    | ↑      |     |     |
|     |     |     |     | j   | temp | middle |     |     |

The `quickSort` method is called on the left subsection of the right section where  $left$  will be equal to 4 as it was upon entry to the method and  $right$  takes on the value  $temp-1$  ;which equals 4. It then calculates  $middle$  as  $4 + 4/2$  as 4 which is the same as  $left$  and  $right$ . The `sort3` method is invoked which does some comparisons, but no swapping as shown below:

|     |     |     |     |        |     |     |     |     |
|-----|-----|-----|-----|--------|-----|-----|-----|-----|
| 1   | 2   | 3   | 4   | 5      | 6   | 8   | 9   | 7   |
| [0] | [1] | [2] | [3] | [4]    | [5] | [6] | [7] | [8] |
|     |     |     |     | ↑      |     |     |     |     |
|     |     |     |     | left   |     |     |     |     |
|     |     |     |     | middle |     |     |     |     |
|     |     |     |     | right  |     |     |     |     |

Although the comparisons on one element are redundant, there are no costly swaps and it is much easier than the quick sort trying to sort one element. Since  $right-left$  is equal to 0 and is less than 3, control is returned to sort the right subsection.

The next recursive call to `quickSort` is for the right subsection and has  $left$  take on the value of  $temp + 1$  which is 6 and  $right$  equal to 8 as it was upon entry to the method. Again,  $middle$  is calculated as  $(6 + 8)/2$  which is 7, `sort3` is invoked, and the right three elements are sorted as follows:

|     |     |     |     |     |     |      |        |       |
|-----|-----|-----|-----|-----|-----|------|--------|-------|
| 1   | 2   | 3   | 4   | 5   | 6   | 7    | 8      | 9     |
| [0] | [1] | [2] | [3] | [4] | [5] | [6]  | [7]    | [8]   |
|     |     |     |     |     |     | ↑    | ↑      | ↑     |
|     |     |     |     |     |     | left | middle | right |

Since  $right-left$  is less than 3, no recursion takes place. Control is returned to the section and then back to the original invocation of `quickSort`, which then returns back to the driver and calling routine with the sorted array as shown above.

As can be seen, the `quickSort` method is highly recursive and it is important to avoid recursion on small data sets. The complete `quickSort` method in the `SortClass` along with the driver, `sort3`, and `swapElements` is as follows:

```
public class SortClass {

    public SortClass() {
    }

    public void quickSort(int[] array, int n) {
        if(n>0)
            quickSort(array,0,n-1);
    }

    public void swapElements(int[] a, int i, int j){
        int temp;
        temp = a[i];
        a[i] = a[j];
        a[j] = temp;
    }

    public void sort3(int[] a, int first, int second, int third) {
        if(a[second] < a[first])
            swapElements(a,second,first);
        if(a[third] < a[first])
            swapElements(a,third,first);
        if(a[third] < a[second])
            swapElements(a,third,second);
    }

    private void quickSort(int[] array, int left, int right) {
        int middle = (left + right) / 2;
        sort3(array,left,middle,right);
        if(right-left >= 3) {
            int i = left, j = right;
            while(i < j) {
                do {
                    i++;
                } while(array[i] <= array[middle]);
                do {
                    j--;
                } while(array[j] >= array[middle]);
                if(i < j)
                    swapElements(array,i,j);
            }
            int temp;
```

```

        if(i < middle && j < middle)
            temp = i;
        else
            if(i > middle && j > middle)
                temp = j;
            else
                temp = middle;
        swapElements(array,temp,middle);
        quickSort(array,left,temp-1);
        quickSort(array,temp+1,right);
    }
}
}

```

The main program would be the same as the one used for the insertion sort in Fig. 9.1 except that the call to the sort should be changed to `sortClass.quickSort(array,n)` and the column heading should be changed to `SystemSystem.out.println("Quick Sort")`. Sample input and output is shown below:

```

Enter an integer or a -1 to stop: 9
Enter an integer or a -1 to stop: 8
Enter an integer or a -1 to stop: 1
Enter an integer or a -1 to stop: 2
Enter an integer or a -1 to stop: 7
Enter an integer or a -1 to stop: 6
Enter an integer or a -1 to stop: 4
Enter an integer or a -1 to stop: 5
Enter an integer or a -1 to stop: 3
Enter an integer or a -1 to stop: -1

```

Quick Sort

```

1
2
3
4

```

5  
6  
7  
8  
9

---

## 9.4 The Radix Sort

The radix sort (or bucket sort) is unlike either of the previous two sorts. Its primary purpose in the past has been to sort physical items such as letters for the post office. Although each of the places where the sorted items are manually placed can be called buckets, on machines that do sorting they are often called pockets. Further, it can also be used to sort items in a computer's memory and displays some interesting characteristics.

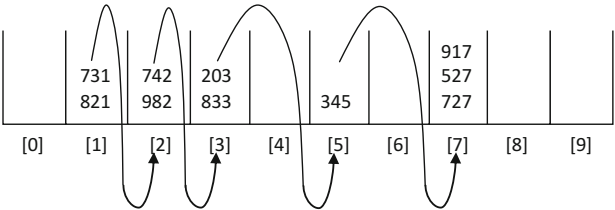
### 9.4.1 General Concepts

Although the radix sort can be used with any base numbering system as will be shown later, it helps to first to examine how the sort works using the base 10 (or decimal) numbering system and ten pockets. Assume that the following numbers are to be sorted:

727 345 982 821 527 833 742 203 731 917

First, given the order above, each number is placed in order into the pocket based upon the least significant digit position as shown below:

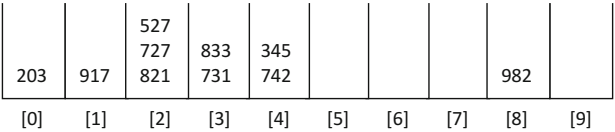
Then each of the numbers are removed or "swept" from the pockets. However, it is imperative that the numbers are removed in the proper order going from left to right and in each pocket from bottom to top as indicated by the arrows below:



The result is the list of numbers as follows:

821 731 982 742 833 203 345 727 527 917

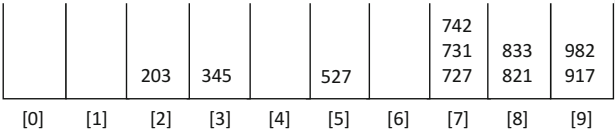
The numbers are then placed into the pockets based on the second digit of the numbers as shown below:



The numbers are again swept from the pockets in the same sequence as before:

203 917 821 727 527 731 833 742 345 982

They are now sorted into the pockets one last time based on the most significant digit as follows:



A final sweep of the pockets using the same sequence results in the correct order of all the numbers:

203 345 527 727 731 742 821 833 917 982

Note that, given a different initial sequence of numbers, the first and second passes would have a different order in each of the pockets, but the final sequence would of course be correct. To illustrate this the reader is encouraged to try a



different first sequence above to see that the process still works correctly and also to try some of the exercises at the end of the chapter.

Again, this process is especially helpful in sorting physical items such as checks according to the bank routing number or checking account numbers that are written in magnetic ink on the check and read based on magnetic ink character recognition (MICR). Also, letters by the postal service can be sorted based upon the zip code of the mailing address read by optical readers using optical character recognition (OCR).

### 9.4.2 More Pockets

Note that the number of pockets does not need to be 10. Using the same numbers from above, what if there were 1000 pockets? Whereas with the example in the previous section using 10 pockets it took three passes to sort the numbers, if there were 1000 pockets there would only need to be one pass. This would be much faster, but does this faster process result in an increase in memory? It depends.

Notice that if there were one thousand pockets but only ten numbers needed to be sorted there is a large amount of wasted space as illustrated below:

|       |     |       |     |       |     |       |     |       |     |
|-------|-----|-------|-----|-------|-----|-------|-----|-------|-----|
|       |     |       |     |       |     |       |     |       |     |
|       |     | 203   |     | 345   |     | 527   |     | 727   |     |
| [000] | ... | [203] | ... | [345] | ... | [527] | ... | [727] | ... |

On the other hand, if there had been close to a thousand numbers to sort, there would be very little wasted space. Although using 1000 pockets appears to use more space, consider if almost one thousand items needed to be sorted with only 10 pockets. In this case, each pocket would need to hold close to 100 items. Although sorting physical items with 10 pockets might not take up as much physical space, it is different when using memory and this topic will be explored further in Sect. 9.4.4.

### 9.4.3 Fewer Pockets

What if instead of more pockets, there were fewer pockets, such as only two pockets? If there are ten pockets, the numbers to be sorted use base 10. However, if there are only two pockets, then the base 10 numbers would need to be converted to binary or base 2 numbers.

As a brief introduction, in base 10 each digit position of a number represents a power of 10. So, for example, the number  $14_{10}$  represents 1 ten and 4 ones. Although this seems fairly obvious, the same holds true for base 2 where each digit position number represents a power of 2. For example, what would the binary

number  $1110_2$  be in base 10? Starting with the right-most position there is a 0 in the ones position, a 1 in the twos position, a 1 in the fours position, and a 1 in the eights position. Adding up each of the powers results in the number  $14_{10}$ .

The conversion from base 10 to base 2 can be accomplished by repetitively dividing a base 10 number by 2 as indicated by the downward arrow below. For example, to convert the number  $14_{10}$  to base 2 would be as follows where  $r$  represents the remainder:

14/2 = 7 r 0

7/2 = 3 r 1

3/2 = 1 r 1

1/2 = 0 r 1

↓

↑

Then taking the reminders in reverse order as indicated by the upward arrow above the number is  $1110_2$ . Although this is just a short introduction, one can always refer to a computer organization or an assembly language book, such as *Guide to Assembly Language* [1] for a complete description. Given the following list of decimal numbers that have been converted to binary, how can they be sorted?

15   02   01   07   05   11   10   12

1111 0010 0001 0111 0101 1011 1010 1100

Just as each digit of base 10 numbers can be put into the correct pocket, the same can happen with base 2 using only 2 pockets, starting with the least significant digit as follows:

|      |      |
|------|------|
|      | 1011 |
|      | 0101 |
| 1100 | 0111 |
| 1010 | 0001 |
| 0010 | 1111 |
| [0]  | [1]  |

Then the numbers would be swept from the pocket from left to right and bottom to top, and again placed into the correct pockets based on the second digit from the right.

|      |      |
|------|------|
|      | 1011 |
|      | 0111 |
| 0101 | 1111 |
| 0001 | 1010 |
| 1100 | 0010 |
| [0]  | [1]  |

Then the next digit:

|      |      |
|------|------|
| 1011 | 0111 |
| 1010 | 1111 |
| 0010 | 0101 |
| 0001 | 1100 |
| [0]  | [1]  |

Then using the most significant digit the last distribution would look as follows:

|      |      |
|------|------|
| 0111 | 1111 |
| 0101 | 1100 |
| 0010 | 1011 |
| 0001 | 1010 |
| [0]  | [1]  |

After sweeping the pockets the order is as follows with the corresponding base 10 numbers below:

|      |      |      |      |      |      |      |      |
|------|------|------|------|------|------|------|------|
| 0001 | 0010 | 0101 | 0111 | 1010 | 1011 | 1100 | 1111 |
| 01   | 02   | 05   | 07   | 10   | 11   | 12   | 15   |

9.4.4 Memory

If the radix sort works well for physical items how could this be used in memory? One way would be to use an array for the pockets. As discussed previously in the case of using many pockets that cover the entire range of numbers, there would be no need to store more than one item per element of the array. But for smaller

number of pockets, with more than one item in each pocket, how would the multiple numbers in each element of the array be represented?

A two-dimensional array might work, but then it is possible that in any given pass some of the numbers could have the same digit in the same position. For example, using 3 digit numbers such as 001 through 991 that all end in the digit 1, each number could end up in the same column on the first pass. So, given 10 rows, each row would need to have 100 columns to hold up to 100 items. The two-dimensional array would be  $10 \times 100$  for a total of 1000 memory locations. The result is that there is no savings in using a 10 by 100 two-dimensional array over a 1000 element one-dimensional array.

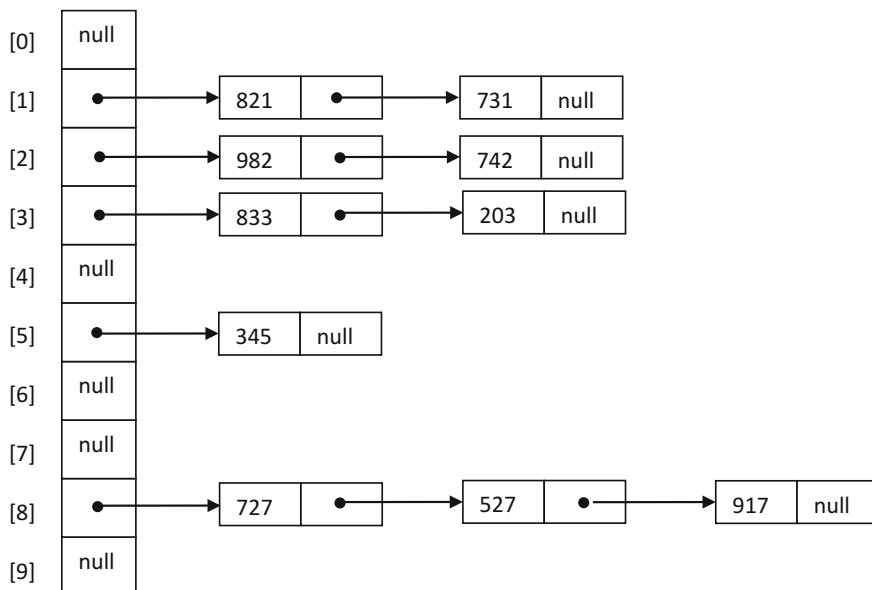
Is there another structure that can vary in size as needed? Recall from Chap. 5, a linked list might serve well here. Does it need to be a linked list of linked lists? It could be, where one linked list represents the pockets. Then each node in the linked list could be the reference to another linked list which represents the items in the pocket. But since the number of pockets is fixed, is it necessary to have a linked list of linked lists? No, instead a one-dimensional array could represent the pockets and each element in the array could reference a linked list. Then each node in the linked lists would represent the items in the pockets and the linked lists could change size as needed. Specifically, the numbers would be stored in a queue using references as discussed in Sect. 7.3. Given the first pass of the three digit numbers in Fig. 9.8 at the beginning of this section, they could be represented in memory as shown in Fig. 9.9 with the numbers being enqueued at the rear located furthest from the array and dequeued from the front which is nearest to the array.

Counting the cells in Fig. 9.9, there are 10 elements for the array of references. Further, each number would require two elements in each node, one for the number and one for a link resulting in 20 memory locations. Also, not shown are the *front* and *rear* references for each queue as part of the `QueueRef` class for another 20 memory locations. Adding up all the locations, the total to store the 10 numbers using a 10-element array of queues is 50 memory locations. What if instead there were 1000 different numbers to be sorted using the 10-element array of queues? There would still be the 10-element array and the 20 *front* and *rear* references. Plus, there would be 2000 memory locations for the 2-element nodes for a total of 2030 memory locations.

However, what if instead of a 10-element array of queues, the above is changed to incorporate a 1000 element array of queues to help cut down on the number of passes? If there were 10 numbers there still would be the 20 memory locations for

|     |     |     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
|     | 731 | 742 | 203 |     |     |     | 917 |     |     |
|     | 821 | 982 | 833 |     | 345 |     | 527 |     |     |
|     |     |     |     |     |     |     | 727 |     |     |
| [0] | [1] | [2] | [3] | [4] | [5] | [6] | [7] | [8] | [9] |

**Fig. 9.8** First pass



**Fig. 9.9** Array of queues using references

the nodes and the 1000 array elements, but there would also be 2000 locations for the `front` and `rear` pointers, for a total of 3020 memory locations. Obviously, this is quite a bit of space for only 10 numbers. Further, what if 1000 different numbers were stored in the 1000-element array of queues? In addition to the 1000 elements of the array and the 2000 locations for `front` and `rear`, each of the 1000 numbers would need to be stored in 2-element nodes form another 2000 memory locations. The result is that there would be a total of 5000 memory locations to store 1000 numbers.

Clearly this last example is expensive and it would be better to just use a simple 1000-element array of integers which would require only 1000 memory locations. However, if only 10 numbers were used in a 1000-element array of integers there would be 990 empty elements as shown in Sect. 9.4.3 which would be a significant amount of wasted space. In analyzing the simple array of integers, if  $n$  is the number of items to be sorted and  $p$  is the number of elements or pockets, then using a simple  $p$  element array results in  $p$  memory locations being used whether  $n$  equals only 1 or  $p$ . This can be wasteful if the  $n$  is small but near optimal should  $n$  be close to  $p$ .

On the other hand, with an array of queues, there were  $3p$  locations being used regardless of the value of  $n$ . In the previous example with  $p$  equal to 10, there were 10 elements in the array and 20 memory locations required for `front` and `rear` for a total of 30. Further, there were 2 memory locations for each item, which equals  $2n$  memory locations, and when  $n$  equaled 10 there were 20 memory

locations for the nodes. The resulting equation for an array of queues is  $3p + 2n$ , and in the previous example there were 50 memory locations.

The result is that if the number of items,  $n$ , to be sorted is close to the number of elements in an array,  $p$ , then there will be very few empty elements in the array of integers and the simple array is worthwhile. However, if there is only a small number of items to be sorted,  $n$ , but the range of the numbers is large, then using an array might result in an inordinate number of empty slots in the array. In this case using a smaller array of queues with more passes will end up saving more memory in spite of the links between nodes as shown above.

Regardless of which is used, a simple array or an array of queues using references, there is also the original array in the main program which has not been considered in the above analysis. Whereas the insertion and quick sort manipulate the numbers in the original array, the radix sort needs to copy data into a second array and then back into the original array for each pass. So even with the simple array with one pass, twice the memory is being used. With an array of queues as discussed above with 1000 numbers, plus an additional array, upwards of six times the memory can be used compared to other sorts. In spite of this, it is useful to examine the code necessary to implement the radix sort and as will be seen it can be one of the fastest sorts under certain circumstances.

### 9.4.5 Radix Sort Program

To create the program, as mentioned above, each element in the array is a queue where the front of the queue is closest to the array and the tail is the furthest node from the array. During each pass through the data, the numbers need to be enqueued at the tail of the queue and when the items are “swept” from the pockets they are dequeued from the front of the queue. As can be seen from Fig. 9.9, this would be a good task for the queues using references developed in Chap. 7. The `QueueRef` class from Fig. 7.2 could be used almost unaltered along with the `Node` class from Fig. 5.1. Since sweeping the data out of each queue will continue until the queue is empty and not any further, the only change that is recommended is the deletion of the error message stating that the queue is empty in the `dequeue` method.

As will be explained later, in order to make the radix sort versatile it would need access to the number of pockets, the number of digits in the largest number, `numDigits`, and the array of queues of type `QueueRef`, all of which would be declared in the `SortClass` class as follows:

```
public class SortClass {
    int numDigits, pockets;
    QueueRef[] qarray;
    // rest of the class
}
```

The constructor for the sort class would need to be sent the number of pockets to allocate `qarray`. Also, each of the elements of the array need to be initialized of type `QueueRef` using a `for` loop also based on the number of pockets, all as shown below:

```
qarray = new QueueRef[pockets];
for(int i=0; i<pockets; i++)
    qarray[i] = new QueueRef();
```

Further, the constructor will need to be sent the largest number to be sorted, `large`, along with the number of pockets. Although it might seem strange to need to know the largest number, the maximum number of digits, `numDigits`, which is also the number of passes, can be determined from these two parameters. Recall that this sort was originally used for sorting items such as postal letters which might have a 5-digit zip code and the maximum number 99,999 would be known. For example, using a smaller two digit number such as 15 and having ten pockets, then only two passes are needed. How can this be accomplished? By repetitively dividing the largest number by the number of pockets until it reaches 0, the number of digits can be determined.

Using the number 15 from above and assuming 10 pockets, 15 divided by 10 is 1. Then dividing 1 by 10 results in a 0 which indicates 2 passes are needed. Although this is fairly easy to determine by just examining the number, what if using the same number, 15, and there are only two pockets? How many passes would be needed? Instead of having to figure it out by hand, the program can perform the same repetitive division to determine that 4 passes are needed. How? The number 15 can be divided by 2, four times, which indicates when using 2 pockets that 4 passes are needed using the binary equivalent number  $1111_2$ .

In order to accomplish this, the variable `numDigits` is initialized to 0. While `large` is greater than 0, it is divided by `pockets` and then `numDigits` is incremented by 1 as follows:

```
numDigits = 0;
while(large > 0) {
    large = large/pockets;
    numDigits++;
}
```

Further, this code will work for any number of pockets. Lastly, the number of pockets needs to be copied to the variable `pockets` in the class so that the radix sort will have access to it as well, as shown in the last line of the complete constructor below:

```

public SortClass(int pockets, int large) {
    qarray = new QueueRef[pockets];
    for(int i=0; i<pockets; i++)
        qarray[i] = new QueueRef();
    numDigits = 0;
    while(large > 0) {
        large = large/pockets;
        numDigits++;
    }
    this.pockets = pockets;
}

```

The `radixSort` method would be sent array containing the integers to be sorted and the number of items in the array, `n`. Some local variables that will be needed are `temp` as a temporary variable, `i`, `j`, and `k` as loop control variables, and `index` which will be explained later.

```

public void radixSort(int[] array, int n) {
    int index=0, temp=-1, i, j, k;
    // rest of the radix sort code
}

```

The number of digits calculated in the constructor from the variable `numDigits` is used in a following outer `for` loop that iterates the required number of passes:

```

for(i=0; i<numDigits; i++){
    // rest of code
}

```

Now for the number of items `n`, it must be determined to which pocket each item must be enqueued. First the `j`th item is removed from the array and stored in `temp`. Then the `i`th digit is extracted from the integer to be used as the index for `qarray`. This is accomplished by repetitively dividing `temp` by `pockets`, similar to determining the number of digits and the conversion to base 2 in Sect. 9.4.3. For example, using the number 325, on the first pass the rightmost digit would be extracted. When 325 is divided by 10, the answer is 32 and the remainder 5 stored in `index` to place the number 325 in the fifth element of `qarray`. On the second pass, the second digit would be extracted. The 32 would be calculated and placed in `temp`, and the loop would iterate again. The 32 divided by 10 is 3 with the remainder of 2 stored in `index` to place the number 325 in the second element of



qarray, and so on. Regardless of the pass, once the index has been determined, array[j] is enqueued into the qarray[index] as shown below:

```
for(j=0; j<n; j++) {
    temp = array[j];
    for(k=0; k<=i; k++) {
        index = temp % pockets;
        temp = temp / pockets;
    }
    qarray[index].enqueue(array[j]);
}
```

After all the items in array have been enqueued into qarray based on the right-most digit, it is time to sweep the items out of qarray back into array. For each of the pockets the items must be dequeued until each pocket is empty. Recall from the dequeue method that when the queue is empty, a-1 is returned. Since there will be at least one number returned, this is a good application for a do-while loop. Further, since the -1 should not be placed into the array, it is placed in temp to be checked prior to placing it into array. As long as the number returned is greater than -1, it is a valid number, and the count j should be incremented.

```
j = 0;
for(k=0; k<pockets; k++)
    do {
        temp = qarray[k].dequeue();
        if(temp > -1){
            array[j] = temp;
            j++;
        }
    } while(temp != -1);
```

Once all the items have swept from qarray back into array, the process continues with the outer for loop for subsequent passes. Putting all the pieces together, the complete class is as follows:

```
public class SortClass {
    private int numDigits, pockets;
    private QueueRef[] qarray;
    public SortClass(int pockets, int large) {
        qarray = new QueueRef[pockets];
        for(int i=0; i<pockets; i++)
            qarray[i] = new QueueRef();
    }
}
```

```

        numDigits = 0;
        while(large > 0) {
            large = large/pockets;
            numDigits++;
        }
        this.pockets = pockets;
    }

    public void radixSort(int[] array, int n) {
        int index=0, temp=-1, i, j, k;
        for(i=0; i<numDigits; i++){
            for(j=0; j<n; j++) {
                temp = array[j];
                for(k=0; k<=i; k++) {
                    index = temp % pockets;
                    temp = temp / pockets;
                }
                qarray[index].enqueue(array[j]);
            }
            j = 0;
            for(k=0; k<pockets; k++)
                do {
                    temp = qarray[k].dequeue();
                    if(temp > -1){
                        array[j] = temp;
                        j++;
                    }
                } while(temp != -1);
        }
    }
}

```

The main program is again similar to the main program for both the insertion sort and the quick sort, except there should be two new variables added `int largest = 65, pockets = 10`; which can be initialized with the suggested values as shown or the user can be prompted for the values. Further, the invoking of the constructor call for the `sortClass` will need to include these two arguments as shown below:

```
SortClass sortClass = new SortClass(pockets, largest);
```

The call to the sort should be changed as follows:

```
sortClass.radixSort(array,n);
```

The heading output prior to the data should be changed to `System.out.println("Radix Sort")` and the spacing of the data output should be changed as well as shown in the sample input and output below:

```
Enter an integer or a -1 to stop: 9
Enter an integer or a -1 to stop: 8
Enter an integer or a -1 to stop: 1
Enter an integer or a -1 to stop: 2
Enter an integer or a -1 to stop: 7
Enter an integer or a -1 to stop: 6
Enter an integer or a -1 to stop: 4
Enter an integer or a -1 to stop: 5
Enter an integer or a -1 to stop: 3
Enter an integer or a -1 to stop: -1
```

Radix Sort

```
1
2
3
4
5
6
7
8
9
```

The speed of the program is based on the number of pockets. If there are the same number of pockets as the largest number to be sorted and a simple array is used, then the radix sort can approach  $O(n)$ . However, if there only a few items to be sorted over a large range of pockets, with more pockets comes the chance of wasted memory.

With fewer pockets comes more passes. For example, if there are 1024 different items to be sorted in the range of 0–1023 and there are only two pockets, then there will need to be 10 passes and the outer loop is  $O(\log n)$ . Why? Because  $2^{10}$  equals 1024 and  $\log_2 1024 = 10$ . Further, the first inner `for` loop is  $O(n)$ . Although the

innermost `for` loop is necessary for integers, if the numbers were stored as strings (as they would be for data like account numbers and zip codes), then the determination of each digit would not require a loop and this inner `for` loop probably does not need to be taken into account. The second `for` loop with the nested `do-while` loop is  $O(n)$  because the two loops are only removing  $n$  items from the array of queues. For example, if there are two pockets and  $n$  numbers are distributed approximately equally between the two, then 2 times  $n/2$  is  $n$ . Now since the two `for` loops in the outer loop are each  $O(n)$ , but are not nested, their speed is  $2n$ , and in Big-O notation this is just  $O(n)$ .

The result is that the outer loop is  $O(\log n)$  and the inner `for` loops are  $O(n)$ , so if there is more than one pass, the radix sort is  $O(n \log n)$ . Although this is the same as the quick sort, as has been previously discussed the disadvantage is the radix sort can use up to six times the memory. Although this can be a hindrance, under certain circumstances the sort can be quite useful and it serves as a nice introduction to various concepts that will be discussed further in Chap. 11 on hashing.

---

## 9.5 Complete Program: Sorting String Items

In this section a list of string items is sorted using the quick sort. In order for the `quickSort` method discussed in Sect. 9.3 to work on strings, the `SortClass` is modified to accept generic types of items to be sorted. Then the `SortClass` can be used to sort any type of data including `String` type. The first line is replaced by

```
public class SortClassGeneric<T extends Comparable <T>>
```

and the type of the items to be sorted, the `int`, in the `SortClass` is replaced by the type parameter `T`. Since references to the objects in a generic class are compared using operators such as `<`, `<=`, and `>=`, not the contents of the objects, the `compareTo` method which is included in the `Comparable` interface is used instead to compare two objects as discussed in Sect. 4.5. The complete definition of `SortClassGeneric` class is shown below:

```
public class SortClassGeneric<T extends Comparable <T>> {

    public SortClassGeneric() {
    }
}
```

```
public void quickSort(T[] array, int n) {
    if(n>0)
        quickSort(array,0,n-1);
}

public void swapElements(T[] a, int i, int j){
    T temp;
    temp = a[i];
    a[i] = a[j];
    a[j] = temp;
}

public void sort3(T[] a, int first, int second, int third) {
    if(a[second].compareTo(a[first]) < 0)
        swapElements(a,second,first);
    if(a[third].compareTo(a[first]) < 0)
        swapElements(a,third,first);
    if(a[third].compareTo(a[second]) < 0)
        swapElements(a,third,second);
}

private void quickSort(T[] array, int left, int right) {
    int center = (left + right) / 2;
    sort3(array,left,center,right);
    if(right-left >= 3) {
        int i = left;
        int j = right;
        while(i < j) {
            do {
                i++;
            } while(i <= j && array[i].compareTo(array[center]) <= 0);
            do {
                j--;
            } while(j >= i && array[j].compareTo(array[center]) >= 0);
            if(i < j)
                swapElements(array,i,j);
        }
        int temp;
        if(i < center && j < center)
            temp = i;
        else
            if(i > center && j > center)
                temp = j;
            else
                temp = center;
        swapElements(array,temp,center);
    }
}
```

```

        temp = center;
        swapElements(array,temp,center);
        quickSort(array,left,temp-1);
        quickSort(array,temp + 1,right);
    }
}
}

```

The main program would basically be the same as the one used for the insertion sort in Fig. 9.1. The new class will be called `QuickSortGeneric`, a variable `str` of type `String` is used to receive a word the user enters instead of a number of type `int`, and instead of using `-1` to stop the input, the user will simply press the enter key to send an empty string to stop the input. To accomplish this, the following line will need to be added,

```

SortClassGeneric <String> sortClassGeneric
    = new SortClassGeneric<String> ();

```

which will create an object of `SortClassGeneric` class called `sortClassGeneric` to allow string values to be sorted. As discussed in Sect. 9.3.4, the call to the sort should be changed to `sortClass.quickSort(array,n)` and the column heading should be changed to `System.out.println(" Quick Sort");`. Finally, the main program for invoking the `quickSort` with a generic type is shown below:

```

import java.util.*;
class QuickSortGeneric {
    public static void main(String[] args){
        Scanner scanner;
        scanner = new Scanner(System.in);
        int i,n;
        String str;
        String[] array = new String[20];
        SortClassGeneric < String > sortClassGeneric
            = new SortClassGeneric < String > ();
        i=0;
        System.out.println();
        System.out.print("Enter a string or press enter to stop: ");
        str = scanner.nextLine();
        while(str.length() != 0) {
            array[i] = str;
            i++;
            System.out.print("Enter a string or press enter to stop: ");

```

```
        str = scanner.nextLine();
    }
    n = i;
    sortClassGeneric.quickSort(array,n);
    System.out.println();
    System.out.println("    Quick Sort");
    System.out.println();
    for(i=0; i<n; i++)
        System.out.println("    " + array[i]);
    System.out.println();
}
}
```

The output when the above code is executed using 15 words is shown below:

```
Enter a string or press enter to stop: variables
Enter a string or press enter to stop: input
Enter a string or press enter to stop: output
Enter a string or press enter to stop: arithmetic
Enter a string or press enter to stop: class
Enter a string or press enter to stop: object
Enter a string or press enter to stop: contour
Enter a string or press enter to stop: selection
Enter a string or press enter to stop: iteration
Enter a string or press enter to stop: array
Enter a string or press enter to stop: recursion
Enter a string or press enter to stop: inheritance
Enter a string or press enter to stop: polymorphism
Enter a string or press enter to stop: exception
Enter a string or press enter to stop: file
Enter a string or press enter to stop:
```

Quick Sort

```
arithmetic
array
class
contour
exception
file
inheritance
input
```

```
iteration
object
output
polymorphism
recursion
selection
variables
```

---

## 9.6 Summary

- The insertion sort is  $O(n^2)$ , but with sorted data it is  $O(n)$  like the bubble sort.
- The quick sort is implemented using recursion which makes it fairly inefficient when trying to sort small sets of numbers or small subsections of numbers.
- Without the assistance of another sort, the quick sort can slow down considerably. One solution is to use the `sort3` method to not only find a possible middle element, but to sort sections of three or fewer elements.
- The quick sort is  $O(n \log n)$  with large sets of data, but without another sort such as the insertion sort on smaller subsets of data from 10 to 20 elements, its speed can be much slower.
- The radix sort can be  $O(n)$  with a large enough number of pockets and  $O(n \log n)$  for a fewer number of pockets. In either case, it uses more memory than other sorts.
- The more pockets in the radix sort, the fewer the number of passes.

---

## 9.7 Exercises (Items Marked with an \* Have Solutions in Appendix B)

1. Modify the do-while loops in the `quickSort` method to allow for duplicate items.
2. Modify the `SortClass` discussed in Sect. 9.3.4, so that for small data set of size 15 or smaller the insertion sort is applied instead of `quickSort`.
3. Convert the following numbers to the bases indicated.



A.  $16_{10} = \underline{\hspace{2cm}}_2$     \*B.  $97_{10} = \underline{\hspace{2cm}}_2$     C.  $245_{10} = \underline{\hspace{2cm}}_2$   
D.  $1101_2 = \underline{\hspace{2cm}}_{10}$     \*E.  $101110_2 = \underline{\hspace{2cm}}_{10}$     F.  $10101011_2 = \underline{\hspace{2cm}}_{10}$

4. Given the following sequence of numbers, show how they would look after one pass of the insertion sort.

58 2 65 96 83 19 74 27 41

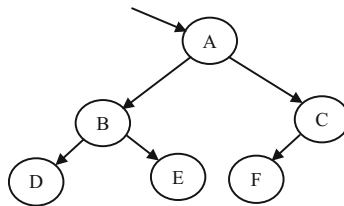
- \*5. Given the sequence of numbers above, show how they would look after one pass of the quickSort method.
- \*6. Given the sequence of numbers above, show how they would look after one pass of the radix sort using 10 pockets.
7. Given the sequence of numbers above, show how they would look after one pass of the radix sort using 2 pockets.
8. Reverse the order of the numbers in Question 4 and show how they would look after one pass of the radix sort using 10 pockets.
9. When is the insertion sort better than the quick sort?
10. How does the choice of the middle item affect the efficiency of the quick sort?

This chapter examines heaps, which are useful in creating priority queues and the heap sort. Priority queues are different than regular queues in that they allow items with the highest priority to be dequeued first. The heap sort is one of the faster sorts like the quick sort and is an  $O(n \log n)$  sort.

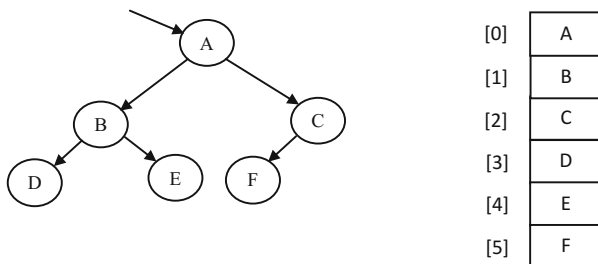
---

## 10.1 General Concepts

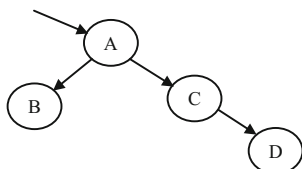
A *heap* is implemented as a binary tree and has some additional characteristics. First, it is implemented as a complete tree. Recall from Chap. 8, that a complete tree is one that at the lowest level, leaf nodes have been filled in from left to right as shown below:



An advantage of a complete tree is that it can readily be implemented in an array as shown below. Notice how the nodes from top to bottom and left to right in the tree are stored in the array from top to bottom.



If it is convenient to store a heap in an array, couldn't all binary trees be stored in arrays? Yes, but it might not be feasible. Consider the following binary tree which is not a complete tree:



If it were to be stored in an array, it would look as follows:

|     |   |
|-----|---|
| [0] | A |
| [1] | B |
| [2] | C |
| [3] |   |
| [4] |   |
| [5] |   |
| [6] | D |

Notice that there are a number of empty spaces in the array, which can be quite expensive in terms of wasted memory especially for large trees. For example, imagine in the above tree that the node containing the D had a right child, how many empty cells would there be? Before answering the question the reader is encouraged to draw the corresponding array. The answer is 10. If there is the potential for wasted space, why can heaps be used in arrays? Again, recall from above that heaps are complete binary trees, thus there is never any wasted space between the nodes in the array.

The second characteristic of a heap is that for each node in the tree, any child node contains a larger number than the parent node. Further, it does not matter whether the right child is larger than the left child as with a binary search tree, just that the children are larger than their parent. This type of heap is known as a *minheap* and the reverse with the smaller children is known as a *maxheap*. Minheaps will be used for the priority queue in Sect. 10.4 and maxheaps will be used for the heap sort in Sect. 10.7.

---

## 10.2 Creating Heaps

There are two ways that a heap can be created. One is to input the data from a user or a file into an array in the order it is read in and then later make the array into a heap. The other is that each time a number is input it can immediately be placed in the proper location to form a heap. If the original order of the data is important and needs to be kept, then the former method might be best, but if the original order is unimportant, then it makes sense to create the heap as the data is input. It is the latter method that will be used with the priority queues and the former with the heap sort.

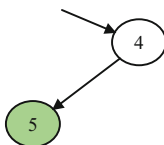
Before looking at the code, it helps to conceptually understand how the data is placed into a heap as it is input. For example, assume that the following numbers will be input:

4 5 2 3 1

The first number input will be placed in the first available spot in the heap which is the root of the tree:

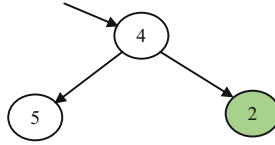


The second number will be placed in the next available spot in the heap as the left child.

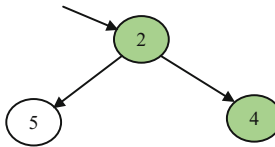


Before continuing, it is compared to its parent to see if it is smaller. Since it is not, it remains in its location. Does it matter that the 5 is to the left of the 4 as would be a problem with binary search trees? No, because again whether a larger number is to the left or right is unimportant. The only important thing is that a larger

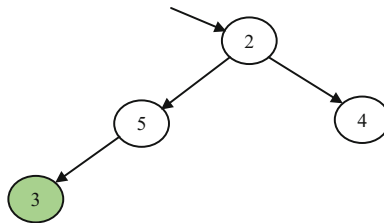
number is a child of a smaller number. Continuing, the 2 would be placed in the next available location as the right child.



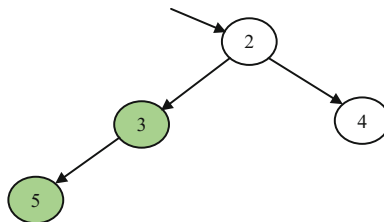
However, note that the above tree is not a heap because there is a smaller number as a child. In order to correct the situation, the 4 and 2 would need to be swapped as follows:



The next number is a 3 and inserted as shown below:

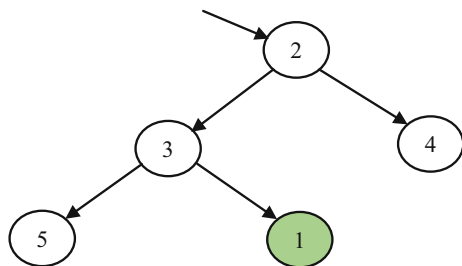


Again, the new number is not in the correct position, so it would be swapped with the 5 as follows:

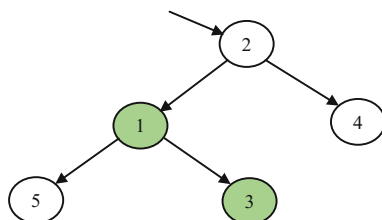


Since the 3 is greater than the 2, it does not need to go further up in the tree. Then the number 1 is inserted as shown in Fig. 10.1.

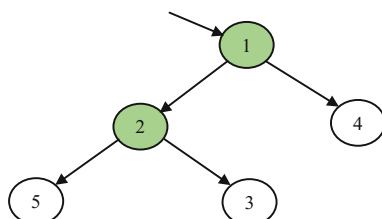
**Fig. 10.1** Heap prior to a double swap



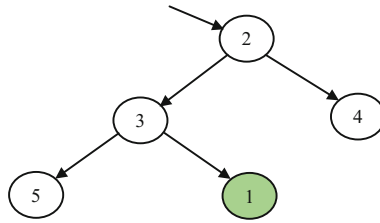
Since the 1 is less than 3, it is swapped with the 3 as shown below:



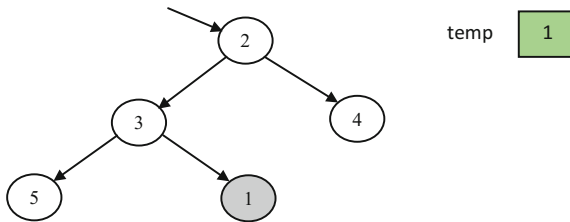
Further, since the 1 is less than 2 it is swapped again as follows:



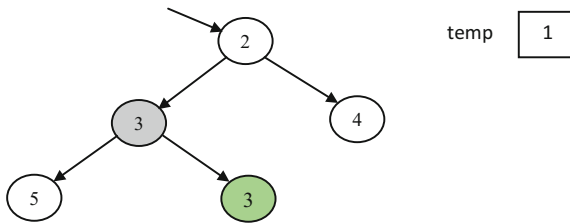
In the case of a larger tree that is many levels deep, say 10, moving a number from the lowest level to the root of the tree would take 9 swaps which would be somewhat time consuming. Is there a way that this could be done faster? Yes, recall the insertion sort in Sect. 9.2 when a number needed to be inserted in the proper location. The simplest way of accomplishing the task was to do a series of swaps as was done above. The alternative was to put the number in a temporary location, move each of the items in the array down, and finally move the new item in the temporary location into the proper location to avoid the large number of swaps. The same strategy could be employed here, where all the items could be moved down in the tree and then the new item could be copied into the proper position. Returning back to the previous tree in Fig. 10.1 when inserting the number 1 and as shown again below:



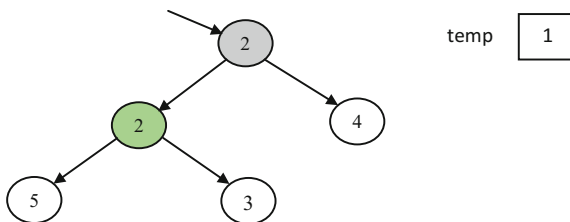
Instead of swapping the 1 with the 3 and then the 2, the 1 is copied into a temporary variable called `temp`. The 1 in the node is shaded in gray to indicate that another number can be copied into that location as shown below:



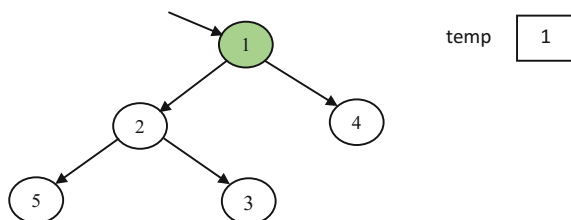
Now instead of comparing the 3 to the location in the tree, it is compared with `temp`, and since the 3 is larger it is copied to the node where the 1 was previously as follows:



Since moving the 3 is really copying, the 3 now occupies both positions. In the above tree, the original 3 in the tree is shown in gray to indicate that although the original copy is still there, the spot is available for a new number from the parent node or `temp`. The 1 in `temp` is now compared to the 2 and the 2 is copied down into the node formerly occupied by the 3 as shown below:



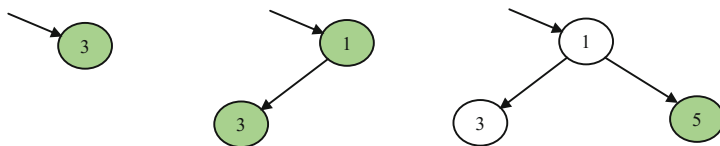
As before, the root node shows the 2 in gray to indicate this spot is available. Since there are no more nodes to compare, the 1 is copied into the root node:



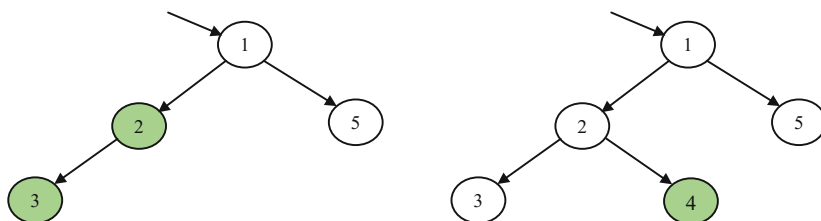
Notice that the tree is again a heap. Although this only save a couple of swaps, in a large heap the savings would be more significant. But what would happen if the original numbers were in a different order? For example, consider the following same numbers but in a different sequence:

3 1 5 2 4

Moving a little more quickly with the figures and without `temp` to save space, the insertions are shown from left to right below. First the 3 is inserted, then the 1 and its eventual placement at the root, followed by the 5:



Next the 2 is inserted and moved to where the 3 was located, and lastly the 4 is inserted:



Even though the same numbers are used, notice that the tree is different. Is it still a heap? Yes, because each of the children are larger than their parents and it is a complete tree. Also recall that when there was a different sequence of numbers with binary search trees that the resulting trees could look different as well. However, binary search trees could be very unbalanced which is not the case with heaps.



Again, although the order of the numbers makes the trees different, they are still complete trees and are still heaps. Some other sequences are left as exercises at the end of the chapter.

---

### 10.3 The `insertItem` and `moveUp` Methods

Now that the basic ideas have been discussed, it is time to create the code to build a heap. There are two methods that will be developed. The first method, `insertItem`, inserts the item at the last location at the bottom of the heap. The second method, `moveUp`, moves the item up into the proper location within the heap.

In the class that the methods will appear, one can assume the existence of the variables for an array and the number of elements `n`, along with the `size` of the heap as shown below:

```
private final int n = 20;
private int array[];
private int size;
```

Further, in the constructor for the class, a new array will be created and `size` of the heap will be set to zero:

```
array=new int[n];
size = 0;
```

#### 10.3.1 The `insertItem` Method

The basic skeleton for the `insertItem` method is as follows with the parameter `item` for the integer to be inserted:

```
public void insertItem(int item) {
    // body of method
}
```

The number sent will be placed into the last location available in the heap. For example, if the heap contains 5 numbers located in positions 0 through 4, `size` will be 5 and that will be the location where the new number in `item` will be placed as follows:

```
array[size] = item;
```

Once the number is in the array, it needs to be move to its correct location by calling the moveUp method.

```
moveUp();
```

Lastly, the size of the array is now one larger than it was so that size needs to be incremented by 1 as follows:

```
size++;
```

Putting all the pieces together results in the following complete method:

```
public void insertItem(int item) {  
    array[size] = item;  
    moveUp();  
    size++;  
}
```

### 10.3.2 The moveUp Method

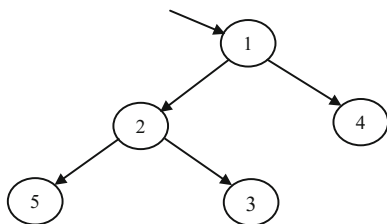
The basic skeleton for the moveUp method is as given below. Since the method will for now only be called by the insertItem method, it is private and has no need for a parameter:

```
private void moveUp() {  
    // body of method  
}
```

Since the number inserted into the heap needs to be moved to its correct position, there will be two local variables, child and parent. Recall that an item is placed in the last position in the heap, so child needs to be initialized to this same location:

```
int child = size;
```

Given child, how can parent be calculated in an array and vice versa? Before answering these questions, consider a previous tree given again below:



In a tree, the children can be accessed by recursively calling a reference to `p.left` and `p.right` and the parent can be accessed by returning to the invoking call as discussed in Chap. 8. However, since the heap can be implemented in an array, what would the array look like? As before, notice that the contents of the tree from top to bottom and left to right fill in the contents of the array below:

|     |   |
|-----|---|
| [0] | 1 |
| [1] | 2 |
| [2] | 4 |
| [3] | 5 |
| [4] | 3 |

In looking at the 1 in `array[0]`, the left child is the 2 in `array[1]` and the right child is the 4 in `array[2]`. At first, it appears that one might just be able to add a 1 or a 2 to the index to gain access to the children. However, look at the 2 in `array[1]`. Its left child is the 5 in `array[3]` and its right child is the 3 in `array[4]`. Clearly just adding a 1 or a 2 would not work.

So, what equation can be used to determine the location of the children? In the latter case multiplying the index 1 by 2 only results in a 2, but adding 1 more to it would result in `array[3]` which is the correct position for the left child. Adding 1 more results in `array[4]` which is the right child. So, the equation to determine the left child is  $2 * \text{parent} + 1$  and the equation to determine the right child is  $2 * \text{parent} + 2$ . Would these equations work for finding the children of `array[0]`? Yes, and the reader is encouraged to do the calculations for this and other potential locations in the array in the exercises.

Having determined the equations for the children, what about the reverse to find the parent of the children? Instead of doing multiplication, division is needed. Looking at the left child 5 in `array[3]`, if the index is divided by 2, the result is `array[1]`, which is correct. Again, recall that an integer divided by an integer is an integer. But with the right child 3 in `array[4]`, if its index is divided by 2 the result is 2 which is incorrect. In order to correct the situation, a 1 needs to be subtracted from child prior to the division. In this example even if a 1 is subtracted from child for the left child, the result is the same, so the same equation,

$(child-1)/2$  can be used to calculate the location of the parent for either the left and right child as shown below:

```
int parent = (child-1)/2;
```

Continuing, since the item inserted into the heap is in the last location, it could be compared with each parent above it. However, as discussed previously this can lead to an excessive number of swaps. Instead, the number should be moved to a temporary memory location `temp` for subsequent comparisons and only one move made at the end of the method.

```
temp = array[parent];
```

Then `temp` needs to be compared to see if it is less than its parent, provided it has not gone beyond the root node in position 0. This needs to be repeated until the proper position is found and this can be accomplished with a `while` loop as follows:

```
while(child > 0 && temp < array[parent]) {  
    //body of loop  
}
```

Note that the check for going beyond location 0 is first in the `while` statement. Should `temp` be smaller than the number in `array[parent]`, the contents of `array[parent]` are copied to the `array[child]` location.

```
array[child] = array[parent];
```

Next, `child` and `parent` need to be updated before returning back to the `while` statement. Specifically, `child` takes on the value of `parent` and `parent` is updated as follows:

```
child = parent;  
parent = (child-1)/2;
```

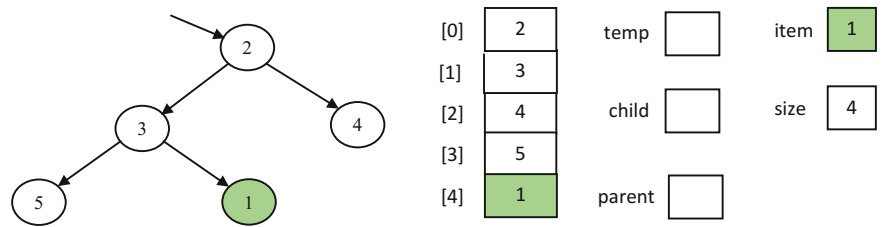
Should the `child` be less than or equal to the root at location 0 or `temp` is greater than or equal to the `array[parent]`, there is no reason to continue looking further up the heap, so iteration stops, and the value in `temp` is copied into its proper location at `array[child]` as follows:

```
array[child]=temp;
```

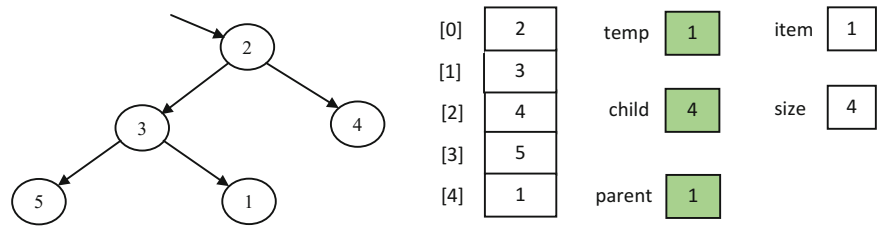
Putting all the pieces above results in the complete method below:

```
private void moveUp() {
    int child = size;
    int parent = (child-1)/2;
    temp = array[child];
    while(child > 0 && temp < array[parent]) {
        array[child] = array[parent];
        child = parent;
        parent = (child-1)/2;
    }
    array[child] = temp;
}
```

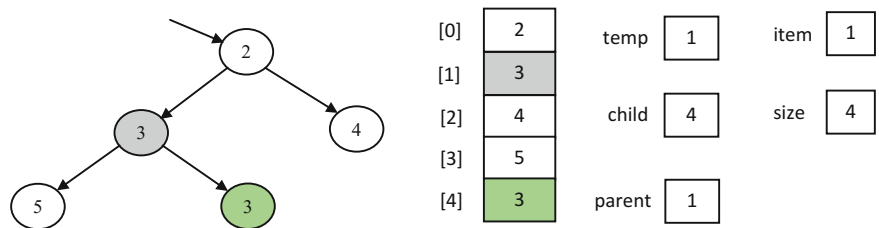
To illustrate the code, assume that the partial heap has already been created as shown below with the tree on the left and the array representation with other variables on the right. Note that `size` contains a 4, and the number 1 has been sent to `item` in the `insertItem` method and assigned to `array[size]`.



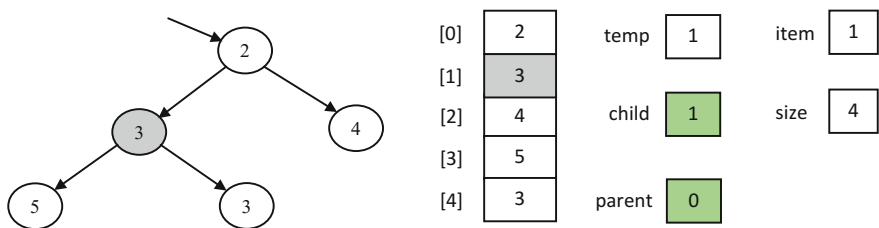
Next, `moveUp` is called, and `child` is set to `size`, `parent` is calculated, and `temp` is set to `array[child]` as shown below:



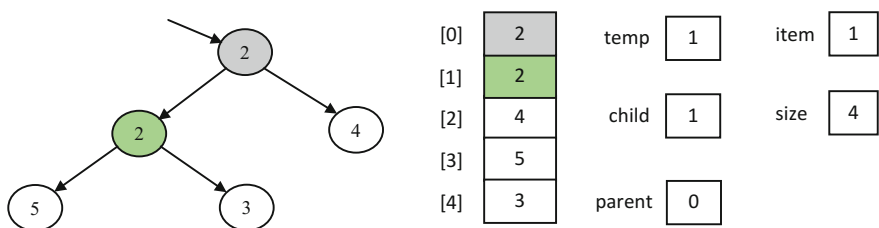
Since `child` is greater than 0 and the 1 in `temp` is less than the 3 in `array[parent]`, the body of the loop is executed. The 3 in `array[parent]` is copied into `array[child]` as shown below:



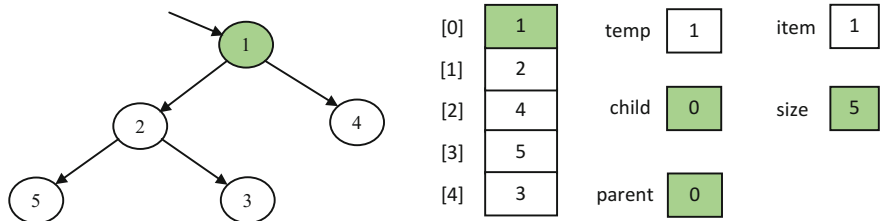
As before, the 3 is still in `array[1]`, but it is shown in gray to indicated that another value could be moved into that location. Now `child` and `parent` are updated as shown below:



Since `child` is greater than 0 and the 1 in `temp` is less than the 2 in `array[parent]`, the 2 is copied from `array[parent]` into `array[child]` and the 2 in `array[parent]` is shaded in gray indicating that a new value could be placed in that location:

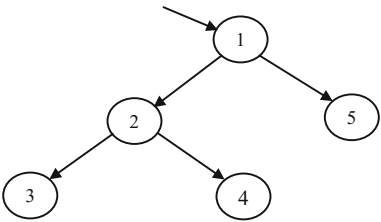


Again, `child` and `parent` are updated, and since `child` is no longer greater than 0 the loop stops iterating and the 1 in `temp` is copied into `array[child]`. The 1 is now in the correct location, which restores the tree to a heap. Lastly, upon returning to the `insertItem` method, `size` is incremented by 1 reflecting the new size of the heap.

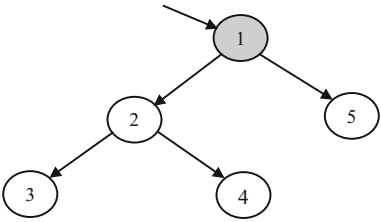


### 10.4   Priority Queues

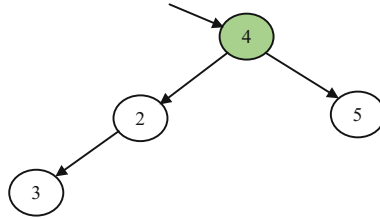
One of the uses for heaps is the creation of *priority queues*. Recall from Chap. 3, that a queue is a FIFO structure. For example, the first person in line for a ticket at a movie theater is the first person admitted to the movie. But there are times when someone or something with a higher priority might be allowed to go first. For example, if a report needs to be on the desk of a manager first thing Monday morning and there are a number of other print jobs in the print queue, the report might be given a higher priority to be printed prior to the other jobs. Different numbers could be assigned to indicate the priority. For example, a lower number could mean a higher priority or vice versa. Using a previous tree and shown again below, note that the smallest number is at the root node indicating it has the highest priority.



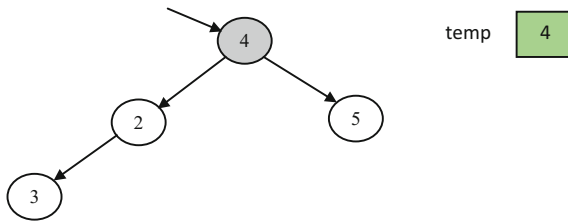
Since 1 represents the highest priority, it will be copied from the priority queue as indicated by the gray shading below:



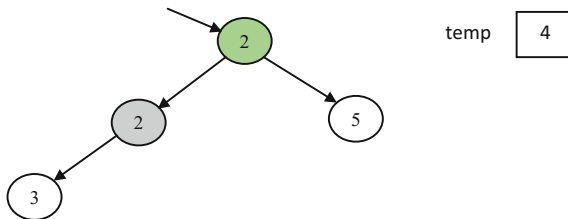
Next, the heap needs to be adjusted so that the next highest priority item is at the root. Since the heap will decrease in size, the item in the last location of the tree is copied into the vacated position at the root as indicated by 4 in green,



In a sense, the 4 in that last position has been “pruned” from the tree and the size of the heap has been decreased by one. As before, the 4 can be swapped down the tree to the correct position, but it would nice to avoid having to do a number of swaps, especially in a large tree. Similar to the `insertItem` method, the 4 can be copied to a temporary location called `temp` as shown in green and its former position shaded in gray to indicate that the space is available as follows:

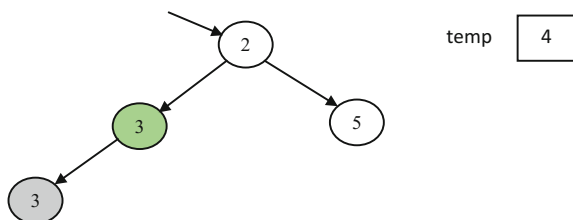


Although the copying of the number from the last position of the heap to both the root and the `temp` variable might seem a little redundant, it is helpful as will be seen later when developing the heap sort. Now it needs to be determined which of the two children should be copied into the root position and become the next highest priority item in the queue. Since 2 is less than the 4 in `temp` and less than 5 in the right child, the 2 is copied into root position as shown below:

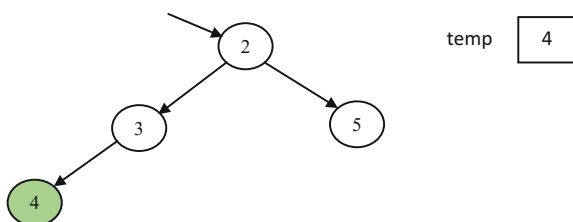




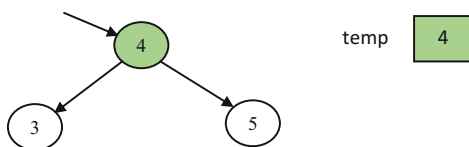
As before, the new location for the 2 is in green and its previous location is shaded in gray. Now since the 3 is less than 4 and there is only one child, the 3 is copied into the previous location of the 2 as follows:



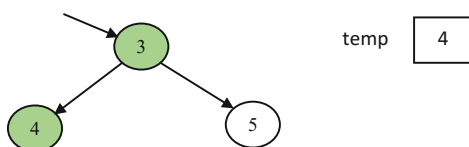
The 4 has now found its proper place in the heap and is placed into the previous location of the 3 as shown below:



Notice that the use of `temp` eliminated a couple of swaps. Although only a small savings here, with a large heap the savings would be more significant. Moving a little more quickly with the figures, this process continues with the removing of the 2 as the next highest priority, the pruning of the 4, and placing it in the root node and `temp`.



The 3 is less than 4 and less than 5, so it is copied to the root. Since there are no more children, the 4 is moved into the position previously occupied by the 3 as shown below:



Now the 3 is removed, and the 5 is copied into the root and `temp`, and its former node pruned from the tree.



Since the 4 is less than 5 and there is only one child, the 4 is moved to the spot held by the 5 and the 5 in `temp` takes the position previously held by the 4.



Now the 4 is removed from the heap. The 5 is copied to the root and `temp`, and its previous node pruned from the tree.



There are no children, so the 5 is copied from `temp` to the root. Although this might seem like a redundant step, it is easier to just perform the copy rather than check to see if the step needs to be performed.

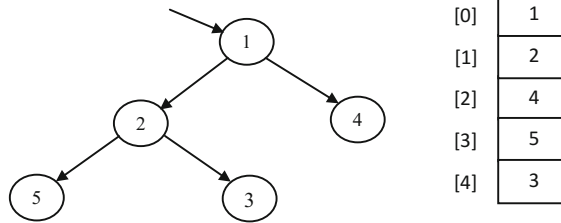


Lastly the 5 is removed, the heap is empty, and the process is complete.

---

## 10.5 The `removeMin` and `moveDown` Methods

So how would code be written to accomplish the above? Although code could be written for dealing with the heap as a binary tree implemented with references, recall that since a heap is a complete tree, it lends itself to being stored in an array. Assuming the data has been input and made into a heap by the `insertItem` and `moveUp` methods in Sect. 10.3, the previous tree with its corresponding array implementation is repeated below:



### 10.5.1 The `removeMin` Method

Although all of the code necessary can be implemented in the `removeMin` method, it is helpful to have some of the code placed in a separate `moveDown` method. The `moveDown` method will be discussed in the next section which can be reused with minimal changes to implement in the heap sort. The `removeMin` method will serve as a driver and its skeleton is as follows:

```
public int removeMin() {  
    // body of method  
}
```

First, the number in `array[0]` is stored in `min` so that it can subsequently be returned to the calling program to be output. This number needs to be saved because there will be a new number at the top of the heap before the `return` statement is executed.

```
int min = array[0];
```

Then as illustrated in the previous figures, the number from the last position of the heap is copied into the first position. However, before doing so `size` is decremented by 1. The reason for this is because the location of the last item is one less than the size of the heap. For example, if `size` is 3, the last item in the heap is in `array[2]`. This can be implemented as follows:

```
array[0]=array[--size];
```

Prior to returning `min` to the invoking program, the `moveDown` method is called to move the new item in the root down to its appropriate position. And lastly, the minimum number representing the highest priority is returned to the invoking program.

```
moveDown();  
return min;
```

Putting all the pieces together results in the following complete method:

```
public int removeMin() {  
    int min = array[0];  
    array[0]=array[--size];  
    moveDown();  
    return min;  
}
```

### 10.5.2 The `moveDown` Method

The heart of the priority queue is the `moveDown` method. Once an item has been moved from the bottom of the queue to the root position, it is necessary to find the proper place for the item in the heap. In doing so, the next highest priority item, the smallest number, will assume its place at the root position in `array[0]`. The skeleton for the method is as follows:

```
private void moveDown() {  
    // body of moveDown method  
}
```

Similar to the `moveUp` method, the `moveDown` method will need the `parent`, `child`, and `temp` variables. But instead of starting at the bottom to move an item up, the variables will need to start at the top to move an item down. The `parent` variable will start at location 0 and `child` will use the equation developed in Sect. 10.3.2 to calculate the left child. The `temp` variable will start with the contents of root at `array[parent]` all as follows:

```
int parent = 0;  
int child = 2*parent+1;  
int temp = array[parent];
```

There are two other variables needed. The variable `flag` is initialized to `false` and will later be set to `true` indicating that there are no more children or that all the numbers further down in the heap are larger than those in `temp` so that the loop should stop iterating. The variable `smallest` is concerned with determining the smallest of two children and is initialized to a negative number under the assumption that all the numbers in the heap will be positive. Both of these variables will be discussed further later in this section.

```
boolean flag=false;
int smallest=-1;
```

Whereas `moveUp` did not let `child` to go past `array[0]`, `moveDown` should not let `child` go past the end of the heap indicated by `size`. Also, to enter the loop the first time, `flag` was set at `false` above and while `flag` is not `true`, the while loop will iterate to find the proper position of `temp` in the heap.

```
while(child < size && !flag) {
    //body of loop
}
```

Then the smaller of the two children needs to be determined to compare with the number in `temp`. The left child calculated above is moved into the variable `smallest`.

```
Smallest = array[child];
```

To ensure that there is a right child and that child does not go off the bottom of the array, `child+1` is checked to ensure that it is less than `size` before determining whether the right child at `array[child+1]` is less than the left child in `array[child]`. If the right is less than the left, `child` is incremented to the right child and the contents of `array[++child]` is copied into `smallest` as shown below:

```
if(child+1 < size && array[child+1] < array[child])
    smallest = array[++child];
```

If `smallest` is less than `temp`, `smallest` is copied into the parent node at `array[parent]` and `parent` takes on the value of `child` in preparation for the next round of comparisons.

```
if(smallest < temp) {
    array[parent] = smallest;
    parent = child;
}
```

Otherwise, there is no need to continue looking further down in the heap and `flag` is set to `true`.

```
else
    flag = true;
```

Then `child` is calculated to take on the value of the left child of the new parent. Even if there are no children or further smaller children, it is redundantly calculated before returning back to the `while` statement. Again, it is easier to just make the calculation on the last time through the loop than trying to determine whether the calculation should be performed.

```
child = 2*parent+1;
```

Should `child` have become greater than or equal to `size`, the loop will stop iterating. If `child` is less than `size` and there are no longer any smaller items in the heap, `flag` will have been set to `true` and the loop will also stop iterating. In either of these cases, `temp` will have found its proper location and will be assigned to `array[parent]`.

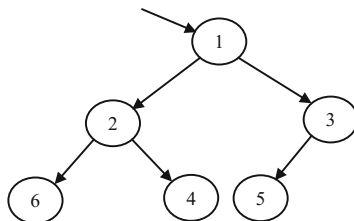
```
array[parent] = temp;
```

Putting all the above together results in the following method:

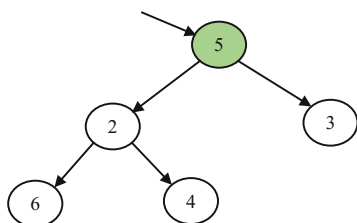
```
private void moveDown() {
    boolean flag = false;
    int smallest = -1;
    int parent = 0;
    int child = 2*parent+1;
    int temp = array[parent];
    while(child < size && !flag) {
        smallest = array[child];
        if(child+1 < size && array[child+1] < array[child])
            smallest = array[++child];
        if(smallest < temp) {
            array[parent] = smallest;
            parent = child;
        }
        else
            flag = true;
        child = 2*parent+1;
    }
    array[parent] = temp;
}
```

### 10.5.3 Walk-Through

As before, it helps to do a partial walk-through of the code to see how it works, starting with the following heap represented as a tree:



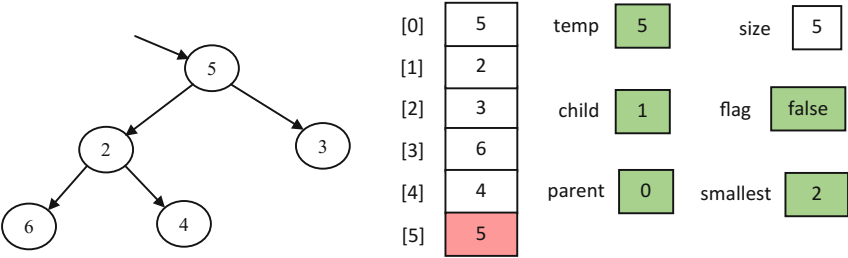
First, the 1 at the root in `array[0]` representing the highest priority is copied into `min` to be subsequently returned to the calling program. Then `size` is decremented by 1 from 6 to 5, so that the last item in the heap can be copied to the root at `array[0]`.



|     |   |      |   |
|-----|---|------|---|
| [0] | 5 | size | 5 |
| [1] | 2 |      |   |
| [2] | 3 |      |   |
| [3] | 6 |      |   |
| [4] | 4 |      |   |
| [5] | 5 | min  | 1 |

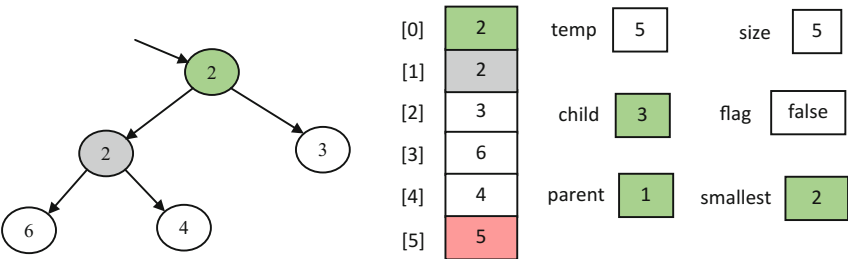
Note that the 5 has been pruned from the tree on the left, but it still exists in `array[5]` on the right. Although it still exists in the array, it is no longer part of the heap since `size` was reduced to 5. To indicate that it is still in the array but not in the heap it is shown in red. This is an important concept that will be utilized later when examining the heap sort in Sect. 10.7.

Then `moveDown` is invoked and all the local variables are initialized in the first five lines of the method. Further, upon encountering the `while` statement, `child` is less than `size` and `!flag` is true. So, the body of the loop is entered and the variable `smallest` takes on the value of `array[child]`.



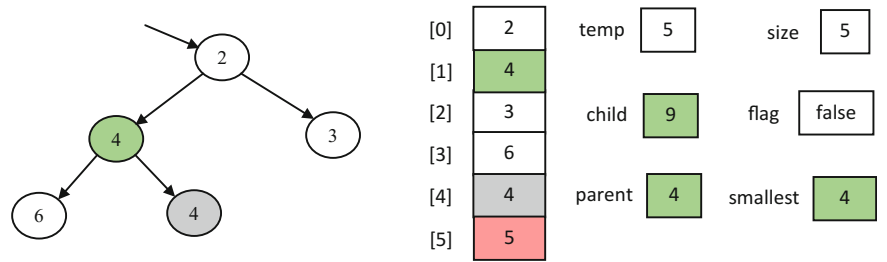
Next, the condition in the first `if` statement is evaluated. Although `child+1` is less than `size`, the right child at `array[child+1]` is not less than the left child at `array[child]`, so the condition is false and the then section of the `if` statement is skipped.

The condition in the second `if` statement is then evaluated. Since `smallest` is less than `temp`, the contents of `smallest` is copied into `array[parent]` and `child` is copied into `parent`. The else section is skipped and `child` is updated as the left child of the new parent as shown below:

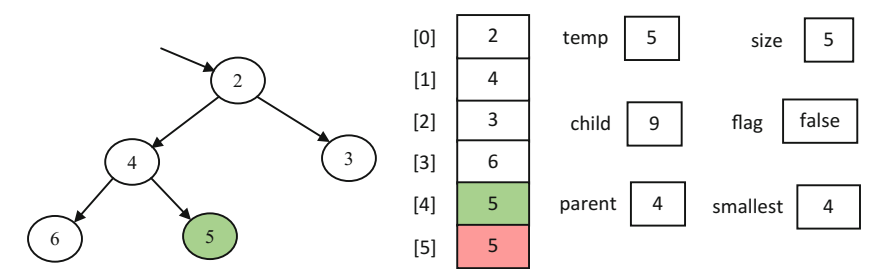


The `while` statement is again evaluated. Since `child` is still less than `size` and `!flag` is still true, the body of the loop is executed again. The variable `smallest` initially takes on the value of 6 in `array[child]` and the first `if` statement is evaluated. Now `child+1` is less than `size` indicating there is a right child so the two children are compared. Since the right child is less than the left child, `child` is incremented so that `array[++child]` is copied into `smallest`. In the second `if` statement, since `smallest` is less than `temp`, it is copied into `array[parent]` and `child` is copied into `parent`. Then `child` is then updated to be the left child of `parent` all as shown below:





Going back to the `while` statement, `child` is no longer less than `temp`, the loop stops and the 5 in `temp` takes its appropriate position in the heap at array `[parent]`.



Control is then returned back to `removeMin` which returns the 1 previously stored in `min` back to the calling program. Note that the tree representation and the array representation above are both heaps. Does the occurrence of the number 5 twice in the array pose a problem? Again, recall although the 5 in red is still part of the array, it is not part of the heap. This completes the first pass of the `moveDown` method and the reader is encouraged to walk through the rest of the passes to help ensure a good understanding of the method.

### 10.6 Test Program

A simple program follows to ensure the above methods work as developed. First, the information needs to be input into an array. As discussed in the introduction to this chapter, there are two ways in which information can be made into a heap. The first is to just create the heap as data is input. The second is to put the data into an array and then convert it into a heap. This section does the former and the latter will be examined in the following section. A simple sentinel control loop is used to prompt for and input the data. Then for each item entered, the `insertItem` method is called which creates the heap. A counter `i` is used to keep track of how many items have been inserted into the heap. Then after all the items have been

input, the program proceeds to output the contents in order of priority using a for loop by repetitively calling `removeMin`. Although similar to the main program used for the insertion sort and quick sort, it is different enough to list it out in its entirety.

```
import java.util.*;
class Ch9Prog {
    public static void main(String[] args){
        Scanner scanner;
        scanner = new Scanner(System.in);
        int i,n,num;
        PriorityQueueClass pQueue = new PriorityQueueClass();
        i=0;
        System.out.println();
        System.out.print("Enter a non-negative integer ");
        System.out.print(" or a -1 to stop: ");
        num = scanner.nextInt();
        while(num > -1) {
            pQueue.insertItem(num);
            i++;
            System.out.print("Enter a non-negative integer ");
            System.out.print(" or a -1 to stop: ");
            num = scanner.nextInt();
        }
        n=i;
        System.out.println();
        System.out.print("Priority Queue = ");
        for(i=0; i<n; i++) {
            num = pQueue.removeMin();
            System.out.print(num + " ");
        }
        System.out.println();
        System.out.println();
    }
}
```

The complete `PriorityQueueClass` is as follows:

```
public class PriorityQueueClass {
    private final int n = 20;
    private int array[];
    private int size;

    public PriorityQueueClass() {
        array=new int[n];
        size = 0;
    }
}
```

```
public void insertItem(int x) {
    array[size] = x;
    moveUp();
    size++;
}

private void moveUp() {
    int child = size;
    int parent = (child-1)/2;
    int temp = array[child];
    while(child > 0 && temp < array[parent]) {
        array[child] = array[parent];
        child = parent;
        parent = (child-1)/2;
    }
    array[child]=temp;
}

public int removeMin() {
    int min=array[0];
    array[0]=array[--size];
    moveDown();
    return min;
}

private void moveDown() {
    boolean flag = false;
    int smallest = -1;
    int parent = 0;
    int child = 2*parent+1;
    int temp = array[parent];
    while(child < size && !flag) {
        smallest = array[child];
        if(child+1 < size && array[child+1] < array[child])
            smallest = array[++child];
        if(smallest < temp) {
            array[parent] = smallest;
            parent = child;
        }
        else
            flag = true;
        child = 2*parent+1;
    }
    array[parent] = temp;
}
}
```

Sample input and output are shown below:

```
Enter a non-negative integer or a -1 to stop: 9
Enter a non-negative integer or a -1 to stop: 8
Enter a non-negative integer or a -1 to stop: 1
Enter a non-negative integer or a -1 to stop: 2
Enter a non-negative integer or a -1 to stop: 7
Enter a non-negative integer or a -1 to stop: 6
Enter a non-negative integer or a -1 to stop: 4
Enter a non-negative integer or a -1 to stop: 5
Enter a non-negative integer or a -1 to stop: 3
Enter a non-negative integer or a -1 to stop: -1

Priority Queue = 1 2 3 4 5 6 7 8 9
```

---

## 10.7 The Heap Sort

As mentioned previously, the *heap sort* is one of the faster sorts. There are two versions of this sort that will be discussed in this section. The first is very easy to implement, but is not as efficient in terms of memory, whereas the second uses less memory and is a little more complicated.

### 10.7.1 The Simplified Heap Sort

In looking at the output of the previous section notice that the output of the data is in ascending order. This feature of the priority queue can be used to sort data. An array is needed in the main program as declared below:

```
int[] array;
array = new int[20];
```

The data could be input directly into a heap as done previously but instead of just outputting the data when it is returned from the `removeMin` method, it could be copied into an array and output at the same time as shown below:

```
System.out.println();
System.out.print("Sorted Array");
System.out.println();
for(i=0; i<n; i++) {
    array[i] = pQueue.removeMin();
    System.out.println("    " + array[i]);
}
```

```
}  
System.out.println();
```

Alternatively, the data could just be stored in the array and output later. The problem is that a minimum of two arrays are needed. One to create the heap in the `PriorityQueueClass` and one for the sorted data in the main program. Is there an alternative that uses only one array? Yes, and it is explored in the next subsection.

### 10.7.2 The Modified Heap Sort

As mentioned above, the problem with the simplified heap sort is that it requires two arrays. How can this problem be resolved? First, instead of having the data inserted into a heap as it is input, the data might have been previously input and processed in its original order before being sorted. So before being sorted, the data in the array is converted into a heap by starting at the roots of lower subtrees and then converting each subtree into a heap. By continuing this process on up the tree to the root of the entire tree, it will have been converted into a heap. Also, as will be explained below, instead of storing the smallest number at the root, a minheap, the largest number should be stored at the root, a maxheap.

Then for the actual sorting, recall from Sect. 10.5.3 that when an item was pruned from a heap, it was not actually removed from the array. In other words, as each item is removed from the heap there is space left over in the array that can be used. It is this space that could be used to store the sorted data removed from the heap. Specifically, as each maximum number is removed from the root it is stored at the bottom of the array item by item until the array is filled with numbers from the largest at the bottom to the smallest at the top. After some minor modifications to `SortClass` and the `moveDown` method there will be a walk-through of both of these processes when creating the `makeHeap` and `heapSort` methods. First, the skeleton for this class along with its constructor are given below:

```
import java.util.*;  
public class SortClass {  
    public SortClass() {  
        }  
    // rest of the methods  
}
```

With data structures like the priority queue, the array is internal to the class. However, with sorts like the insertion sort and quick sort, the array remains in the main program. Since the array is in the calling program, the methods in the `SortClass` will need parameters. The array can be sent as a parameter along with the starting location and the number of items in the array.

The `moveDown` routine will need only a few changes. First, as mentioned above, it will need three parameters. The first one will be `array` sent from the main program. As briefly discussed above, since `moveDown` will need the ability to move an item down not only from the root of the heap, but from any of its subroots, `parent` will no longer be a local variable initialized to 0, but rather the second parameter. Lastly, since the array is in the calling program, the number of items will be the third parameter, `size`.

Other changes include the reversing of the less than symbols in the two `if` statements to greater than symbols, since the code is no longer finding the smaller of the two children, but rather the larger of the two children. Further, it is helpful to change the name of the variable `smallest` to `largest` to help with readability. The result is that the `moveDown` method with the changes in bold is as follows:

```
private void moveDown(int[] array, int parent, int size) {

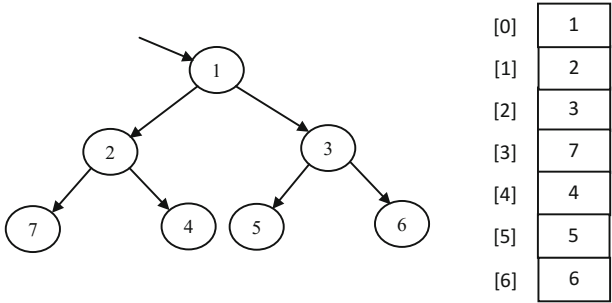
    int largest = -1;
    //int parent = 0;

    int child = 2*parent+1;
    int temp = array[parent];
    while(child < size && !flag) {
        largest = array[child];
        if(child+1 < size && array[child+1] > array[child])
            largest = array[++child];
        if(largest > temp) {
            array[parent] = largest;
            parent = child;
        }
        else
            flag = true;
        child = 2*parent+1;
    }
    array[parent] = temp;
}
```

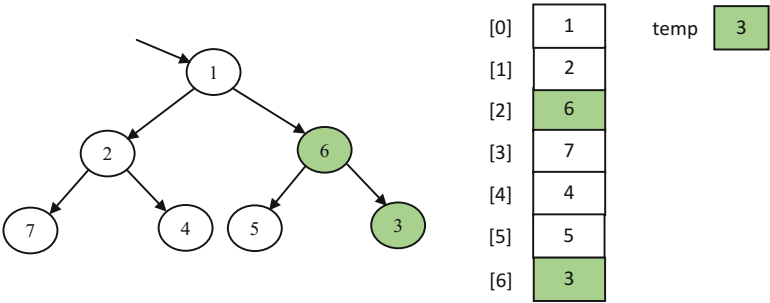
After the above few changes, the two other methods that need to be created for the heap sort can now be developed.

### 10.7.3 The `makeHeap` Method

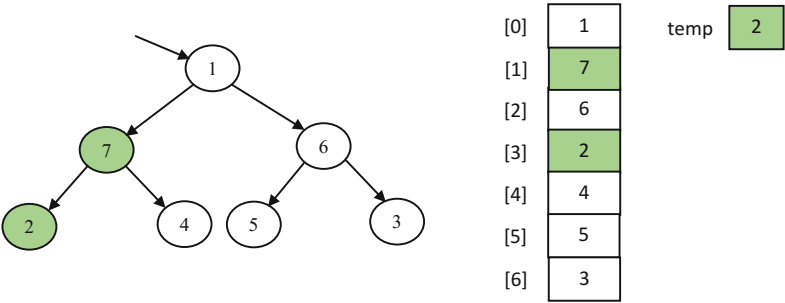
The first thing that needs to be done is convert the items in the array into a heap, specifically a maxheap. Although this could be done one item at a time as was done when inputting the data in Sect. 10.6, a heap can be created if each subtree is made into a heap as discussed briefly in Sect. 10.7.2. For example, consider the following tree, which happens to be initially a minheap:



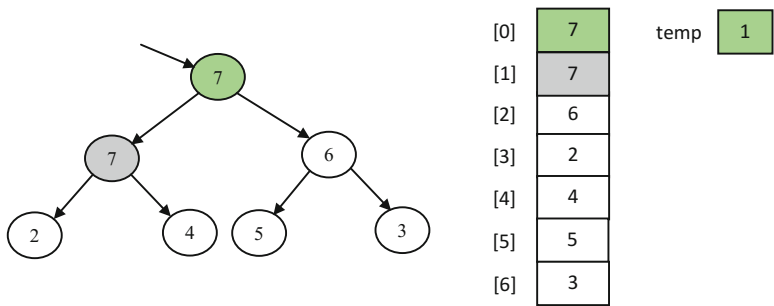
The lower right subtree can be made into a maxheap by copying the 3 into temp, determining the largest of the two children which in this case is the 6 in the right child, and copying it to the root of the subtree, and copying the 3 in temp into the right child as shown below:



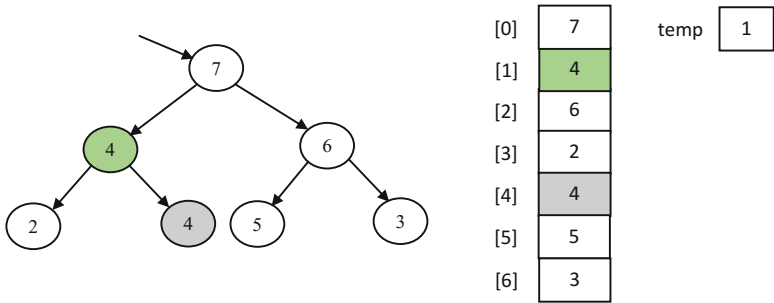
Then the lower left subtree can also be made into a heap by copying the 2 into temp, finding the largest of its two children the 7, moving it to the root of the subtree, and copying the 2 in temp to the left child as follows:



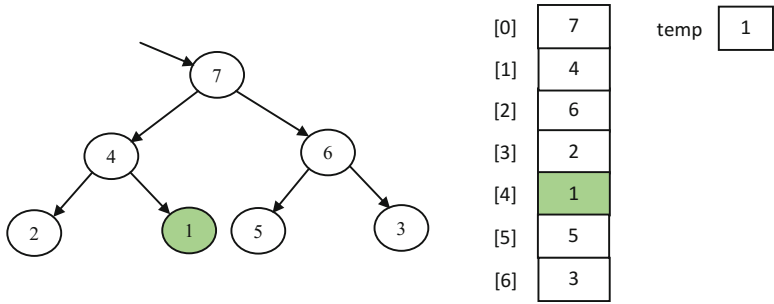
Now the 1 in the root for the entire tree can be copied into temp and the largest of the two children can be copied into the root position.



Then the larger of the two children of the left subtree where the 7 was located, the 4, can be copied into the sub-root position.



Lastly, the 1 in `temp` has been moved down to its correct position resulting in the following heap:



Although creating the code for this may appear to be complicated, it is really rather simple, since the `moveDown` routine has already been modified. The above is accomplished by noting that even though there were seven nodes in the tree, there are only three roots and their corresponding trees. These three roots are located at `array[0]` through `array[2]`. Given a size of 7 for `n`, how can these locations be



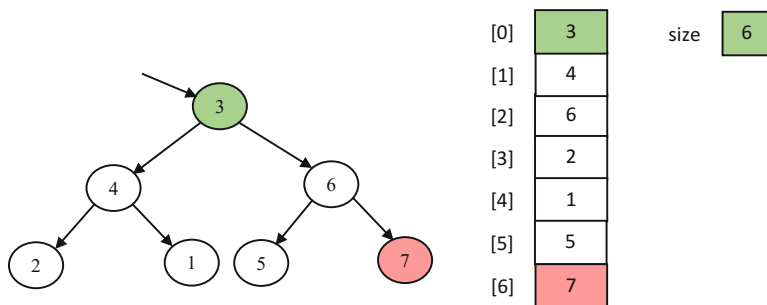
calculated? In this case,  $7/2$  results in 3 and subtracting 1 results in 2. Would this work for different values of  $n$ , such as 6? The equation would be  $6/2-1$  which is also 2. This would still allow the building of a heap for `array[2]` which would only have one child. In one more case, with  $n$  being a 5, the result would be  $5/2-1$  equaling a 1 to build a heap for the root at `array[1]`. But what about the node at `array[2]`? Since it would no longer have any children for an array of size 5, this would not be a problem. The resulting method is as follows:

```
public void makeHeap(int array[], int n) {
    for (int i = n/2-1; i >= 0; i--)
        moveDown(array, i, n);
}
```

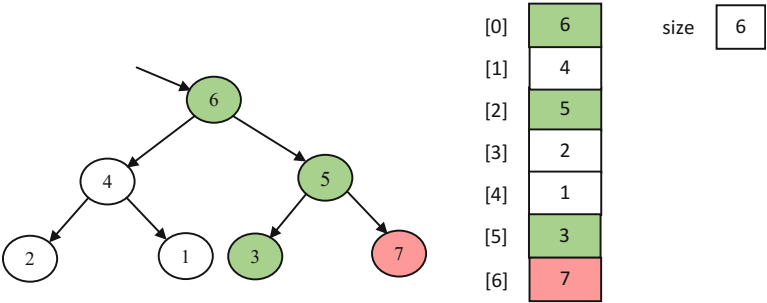
The method is `private`, but could be made `public` if needed for use in other programs. As will be seen later, the three arguments for the `moveDown` method, specifically the second argument `parent`, allows for the different locations of the root that will be moved down in the heaps.

### 10.7.4 The heapSort Method

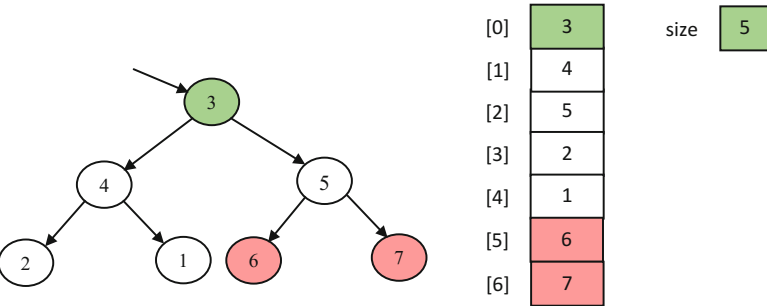
Once the heap has been made, it can now be sorted. Without showing all of the variables as was done previously, the root and the last position in the heap can be swapped. The swap can be performed by calling `swapElements` which was developed in Sect. 9.3.3. The `size` can be decremented by 1, thus pruning the 7 from the heap as shown below:



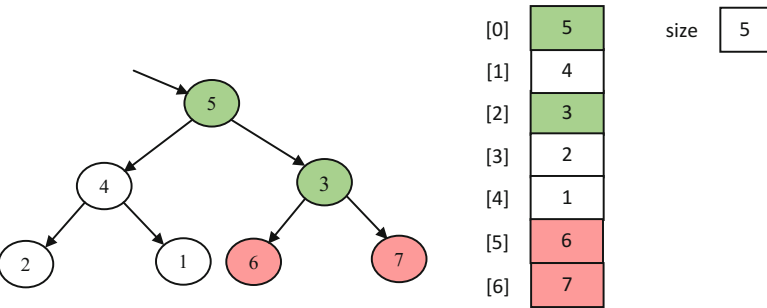
As discussed before, the last item in the array is not the last item in the heap. Also, instead of having it pruned from the tree representation as done previously, it will remain in the tree to show its relative location in comparison with the array. By calling `moveDown`, the 3 at the root can find its appropriate place in the heap and the largest number in the heap, the 6, moves up to the root:



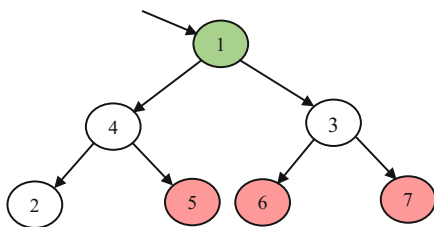
Then the 6 and 3 can be swapped, the 6 is pruned from the heap, and `size` is decremented to 5 as follows:



The 3 is then moved down to its appropriate location in the heap and the 5 to the root.

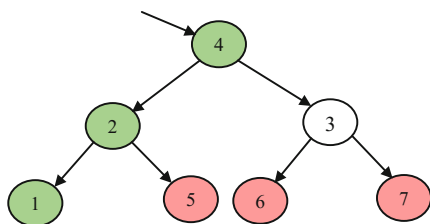


Then the 5 and 1 are swapped, the 5 pruned from the heap, and `size` decremented yet again:



|     |   |                                                                    |
|-----|---|--------------------------------------------------------------------|
| [0] | 1 | size <span style="border: 1px solid black; padding: 2px;">4</span> |
| [1] | 4 |                                                                    |
| [2] | 3 |                                                                    |
| [3] | 2 |                                                                    |
| [4] | 5 |                                                                    |
| [5] | 6 |                                                                    |
| [6] | 7 |                                                                    |

And again, the 1 will find its appropriate location and the largest number 4 will be at the root.



|     |   |                                                                    |
|-----|---|--------------------------------------------------------------------|
| [0] | 4 | size <span style="border: 1px solid black; padding: 2px;">4</span> |
| [1] | 2 |                                                                    |
| [2] | 3 |                                                                    |
| [3] | 1 |                                                                    |
| [4] | 5 |                                                                    |
| [5] | 6 |                                                                    |
| [6] | 7 |                                                                    |

By now the pattern should be discernable. Each time the largest number is at the root, it is swapped with the last item in the heap, it is pruned from the heap, the item at the root is moved down to its appropriate location, and a new larger number is at the root. This process continues until all the items have been pruned from the heap and filled in with the largest number at the bottom of the array and the smallest at the top, thus sorting the numbers.

How is this accomplished? The remaining code is relatively simple, since all of the other methods have already been written. Of course, `makeHeap` must be called to ensure that the array is made into a maxheap.

```
makeHeap(array,n);
```

Then using a loop and starting at the bottom of the heap at location `n-1`, the process continues until reaching one less than the root. Why one less than root? Because once the second to the last number has been put in its proper location, all that remains is the smallest number in `array[0]` and it does not need to be sorted.

```
for(int i=n-1; i>0; i--) {
    // body of loop
}
```

Each time through the loop the 0th location and the *i*th location need to be swapped by calling `swapElements` which is the same routine developed in Chap. 9.

```
swapElements(array,0,i);
```

Once swapped, the number at the root at `array[0]` needs to be moved down to its appropriate location in the array which sends the largest number to the root.

```
moveDown(array,0,i);
```

Putting all the pieces together into a complete method results in the following:

```
public void heapSort(int array[], int n) {  
    makeHeap(array,n);  
    for(int i=n-1; i>0; i--) {  
        swapElements(array,0,i);  
        moveDown(array,0,i);  
    }  
}
```

That is all there is to the `heapSort` method. The complete class is as follows:

```
import java.util.*;  
public class SortClass {  
  
    public SortClass() {  
    }  
  
    private void swapElements(int[] a, int i, int j) {  
        int temp;  
        temp = a[i];  
        a[i] = a[j];  
        a[j] = temp;  
    }  
  
    private void moveDown(int[] array, int parent, int size) {  
        boolean flag = false;  
        int largest = -1;  
        int child = 2*parent+1;
```

```

        int temp = array[parent];
        while(child < size && !flag) {
            largest = array[child];
            if(child+1 < size && array[child+1] > array[child])
                largest = array[++child];
            if(largest > temp) {
                array[parent] = largest;
                parent = child;
            }
            else
                flag = true;
            child = 2*parent+1;
        }
        array[parent] = temp;
    }

    private void makeHeap(int array[], int n) {
        for (int i = n/2-1; i>=0; i--)
            moveDown(array,i,n);
    }

    public void heapSort(int array[], int n) {
        makeHeap(array,n);
        for(int i=n-1; i>0; i--) {
            swapElements(array,0,i);
            moveDown(array,0,i);
        }
    }
}

```

---

## 10.8 Test Program and Output

The test program is similar to the previous test programs for the insertion sort in Fig. 9.1. The only changes are to instead invoke the heap sort method using `sortClass.heapSort(array,n)`, change the column heading for the output `System.out.println("Heap Sort")`, and change the format of the output if needed. Sample input and output are as follows:

```

Enter an integer or a -1 to stop: 9
Enter an integer or a -1 to stop: 8
Enter an integer or a -1 to stop: 1
Enter an integer or a -1 to stop: 2

```

```
Enter an integer or a -1 to stop: 7
Enter an integer or a -1 to stop: 6
Enter an integer or a -1 to stop: 4
Enter an integer or a -1 to stop: 5
Enter an integer or a -1 to stop: 3
Enter an integer or a -1 to stop: -1
```

Heap Sort

- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8
- 9

### 10.9 Complete Program: Priority Queues for Printers

In this section a simple printer queue is simulated using a priory queue. If there are several files in the queue, the printer will print the file with the highest priority first. Consider the following set of jobs with the name of the file and the ranking of the person who sent the file.

| Name of the file | Ranking         |
|------------------|-----------------|
| prog1.java       | Corps de Ballet |
| prog2.lisp       | Soloist         |
| prog3.c          | Corps de Ballet |
| prog4.pl         | Principal       |
| prog5.a          | Apprentice      |

Classical ballet companies have a ranking system. The Principal is the highest, then Soloist, followed by Corps de Ballet, and lastly Apprentice. If the file sent by the Principal is in the queue, then that file will be printed first. To make it simple all the files are submitted at the beginning. The program will place them in the proper order using the ranking and print the contents of the priority queue.

Each entry of the queue will consist of the name of the file and ranking of the person who sends the file to the printer. Therefore, the `FileWithPriority` class will be defined to keep track of this information. A constructor will accept values

for two private data members, `fileName` and `ranking`, and assign them to the appropriate data members which have a corresponding accessor and mutator.

Because the files are compared with one of the data members, `ranking`, and placed in the queue, the `compareTo` method needs to be written. The `compareTo` method is included in the `Comparable` interface that `FileWithPriority` class implements. As discussed previously in Sect. 6.7, the method returns a negative integer, zero, or a positive integer if the ranking of the object that is calling the method is higher than, equal to, or lower than the one of the specified object, respectively. To accomplish the comparisons of rankings, the `getRankingNumber` method will take the ranking and assign the integer to it. For example, the ranking `Principal` is given a value of 1, and the `Apprentice` is given a value of 4. The code below is the `getRankingNumber` method:

```
private int getRankingNumber(String ranking) {
    int rankingNumber;
    if(ranking.equals("Principal"))
        rankingNumber = 1;
    else
        if(ranking.equals("Soloist"))
            rankingNumber = 2;
        else
            if(ranking.equals("Corps de Ballet"))
                rankingNumber = 3;
            else
                if(ranking.equals("Apprentice"))
                    rankingNumber = 4;
                else
                    rankingNumber = 5;
    return rankingNumber;
}
```

Because this method is used in the `compareTo` method internally it is declared as `private`, and the definition of the `compareTo` method is shown below:

```
public int compareTo(FileWithPriority otherFile) {
    int result;
    if(getRankingNumber(ranking)
        < getRankingNumber(otherFile.getRanking()))
        result = -1;
    else
        if(getRankingNumber(ranking)
            > getRankingNumber(otherFile.getRanking()))
            result = 1;
        else
```

```
        result = 0;
    return result;
}
```

Lastly, the `toString` method is added at the end to return a string representation of the contents of the data members of an object. The following is the method that will return the name of the file and the ranking of the person who sent the file:

```
public String toString() {
    return ("File name: " + fileName + " sent by " + ranking);
}
```

Putting pieces together, the complete code defining the class for a `FileWithPriority` object is shown below:

```
public class FileWithPriority implements Comparable<FileWithPriority> {
    private String fileName;
    private String ranking;

    public FileWithPriority(String fileName, String ranking) {
        setFileName(fileName);
        setRanking(ranking);
    }
    public String getFileName() {
        return fileName;
    }

    public String getRanking() {
        return ranking;
    }

    public void setFileName(String fileName) {
        this.fileName = fileName;
    }

    public void setRanking(String ranking) {
        this.ranking = ranking;
    }

    public int compareTo(FileWithPriority otherFile) {
        int result;
        if (getRankingNumber(ranking)
            < getRankingNumber(otherFile.getRanking()))
            result = -1;
        else
```



```

        if (getRankingNumber (ranking)
            > getRankingNumber (otherFile.getRanking ()))
            result = 1;
        else
            result = 0;
        return result;
    }

    private int getRankingNumber (String ranking) {
        int rankingNumber;
        if (ranking.equals ("Principal"))
            rankingNumber = 1;
        else
            if (ranking.equals ("Soloist"))
                rankingNumber = 2;
            else
                if (ranking.equals ("Corps de Ballet"))
                    rankingNumber = 3;
                else
                    if (ranking.equals ("Apprentice"))
                        rankingNumber = 4;
                    else
                        rankingNumber = 5;
        return rankingNumber;
    }

    public String toString () {
        return ("File name: " + fileName + " sent by " + ranking);
    }
}

```

In order for the `PriorityQueueClass` discussed in Sect. 10.4 to accept the objects of the `FileWithPriority` class, the `PriorityQueueClass` is modified to accept generic types of items. The first line is replaced by `public class PriorityQueueClassGeneric <T extends Comparable<T>>`. Also, the type of the items in the queue, the `int` in the `PriorityQueueClass`, is replaced by the type parameter `T`. Integer comparisons using `<` are changed to the `compareTo` method because two objects are compared using a data member, `ranking`. The complete definitions of `SortClassGeneric` class is shown below:

```

public class PriorityQueueClassGeneric<T extends Comparable<T>> {
    private final int n = 20;
    private T[] array;
    private int size;

```

```
public PriorityQueueClassGeneric() {
    array=(T[]) new Comparable[n];
    size = 0;
}

public void insertItem(T x) {
    array[size] = x;
    moveUp();
    size++;
}

private void moveUp() {
    int child = size;
    int parent = (child-1)/2;
    T temp = array[child];
    while(child > 0 && temp.compareTo(array[parent]) < 0) {
        array[child] = array[parent];
        child = parent;
        parent = (child-1)/2;
    }
    array[child]=temp;
}

public T removeMin() {
    T min=array[0];
    array[0]=array[--size];
    moveDown();
    return min;
}

public T peekMin() {
    return array[0];
}

public void copyQueue(T[] otherArray) {
    otherArray=(T[]) new Comparable[n];
    for(int i=0; i<size; i++)
        otherArray[i] = array[i];
}

private void moveDown() {
    boolean flag = false;
    T smallest = null;
    int parent = 0;
    int child = 2*parent+1;
```

```

    T temp = array[parent];
    while(child < size && !flag) {
        smallest = array[child];
        if(child+1<size && array[child+1].compareTo(array[child])<0)
            smallest = array[++child];
        if(smallest.compareTo(temp) < 0) {
            array[parent] = smallest;
            parent = child;
        }
        else
            flag = true;
        child = 2*parent+1;
    }
    array[parent] = temp;
}
}

```

In the main program an object of the `PriorityQueueClassGeneric` is created which holds objects for the priority queue that consists of a name of the file and a ranking. This information is read from the file called `files.txt` with the filename and rank on alternating lines. The `insertItemToPriorityQueue` method handles the file input and five files are inserted to the priority queue. Contents of the queue are printed at the end. Below is the complete main program.

```

import java.util.*;
import java.io.*;
class PrinterPriorityQueue {

    public static void main(String[] args) throws IOException {
        PriorityQueueClassGeneric<FileWithPriority> printQueue
            = new PriorityQueueClassGeneric<FileWithPriority>();
        insertItemToPrintQueue(printQueue);
        System.out.println("Printer Queue: ");
        for(int i=0; i<5; i++)
            System.out.println(printQueue.removeMin().toString());
    }

    public static void insertItemToPrintQueue(PriorityQueueClassGeneric
        <FileWithPriority> printQueue) throws IOException {
        String fileName, rank;
        Scanner inFile = new Scanner(new File("files.txt"));
        while(inFile.hasNextLine()) {
            fileName = inFile.nextLine();
            rank = inFile.nextLine();
            FileWithPriority file = new FileWithPriority(fileName, rank);

```

```
        printQueue.insertItem(file);  
    }  
}  
}
```

The output when the above code is executed is shown below. Notice that the priority queue is arranged by ranking, and files will be printed in priority order.

```
Printer Queue:  
File name: prog4.pl sent by Principal  
File name: prog2.lisp sent by Soloist  
File name: prog1.java sent by Corps de Ballet  
File name: prog3.c sent by Corps de Ballet  
File name: prog5.a sent by Apprentice
```

---

## 10.10 Summary

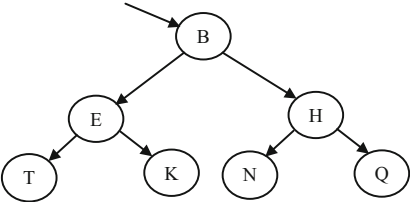
- A heap can be represented as a tree using references or an array.
- A heap is implemented as a complete tree.
- When using an array for a heap starting at location 0, the location of the left child can be found by using the equation:  $2 * \text{parent} + 1$ . The location of the right child can be determined using the equation:  $2 * \text{parent} + 2$  and parent can be determined by the equation  $(\text{child} - 1) / 2$ .
- Priority queues can be implemented using a heap and are different than regular queues because they allow items to violate the FIFO rule.
- The simplified heap sort can be implemented using a priority queue but requires the use of an additional array.
- The modified heap sort is implemented without an extra array.
- The heap sort is  $O(n \log n)$  like the quick sort.

---

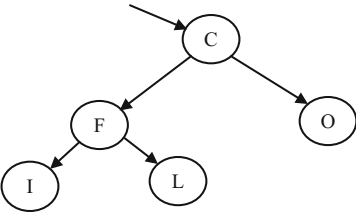
## 10.11 Exercises (Items Marked with an \* Have Solutions in Appendix B)

1. Given the following tree representations of heaps, draw the corresponding heaps as arrays.

A.



\*B.



2. Given the following array representations of heaps, draw the corresponding tree representations:

\*A.

|     |   |
|-----|---|
| [0] | D |
| [1] | G |
| [2] | J |
| [3] | V |
| [4] | M |
| [5] | P |
| [6] | Q |

B.

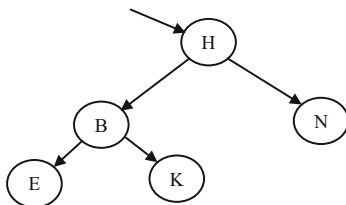
|     |   |
|-----|---|
| [0] | F |
| [1] | H |
| [2] | L |
| [3] | W |
| [4] | O |
| [5] | Q |
| [6] | R |

3. Given the following array representation of a heap, answer the following questions:

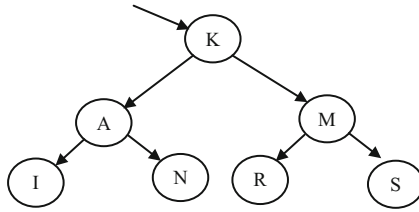
|     |   |
|-----|---|
| [0] | E |
| [1] | I |
| [2] | K |
| [3] | X |
| [4] | N |
| [5] | R |
| [6] | S |

- What is the left child of the X in `array[3]`?
  - What is the right child of the N in `array[4]`?
  - \*C. What is the parent of R in `array[5]`?
- Instead of having the root node of a heap in `array[0]`, assume that it begins in `array[1]`. What are the new equations for the left child, right child, and parent?
  - Given the following sequence of numbers create the corresponding minheap using tree representations as each number is input as done with the priority queue. Show the steps when adding each node to the heap.
    - 5 3 8 7 2 4 9 6 1
    - 9 8 5 6 2 4 7 1 3
    - 3 8 4 7 9 5 2 1 6
  - Given the following complete trees, create the corresponding maxheaps using the method from the heap sort. Show the steps as each subtree is made into a heap.

A.



B.



7. Rewrite the priority queue using large numbers instead of small numbers to indicate a higher priority.
8. Rewrite the heap sort to sort the numbers in descending order instead of ascending order.

Some of the most common operations applied on data are insertions, deletions, and searches. *Hashing* is a technique that can be used to perform these operations efficiently. This chapter is a brief introduction to hashing.

---

## 11.1 General Concepts

In an ideal world, all items are unique and in any given list of items, they would be stored at a position that matches the item. Then, the search for any item in such a list would be one access to the list. The data structure that can be used for this technique is called a hash table and it is normally an array of a fixed size containing various items. Consider a roster for a course. An instructor might keep track of student ID number, student's name, and attendance. The student ID number is unique to every student, and name and attendance are the information associated with it. Information about each student would be stored in a particular location of an array using a student ID which is called a *key*. Then a search for a student could be performed using the key. To make it simple, in the following example, only the key is stored in the array. In later sections, information associated with the key will also be stored in the hash table.

Assume keys  $\{3, 1\}$  and an array of size 5. The key 3 goes into the array location 3 and the 1 is stored in location 1 as shown in the following figure:



|     |   |
|-----|---|
| [0] |   |
| [1] | 1 |
| [2] |   |
| [3] | 3 |
| [4] |   |

Using the above array, what if keys to be inserted are {3, 1, 9, 7}? There is no location 9 or 7 in the array. In this case a larger array could be used. The following figure shows the insertion of 4 keys in an array of size 10.

|     |   |
|-----|---|
| [0] |   |
| [1] | 1 |
| [2] |   |
| [3] | 3 |
| [4] |   |
| [5] |   |
| [6] |   |
| [7] | 7 |
| [8] |   |
| [9] | 9 |

The key 7 goes to the location 7 and the 9 is stored in 9. What about a list of keys such as {3, 1, 9, 7, 8764, 125, 950}? An array of size 10,000 could be created and all keys would be placed in the matching cell. However, storing 7 items in the array of size 10,000 is not an efficient way of using memory. Is there any way keys that are greater than 9 can be converted to numbers between 0 and 9 so that they can be stored in the array of size 10? The answer is a modulo operation. Recall that the

modulo operator  $\%$  returns the remainder after a division of one number by another. Since  $8764\% 10 = 4$ , the key 8764 will be stored in location 4 of the array. The key 125 would go to the location 5 because  $125\% 10 = 5$  and 950 to location 0 as shown in the following figure:

|     |      |
|-----|------|
| [0] | 950  |
| [1] | 1    |
| [2] |      |
| [3] | 3    |
| [4] | 8764 |
| [5] | 125  |
| [6] |      |
| [7] | 7    |
| [8] |      |
| [9] | 9    |

As can be seen, each key is mapped into the appropriate cell and the mapping is called a *hash function*. One of the simplest hash functions uses the modulo operation and it is implemented in Sect. 11.5.

---

## 11.2 Mid-square Method

In this section another hash function called the mid-square method is introduced. This hash function will square the key and then take the appropriate number of digits from the middle of the square in order to obtain an array index. For example, if the size of the hash value is to be a 2-digit integer and the key is 351, first it is multiplied by itself producing 123201. Then the middle 2 digits 32 can be extracted and used to store the key 351 at location 32. Using the same procedure, the key 657 would go to the location 16, because  $657 \times 657 = 431649$  and the middle two digits are 16. If the key is 167, the hash value could be 78 or 88 since  $167 \times 167 = 27889$  depending on the algorithm. The implementation below extracts the 78:

```
private int midSquareHash(int key) {  
    String str = String.valueOf(key*key);  
    str = str.substring(str.length()/2-1, str.length()/2 + 1);  
    return Integer.valueOf(str);  
}
```

The method multiplies the key by itself, converts the square to the `String` type using the `valueOf` method defined in the `String` class, and extracts the middle two numbers. For the key 657, value of `str` is 431649 since  $657 \times 657 = 431649$ . Recall that the first argument of the `substring` method is the starting position of the substring of the original string which is 2 in this example. This is because the return value of `str.length()` is 6 which is the total number of characters. Dividing 6 by 2 gives 3, and subtracting one from it results in 2. The second argument of the `substring` method is the ending position + 1. The `str.length()/2` gives the ending position which is 3 since  $6/2 = 3$  and 1 is added to it. A string 16 is returned from the `substring` method because the 1 is at position 2 and the 6 is at position 3 of the original string, 431649. Finally, the `midSquareHash` method returns the result as an integer using the `valueOf` method in the `Integer` class. If there is an odd number of digits in the square, the middle number and the one before are extracted. Therefore, 78 will be returned for the key 167 as mentioned above.

The modulo operation and mid-square method can also be used with a key of `String` type after converting a string into an integer value, and this conversion will be discussed in the next section. An implementation of the mid-square method for string keys is left as an exercise at the end of the chapter.

---

## 11.3 Hash Function for Strings

As discussed in Sect. 11.1, keys are not always integers that can be matched with the indexes of the array. When keys are strings, one option is to add up the Unicode values of the characters in the string and use the modulo operation or other hash functions to obtain a hash value. For example, consider the word “kaminari.” The Unicode value for ‘k’ in decimal is 107, ‘a’ is 97, ‘m’ is 109, ‘i’ is 105, ‘n’ is 110, and ‘r’ is 114. The sum of the Unicode values of 8 letters is  $107 + 97 + 109 + 105 + 110 + 97 + 114 + 105$  equals 844. Using the modulo function and if the size of the hash table is 5, then ‘kaminari’ could be stored at location number 4 because  $844 \% 5 = 4$ . The following method calculates the hash values for a string using the Unicode values and the modulo operation. Assume that the name of the hash table is `harray` and it is an array of type `int`.

```
private int stringHash(String key) {  
    int keyValue = 0;
```

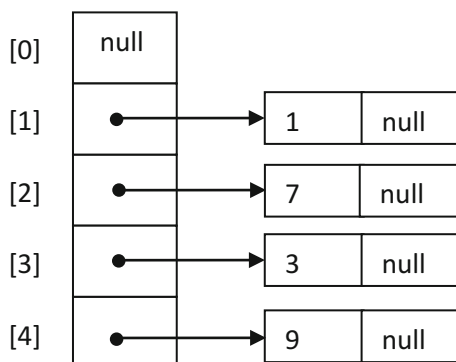
```
for(int i = 0; i<key.length(); i++)  
    keyValue = keyValue + key.charAt(i);  
return keyValue % harray.length;  
}
```

Notice that in the line, `keyValue = keyValue + key.charAt(i)`, the Unicode values of the characters are summed.

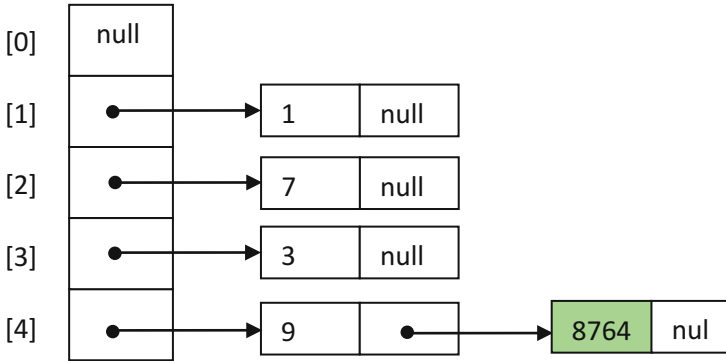
---

## 11.4 Collision Resolution

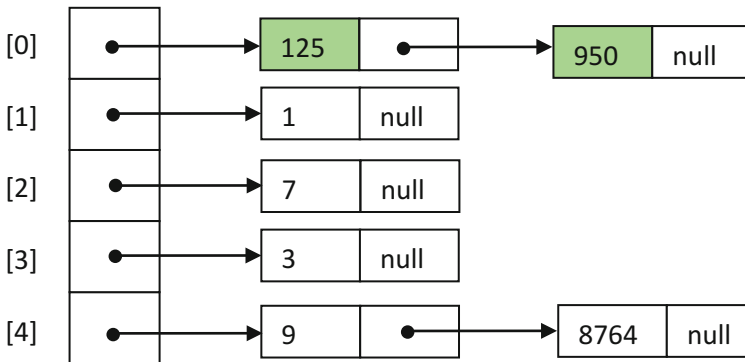
What happens when 7 keys {3, 1, 9, 7, 8764, 125, 950} from Sect. 11.1 are inserted into the hash table of size 5 using a modulo hash function? The 3 goes to the location 3, 1 to 1, 9 to 4 because  $9\%5 = 4$ , and 7 to 2 since  $7\%5 = 2$ . But what about 8764? Where should it go? The 8764 hashes to 4 because  $8764\%5 = 4$ ; however, the key 9 is already at the location 4. Since there are only 5 cells in the array, more than one key could hash to the same value, which is known as a *collision*. There needs to be a mechanism to deal with this kind of situation. One solution is to keep a list of all keys that hash to the same value together using a linked list. This is known as *separate chaining*. The figure below shows a hash table that uses a linked list to handle collisions and has the first 4 keys from above already inserted.



When the 8764 is to be inserted, a new node is added to the linked list at the hash table location 4 as illustrated in the following figure.



After inserting all 7 keys, the hash table looks like the one below.



Notice that the linked lists are sorted when the new node is added. Although data could be placed at the end of each list as in the radix sort discussed in Sect. 9.4, this would make the list unordered and searching would be slowed. Instead, the data could be sorted as in an ordered linked list described in Sect. 6.1 to make retrieving data quicker and implementation is discussed in the next Section.

## 11.5 Implementation of a Hash Table

A hash table using a modulo operation is implemented in this section. The separate chaining hash table can be written with either a sorted or an unsorted list. Since the sorted linked list class was already implemented in Chap. 6, the `LinkedList` class in Fig. 6.3 is used here. An implementation of a separate chaining hash table using an unordered linked lists is left as an exercise at the end of the chapter.

```

public class ChainingHashTable {
    private LinkedList[] harray;

    public ChainingHashTable(int size) {
        harray = new LinkedList[size];
        for(int i=0; i<harray.length; i++)
            harray[i] = new LinkedList();
    }

    public void insert(int n) {
        if(harray[modHash(n)].insert(n))
            System.out.println("item, " + n + ", found, not added");
    }

    public void remove(int n) {
        if(!harray[modHash(n)].delete(n))
            System.out.print("item, " + n + ", not found, not removed");
    }

    public void printHashTable() {
        for(int i=0; i<harray.length; i++) {
            System.out.print("harray[" + i + " ] ");
            harray[i].print();
        }
    }

    private int modHash(int key) {
        return key % harray.length;
    }
}

```

**Fig. 11.1** ChainingHashTable class

The structure of the hash table will be an array of linked lists called `harray`. The private `modHash` method uses a key as a parameter to compute a hash function and returns the hash value. The constructor simply creates an array with null values. The `insert` method adds the key to the correct location of the hash table using the hash value by calling the `insert` method defined in the `LinkedList` class. The `remove` method deletes the key if it exists in the hash table by calling the `delete` method in the `LinkedList` class. The `printHashTable` method outputs the contents of the hash table. The complete `ChainingHashTable` class is shown in Fig. 11.1.

The main program would be similar to the one used for the insertion sort in Fig. 9.1. The new class is called `TestChainingHashTable`, and a hash table of size 5 is created using the following statement:

```
ChainingHashTable hashTable = new ChainingHashTable(5);
```

Instead of simply assigning the number to an array location as done in Fig. 9.1, the method `hashTable.insert(number)` is used inside the while loop. The variable `i` is removed from the while loop since there is no reason to keep track of the position of the array when using a hash table. Further, the column heading should be changed to `System.out.println("Hash Table")`. Also, as a

replacement of the `for` loop to print the contents of the array, the `printHashTable` method is used instead. Finally, the main program to create a hash table with any number of keys is shown below:

```
import java.util.*;
class TestChainingHashTable {
    public static void main(String[] args){
        Scanner scanner;
        scanner = new Scanner(System.in);
        int number;
        ChainingHashTable hashTable = new ChainingHashTable(5);
        System.out.println();
        System.out.print("Enter an integer or a -1 to stop: ");
        number = scanner.nextInt();
        while(number > -1) {
            hashTable.insert(number);
            System.out.print("Enter an integer or a -1 to stop: ");
            number = scanner.nextInt();
        }

        System.out.println();
        System.out.println("Hash Table");
        System.out.println();
        hashTable.printHashTable();
    }
}
```

The input and output when the above code is executed using 9 keys is shown below:

```
Enter an integer or a -1 to stop: 9
Enter an integer or a -1 to stop: 8
Enter an integer or a -1 to stop: 1
Enter an integer or a -1 to stop: 7
Enter an integer or a -1 to stop: 12
Enter an integer or a -1 to stop: 6
Enter an integer or a -1 to stop: 4
Enter an integer or a -1 to stop: 5
Enter an integer or a -1 to stop: 3
Enter an integer or a -1 to stop: -1
```

Hash Table

```
harray[0] List: 5
harray[1] List: 1 6
harray[2] List: 7 12
harray[3] List: 3 8
harray[4] List: 4 9
```

Next, the performance of a separate chaining hash table with an ordered linked list as discussed above is analyzed. An analysis of a hash table with an unordered linked list is left as an exercise at the end of the chapter. Assume that there are  $n$  items and that the `size` is the number of cells in the array. If there is a good hash function which distributes items evenly among the cells, each cell would have same number of items,  $n/\text{size}$ . Then a successful search with a separate chaining hash table depends on both  $n$  and `size` and it would take on average  $(n/\text{size})/2$  iterations. An unsuccessful search also takes  $(n/\text{size})/2$  iterations. In both cases, the searches on average would stop in the middle, therefore, the  $(n/\text{size})$  is divided by 2. Because the search has to take place in order to find the location of the item for the insertions and deletions, the analysis of both operations is the same as the searches as long as the good hash function is used.

If the `size` of the array is kept the same as the number of items  $n$  or larger, then  $n/\text{size} \leq 1$  becomes a constant run time for searches, insertions, and deletions. In the worst case, all the items are hashed into one location of the hash table, meaning the length of the chain is  $n$ . Then, all three operations take linear time.

As mentioned earlier, hashing can be efficient; however, if one cannot assume the hash function would work well with the given data sets, it might be better to use other data structures such as trees which guarantee the logarithmic worst case performance. Also, notice that items in a hash table are not sorted although each chain may be in order. If having items in the entire table sorted is important for an application, other data structures such as trees may be a better choice.

---

## 11.6 Implementation of Hash Tables Using the Java API

As discussed in Chap. 1, there exist in the Java programming language various libraries that contain a number of data structures known as the Java Application Program Interface (API). Stacks, queues, lists, trees, and hash tables discussed in this book can be implemented using classes in the standard libraries. In this section hash tables are implemented using the Java API to show how predefined classes can be used.

There are 7 classes in the Java API that implement hash tables. The most common ones are the `HashMap` and the `HashSet` classes. In each entry of the `HashMap` object, a key and information associated with the key, called a `value`,



are stored separately. The entry of the `HashSet` object is called an element, which consists of both the key and the value. The `HashMap` does not allow duplicate keys; however, it allows duplicate values associated with the key. On the other hand, the `HashSet` does not allow duplicate elements in the set. The following sections examine the `HashMap` and the `HashSet` classes, and implements simple hash tables.

### 11.6.1 The `HashMap` Class

The `HashMap` consists of two entries: a key and a value. Both a key and a value can be an instance of any class. As mentioned above, no duplicate keys are allowed in the `HashMap`; however, it allows duplicate values. A key is used to locate a value associated with the key. For example, in the case of maintaining horses' information the key could be an ID number and the value could be a name and a breed of the horse.

In the Java API `HashMap`, converting a key into a position of a hash table is accomplished first by calling the predefined `hashCode` method on a given key. The purpose of this method is to perform a hash function on the key and return an `int` value. This value is then transformed into a location in the hash table. If the key is an object of the `Integer` class, then the `hashCode` method simply returns the contents of the `Integer` object which is an `int` value. If the key is a `String` value, the `hashCode` method for a string defined in the Java API will return an `int` value. In both cases, the modulo operation is used to find the position of the hash table. Programmers can implement their own `hashCode` method to use either with the `HashMap` class or the `HashSet` class, which will be discussed in Sect. 11.6.3.

As mentioned in Sect. 11.5, the efficiency of the insertions, deletions, and searches on a hash table depends on the size of the table and the number of items. An object of `HashMap` has two parameters, a capacity and a maximum load factor. The capacity is the number of cells in the hash table and the maximum load factor is a value which indicates how full the hash table is before its capacity must be increased. The load factor is calculated by taking the number of entries in the hash table divided by its capacity. The default constructor of the `HashMap` class will initially create a hash table with the capacity of 16 and the maximum load factor of 0.75. When the number of entries in the hash table exceeds the product of the maximum load factor and the capacity at the time, the hash table is automatically rebuilt to double its capacity so that the chains in each cell do not become too long. If the hash table starts with the capacity of 16 and the maximum load factor of 0.75, then when the number of entries in the hash table becomes 13, the hash table will be expanded since  $16 \times 0.75 = 12$ . Other overloaded constructors can accept arguments so that programmers can choose the appropriate values depending on the application.

In order to insert the items in the hash table using the `HashMap`, the `put` method is used. The `put` method accepts two parameters, the key and the value.

The key is used to determine the location in the hash table and the value is the information associated with the key. It then returns the previous value linked to the key or null if the key was not in the hash table. If the key already exists in the hash table, the old value will be replaced with the new value. The `remove` method accepts a key, removes the entire entry, and returns the previous value associated with the key, or returns null if the key was not in the hash table. The `get` method accepts a key and returns the value linked to the key, or null if the key was not in the hash table.

The following code creates an object of the `HashMap` class named `horseMap` and information for 3 horses is entered. The 5 digit ID number is used for the key and the name of the horse is the value associated with the key. In order to use the `HashMap` class, the `java.util` package needs to be imported. Both the key and the value have to be a reference type, therefore, an `int` value is converted into an object containing an integer value using a wrapper class, `Integer`, and a string is by default an object.

```
import java.util.*;

public class HashMapHorse {
    public static void main(String[] args) {

        HashMap horseMap = new HashMap();
        horseMap.put(new Integer(15937), "Midnight Swirl");
        horseMap.put(new Integer(51303), "Krugerrand Dancer");
        horseMap.put(new Integer(75986), "Currency Kid");
        System.out.println(horseMap);
    }
}
```

The following is the output from the above program:

```
{15937 = Midnight Swirl, 75986 = Currency Kid, 51303 = Krugerrand Dancer}
```

It shows the association of the keys and values. Because of the nature of the hash table, items are neither sorted by keys nor values. Using the default hash table of size 16, since  $15937 \% 16 = 1$ , the information of the horse with ID number 15937 is stored into the location at 1. The horse with 75986 is saved at 2 since  $75986 \% 16 = 2$ . Finally, the horse with 51303 is at 7 since  $51303 \% 16 = 7$ . As mentioned before, a hash table is good on average for insertion, deletions, and searches; however, the unsorted feature is one of the weakness of hash tables.

Returning to the main program, what if there were duplicate values? The following code segment shows two horses having the same name but different keys.

```
import java.util.*;

public class HashMapHorse {
    public static void main(String[] args) {

        HashMap horseMap = new HashMap();
        horseMap.put(new Integer(15937), "Midnight Swirl");
        horseMap.put(new Integer(51303), "Krugerrand Dancer");
        horseMap.put(new Integer(15937), "Currency Kid");
        System.out.println(horseMap);
    }
}
```

The output below shows that this is allowed as long as they have different ID numbers.

```
{15937 = Krugerrand Dancer, 75986 = Currency Kid, 51303 = Krugerrand Dancer}
```

### How about duplicate keys?

```
import java.util.*;

public class HashMapHorse {
    public static void main(String[] args) {

        HashMap horseMap = new HashMap();
        horseMap.put(new Integer(15937), "Midnight Swirl");
        horseMap.put(new Integer(51303), "Krugerrand Dancer");
        horseMap.put(new Integer(15937), "Currency Kid");
        System.out.println(horseMap);
    }
}
```

The output from the above program shown below indicates that the old value of the key 15937 was replaced by the new value entered, meaning Midnight Swirl was replaced by Currency Kid. The most recent duplicate is the one stored in the hash table.

```
{15937 = Currency Kid, 51303 = Krugerrand Dancer}
```

Other useful methods in the `HashMap` class include `keySet` and `values`. The `keySet` method uses no parameters and returns the set of keys in the hash table. The `values` method does not require any parameters and returns the set of values. These methods can be used to output the contents of the hash table. In order to traverse elements in the set, an enhanced `for` loop can be used. First, the set of key values are extracted from the hash table using the `keySet` method as shown in the following statement:

```
Set<Integer> keys = horseMap.keySet();
```

Because the `Set` is a generic class, the type of the elements, `Integer`, was specified in the angle brackets. In order for the loop to iterate over every element in the set, a variable is declared in the enhanced `for` loop to refer to each element. The variable `id` of type `Integer` would hold the current value from the set, `keys`. In each iteration, the `get` method is used with `id` to get the name of the horse as shown below:

```
for(Integer id:keys)
    System.out.println("ID " + id + " belongs to "
                       + horseMap.get(id));
```

With the three horses in the hash table, the output from the above code segment is shown below. It prints each horse's information on a separate line.

```
ID15937 belongs to Midnight Swirl
ID 75986 belongs to Currency Kid
ID 51303 belongs to Krugerrand Dancer
```

The program below uses an enhanced `for` loop to output the contents of the hash table, and also shows the use of the `remove` and the `get` methods.

```
import java.util.*;

public class HashMapHorse {
    public static void main(String[] args) {

        HashMap horseMap = new HashMap();

        horseMap.put(new Integer(15937), "Midnight Swirl");
        horseMap.put(new Integer(51303), "Krugerrand Dancer");
        horseMap.put(new Integer(75986), "Currency Kid");
```

```
        System.out.println("ID 51303 belongs to "
                           + horseMap.get(51303));

        System.out.println();

        System.out.println("Removing horse with ID 15937");
        System.out.println();
        horseMap.remove(15937);

        System.out.println("Hash table contains: ");
        Set<Integer> keys = horseMap.keySet();
        for(Integer id:keys)
            System.out.println("ID " + id + " belongs to "
                               + horseMap.get(id));
    }
}
```

The output from the above code is:

```
ID 51303 belongs to Krugerrand Dancer
```

```
Removing horse with ID 15937
```

```
Hash table contains:
```

```
ID 75986 belongs to Currency Kid
```

```
ID 51303 belongs to Krugerrand Dancer
```

For more information about the `HashMap` class, refer to the [References and Useful Websites](#) section at the end the text which has the link to the Java API specification document at the Oracle website.

### 11.6.2 The HashSet Class

Whereas the `HashMap` is a collection of keys and values, the `HashSet` can contain only elements, so there are no duplicates. For example, if the hash set contains a set of integers {15937, 51303, 75986} and the number 51303 is to be added, the set would not change, since the 51303 already exists in the set and will not be added. Similar to the `HashMap` discussed in Sect. 11.6.1, an element can be an instance of any class. Also, like the `HashMap`, the default constructor will create a new empty

set of size 16 and a maximum load factor of 0.75. When the number of elements increases and the number of elements in the hash table exceeds the product of the maximum load factor and the capacity described in Sect. 11.6.1, the hash set is expanded automatically.

Useful methods in the `HashSet` class include the `add`, `remove`, and `contains` methods. The `add` method takes an element as a parameter and adds it to the set if the set does not contain that element. If the element is added successfully it returns `true`, otherwise it returns `false`. The `remove` method takes an element as a parameter and removes it from the set if it is present. If the element is removed successfully it returns `true`, otherwise it returns `false`. The `contains` method takes an element as a parameter and returns `true` if the set contains the element, otherwise it returns `false`. The following code inserts three numbers into a hash set and prints the contents:

```
import java.util.*;

public class HashSetHorse {
    public static void main(String[] args) {

        HashSet horseSet = new HashSet();
        horseSet.add(new Integer(15937));
        horseSet.add(new Integer(51303));
        horseSet.add(new Integer(75986));
        System.out.println(horseSet);
    }
}
```

Similar to the `HashMap`, it requires that the `java.util` package be imported in order to use the `HashSet` class. Since the element to be added needs to be a reference type, an integer is converted into an object containing an integer value using a wrapper class, `Integer`. The output from the above program shown below indicates that three elements are added to the set.

```
[15937, 75986, 51303]
```

The program below shows the use of the `remove` and `contains` methods.

```
import java.util.*;

public class HashSetHorse {
    public static void main(String[] args) {
```

```
HashSet horseSet = new HashSet();
horseSet.add(new Integer(15937));
horseSet.add(new Integer(51303));
horseSet.add(new Integer(75986));

System.out.println("Hash table contains: ");
System.out.println(horseSet);
System.out.println();

System.out.println("ID 15937 exists: "
                  + horseSet.contains(15937));
System.out.println();

System.out.println("ID 12345 exists: "
                  + horseSet.contains(12345));
System.out.println();

System.out.println("Removing ID 15937");
System.out.println();
horseSet.remove(15937);

System.out.println("Hash table contains: ");
System.out.println(horseSet);
System.out.println();
    }
}
```

Here is the output from the above program:

```
Hash table contains:
[15937, 75986, 51303]

ID 15937 exists: true

ID 12345 exists: false

Removing ID 15937

Hash table contains:
[75986, 51303]
```

For more information about the `HashSet` class, again refer to the References and Useful Websites section at the end of the text which has the link to the Java API specification document at the Oracle website.

### 11.6.3 The hashCode and Equals Methods

As mentioned before, choosing a good hash function is critical for getting good performance on insertions, deletions, and searches. Instead of using a default hash function from the Java API, a different one which is appropriate for a particular application can be implemented by programmers and used with the `HashSet` and the `HashMap` classes. Also, the `equals` method can be added by programmers to check the equality of two objects.

In the following example, the `HashSet` class is used to store objects of the `Horse` class, which keeps the name and the breed as data members. The `Horse` class has a constructor and each data member has a corresponding accessor and mutator. The `toString` method returns a string representation of the contents of the data members of an object. The `hashCode` and the `equals` methods are overridden methods that were inherited from the `Object` class. In the `hashCode` method, the name of the horse is used as a key to get a hash value. As described in Sect. 11.3, the Unicode value of each character of the name is added in the `hashCode` method. The return value from the `hashCode` method is then automatically divided by the number of cells in the hash table to find the exact location. The `hashCode` method is shown below:

```
public int hashCode() {  
    int hashcode = 0;  
    for(int i = 0; i<name.length(); i++)  
        hashcode = hashcode + name.charAt(i);  
    return hashcode;  
}
```

The `equals` method compares contents of each data member of two objects and if they are the same it returns true, otherwise, it returns false. Because the parameter of the `equals` method, `obj`, can be any class, before the comparison takes place, it has to be converted to the `Horse` type using a typecast in Line 3 of the following code:

```
public boolean equals(Object obj) {  
    boolean result = false;  
    Horse otherHorse = (Horse) obj;  
    if(otherHorse.name.equals(this.name)  
        && otherHorse.breed.equals(this.breed))  
        result = true;  
    return result;  
}
```

// Line 1  
// Line 2  
// Line 3  
// Line 4  
// Line 5  
// Line 6  
// Line 7  
// Line 8

The complete `Horse` class is shown in Fig. 11.2.



**Fig. 11.2** Horse class

```
public class Horse {  
  
    private String name, breed;  
  
    public Horse(String name, String breed) {  
        this.name = name;  
        this.breed = breed;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public String getBreed() {  
        return breed;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    public void setBreed(String breed) {  
        this.breed = breed;  
    }  
  
    public String toString() {  
        return (name + ", " + breed);  
    }  
  
    public int hashCode() {  
        int hashCode = 0;  
        for(int i=0; i<name.length(); i++)  
            hashCode = hashCode + name.charAt(i);  
        return hashCode;  
    }  
  
    public boolean equals(Object obj) {  
        boolean result = false;  
        Horse otherHorse = (Horse) obj;  
        if(otherHorse.name.equals(this.name)  
            && otherHorse.breed.equals(this.breed))  
            result = true;  
        return result;  
    }  
}
```

The main program creates an object of the `HashSet` class and the information for 3 horses is inserted. An object of `Horse` class is created as an argument of the `add` method. At the end, the contents of the hash table is output.

```
import java.util.*;

public class HashSetHorse {
    public static void main(String[] args) {

        HashSet horseSet = new HashSet();
        horseSet.add(new Horse("Midnight Swirl", "Quarter Horse"));
        horseSet.add(new Horse("Krugerrand Dancer", "Thoroughbred"));
        horseSet.add(new Horse("Currency Kid", "Thoroughbred"));
        System.out.println(horseSet);
    }
}
```

The output from the above code is shown below.

```
[Krugerrand Dancer, Thoroughbred,
Currency Kid, Thoroughbred,
Midnight Swirl, Quarter Horse]
```

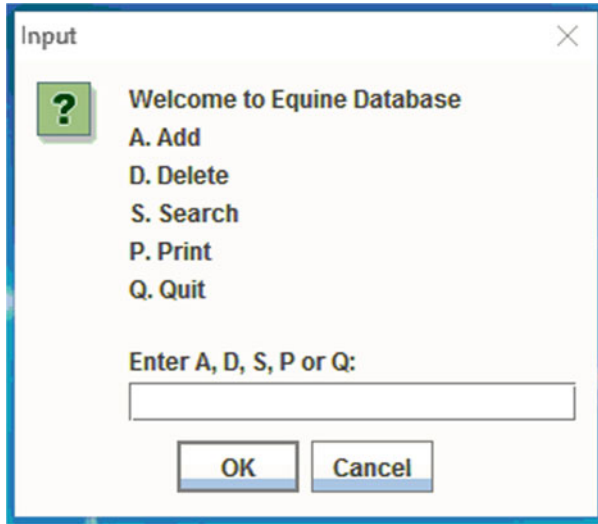
The Horse class including the hashCode and the equals methods can be used with the HashMap class the same way as done with HashSet class.

---

## 11.7 Complete Program: Hash Table Using the HashMap Class

A database system consists of a collection of related data and software to manipulate that data. An example includes library catalogs which contain information such as titles, authors, publishers, and years of books and journals. Another example are schools that keep track of information on students, instructors and courses. Also, companies need to deal with a vast amount of data on customers, products, and employees. In database systems, files of data are typically stored on magnetic disks, and data must be well-organized so that it can be accessed efficiently.

In this section, a small database which keeps horses' information is simulated using the HashMap class discussed in Sect. 11.6.1 and a simple GUI interface. Instead of having just a string data, an object of the Horse class that includes the horse's name and breed shown in Fig. 11.2 is used here. The program will be written so that when it is executed, the menu shown below will be displayed. Users will be able to enter one of the options in the text box. This is accomplished by using an input dialog box defined in the Java API JOptionPane class discussed in Appendix A of *Guide to Java* [2], or refer to the References and Useful Websites section at the end of this text which has the link to the Java API specification document at the Oracle website.



The Add option will ask users to enter an ID number, which is a key, and values such as the name and breed of a horse, which are input inserted into the hash table. The Delete option will ask users for an ID number, and information about that horse will be removed from the hash table. The Search option will also ask for an ID number, and the name and breed of the horse will be output. The Print option simply outputs all the contents in the hash table, and the Quit option will stop the execution of the program. A do-while loop in the main method will repeat displaying the menu and executing the options selected until the user enters Q in the text box. Besides the main method, the EquineDB class has 5 more private methods for each option such as the add, the delete, the search, and the print, along with the `getSelection` method which displays the menu.

```
import javax.swing.*;
import java.util.*;

class EquineDB {
    public static void main(String[] args) {
        HashMap horseMap = new HashMap();
        char option;
        do {
            option = getOption();
            switch(option) {
                case 'A': add(horseMap);
                           break;
                case 'D': delete(horseMap);
                           break;
```

```

        case 'S': search(horseMap);;
                break;
        case 'P': print(horseMap);;
                break;
    }
} while(option != 'Q');
System.exit(0);
}

private static char getOption() {
    char option;
    String str;
    str = "Welcome to Equine Database\n"
        + "A. Add\n"
        + "D. Delete\n"
        + "S. Search\n"
        + "P. Print\n"
        + "Q. Quit\n"
        + "\nEnter A, D, S, P or Q:\n";
    option = JOptionPane.showInputDialog(null, str).charAt(0);
    return option;
}

private static void add(HashMap horseMap) {
    int id;
    String name, breed;
    id = Integer.parseInt(JOptionPane.showInputDialog(null,
        "Enter ID number: "));
    name = JOptionPane.showInputDialog(null, "Enter name: ");
    breed = JOptionPane.showInputDialog(null, "Enter breed: ");
    horseMap.put(id, new Horse(name, breed));
}

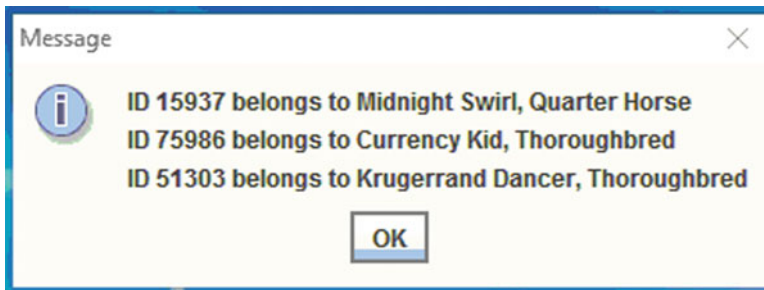
private static void delete(HashMap horseMap) {
    int id;
    id = Integer.parseInt(JOptionPane.showInputDialog(null,
        "Enter ID number: "));
    horseMap.remove(id);
}

private static void search(HashMap horseMap) {
    int id;
    id = Integer.parseInt(JOptionPane.showInputDialog(null,
        "Enter ID number: "));
    JOptionPane.showMessageDialog(null, "ID " + id

```

```
        + " belongs to " + horseMap.get(id));  
    }  
  
    private static void print(HashMap horseMap) {  
        String str = "";  
        Set<Integer> keys = horseMap.keySet();  
        for(Integer id:keys)  
            str = str + "ID " + id + " belongs to "  
                + horseMap.get(id) + "\n";  
        JOptionPane.showMessageDialog(null, str);  
    }  
}
```

When the above program is executed, and after three horses are entered in the database, the output from the print method looks as follows:



---

## 11.8 Summary

- Insertions, deletions, and searches can be performed efficiently using hashing, if the hash function is carefully chosen.
- A hash function returns the position of the key in the list to be stored or found.
- Ideally, a hash function should be simple to compute and ensure that any two distinct keys get different cells.
- A good hash function distributes keys evenly among the cells.
- There are many different hash functions such as the modulo operation and the mid-square method.
- When a key is hashed to the same value as an existing key, then it is called a collision.

- A solution for collisions in a hash table is separate chaining.
- Hash tables can be implemented using the Java API `HashMap` or `HashSet` class.
- The `hashCode` method in the Java API can be overridden.

---

## 11.9 Exercises (Items Marked with an \* Have Solutions in Appendix B)

1. Given the input {1982, 3411, 333, 632, 1090, 459, 379, 238} and a hash function, `key mod 10`, draw the resulting hash table using a separate chaining.
- \*2. Repeat the previous question using an array of size 7.
3. Given the input {9, 140, 35, 20, 49, 508, 22, 27} and a hash function, `key mod 11`, draw the resulting hash table using a separate chaining.
4. Repeat the previous question using an array of size 13.
- \*5. Write a method called `contains` which could be added to the `ChainingHashTable` class in Fig. 11.1. The `contains` method accepts one integer and returns true if the integer is in the hash table. Otherwise, it returns false.
6. Write a method called `size` which could be added to the `ChainingHashTable` class in Fig. 11.1. The `size` method does not accept any parameters and returns the number of items in the hash table.
7. Write a method called `isEmpty` which could be added to the `ChainingHashTable` class in Fig. 11.1. The `isEmpty` method does not accept any parameters and returns true if the hash table is empty. Otherwise, it returns false.
8. Write a program to insert 1000 random numbers in a separate chaining hash table using a hash function, `key mod 15`, and see if keys are distributed evenly in the hash table.
9. Implement the separate chaining hash tables discussed in Sect. 11.4 using an unordered linked list.
10. Implement the separate chaining hash tables discussed in Sect. 11.4 using a two-dimensional array instead of a singly linked list.
11. Discuss a disadvantage of implementing a separate chaining hash table using an array instead of a singly linked list.
12. Analyze the performance of the separate chaining hash tables discussed in Sect. 11.4 using an unordered linked list.
13. Explain how hashing could be implemented for a collection of keys in which keys were not unique. For example, several people may be identified by the same name.
14. List applications that might be appropriate to implement using hash tables instead of binary trees or simple arrays.
15. Implement a hash function using the mid-square method for string keys.

- \*16. Assume that the `hashTable` is an object of the Java API `HashMap` class and it is empty. What is the effect of each of the following method calls if the statements are executed in this order?

```
hashTable.put(new Integer(110), "while"); // Line 1
hashTable.put(new Integer(8), "double"); // Line 2
hashTable.put(new Integer(110), "while"); // Line 3
hashTable.put(new Integer(8), "int"); // Line 4
hashTable.remove(43); // Line 5
hashTable.remove(110); // Line 6
```

17. Implement a hash table using the Java API `HashMap` class which contains book information. The ISBN is the key and the book title, the author, the publisher, and year are associated with the ISBN number.
18. Write a complete program which stores the number of occurrences of words in a document

using the Java API `HashMap` class.

19. Assume that the `hashTable` is an object of the Java API `HashSet` class and it is empty. What is the effect of each of the following method calls if the statements are executed in this order?

```
hashTable.put(add Integer(110)); // Line 1
hashTable.put(add Integer(8)); // Line 2
hashTable.put(add Integer(110)); // Line 3
hashTable.remove(43); // Line 4
hashTable.remove(110); // Line 5
```

20. Implement a hash table using the Java API `HashSet` class which co book information such as the ISBN, the book title, the author, the publisher, and year.

---

## Appendix A: Glossary

All the terms in *italics* in the text can be found in the index, and some of these terms (including abbreviations) can be found here in the glossary. The descriptions of the terms in this glossary should not be used in lieu of the complete descriptions in the text, but rather they serve as a quick review. Should a more complete description be needed, the index can guide the reader to the appropriate pages where the terms are discussed in more detail.

|                        |                                                                                                                                                                                                                             |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ADT                    | Abstract Data Type.                                                                                                                                                                                                         |
| API                    | Application Program Interface.                                                                                                                                                                                              |
| Big O                  | A notation used to compare the relative order of magnitude of algorithms.                                                                                                                                                   |
| Binary expression tree | A binary tree used to store expressions such as arithmetic expressions.                                                                                                                                                     |
| Binary search tree     | A binary tree where if storing integers, all integers in left subtrees are smaller than the subroots and all integers in the right subtrees is larger than the subroots. Other types and orderings can be used as well.     |
| Binary tree            | A tree that has no more than two branches.                                                                                                                                                                                  |
| CPU scheduler          | The part of the operating system that schedules which program will be run next in the central processing unit.                                                                                                              |
| Data structures        | The different ways data can be stored along with the specific operations that can be performed on the data.                                                                                                                 |
| Doubly linked list     | A linked list where each node has both a next and back reference eliminating the need for a separate previous reference.                                                                                                    |
| FIFO                   | First-in, first-out as with a queue.                                                                                                                                                                                        |
| Generic types          | The ability to use different types with a given data structure.                                                                                                                                                             |
| GUI                    | Graphical User Interface.                                                                                                                                                                                                   |
| Hashing                | A technique using a key to provide direct access to a table for efficient searches, deletions, and insertions.                                                                                                              |
| Heap                   | A binary tree implemented as a complete tree with the children being either smaller or larger than its parent. When each parent is smaller than its children it is known as a minheap and if each parent larger, a maxheap. |



|                |                                                                                                                                                                                                                                                         |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Heap sort      | A very fast sort utilizing a heap and on average is $O(n \log n)$ .                                                                                                                                                                                     |
| IDE            | Integrated Development Environment.                                                                                                                                                                                                                     |
| Insertion sort | A relatively easy sort to write and good for smaller sets of data. On average, it is $O(n^2)$ and with data that is already in order, $O(n)$ .                                                                                                          |
| LIFO           | Last-in, first-out as with a stack.                                                                                                                                                                                                                     |
| Linked list    | A list that is usually implement using reference and objects.                                                                                                                                                                                           |
| List           | A data structure where items can be inserted at any location. Lists can be ordered or unordered.                                                                                                                                                        |
| MICR           | Magnetic Ink Character Recognition.                                                                                                                                                                                                                     |
| OCR            | Optical Character Recognition.                                                                                                                                                                                                                          |
| Priority queue | Based on a heap allows the item with the highest priority to be removed first.                                                                                                                                                                          |
| Queue          | A data structure having the first-in, first-out property.                                                                                                                                                                                               |
| Quick sort     | A highly recursive sort that is good with large sets of data. It is one of the fastest sorts and on average it is $O(n)$ . When trying to sort smaller subsets of data it helps to have the quick sort call another sort such as the insertion sort.    |
| Radix sort     | Sometimes known as the bucket sort and is very good for sorting physical items such as postal letters and checks. When sorting items in memory it can be one of the fastest sorts at $O(n)$ , but in doing so it can use a very large amount of memory. |
| RPN            | Reverse Polish notation.                                                                                                                                                                                                                                |
| Stack          | A data structure having the last-in, first-out property.                                                                                                                                                                                                |
| Tree           | A nonlinear data structure that can have any number of branches.                                                                                                                                                                                        |
| UML            | Unified Modeling Language.                                                                                                                                                                                                                              |

---

# Appendix B: Answers to Selected Exercises

## Chapter 1

- 1.A. 1 ( $x=0$ )  
1 ( $i=0$ )  
 $3 \times 1$  ( $i<3$ ) = 3  
 $3 \times 3$  ( $x=x+i*2$ ) = 9  
 $3 \times 2$  ( $i++$ ) = 6  
1 ( $i=0$ )  
-----  
Total: 21
- 2.A.  $O(n)$
- 3.A.  $O(n)$

## Chapter 2

|        |    |
|--------|----|
| [2]    | 5  |
| 1. [1] | 5  |
| [0]    | 78 |

- 3.
- ```
public int peek() {  
    return sarray[top-1];  
}
```

10. Both the `PalindromeVersion1` and `PalindromeVersion2` classes are  $O(n)$ .

## Chapter 3

1.

|     |     |     |     |
|-----|-----|-----|-----|
|     | 5   | 79  | 5   |
| [0] | [1] | [2] | [3] |

3.A.

count: 3, front: 3, rear: 1

4.

```
public int peek() {
    return qarray[front];
}
```

## Chapter 4

1.

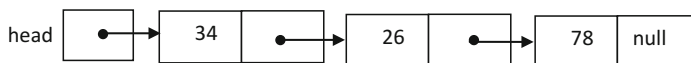
|     |     |     |     |  |
|-----|-----|-----|-----|--|
| 5   | 78  | 79  |     |  |
| [0] | [1] | [2] | [3] |  |

3.

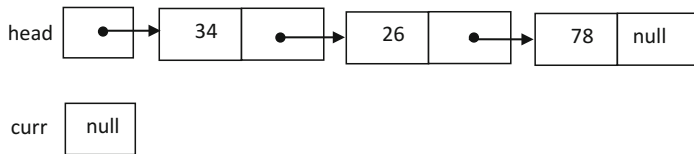
```
public boolean deleteAt(int position) {
    boolean deleted=false;
    if(!empty()){
        for(int j=position; j<count-1; j++)
            larray[j] = larray[j+1];
        count--;
        deleted = true;
    }
    else
        System.out.println("List is Empty");
    return deleted;
}
```

## Chapter 5

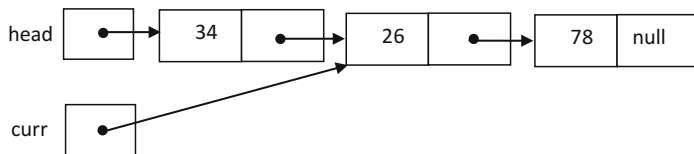
1.A.



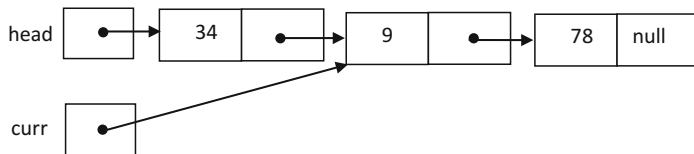
1.B. `Node curr;` declaration of a variable `curr` of type `Node`.



`Curr = head.getNext () ;` making `curr` reference the node containing data 26.



`Curr.setData (9) ;` change the value of the node containing 26 to 9.



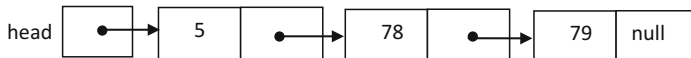
2.B. invalid statement, cannot call the `getNext` method after calling the `getData` method.

3.B. (320)

4.B. `head = head.getNext ().getNext ()`  
or `head = current.getNext ().getNext ()`

## Chapter 6

1.



4.

```
public boolean deleteTail() {
    Node ptr = head;
    boolean deleted = false;
```

```

if(head != null) { // list is empty
    deleted = true;
    if(head.getNext() != null) { // list has more than 1 items
        while(ptr.getNext().getNext() != null) {
            ptr = ptr.getNext();
        }
        ptr.setNext(null);
    }
    else // list has one item
        head = null;
}
return deleted;
}

```

7.

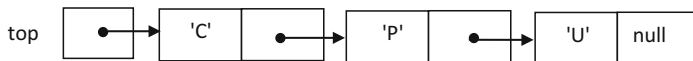
```

public boolean isEmpty() {
    return head == null;
}

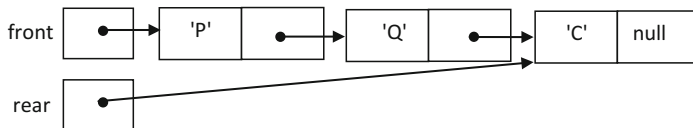
```

## Chapter 7

2.

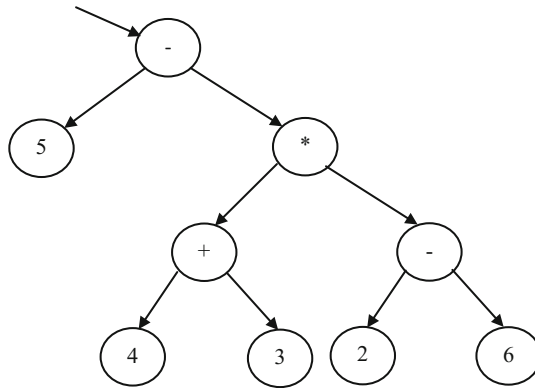


4.



## Chapter 8

2.D.



8.

```

public void traverseBSTPreorder() {
    System.out.print("The preorder is:");
    if(root!=null)
        traversePreorder(root);
    else
        System.out.print(" " + "Empty");
    System.out.println();
}

private void traversePreorder(Node ptr) {
    System.out.print(" " + ptr.data);
    if(ptr.left != null)
        traversePreorder(ptr.left);
    if(ptr.right != null)
        traversePreorder(ptr.right);
}

```

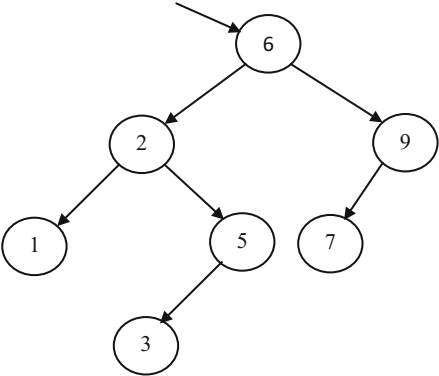
12.

```

private int findMinRec(Node p) {
    int data = -1;
    if(p != null)
        if(p.left == null)
            data = p.data;
        else
            data = findMinRec(p.left);
    return data;
}

```

18.



**Chapter 9**

3.B. 0110001

3.E. 46

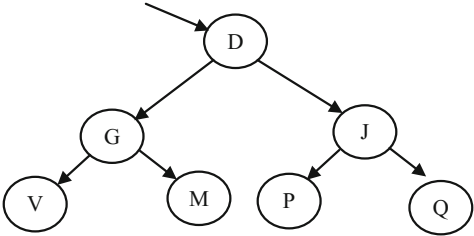
5. 41 32 27 19 58 96 74 65 83

6. 41 32 83 74 65 96 27 58 19

**Chapter 10**

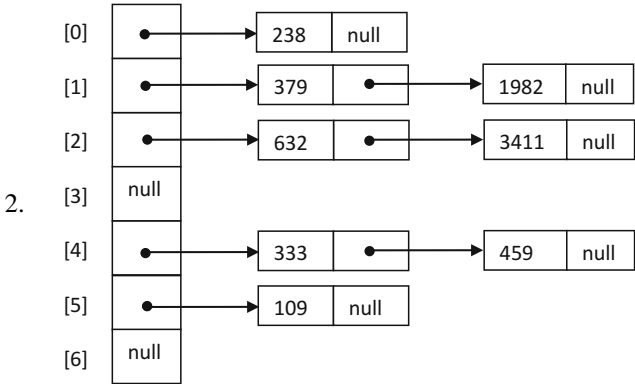
|      |     |   |
|------|-----|---|
| 1.B. | [0] | C |
|      | [1] | F |
|      | [2] | O |
|      | [3] | I |
|      | [4] | L |

2.A.



3.C. K

Chapter 11



5. The search method in LinkedList class in Figure 5.X needs to be change to a public method.

```
public boolean contain(int key) {
    boolean result = false;
    int i = 0;
    while(i<harray.length && result == false) {
        if(harray[i].search(key) == true)
            result = true;
        i++;
    }
    return result;
}
```



16.

Line 1: `while` was addedLine 2: `double` was added

Line 3: no change

Line 4: `double` was replaced by `int`

Line 5: no change

Line 6: `while` was removed

---

## References and Useful Websites

### References

1. Streib JT (2011) Guide to assembly language: a concise introduction. Springer, London
2. Streib JT, Soma T (2014) Guide to java: a concise introduction to programming. Springer, London

### Useful Websites

- “Java™ Platform, Standard Edition 8 API Specification,” List of classes and packages in Java 8. <http://docs.oracle.com/javase/8/docs/api/index.html>
- “Package javax.swing,” information on `javax.swing` package discussed in Chapter 6. <https://docs.oracle.com/javase/8/docs/api/javax/swing/package-summary.html>
- “Package java.awt,” information on `java.awt` package discussed in Chapter 6. <https://docs.oracle.com/javase/8/docs/api/java/awt/package-summary.html>
- “Class HashMap,” information on `HashMap` class discussed in Chapter 10. <https://docs.oracle.com/javase/8/docs/api/java/util/HashMap.html>
- “Class HashSet,” information on `HashSet` class discussed in Chapter 10. <https://docs.oracle.com/javase/8/docs/api/java/util/HashSet.html>
- “Class JOptionPane,” information on dialog boxes discussed in Chapter 10. <http://docs.oracle.com/javase/8/docs/api/javax/swing/JOptionPane.html>

---

# Index

## A

- Abstract Data Type (ADT), 8
- Abstraction, 8
  - data abstraction, 8
  - procedural abstraction, 8
- Algorithm analysis, 2, 4, 6, 7
- Application Programming Interface (API), 12
- Array versus references, 186
- Assembly language, 97

## B

- Big O, 3
  - comparison, 7
- Binary expression tree, 201–203, 205, 207, 216, 222
  - creating from prefix expression, 216, 218, 244
  - inorder traversal, 222–225
  - preorder traversal, 203, 205, 206, 208, 222
- Binary search trees, 221, 248
  - creating, 226, 229
  - finding minimum, 230
  - inorder traversal, 222, 223, 225
  - removing an item, 231
- Binary trees, 199, 201, 221, 294
  - child, 200
  - complete tree, 199
  - depth, 199
  - directed edges, 199
  - edges, 199
  - full tree, 200
  - generic if for recursion, 220, 222
  - height, 199
  - nodes, 199
  - nonterminal nodes, 199
  - parent, 200

- root, 199
- terminal nodes, 199

## C

- CompareTo, 99, 100, 163, 330, 332, 334
- CPU scheduler, 67

## D

- Data structures, 1
- Dynamic, 8

## E

- Equals, 129, 157, 163, 165, 193
- Event-driven programs, 187

## F

- FIFO, *See* Queues

## G

- Generic types, 32, 33, 41
  - typecasting, 33
  - type parameter, 32
- Graphical User Interface (GUI), 187

## H

- Heaps, 294, 299, 316, 335
  - concepts, 293
  - creating, 293, 295, 320, 323
  - maxheap, 295
  - minheap, 295
  - priority queues, *See* Priority queues
- Heap sort, 307, 310, 314, 319, 321, 328, 335, 338
  - makeHeap, 324
  - program, 321, 328, 329
  - simplified heap sort, 319

**I**

Infix, 37, 202, 203, 211, 213, 215, 216, 220

Insertion sort

analysis, 231

Integrated Development Environment (IDE), 13

Interface, 8, 10–12, 15

**L**

LIFO, *See* Stacks

Linked list, ordered, 135, 137, 138, 140, 142, 145, 147, 157–159, 163, 169, 172

delete, 151

doubly linked list, 157, 158, 160

inner Node class, 160, 167

insert, 136, 138, 140, 142–146

Linked list, unordered

links, 115, 117, 120, 140

node class, 114, 115

object, 109, 112, 114

output, 114, 121, 122, 124–126

iterative, 130, 137

recursive, 127

references, 109, 111

Lists, array based, 77, 81, 105

delete, 87

empty, 87, 89

full, 82, 83

insert, 78

search, 91

**M**

Magnetic Ink Character Recognition (MICR), 275

**O**

Optical Character Recognition (OCR), 275

**P**

Postfix, 37–40, 202, 216, 247

Prefix, 37, 38, 41, 47, 202, 203, 205, 206, 211, 216, 220

Priority queues, 293, 295, 306, 329, 335

concepts, 293

remove, 309

walk-through, 314, 320

**Q**

Queues, 49, 51, 67, 73, 77, 81

array based, 45, 77, 78, 82

circular queue, 55

dequeue, 50, 56, 57, 60

empty, 58

enqueue, 50–52, 54–56, 60

first-in, first-out (FIFO), 49

full, 58, 62

priority queues, *See* Priority queues

Queues, reference based, 173, 178, 183

dequeue, 198

empty, 180

enqueue, 182

full, 177

Quick sort

concepts, 255, 273

potential problems, 294, 302

quickSort, 258, 260

sort3, 258

swapElements, 258

**R**

Radix sort, 273, 277, 280

analysis, 280

binary conversion, 275, 276

concepts, 255, 273

fewer pockets, 275, 285

memory, 273, 275, 277

more pockets, 275, 285

program, 272, 280

Round-Robin (RR) scheduling algorithm, 67

**S**

Sorting, 320, 364

heap sort, *See* Heap sort

insertion sort, 231, 249

analysis, 231

quick sort, *See* Quick sort

radix sort, *See* Radix sort

Stacks, array based, 17, 37, 49, 60, 62

empty, 25

full, 27

infix, 37

last-in, first-out (LIFO), 17

pop, 17

push, 17

reversing integers, 29

Stacks, reference based, 173

empty, 175

full, 176

pop, 176

push, 173, 175

reversing integers, [177](#)  
Static, [8](#)

## T

ToString, [165](#)  
Trees, [199](#)

## U

Unified Modeling Language (UML), [11](#)

## W

Wrapper class, [33](#), [36](#), [37](#)